

Machine Learning & Deep Learning

The Complete Guide - From Beginner to Expert

Foundations · Classical ML · Deep Learning · Transformers · Generative Models · Quantization & Pruning · On-Device Training · Reinforcement Learning · Time Series · Recommenders · Vision · NLP · Speech · Distributed Systems · Causal Inference · MLOps

Contents	46 chapters in 6 parts, plus 3 appendices
Level	Absolute beginner to research practitioner
Style	Intuition first, then the mathematics, then working code
Worked examples	Every core algorithm is computed by hand on real numbers you can verify with a calculator
Code	Python / NumPy / PyTorch (framework-agnostic where possible)

How To Use This Book

This book is written to be read front to back by someone who has never trained a model, and to be used as a reference by someone who trains them for a living. Every chapter follows the same three-beat rhythm: first the **intuition** in plain language, then the **mathematics** written out step by step with no skipped algebra, then **code** that you can run.

Nothing is assumed beyond high-school algebra. Everything else - vectors, matrices, derivatives, probability - is built up in Chapter 2 and used consistently afterwards.

The six parts

Part	Chapters	What you get out of it
I - Foundations	1-4	Vocabulary, the mathematics you actually need, what 'learning' means formally, and how to prepare data.
II - Classical ML	5-13	Linear and logistic regression, trees, SVMs, boosting, clustering, PCA, and how to measure a model honestly. Still the right answer for most tabular problems.
III - Deep Learning Core	14-20	Neurons, backpropagation derived by hand, activations, optimizers, normalization, regularization, and a practical debugging playbook.
IV - Architectures	21-27	CNNs, RNN/LSTM, attention and Transformers, LLMs, VAEs/GANs/diffusion, graph networks, and self-supervised learning.
V - Expert Topics	28-35	Quantization, pruning and sparsity, distillation, on-device and federated training, reinforcement learning, uncertainty and robustness, MLOps, and how to do research.
VI - Domains, Theory and Systems	36-46	Why generalization works at all, probabilistic modelling, forecasting, recommenders, detection and segmentation, NLP and retrieval, speech and multimodal models, GPUs and distributed training, causal inference and A/B testing, privacy and security, and a complete thirteen-stage project walkthrough.

Reading paths

- **Complete beginner (about 6 months, part-time):** Chapters 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 13 -> 9 -> 11, then Part III in order. Do the exercises at the end of every chapter before moving on.
- **Starting a real project today:** read Chapter 46 first - it is the thirteen-stage procedure end to end - and follow its references back into the chapters as each stage needs them.
- **Programmer who wants deep learning fast:** skim 1-3, read 4, then jump to 14-20, then pick the architecture chapter that matches your data (21 and 40 for images, 22-23 for sequences, 26 for graphs, 38 for time series).
- **Practitioner shipping to devices:** Part III as refresher, then 28-31 (quantization, pruning, distillation, on-device training), 43 (hardware and memory arithmetic), 34 (MLOps) and 46 (the full project walkthrough).
- **Working on a specific domain:** go straight to its chapter and read backwards - 38 time series, 39 recommenders, 40 vision beyond classification, 41 NLP and retrieval, 42 speech and multimodal.
- **Training or serving large models:** 23-24, then 43 (memory, parallelism, MFU, serving) and 36 (scaling laws).
- **Deciding rather than predicting:** 45 (causal inference and A/B testing) with 13 (evaluation) - the pair that prevents the most expensive mistakes in industry.
- **Interview preparation:** 3, 5, 6, 13, 15, 17, 21, 23, 36, plus the glossary in Appendix B.

Conventions used throughout

Symbol / style	Meaning
x, y	Scalars in italics in the maths; monospace in the code.
$\mathbf{x}$ (bold)	A vector; by default a column vector of shape $(d, 1)$.

Symbol / style	Meaning
X (bold capital)	A matrix; the design matrix has shape (n, d): n rows = samples, d columns = features.
theta, w, b	Learnable parameters: generic parameters, weights, bias.
L, J	Per-sample loss L, and full objective / cost J averaged over data.
eta (or lr)	Learning rate.
Coloured boxes	KEY IDEA = the one thing to remember. MATH = a derivation. INTUITION = the mental picture. PRACTICAL TIP = what to actually do. PITFALL = a mistake people really make. EXPERT CORNER = depth you can skip on a first read.

Worked examples

Explanations are cheap; arithmetic is not. Wherever an algorithm has a core computation, this book performs it on real numbers and shows every intermediate value, so you can check the claim rather than accept it. Among them: least squares solved by hand on five houses; a gradient-descent trace step by step; a logistic regression trained on four points; an exhaustive decision-tree split search; three rounds of gradient boosting on six numbers; k-means and PCA computed in full; every classification metric derived from one confusion matrix; a complete forward and backward pass through a small network with a numerical gradient check; one Adam update; convolution and receptive-field arithmetic; attention computed on three tokens; and eight weights quantized to INT8 with the exact error.

LEARN BY REBUILDING

Every algorithm in Parts II and III is presented so that you can reimplement it in NumPy in under 100 lines. Do that at least once for linear regression, logistic regression, a decision tree, and a two-layer network trained with your own backpropagation. Nothing else produces the same level of understanding - libraries hide exactly the parts that matter.

How this edition connects the book to the thesis project

This edition keeps the whole original book and adds, inside every section, an explanation for a reader who is starting from zero and a direct link to the thesis project. You do not need to jump to a separate part: each time a concept appears, the text right next to it says what the concept means in everyday words, whether our project uses it, and where.

Where the project numbers come from. (1) Results of the model -- confusion matrix, confidence, loss curves, the dead zone -- come from the project's own C trainers (`train_c.c`, `train_c_sym.c`), compiled and run unchanged on the real data; their logs are kept in the book's build folder. (2) The 10-seed averages come from the project's own C sweeps (`run_matrix.py` drives the C binaries) as recorded in its README. (3) Facts about the data (class counts, channel statistics, walking frequency) were read with NumPy from the same .npy files the C trainer reads. (4) A few methods the project does not implement (decision trees, random forests, k-means, SVM, ...) are illustrated with scikit-learn in Python; those numbers carry a grey note saying so. (5) "FROM OUR C CODE" boxes show lines copied from the project's source files at build time.

Citations such as [4] or [1, Ch. 6] refer to the numbered References at the end of the book. Numbers marked "measured" were computed from the project's real data files or taken from the project's logged experiments.

THE TWO NEW BOXES YOU WILL SEE IN EVERY SECTION

FROM ZERO (green-blue): the idea of the section again, slowly, with every word defined and a small example with numbers. Read it before the original text if the section feels hard.

IN OUR PROJECT (green = used, orange = partly used, grey = not used): if the project uses the concept, it shows exactly where (which file, which layer) and the real numbers; if not, it says why not and what the project does instead. **WORKED ON OUR PROJECT** boxes are longer calculations with the project's data.

MEET THE PROJECT — READ THIS ONCE, IT IS USED EVERYWHERE

The task. A phone worn on the waist measures how the body moves. From 2.56 seconds of these measurements the program must say what the person is doing: WALKING, walking UPSTAIRS, walking DOWNSTAIRS, SITTING, STANDING or LAYING. This is called human activity recognition (HAR). The data are the public UCI HAR dataset [9].

The data. The phone has an accelerometer (measures acceleration, including gravity) and a gyroscope (measures rotation), each along 3 axes. The dataset gives 9 signals ("channels"): body acceleration x/y/z, gyroscope x/y/z, total acceleration x/y/z. Each signal is measured 50 times per second. The recording is cut into **windows** of 128 measurements (2.56 s). One window is therefore a small table of 9 rows x 128 columns = 1,152 numbers, and it carries one **label** (the activity). There are 6617 training windows, 735 validation windows and 2947 test windows. The test windows come from 9 people who are not in the training data.

The model. A small neural network of the kind called a 1-D convolutional network (explained in Chapter 21). It has 10,214 adjustable numbers ("parameters") and turns one window into 6 scores, one per activity. It is trained twice with the same settings: in Python with PyTorch on a PC (the reference), and in C with the ODT library written for the thesis. Both reach about 90% correct answers on the test people [35].

The board. The C version is meant to run -- and to *learn* -- on an STM32F746 microcontroller: 216 MHz, 320 KB of working memory (RAM), 1 MB of program memory (flash). Training on such a small chip is the point of the thesis. To make it fit, the numbers are stored as 8-bit integers instead of 32-bit decimals ("quantisation", Chapter 28, [3]) and most connections are switched off while training keeps rewiring them ("RigL sparsity", Chapter 29, [4]).

The rest of the library. The ODT library also contains layers and examples that the HAR model does not use but other project examples do: an ECG autoencoder (`ecg_anomaly_ae`), keyword spotting from speech (`kws_mfcc`, `kws_raw`), MNIST digits, a continual-learning version of HAR with a PCA-based replay memory (`train_c_continual`), GroupNorm, LayerNorm, Dropout, AdamW, learning-rate schedules and a mean-squared-error loss [35]. When a concept is used only there, the IN OUR PROJECT box says so.

Table of Contents

1. What Machine Learning Really Is	17
1.1 A worked micro-example before any theory	17
1.2 The vocabulary, defined once and used forever	18
1.3 The four learning paradigms	19
1.4 Where deep learning fits	20
1.5 The taxonomy at a glance	21
1.6 A first end-to-end program	22
1.7 Why machine learning works now and not in 1990	23
1.8 A day in the life of a machine-learning project	24
1.9 Reading the training loop as a sentence	24
1.10 What can go wrong (a preview of Chapter 3)	25
2. The Mathematical Toolkit You Actually Need	27
2.1 How to read the mathematics in this book	27
2.2 Linear algebra: the language of data	28
2.3 Calculus: how the loss reacts to a parameter	30
2.4 Probability: reasoning under uncertainty	31
2.5 Worked example: matrix shapes in a real layer	33
2.6 Worked example: a derivative you can check by hand	34
2.7 Worked example: Bayes' rule as counting	34
2.8 Optimisation: moving downhill	35
3. The Learning Problem: Generalization, Bias and Variance	38
3.1 The formal setup	38
3.2 Overfitting and underfitting, seen on a curve	38
3.3 The bias-variance decomposition, derived	40
3.4 Splitting data honestly	41
3.5 Data leakage: the silent score inflator	42
3.6 Learning curves: the diagnostic you should always plot	43
3.7 Overfitting made concrete: one dataset, three models	43
3.8 Cross-validation, step by step	44
3.9 How much data do I need?	45
3.10 No free lunch, and what it means for you	46
4. Data: Collection, Cleaning, Features and Splits	47
4.1 Types of data and what they demand	47
4.2 Cleaning: missing values, outliers, duplicates	47
4.3 Scaling and transformation	48
4.4 Encoding categorical variables	49
4.5 Feature engineering that pays	50

4.6 Class imbalance	50
4.7 Data augmentation	51
4.8 A worked cleaning session	52
4.9 Windowing time series, correctly	52
4.10 Dataset documentation and ethics	53
5. Linear Regression, Derived Completely	56
5.1 The model	56
5.2 The loss and where it comes from	56
5.3 Solution 1: the normal equations (closed form)	57
5.4 Solution 2: gradient descent (the scalable way)	58
5.5 Interpreting and validating a linear model	59
5.6 Polynomial and basis-function regression	59
5.7 The five houses, solved completely by hand	60
5.8 The same fit by gradient descent, step by step	61
5.9 From least squares to probability	62
6. Logistic and Softmax Regression	63
6.1 From linear score to probability	63
6.2 The loss: binary cross-entropy	63
6.3 Multiclass: softmax regression	64
6.4 Decision boundary, odds, and interpretation	66
6.5 Logistic regression on four points, by hand	67
6.6 Sigmoid, logit and probability: three views of one number	67
6.7 Practical matters	68
7. Regularization and Model Selection	70
7.1 Penalty-based regularisation	70
7.2 Other forms of regularisation	71
7.3 Choosing hyperparameters	72
7.4 Model selection criteria without a validation set	73
7.5 A disciplined selection protocol	73
8. Instance-Based and Probabilistic Models	75
8.1 k-Nearest Neighbours	75
8.2 Naive Bayes	76
8.3 Linear and quadratic discriminant analysis	77
9. Decision Trees	78
9.1 How a split is chosen	78
9.2 Controlling growth	79
9.3 Strengths, weaknesses, and the properties that matter downstream	79
9.4 A complete split search on a tiny dataset	80

9.5 Reading a tree out loud	81
9.6 Feature importance - and how it misleads	81
10. Support Vector Machines and Kernels	83
10.1 Maximum margin classification	83
10.2 The dual problem and the kernel trick	84
10.3 Practical guidance	84
11. Ensembles: Bagging, Random Forests and Boosting	86
11.1 Why averaging works	86
11.2 Bagging and random forests	86
11.3 Boosting, from AdaBoost to gradient boosting	87
11.4 Modern implementations	88
11.5 Gradient boosting on six points, three rounds	88
11.6 Bagging versus boosting, side by side	89
11.7 Stacking and blending	90
12. Unsupervised Learning and Dimensionality Reduction	91
12.1 Clustering	91
12.2 Dimensionality reduction	92
12.3 k-means, iteration by iteration	93
12.4 PCA on ten points, computed in full	94
12.5 Anomaly detection	95
13. Evaluation: Metrics, Calibration and Imbalance	97
13.1 Classification: the confusion matrix and everything derived from it	97
13.2 Threshold-free metrics: ROC and PR curves	98
13.3 Choosing the operating threshold	99
13.4 Regression metrics	99
13.5 Probability calibration	100
13.6 One confusion matrix, every metric computed	101
13.7 Reading a ROC curve and a PR curve of the same model	102
13.8 Statistical significance and reporting	102
13.9 Slice-based evaluation	103
14. From a Single Neuron to a Multilayer Network	106
14.1 The artificial neuron	106
14.2 The multilayer perceptron	106
14.3 The universal approximation theorem, honestly stated	107
14.4 Counting parameters and cost	108
14.5 Your first network, twice	108
15. Backpropagation, Derived and Implemented	110

15.1 The four equations	110
15.2 Why reverse mode, and what it costs	111
15.3 A complete NumPy implementation	112
15.4 Gradient checking	113
15.5 Backpropagation on real numbers, end to end	113
15.6 What ReLU's gate does to the gradient	115
15.7 Vanishing and exploding gradients	116
15.8 Automatic differentiation in practice	116
16. Activations, Initialization and Loss Functions	118
16.1 Activation functions	118
16.2 Weight initialisation	119
16.3 Loss functions, and what each one assumes	120
17. Optimization Algorithms and Learning-Rate Schedules	123
17.1 The algorithms, in the order they were invented	123
17.2 Learning-rate schedules	125
17.3 Batch size, learning rate, and their interaction	126
17.4 Finding the learning rate	127
17.5 One Adam update, arithmetic included	127
17.6 Choosing a learning rate: what each failure looks like	128
17.7 Momentum, seen as a ball on a surface	129
17.8 Gradient clipping and stability	130
18. Normalization Layers	131
18.1 Batch normalisation	131
18.2 The alternatives, and how they differ	132
18.3 Placement: pre-norm versus post-norm	133
19. Regularization for Deep Networks	134
19.1 Dropout	134
19.2 Weight decay	135
19.3 Early stopping	135
19.4 Augmentation and label-level regularisers	135
19.5 Ensembling for deep networks	137
19.6 A regularisation recipe by data size	137
20. Training in Practice: A Debugging Playbook	139
20.1 Start correctly	139
20.2 Symptom-to-cause table	139
20.3 Instrumentation worth having from day one	140
20.4 Mixed precision and throughput	141
20.5 Reproducibility	141

21. Convolutional Neural Networks	144
21.1 The convolution operation	144
21.2 The standard building blocks	145
21.3 Receptive field - the quantity to reason about	146
21.4 Convolution arithmetic worked through	146
21.5 Cost of one real layer, counted exactly	147
21.6 Receptive field, computed layer by layer	148
21.7 A short history worth knowing	148
21.8 Beyond classification	149
21.9 Transfer learning - the default workflow for vision	149
22. Sequence Models: RNN, LSTM and GRU	151
22.1 Why plain RNNs fail on long sequences	151
22.2 LSTM: a cell state with gates	151
22.3 Sequence-to-sequence and the birth of attention	152
22.4 When to still use an RNN in the 2020s	153
23. Attention and the Transformer	154
23.1 Scaled dot-product attention	154
23.2 Multi-head attention	154
23.3 The other components	155
23.4 Positional information	155
23.5 Complexity and the efficiency ladder	156
23.6 Implementing attention	156
23.7 Attention computed on three tokens	157
23.8 Why causal masking makes parallel training possible	158
23.9 Sizing a Transformer block	159
23.10 The three architectural families	159
23.11 Transformers outside text	160
24. Large Language Models	161
24.1 Tokenization	161
24.2 Pretraining	161
24.3 Post-training: from predictor to assistant	162
24.4 Parameter-efficient fine-tuning	162
24.5 Inference: what actually costs money	163
24.6 What 'next token prediction' actually looks like	163
24.7 The KV cache, sized with real numbers	164
24.8 Prompting, from worst to best	165
24.9 Using LLMs well	165
25. Generative Models: Autoencoders, VAEs, GANs and Diffusion	167

25.1 Autoencoders	167
25.2 Variational autoencoders	167
25.3 Generative adversarial networks	168
25.4 Diffusion models	169
25.5 The other families, briefly	170
25.6 Evaluating generative models	170
26. Graph Neural Networks	171
26.1 Message passing	171
26.2 Tasks and readouts	171
26.3 The characteristic problems	172
27. Self-Supervised and Transfer Learning	174
27.1 Pretext tasks by modality	174
27.2 Contrastive learning in detail	175
27.3 Transfer learning strategies	175
27.4 Semi-supervised learning	176
28. Efficient Deep Learning I: Quantization	178
28.1 Why it works and why it pays	178
28.2 The mathematics of affine quantization	179
28.3 Granularity and what it costs	180
28.4 Post-training quantization (PTQ)	180
28.5 Quantization-aware training (QAT)	181
28.6 Quantizing eight real weights, by hand	183
28.7 What happens as the bits come off	184
28.8 Asymmetric quantization for activations, worked	185
28.9 The whole INT8 layer, arithmetic included	186
28.10 What breaks, and how to fix it	186
28.11 Reporting quantization results honestly	187
29. Efficient Deep Learning II: Pruning and Sparsity	188
29.1 Structured versus unstructured	188
29.2 What to prune: saliency criteria	189
29.3 When to prune: the four schedules	189
29.4 The lottery ticket hypothesis	190
29.5 Pruning in practice	190
29.6 Pruning a small layer, weight by weight	192
29.7 How far can you actually prune?	193
29.8 Compression stacked, with the numbers	193
29.9 Sparsity in large language models	194
29.10 Combining compression techniques	194

30. Efficient Deep Learning III: Distillation, NAS and Efficient Design	196
30.1 Knowledge distillation	196
30.2 Efficient architecture design	197
30.3 Neural architecture search	197
30.4 Choosing a compression strategy	198
31. On-Device and Federated Training	199
31.1 Why train on the device at all	199
31.2 The resource wall	199
31.3 Techniques that make on-device training possible	200
31.4 Federated learning	201
31.5 The deployment stack for edge machine learning	202
32. Reinforcement Learning	204
32.1 The formalism: Markov decision processes	204
32.2 The two families	204
32.3 Q-learning and DQN	205
32.4 Policy gradients and PPO	205
32.5 A gridworld you can solve on paper	206
32.6 Why reinforcement learning is harder than supervised learning	207
32.7 Where RL is worth it	207
33. Uncertainty, Robustness and Interpretability	209
33.1 Two kinds of uncertainty	209
33.2 Estimating uncertainty in deep networks	209
33.3 Out-of-distribution detection and drift	210
33.4 Adversarial robustness	211
33.5 Interpretability	212
33.6 Fairness	213
34. MLOps: Shipping and Keeping Models Alive	214
34.1 The lifecycle	214
34.2 Versioning everything	214
34.3 Serving patterns	215
34.4 Monitoring	215
34.5 Retraining	216
34.6 Testing machine learning systems	216
34.7 Cost and carbon	217
35. Doing Research and Reading the Literature	218
35.1 Reading a paper efficiently	218
35.2 Running an experiment that survives scrutiny	218
35.3 Where the field is heading	219

36. Learning Theory: Why Generalization Is Possible At All	221
36.1 Empirical risk minimization, stated precisely	221
36.2 A single hypothesis: Hoeffding's inequality	222
36.3 Finite hypothesis classes: the union bound	222
36.4 Infinite classes: shattering and VC dimension	223
36.5 Rademacher complexity and margin bounds	224
36.6 Where the classical story breaks: double descent	224
36.7 Scaling laws: the empirical theory that replaced the bounds	225
36.8 What theory gives you, and what it does not	226
37. Probabilistic Machine Learning: Bayes, EM, HMMs and Gaussian Processes	228
37.1 The Bayesian recipe in one page	228
37.2 Conjugacy, worked on real numbers	228
37.3 Latent variables and the EM algorithm	229
37.4 Hidden Markov models and dynamic programming	231
37.5 Gaussian processes: distributions over functions	231
37.6 When the integral is intractable: VI and MCMC	232
37.7 Choosing your weapon	233
38. Time Series and Forecasting	235
38.1 What makes a time series different	235
38.2 Decomposition: trend, seasonality, remainder	235
38.3 Stationarity, differencing, ACF and PACF	236
38.4 The classical families	237
38.5 The machine-learning approach: supervised windows	237
38.6 Deep learning for forecasting	238
38.7 Backtesting: the only evaluation that counts	239
38.8 Metrics, and the one that is quietly broken	239
38.9 Hierarchies, intermittency and anomalies	240
39. Recommender Systems	241
39.1 The industrial shape of the problem	241
39.2 Feedback data and what it really means	241
39.3 Neighbourhood collaborative filtering	242
39.4 Matrix factorization, the model that won Netflix	242
39.5 Neural retrieval and sequential models	243
39.6 Ranking and its metrics	244
39.7 The problems that only recommenders have	245
40. Computer Vision Beyond Classification	247
40.1 The task taxonomy	247
40.2 IoU, the quantity everything is built on	247

40.3 Detector families	248
40.4 Segmentation	249
40.5 Mean average precision, computed honestly	249
40.6 Video, tracking and 3D	250
40.7 Training and deploying detectors in practice	250
41. Natural Language Processing: From Counting Words to Retrieval	252
41.1 The classical pipeline, and why it still matters	252
41.2 Counting: TF-IDF and BM25	252
41.3 Word embeddings: dense meaning from co-occurrence	253
41.4 Tasks with structure	254
41.5 Retrieval-augmented generation, done properly	254
41.6 Evaluating language output	255
42. Speech, Audio and Multimodal Models	257
42.1 From waveform to features	257
42.2 Recognition: CTC, attention and transducers	258
42.3 Speaking, separating and self-supervising	259
42.4 Multimodal models: one space for two modalities	259
43. Systems and Scale: Hardware, Memory and Distributed Training	261
43.1 What a GPU actually is, for our purposes	261
43.2 Training memory, accounted exactly	262
43.3 Precision	262
43.4 The four axes of parallelism	263
43.5 Measuring whether your run is any good	264
43.6 Serving: a different bottleneck	265
44. Causal Inference and Experimentation	267
44.1 Prediction is not intervention	267
44.2 Running an experiment that means something	267
44.3 When you cannot randomize	269
44.4 Heterogeneous effects and uplift modelling	270
45. Privacy, Security and Governance	271
45.1 The threat model	271
45.2 Differential privacy, precisely enough to use	271
45.3 Handling personal data	272
45.4 Securing a deployed model, and an agent	273
45.5 Governance: the paperwork that is actually load-bearing	274
46. A Complete Project, Step by Step	276
46.1 Stage 0 - Frame the problem before touching data	276

46.2 Stage 1 - Set up the repository and the environment	277
46.3 Stage 2 - Get the data, document it, and split it first	278
46.4 Stage 3 - Explore, on the training split only	279
46.5 Stage 4 - Establish baselines before modelling	280
46.6 Stage 5 - Build the preprocessing pipeline as one object	281
46.7 Stage 6 - Model iteration: two tracks	282
46.8 Stage 7 - Tune only what matters	282
46.9 Stage 8 - Evaluate like a sceptic	283
46.10 Stage 9 - Compress to the deployment budget	284
46.11 Stage 10 - Package the artefact	284
46.12 Stage 11 - Deploy carefully	285
46.13 Stage 12 - Monitor, and plan for decay	286
46.14 Worked project A - a tabular classifier, end to end	286
46.15 Worked project B - a deep model for sensor data, on-device	288
46.16 Time budget and the failure timeline	289
46.17 The one-page master checklist	289
APPENDICES	290
Appendix A. Mathematical Reference	290
Linear algebra	291
Matrix calculus (denominator layout)	291
Probability	291
Information theory	291
Key derivatives used in this book	291
Complexity cheat-sheet	292
Useful numbers	292
Appendix B. Glossary	292
Appendix C. Study Roadmap, Projects and Resources	296
A 30-week study plan	296
Portfolio projects worth building	297
Books	297
Courses, tools and venues	297
References	299

PART I - Foundations

What learning from data means, the mathematics behind it, and how to set up a problem so that the answer is trustworthy.

1. What Machine Learning Really Is

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Everything in this chapter has a concrete counterpart in the thesis: the "examples" are the 6617 training windows, the "labels" are the six activities, the "model" is the 10,214-parameter network, and the "loop" is the training loop that the thesis wants to run on the STM32 board itself.

A traditional program is a list of rules written by a person. You want to detect spam, so you write: *if the subject contains 'FREE MONEY', mark it as spam*. That works until the spammers write 'F R E E MONEY'. You add another rule. They adapt again. After two years you have 4,000 brittle rules and no one understands them.

Machine learning inverts the arrow. Instead of writing the rules, you collect **examples** - 50,000 emails, each labelled spam or not-spam - and you write a program that **searches for the rules by itself**. The output of that search is a **model**: a function with numbers inside it that turns an input into a prediction.

CLASSICAL PROGRAMMING	MACHINE LEARNING
-----	-----
data ----+ --> [program] --> answers	data ----+ --> [LEARNING] --> program
rules ----+ Humans supply the rules.	answers ----+ (the model) Humans supply the answers; the machine writes the rules.

Figure 1.1 - The defining inversion of machine learning.

THE ONE-SENTENCE DEFINITION

Machine learning is the practice of fitting a parameterised function to data by minimising a measure of error, in the hope that the fitted function also works on data it has never seen. Everything else in this book - trees, transformers, diffusion models - is a choice about the shape of that function and the way you search for its parameters.

1.1 A worked micro-example before any theory

Suppose you have five houses and you want to predict price from size.

FROM ZERO — the idea explained step by step

A normal program is a recipe that a person writes: "if this, then that". Machine learning turns this around. You give the computer many **examples** together with the right **answers**, and the computer searches for the recipe by itself.

The recipe it finds is called a **model**. A model is just a formula with some numbers inside it that are not fixed yet. These numbers are called **parameters**. "Learning" means trying different values for the parameters until the formula gives answers close to the right ones.

Tiny example: the book uses $\text{price} = w \times \text{size} + b$ for five houses. w and b are the two parameters. With $w = 2.55$ and $b = 20$, a 100 m² house gets $2.55 \times 100 + 20 = 275$ (thousand EUR). Nobody typed "2.55"; the search found it.

IN OUR PROJECT — used

Our project does exactly this, only with more numbers. Instead of house size, the input is one window of sensor data (1,152 numbers). Instead of a price, the output is one of six activities. Instead of 2 parameters, the model has 10,214.

Even a one-parameter rule can be learned from our data: searching all thresholds on the training windows finds "if total_acc_x_std is at most 0.0709 the person is still, otherwise moving". On the 9 unseen test people this learned rule is right in 0.9993 of the windows (measured). Nobody told the program that 0.07 is the right number -- the data did.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Size (m ²)	50	70	90	110	130
Price (k EUR)	150	195	260	300	355

You guess the relationship is a straight line, $\text{price} = w * \text{size} + b$. The pair (w, b) are the **parameters**. Learning here means: try many values of w and b , and keep the pair whose predictions are closest to the five real prices. 'Closest' has to be defined numerically - that definition is the **loss function**. The usual choice is mean squared error:

$$J(w, b) = (1/n) * \sum_i (w * \text{size}_i + b - \text{price}_i)^2$$

For this data the best fit is roughly $w = 2.55$, $b = 20$. That model now predicts a 100 m² house at about 275k. You never wrote the rule 'each square metre costs 2,550 EUR' - the data implied it and the optimiser found it. That is the entire idea, and every later chapter is a richer version of this loop:

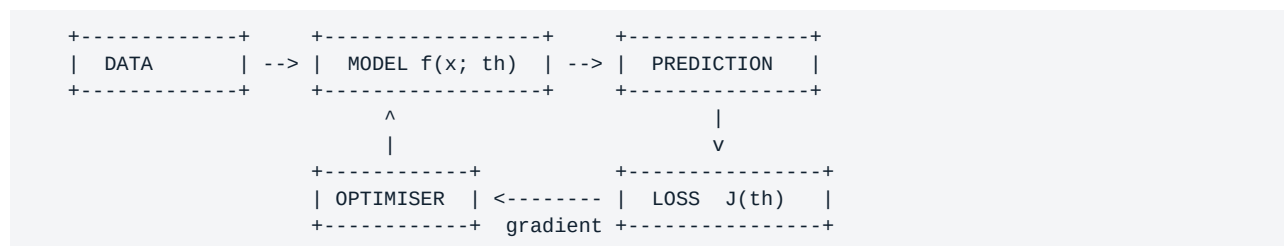
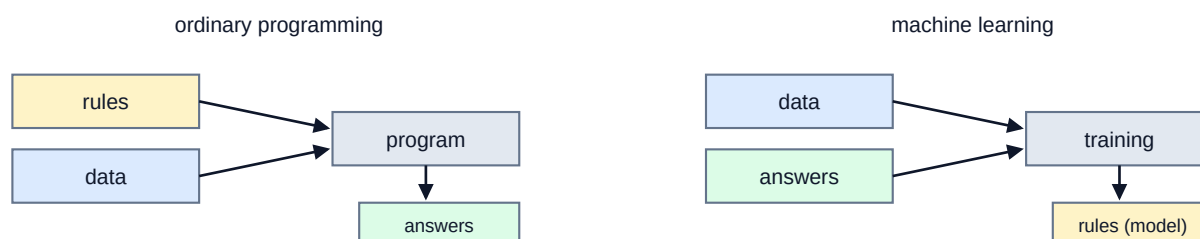


Figure 1.2 - The universal training loop: predict, score, adjust, repeat.

1.2 The vocabulary, defined once and used forever



Graph 1.1 — The one idea behind machine learning. Ordinary programming: you write the rules, the computer applies them to data. Machine learning: you give data and the right answers, and the computer works out the rules.

Term	Meaning	In the house example
Sample / instance	One row of data, one thing you make a prediction about.	One house.
Feature	One measured input variable. d features per sample.	Size in m ² .
Feature vector x	All features of one sample stacked together.	[50]
Label / target y	The correct answer you want to predict.	150k
Design matrix X	All samples stacked: shape (n, d) .	5 x 1 matrix
Model $f(x; \theta)$	The parameterised function producing predictions.	$w * x + b$
Parameters θ	Numbers learned from data.	w and b

Term	Meaning	In the house example
Hyperparameters	Numbers you choose before training, not learned from the training loss.	Learning rate, degree of polynomial
Loss L	Error on one sample.	$(\text{pred} - \text{price})^2$
Cost / objective J	Average loss over a dataset, plus any penalties.	MSE over 5 houses
Training	Searching parameter space to minimise J.	Fitting w, b
Inference	Running the trained model on new inputs.	Pricing a new listing
Generalisation	Accuracy on data not used for training.	Pricing houses you never saw

FROM ZERO — the idea explained step by step

Machine learning has a small vocabulary that is used everywhere. **Sample**: one thing you make a prediction about. **Feature**: one number that describes the sample. **Label**: the right answer for a sample. **Model**: the formula with adjustable numbers. **Parameter**: one adjustable number inside the model. **Loss**: one number that says how wrong the model is on a sample (smaller is better). **Training**: changing the parameters to make the loss small. **Training set** and **test set**: the examples used for learning, and other examples kept aside to check the result honestly.

Think of a student preparing for an exam. The exercises with solutions are the training set; the real exam with new questions is the test set. A student who memorised the solutions without understanding fails the exam.

IN OUR PROJECT — used

In our project: a **sample** is one window of 2.56 s. Its **features** are the 1,152 sensor values (9 signals x 128 time steps). Its **label** is one of WALKING, UPSTAIRS, DOWNSTAIRS, SITTING, STANDING, LAYING, stored as a number 0 to 5. The **model** is the 1-D CNN; its **parameters** are 10,214 weights and biases. The **loss** is cross-entropy (Chapter 6). The **training set** has 6617 windows from 21 people, the **test set** 2947 windows from 9 other people [9, 35].

1.3 The four learning paradigms

FROM ZERO — the idea explained step by step

There are four ways a machine can learn, depending on what information you have.

Supervised: every example comes with its answer (a photo AND the word "cat"). **Unsupervised**: only the examples, no answers; the machine looks for groups or patterns by itself. **Self-supervised**: no answers are given, so the machine hides part of each example and learns to predict the hidden part. **Reinforcement**: the machine acts, and receives a reward or a penalty afterwards, like training a dog.

IN OUR PROJECT — used

Supervised -- this is our project. Every training window has its activity label.

Unsupervised -- not used for training, but it works on our data: grouping the training windows into 2 groups without looking at labels puts moving and still windows into separate groups with purity 0.9986 (measured, Chapter 12).

Self-supervised and **reinforcement** learning are not used in the thesis.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

1. Supervised learning - you have the answers

Every training sample carries a label. The model learns a mapping $x \rightarrow y$. Two sub-types dominate:

IN OUR PROJECT — 1. Supervised learning - you have the answers

Our task is supervised **classification**: the answer is one of 6 classes, not a number. The labels come with the UCI HAR dataset; volunteers performed each activity on command, so the labels are reliable [9].

- **Regression**: y is a continuous number. Price, temperature, remaining battery life, time-to-failure. Measured with MSE, MAE, R^2 .
- **Classification**: y is one of K discrete classes. Spam / not spam, which of 1,000 objects is in this photo, which of 5 activities a wearable sensor is recording. Measured with accuracy, precision, recall, F1, AUC.

Supervised learning is by far the most reliable paradigm, and also the most expensive: someone has to produce the labels. A rule of thumb from industry is that 60-80% of the effort on a real supervised project is spent obtaining, cleaning and auditing labels, not on modelling.

2. Unsupervised learning - structure without answers

Only x is available. The goal is to find structure: **clusters** of similar samples (k-means, GMM), a **low-dimensional** description (PCA, autoencoders), a **density** model of where data lives (which also gives you anomaly detection), or **associations** between items.

3. Self-supervised learning - the answers hide in the data

Take unlabelled data and invent a task whose label is part of the data itself: hide a word and predict it, hide a patch of an image and reconstruct it, ask whether two augmented crops came from the same photo. This is how every modern large model - GPT-class language models, CLIP, DINO, wav2vec - is pretrained. It is technically supervised learning with free labels, and it is the single biggest reason deep learning scaled: it removed the labelling bottleneck. Chapter 27 covers it in depth.

4. Reinforcement learning - learning from consequences

An **agent** takes **actions** in an **environment**, receives a scalar **reward**, and must learn a **policy** that maximises cumulative reward. There is no labelled correct action, only delayed and noisy feedback, and the agent's own behaviour determines the data it sees. Robotics, game playing, and the alignment stage of large language models (RLHF) all live here. Chapter 32.

WHICH PARADIGM IS MY PROBLEM?

Ask what you can actually collect. If you can collect input-output pairs, use supervised learning - it is the most sample-efficient and the easiest to evaluate. If you can collect only inputs, use self-supervised pretraining and then fine-tune on the few labels you can afford. Use reinforcement learning only when the decision changes the future data (control, sequential decisions); otherwise it makes an easy problem hard.

1.4 Where deep learning fits

Deep learning is not a separate field from machine learning; it is the subset in which the model is a neural network with many layers, and in which the **features are learned rather than designed**. That is the whole distinction, and it is a big one:

FROM ZERO — the idea explained step by step

Deep learning is one kind of machine learning. The model is a **neural network**: many simple calculation units arranged in layers, each layer feeding the next. "Deep" means many layers.

The big difference to older methods: with older methods a person first decides which summary numbers ("features") describe the data, and the model only combines them. A deep network receives the raw data and learns its own features.

IN OUR PROJECT — used

We can compare both ways on our own data. Hand-made features -- the mean and the standard deviation of each of the 9 signals, 18 numbers per window -- fed into a simple classifier give 0.7869 accuracy on the test people (measured). The CNN, which receives the raw 1,152 numbers and learns its own features, reaches about 0.90 [35].

So on our task the learned features are worth about 11 percentage points. This is why the thesis uses a small deep network and not a classical model.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

```

CLASSICAL ML PIPELINE
raw data -> [ hand-designed features ] -> [ learned classifier ] -> y
           ^ engineered by a human expert over months
DEEP LEARNING PIPELINE
raw data -> [ layer1 -> layer2 -> ... -> layerN -> classifier ] -> y
           ^ every stage learned jointly from the same loss

```

Figure 1.3 - Feature engineering versus representation learning.

In a convolutional network trained on photographs, the first layer ends up detecting edges, the next combines edges into corners and textures, the next into object parts, and the last into whole objects. Nobody programmed that hierarchy; it is what minimising the classification loss produces. This is called **representation learning**, and it is why deep learning wins on images, audio, text and video, where useful features are hard for humans to write down.

DEEP LEARNING IS NOT ALWAYS THE ANSWER

On small or medium tabular datasets - the most common kind of data in industry - gradient-boosted trees (Chapter 11) usually beat neural networks, train in seconds instead of hours, need almost no tuning, and are easier to explain to a regulator. Reach for deep learning when the input is perceptual (pixels, waveforms, tokens), when you have a lot of data, or when you can start from a pretrained model.

1.5 The taxonomy at a glance

Paradigm	Input	Typical models	Typical use
Supervised - regression	x, y in R	Linear/ridge, gradient boosting, MLP	Price, demand, RUL
Supervised - classification	x, y in {1..K}	Logistic regression, trees, CNN, Transformer	Spam, diagnosis, vision
Unsupervised - clustering	x only	k-means, GMM, DBSCAN, HDBSCAN	Segmentation, grouping
Unsupervised - dim. reduction	x only	PCA, t-SNE, UMAP, autoencoder	Visualisation, compression
Unsupervised - density	x only	GMM, normalising flow, diffusion	Anomaly detection, generation
Self-supervised	x + invented task	BERT/GPT objectives, SimCLR, MAE	Pretraining foundation models
Reinforcement	state, action, reward	DQN, PPO, SAC, AlphaZero	Control, games, RLHF

FROM ZERO — the idea explained step by step

The taxonomy is a map of problem types. **Classification**: predict a category. **Regression**: predict a number. **Clustering**: find groups. **Dimensionality reduction**: describe data with fewer numbers. **Forecasting**: predict the future of a time series. Each type has its own losses, metrics and usual models.

IN OUR PROJECT — used

Our project is multi-class classification of fixed-length time series (6 classes, windows of 128 steps).

Other problem types appear in the rest of the project library: the ECG example is anomaly detection with an autoencoder (reconstruction, Chapter 25), and the continual-learning HAR example uses PCA (dimensionality reduction, Chapter 12) to remember old data [35].

1.6 A first end-to-end program

Here is a complete, honest supervised-learning workflow in 25 lines. Read it now even if the details are unfamiliar; every line is explained in the next three chapters.

FROM ZERO — the idea explained step by step

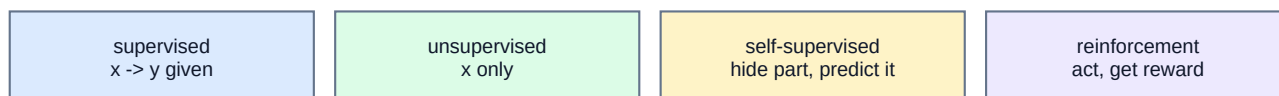
A complete machine-learning program always has the same steps: load the data, split it into training and test parts, choose a model, train it on the training part, and measure it once on the test part.

The order matters: the test part must be put aside first and not looked at until the end, otherwise the final measurement is too optimistic.

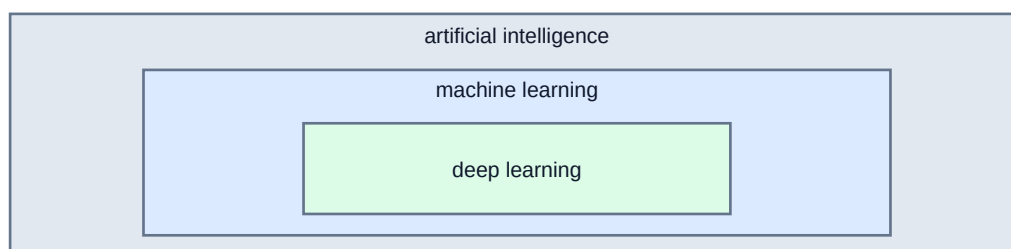
IN OUR PROJECT — used

Our project has these steps as files [35]: `prepare_data.py` (download, cut into windows, split, save), `train_pytorch.py` (train the reference model and write a log), `train_c.c` (the same training in C), `compare.py` (check that C and PyTorch reach the same result).

The test split is fixed by the dataset (9 people) and never used for decisions -- exactly the discipline of this section.



Graph 1.2 — The four ways a model can learn, by what it is given. Supervised: examples with answers. Unsupervised: examples only, find structure. Self-supervised: make answers from the data itself. Reinforcement: act, and learn from rewards.



Graph 1.3 — Where deep learning fits. Deep learning is one kind of machine learning (models made of many layers), and machine learning is one way of doing artificial intelligence.

```

import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
X, y = load_breast_cancer(return_X_y=True)      # 569 samples, 30 features
# 1. Hold out a test set FIRST and do not look at it again until the end.
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)
# 2. Scaling belongs INSIDE the pipeline so it is refit on each CV fold.
model = make_pipeline(StandardScaler(),
    LogisticRegression(max_iter=5000, C=1.0))
# 3. Estimate generalisation using cross-validation on the training set.
cv = cross_val_score(model, X_tr, y_tr, cv=5, scoring='roc_auc')
print(f'CV AUC: {cv.mean():.3f} +/- {cv.std():.3f}')
# 4. Refit on all training data, then evaluate ONCE on the test set.
model.fit(X_tr, y_tr)
proba = model.predict_proba(X_te)[: , 1]
print('test AUC:', round(roc_auc_score(y_te, proba), 3))
print(classification_report(y_te, (proba > 0.5).astype(int)))

```

Listing 1.1 - The shape of every honest supervised experiment.

Four habits in that listing separate practitioners from beginners: the test set is split off **before** anything else happens; preprocessing is inside the pipeline so it cannot leak information across folds; the model is selected using cross-validation, not the test set; and the test set is touched exactly once.

1.7 Why machine learning works now and not in 1990

The core algorithms are old: least squares is from 1805, the perceptron from 1958, backpropagation was popularised in 1986, and convolutional networks recognised digits commercially in the early 1990s. Nothing conceptual was missing. Three practical things changed, and it is worth knowing which one you are short of when a project stalls.

FROM ZERO — the idea explained step by step

Machine learning is old, but it only became practical recently because three things arrived together: large datasets, fast computers, and better training methods.

Each of the three can be the thing that holds a project back.

IN OUR PROJECT — used

For our project the limiting ingredient is **compute and memory on the device**, not data or methods. The data (UCI HAR, 2013 [9]) and the methods (CNNs, SGD) are standard; the new part is training inside 320 KB of RAM [3, 5]. That is why most of the thesis is about saving memory: 8-bit numbers and sparse connections.

Ingre dient	Then	Now	Why it mattered
Data	Thousands of hand-collected samples	Billions of images, trillions of text tokens	Large models need large data; the internet supplied it
Comp ute	A workstation doing millions of operations per second	A GPU doing 10^{14} operations per second	Training that took a year now takes an hour, so you get hundreds of attempts instead of one
Metho d	Sigmoid units, random initialisation, plain SGD	ReLU, He initialisation, batch norm, residual connections, Adam	These made deep networks trainable at all - before them, depth beyond a few layers simply did not converge

WHICH OF THE THREE IS YOUR BOTTLENECK?

If more data reliably improves your validation score, you are data-bound: buy labels or use self-supervision. If your model has already fitted the training set perfectly and you cannot afford a bigger one, you are compute-bound. If training is unstable, diverging, or plateauing far above a reasonable loss, you are method-bound - and Part III is the chapter list for that.

1.8 A day in the life of a machine-learning project

Textbooks present modelling as the main activity. It is not. Here is where the time actually goes on a typical supervised project, and the chapter that covers each stage.

FROM ZERO — the idea explained step by step

In a real project, most time goes into preparing data, checking results and fixing mistakes -- not into choosing a clever model.

IN OUR PROJECT — used

This was true for the thesis: much of the work was making the C code give the same numbers as PyTorch, fixing random seeds, measuring memory on the board, and handling 8-bit rounding. The model itself is a standard small CNN [35].

Stage	Share of effort	What it involves	Chapter
Framing the problem	5%	Deciding what to predict, what a prediction is worth, and what a mistake costs	1, 13
Getting and labelling data	35%	Collection, joins, consent, labelling guidelines, adjudicating disagreements	4
Cleaning and features	25%	Missing values, outliers, encodings, aggregations, leakage audits	4
Modelling	10%	Baseline, then a stronger model, then tuning	5-27
Evaluation	10%	Metrics, slices, calibration, error analysis	3, 13
Deployment and monitoring	15%	Serving, drift, retraining, incident response	34

THE MOST COMMON BEGINNER MISTAKE

Spending three weeks on architectures before spending three days looking at the data. Print fifty random rows. Look at fifty random images with their labels. Find the ten samples your baseline gets most wrong and read them one by one. Almost every project has a data problem hiding in plain sight, and no architecture fixes a wrong label.

1.9 Reading the training loop as a sentence

Every training run, in every framework, in every architecture in this book, is the same five-line sentence. If you can narrate these five lines you can read any deep-learning codebase.

FROM ZERO — the idea explained step by step

Every training program repeats one sentence: take a small group of examples (a **batch**), let the model predict (**forward**), measure how wrong it is (**loss**), work out in which direction each parameter should move (**backward**, the gradient), and move every parameter a little in that direction (**update**). One pass over all training examples is one **epoch**.

IN OUR PROJECT — used

In our project the sentence runs with batches of 64 windows. 6617 windows make 103 full batches (the last 25 windows are dropped), so one epoch is 103 updates. Training lasts 20 epochs: $20 \times 103 = 2,060$ updates. Each update moves the parameters with learning rate 0.01 and momentum 0.9 (Chapter 17) [35].

In the C code the same five verbs are the functions of the training loop: load batch, forward through the layers, cross-entropy loss, backward through the layers, optimiser step.

```
for xb, yb in loader:           # 1. take a batch of examples
    pred = model(xb)             # 2. FORWARD: guess the answers
    loss = lossf(pred, yb)       # 3. SCORE: how wrong were the guesses?
    loss.backward()              # 4. BACKWARD: how should each weight change?
    opt.step()                   # 5. UPDATE: change every weight a little
    opt.zero_grad()              # then forget the old gradients
```

Listing 1.2 - The five verbs: batch, forward, score, backward, update.

Chapters 14-17 explain each verb in full: forward is a stack of matrix multiplications and non-linearities; score is the negative log-likelihood of your data under an assumed noise model; backward is the chain rule applied right to left; and update is a small step downhill with a per-parameter step size. Everything else - convolutions, attention, diffusion - changes only what happens inside step 2.

1.10 What can go wrong (a preview of Chapter 3)

- **Overfitting:** the model memorises the training data, including its noise, and fails on new data. The classic symptom is 99% training accuracy and 62% test accuracy.

FROM ZERO — the idea explained step by step

Four classic mistakes: **overfitting** (learning the training examples by heart), **underfitting** (a model too simple to learn anything), **data leakage** (information from the test examples sneaks into training), and **distribution shift** (the real world differs from the training data).

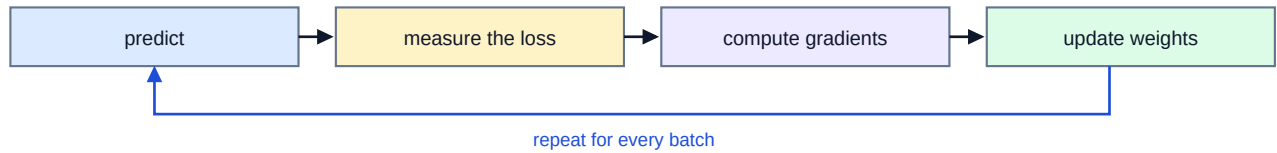
IN OUR PROJECT — used

Two of these are real in our project. **Distribution shift:** the test people are new people, and they move differently. **Leakage:** the project's validation set takes random windows from the same 21 training people, and neighbouring windows share half of their samples, so the validation score is too optimistic (0.902 on validation versus 0.7869 on the test people for the same simple model, measured). Chapter 3 explains both.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- **Underfitting:** the model is too simple to capture the pattern - a straight line through a curved relationship. Both training and test error are high.
- **Data leakage:** information about the answer sneaks into the features. Scaling before splitting, using a future value as a feature, or having the same patient in both train and test all inflate your score and produce a model that fails in production.
- **Distribution shift:** the world changes after you train. Prices move, sensors drift, users behave differently. A model is a photograph of the past.
- **Optimising the wrong metric:** 99% accuracy on a dataset where 99% of samples are negative means your model learned to say 'no'.

Exercises



Graph 1.4 — The training loop as a sentence: make a prediction, measure how wrong it is, work out which way to change each weight, change them a little, and repeat.

1 Fit $\text{price} = w \cdot \text{size} + b$ on the five houses above by hand: compute J for $(w, b) = (2.0, 50), (2.55, 20), (3.0, 0)$ and say which is best. 2 For each of these, name the paradigm and the metric you would use: predicting tomorrow's electricity demand; grouping customers with no labels; teaching a drone to land; filling in masked words in Wikipedia. 3 Run Listing 1.1. Then deliberately break it by scaling X before the split and observe how much the test AUC changes - this is leakage in miniature.

2. The Mathematical Toolkit You Actually Need

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The mathematics of this chapter is exactly the mathematics the C code of the project computes: matrix products in every layer, derivatives in the backward pass, probabilities in the softmax output, and gradient descent in the optimiser. Each section below points to the place in the project where it happens.

You do not need a mathematics degree. You need fluency with four things: **linear algebra** (how data and parameters are stored and multiplied), **calculus** (how a loss changes when a parameter moves), **probability** (how to talk about uncertainty and where loss functions come from), and **optimisation** (how to move parameters downhill). This chapter builds all four from scratch, with machine-learning meaning attached to every object.

2.1 How to read the mathematics in this book

Mathematical notation is compressed English. This section decompresses the five symbols that do most of the work, so that no equation later in the book is opaque.

FROM ZERO — the idea explained step by step

Mathematical formulas are short ways of writing sentences. The symbol $\sum_i x_i$ means "add up all the x values". A letter in bold or capital (X) usually means a whole table of numbers (a matrix); a small letter (x) means one number or one list of numbers.

The trick: read each symbol aloud, and replace it by a small concrete example with 2 or 3 numbers. Then the formula becomes arithmetic you can check.

IN OUR PROJECT — used

In our project, X for a batch is a block of shape (64, 9, 128): 64 windows, 9 signals, 128 time steps. The shape tells you how many numbers there are: $64 \times 9 \times 128 = 73,728$ values = 294,912 bytes as 32-bit floats.

Writing the shape next to every formula is the habit that makes the C code of the project checkable: every buffer size in the code is such a product of shapes [35].

Symbol	Read it aloud as	Example	Meaning of the example
$\sum_i x_i$	the sum over i of x sub i	$\sum_i x_i$	Add up every element of the list x
$\prod_i x_i$	the product over i	$\prod_i p_i$	Multiply all the probabilities together
$\operatorname{argmin}_w f(w)$	the w that makes f smallest	$\operatorname{argmin}_w J(w)$	The parameters with the lowest loss - not the loss value itself, the PARAMETERS
$E[X]$	the expected value of X	$E[\text{loss}]$	The average loss if you could see infinite data
$x \sim D$	x is drawn from the distribution D	$\text{eps} \sim N(0, 1)$	The noise is a random draw from a standard normal
dJ/dw	the derivative of J with respect to w	$dJ/dw = 3$	If w increases by 0.01, J increases by about 0.03
$\ x\ $	the norm, i.e. the length of x	$\ w\ ^2$	The squared length of the weight vector
$a := b$ or $a \leftarrow b$	a is defined as / set to b	$w \leftarrow w - \text{eta } g$	Overwrite w with the new value

THE TRICK THAT MAKES EQUATIONS READABLE

Whenever you meet an unfamiliar equation, do two things. First, say out loud what each symbol IS - a scalar, a vector, a matrix, and of what shape. Second, substitute the smallest possible concrete case: one sample, two features, one output. Almost every equation in machine learning becomes obvious at $n = 1$, $d = 2$, and the general case is only bookkeeping on top of that.

2.2 Linear algebra: the language of data

FROM ZERO — the idea explained step by step

A **scalar** is one number. A **vector** is a list of numbers, like (2, 5, 1). A **matrix** is a table with rows and columns. A **tensor** is the general word for a block of numbers with any number of dimensions.

Two operations do almost all the work. The **dot product** of two lists multiplies them position by position and adds the results: $(1, 2) \cdot (3, 4) = 1 \times 3 + 2 \times 4 = 11$. A **matrix product** is many dot products at once: each row of the first table with each column of the second.

IN OUR PROJECT — used

Every layer of our network is built on these operations. The last layer turns 64 numbers into 6 class scores: each score is the dot product of the 64 inputs with one row of a 6 x 64 weight table, plus a bias. That is a matrix product with a (6, 64) matrix.

The tensor type of the ODT library (tensor_t) stores exactly such blocks: a pointer to the numbers, a shape, and a quantisation description [35].

FROM OUR C CODE — src/tensor/TensorConversion.c, lines 274-286: the L-infinity norm sets the quantisation scale

What it does. findAbsMaxFloat returns the largest absolute value in the tensor -- the L-infinity norm. Dividing it by qMax (the largest integer, e.g. 127 for 8 bits or 2047 for 12 bits) gives the size of one integer step.

Example. If the largest $|w|$ is 0.4 and $qMax = 127$, $scale = 0.4 / 127 = 0.0031496$. A tensor of all zeros would give scale 0, so the code uses 1 instead (division by zero is avoided).

```
float absMax = findAbsMaxFloat(inputTensor->data, numberOfElements);

symInt32QConfig_t *symInt32QC = outputTensor->quantization->qConfig;
uint8_t qMaxBits = symInt32QC->qMaxBits;

const float qMax = powf(2, (float)qMaxBits - 1) - 1;
const float qMin = -powf(2, (float)qMaxBits - 1);

float scale;
if (absMax == 0.f) {
    scale = 1.f;
} else {
    scale = absMax / qMax;
```

Scalars, vectors, matrices, tensors

Object	Notation	Shape	In ML
Scalar	x	()	A learning rate, a single loss value
Vector	x (bold)	(d,)	One sample's features; one layer's biases
Matrix	X (bold cap)	(n, d)	A batch of n samples; a weight matrix
3-tensor	T	(N, H, W)	A batch of greyscale images
4-tensor	T	(N, C, H, W)	A batch of colour images in PyTorch order

A **tensor** in deep-learning software is simply an n-dimensional array with an attached gradient machinery. The word carries no physics meaning here. What matters in practice is **shape discipline** : most bugs in deep learning are shape bugs, and the fastest debugging habit you can build is printing `tensor . shape` at every step.

The three products you must never confuse

Dot product	$\mathbf{a} \cdot \mathbf{b} = \sum_i \mathbf{a}_i \mathbf{b}_i$	-> a scalar
Matrix-vector	$(\mathbf{A} \mathbf{x})_i = \sum_j \mathbf{A}_{ij} \mathbf{x}_j$	-> a vector
Elementwise	$(\mathbf{a} * \mathbf{b})_i = \mathbf{a}_i \mathbf{b}_i$	-> same shape

Matrix multiplication $C = A \cdot B$ with A of shape (m, k) and B of shape (k, n) gives C of shape (m, n) , where $C_{ij} = \sum_k A_{ik} B_{kj}$. The inner dimensions must match; the outer ones survive. Reading that rule as $(m, k) \times (k, n) \rightarrow (m, n)$ and checking it mentally before every line you write will save you hours.

A NEURAL NETWORK LAYER IS ONE MATRIX MULTIPLY

A fully connected layer computing $z = Wx + b$ for a single sample becomes $Z = XW^T + b$ for a whole batch, where X is (n, d) and W is $(units, d)$. The reason GPUs transformed this field is that this single operation - dense matrix multiplication - is exactly what they do thousands of times faster than a CPU.

Useful identities you will meet again in the backpropagation chapter:

$(A \cdot B)^T = B^T \cdot A^T$	
$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	(associative)
$A \cdot (B + C) = A \cdot B + A \cdot C$	(distributive)
$A \cdot B \neq B \cdot A$	(NOT commutative)

Norms - how big is this vector?

L1 norm	$\ x\ _1 = \sum_i x_i $	-> promotes sparsity
L2 norm	$\ x\ _2 = \sqrt{\sum_i x_i^2}$	-> ordinary length
L-inf norm	$\ x\ _{\infty} = \max_i x_i $	-> worst single component

IN OUR PROJECT — Norms - how big is this vector?

Yes, we use norms -- three of them, in three different places.

L-infinity norm (the largest absolute value in a list): the int8 quantisation scale of a weight tensor is $s = \max|w| / 127$, i.e. the L-inf norm divided by 127. For conv3 with largest weight 0.4: $s = 0.4 / 127 = 0.0031496$. Every weight is then stored as the integer round(w / s) [10, 35].

L1 idea, per weight: RigL (Chapter 29) ranks weights by their absolute value $|w|$ and switches off the smallest ones; the gradients of switched-off weights are ranked by $|g|$ to choose which to switch on [4].

L2 norm, squared: the mean-squared-error loss (used by the ECG autoencoder example) is the squared L2 norm of the error divided by the number of values; AdamW's weight decay pulls on the squared L2 norm of the weights [13, 35].

L2 is the default: it is smooth, differentiable everywhere, and gives Euclidean distance $\|a - b\|_2$. L1 is not differentiable at zero, and that corner is precisely why L1 regularisation drives coefficients exactly to zero (Chapter 7). L-infinity appears in adversarial robustness, where an attacker may perturb every pixel by at most epsilon (Chapter 33).

Geometry: angles, projections, similarity

$$a \cdot b = \|a\| \|b\| \cos(\theta) \quad \Rightarrow \quad \cos_{\text{sim}}(a, b) = (a \cdot b) / (\|a\| \|b\|)$$

IN OUR PROJECT — Geometry: angles, projections, similarity

Partly. Cosine similarity is not computed in the HAR model. The dot product it is built on is used in every layer.

Cosine similarity ignores magnitude and compares direction only; it is the standard way to compare embeddings, and it is the numerator inside attention (Chapter 23) and inside every vector database. Two vectors are **orthogonal** when their dot product is zero - they share no information in the linear sense.

Matrix decompositions that carry real meaning

- **Eigendecomposition** $A = Q \cdot L \cdot Q^{-1}$: for a symmetric matrix, Q is orthogonal and L diagonal. Eigenvectors are directions the matrix only stretches; eigenvalues are the stretch factors. The eigenvalues of the **Hessian** of your loss describe the curvature of the optimisation landscape, and their ratio (the **condition number**) predicts how badly plain gradient descent will zig-zag.

IN OUR PROJECT — Matrix decompositions that carry real meaning

Used in the continual-learning example. The PCA replay memory (src/continual_learning/PpcaReplay.c) keeps a rank-8 description of old HAR windows; the eigen-decomposition is computed by src/arithmetic/JacobiEig.c. Rank 8 instead of 1,152 numbers per direction is what makes the memory fit [35].

- **Singular value decomposition** $A = U S V^T$: exists for every matrix. Keeping only the top k singular values gives the best possible rank- k approximation (Eckart-Young theorem). This one fact powers PCA (Chapter 12), latent semantic analysis, recommender systems, and LoRA fine-tuning of large models (Chapter 24).
- **Rank** : the number of linearly independent directions a matrix actually uses. Low-rank structure is what makes compression possible; 'low-rank adaptation' means updating a big weight matrix with a product of two thin ones.

```
import numpy as np
A = np.random.randn(200, 50)
U, s, Vt = np.linalg.svd(A, full_matrices=False)
k = 10
A_k = (U[:, :k] * s[:k]) @ Vt[:k]          # best rank-10 approximation
err = np.linalg.norm(A - A_k) / np.linalg.norm(A)
print(k, 'components keep', round(100*(1-err), 1), '% of the energy')
# Storage: 200*50 = 10,000 numbers -> (200+50)*10 = 2,500 numbers
```

Listing 2.1 - Low-rank approximation: the mathematical core of PCA and LoRA.

2.3 Calculus: how the loss reacts to a parameter

FROM ZERO — the idea explained step by step

A **derivative** answers: if I change this input a tiny bit, how much does the output change? For $f(x) = x^2$ at $x = 3$, increasing x to 3.001 makes f go from 9 to 9.006001: the output changes about 6 times as much, so the derivative is 6 ($= 2x$).

A model has many parameters, so we need one derivative per parameter. The list of all these derivatives is the **gradient**. It points in the direction in which the loss grows fastest; training moves the parameters the opposite way.

The **chain rule** says how derivatives pass through several steps: if a change in x changes u , and a change in u changes y , multiply the two rates. A neural network is a long chain of steps, so the chain rule is how the error at the end is traced back to every parameter.

IN OUR PROJECT — used

The backward pass of our C library is the chain rule written out for each layer [35]. Each layer receives "how much the loss changes when my output changes" and computes (1) the same thing for its input, to pass back, and (2) the gradient of its own weights.

The gradient of one weight is the product of its input value and the error arriving from above: $g = x \times \delta$ (Chapter 15). Its size matters in the thesis: with 8-bit weights a gradient smaller than 0.1575 cannot move a weight at learning rate 0.01 (the dead zone, Chapter 28) [3, 11].

Derivative, partial derivative, gradient

The derivative df/dx is the rate of change of f at a point: move x by a tiny amount h and f moves by about $h \times df/dx$. With many inputs, the **partial derivative** dJ/dw_i measures the effect of one parameter with all others frozen. Collecting all partials gives the **gradient** :

```
grad J(w) = [ dJ/dw_1 , dJ/dw_2 , ... , dJ/dw_d ]
```

THE GRADIENT IS THE UPHILL DIRECTION

Stand on a hillside in fog. The gradient points in the direction of steepest ascent, and its length says how steep. To minimise a loss you therefore step in the direction of the **NEGATIVE** gradient. That single sentence is the whole of gradient descent, and by extension most of modern machine learning.

The chain rule - the engine of deep learning

If $y = f(u)$ and $u = g(x)$, then $dy/dx = (dy/du) * (du/dx)$. For a network that computes a composition of L layers, the derivative of the loss with respect to an early weight is a product of L local derivatives. Backpropagation (Chapter 15) is nothing more than an efficient, right-to-left evaluation of that product, reusing shared subexpressions.

$$J = L(a_L), \quad a_L = f_L(a_{L-1}), \quad \dots, \quad a_1 = f_1(x)$$

$$dJ/dW_k = dJ/da_L * da_L/da_{L-1} * \dots * da_{k+1}/da_k * da_k/dW_k$$

Two immediate consequences, both central to Part III: if the local factors are consistently smaller than 1, the product shrinks exponentially with depth (**vanishing gradients**); if consistently larger than 1, it explodes (**exploding gradients**). Residual connections, careful initialisation and normalisation layers all exist to keep that product near 1.

Jacobians, Hessians and what curvature buys you

- **Jacobian** J : the matrix of all first partials of a vector-valued function, shape (outputs, inputs). Automatic differentiation never builds it explicitly; it computes Jacobian-vector products instead, which is why backprop costs about the same as a forward pass.
- **Hessian** H : the matrix of second partials, shape (d, d) . Positive definite H means a local minimum; mixed signs mean a saddle point. In high dimensions saddles vastly outnumber local minima, which is the modern explanation of why gradient descent works so well on non-convex networks.
- **Second-order methods** (Newton, L-BFGS, K-FAC) use curvature to choose the step size per direction. They converge in fewer iterations but cost $O(d^2)$ or $O(d^3)$ per step; with d in the billions they are impractical, which is why Adam - a cheap diagonal approximation - dominates.

2.4 Probability: reasoning under uncertainty

FROM ZERO — the idea explained step by step

Probability measures how likely something is, from 0 (impossible) to 1 (certain). **Conditional probability** $P(A | B)$ means "the probability of A if we already know B".

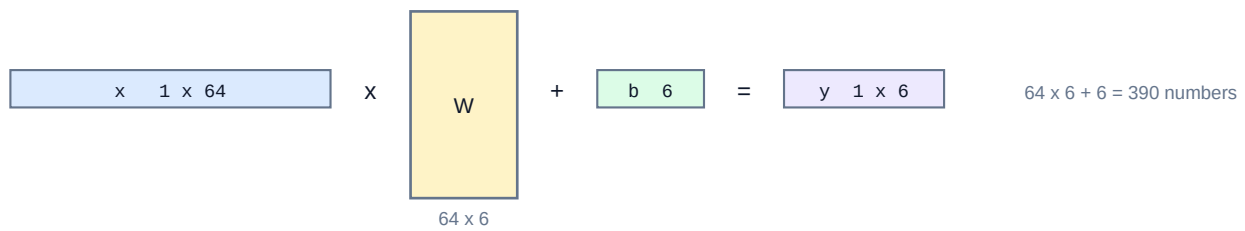
A **distribution** lists the probability of every possible outcome, and the probabilities add up to 1. **Maximum likelihood** means: choose the parameters that make the observed data most probable. Most loss functions come from this idea.

IN OUR PROJECT — used

Our network ends with a **softmax** layer, which turns 6 scores into a probability distribution over the 6 activities, e.g. (0.52, 0.19, 0.07, 0.07, 0.07, 0.07). The loss is the negative logarithm of the probability given to the true activity -- maximum likelihood written as a loss (Chapter 6) [1, Ch. 6].

The training data themselves give simple probabilities: the fraction of LAYING windows is 0.1906.

The rules



Graph 2.1 — Matrix shapes in a real layer: the six-class head of the HAR model. A row of 64 features times a 64 x 6 weight matrix, plus 6 biases, gives 6 scores. The inner sizes (64 and 64) must match; the outer ones (1 and 6) give the result's shape.

Sum rule	$P(A) = \sum_B P(A, B)$
Product rule	$P(A, B) = P(A B) P(B)$
Bayes rule	$P(H D) = P(D H) P(H) / P(D)$
Independence	$P(A, B) = P(A) P(B)$

Bayes' rule is how you turn a **likelihood** (how probable is this data if the hypothesis holds) into a **posterior** (how probable is the hypothesis given the data). Read it as: *posterior is proportional to likelihood times prior*.

THE CLASSIC MEDICAL-TEST CALCULATION

A disease affects 1 in 1,000 people. A test has 99% sensitivity and 99% specificity. You test positive - what is the probability you are ill? $P(D)=0.001$, $P(+|D)=0.99$, $P(+|\text{not } D)=0.01$. Then $P(+) = 0.99 \cdot 0.001 + 0.01 \cdot 0.999 = 0.01098$, so $P(D|+) = 0.00099 / 0.01098 = 9.0\%$. Ninety-one percent of positives are false. This is the same arithmetic as precision on an imbalanced classification problem (Chapter 13) - and the same reason a 99%-accurate fraud detector can be useless.

Distributions you will meet

Distribution	Support	Parameters	Where it appears in ML
Bernoulli	{0,1}	p	Binary labels; the output of a sigmoid
Categorical	{1..K}	$p_1..p_K$	Multiclass labels; softmax output
Binomial	0..n	n, p	Counts of successes; A/B tests
Gaussian	R	μ, σ^2	Noise models, weight init, VAEs, diffusion
Laplace	R	μ, b	The prior behind L1 regularisation
Poisson	0,1,2..	lambda	Event counts; click and arrival models
Exponential	R+	lambda	Waiting times, survival analysis
Uniform	[a,b]	a, b	Random search, dropout masks, init ranges
Beta / Dirichlet	simplex	alpha	Priors over probabilities; topic models

Expectation, variance, covariance

$$\begin{aligned}
 E[X] &= \sum_x x \cdot P(x) && \text{(mean)} \\
 \text{Var}[X] &= E[(X - E[X])^2] = E[X^2] - E[X]^2 \\
 \text{Cov}[X, Y] &= E[(X - E[X])(Y - E[Y])] \\
 \text{Corr}[X, Y] &= \text{Cov}[X, Y] / (\text{sd}(X) \text{sd}(Y)) && \text{in } [-1, 1]
 \end{aligned}$$

Linearity of expectation, $E[aX + bY] = aE[X] + bE[Y]$, holds even when X and Y are dependent, and it is used constantly - for instance to show that a minibatch gradient is an unbiased estimate of the full-dataset gradient, which is the entire justification for stochastic gradient descent.

Maximum likelihood: where loss functions come from

Assume your data was generated by a model with parameters θ . The **likelihood** is the probability of the observed data under that model. Maximum-likelihood estimation picks θ to maximise it; equivalently it

minimises the negative log-likelihood, because logs turn products into sums and are monotone:

```
theta_MLE = argmax_theta PROD_i p(y_i | x_i; theta)
           = argmin_theta -SUM_i log p(y_i | x_i; theta)
```

IN OUR PROJECT — Maximum likelihood: where loss functions come from

Our cross-entropy loss is exactly this: minimising $-\log p(\text{true class})$ over the training windows is the same as maximising the probability the model gives to the observed labels. `src/loss_functions/CrossEntropy.c` computes it [35].

TWO DERIVATIONS YOU SHOULD BE ABLE TO DO FROM MEMORY

Assume Gaussian noise, $y = f(x) + \text{eps}$ with $\text{eps} \sim N(0, \sigma^2)$. The negative log-likelihood becomes $\text{SUM}_i (y_i - f(x_i))^2 / (2 \sigma^2)$ plus a constant - that is MEAN SQUARED ERROR. Assume a Bernoulli label with $p = \text{sigmoid}(z)$. The negative log-likelihood becomes $-\text{SUM}_i [y \log p + (1-y) \log(1-p)]$ - that is CROSS-ENTROPY. Loss functions are not arbitrary; each one encodes an assumption about the noise in your labels.

Information theory in one page

Entropy	$H(p) = -\text{SUM}_x p(x) \log p(x)$
Cross-entropy	$H(p, q) = -\text{SUM}_x p(x) \log q(x)$
KL divergence	$KL(p q) = \text{SUM}_x p(x) \log(p(x)/q(x)) \geq 0$
Relationship	$H(p, q) = H(p) + KL(p q)$

IN OUR PROJECT — Information theory in one page

Cross-entropy of a uniform guess over 6 classes is $\log(6) = 1.792$ -- the value our training loss starts from. After 20 epochs the test loss is about 0.33 (float model, 10 seeds) [35].

Entropy measures average surprise, in bits if the log is base 2. Cross-entropy measures the cost of encoding data from p using a code built for q . Since $H(p)$ is fixed by the data, **minimising cross-entropy is exactly minimising KL divergence** between the true label distribution and your model - which is why it is the default classification loss. KL is not symmetric, and that asymmetry is the difference between mode-seeking and mode-covering behaviour in variational inference and GAN training (Chapter 25).

2.5 Worked example: matrix shapes in a real layer

Suppose a batch of 4 samples, each with 3 features, entering a layer with 2 output units. Track the shapes:

X (4, 3)	four samples, three features each
W (2, 3)	two units, each with three weights
b (2,)	one bias per unit

FROM ZERO — the idea explained step by step

Before computing anything, write down the shape of every block of numbers. Shapes must fit: a $(m \times k)$ table can only be multiplied with a $(k \times n)$ table, and the result is $(m \times n)$. Checking shapes finds most mistakes before a single number is computed.

IN OUR PROJECT — used

The biggest layer of our network, conv3, has weights of shape (64, 32, 3): 64 output signals, 32 input signals, kernel length 3 -- 6,144 numbers. Viewed as a matrix it is (64 x 96). It is applied at 32 time positions, so the layer computes $(64 \times 96) \times (96 \times 32) \rightarrow (64 \times 32)$: $64 \times 96 \times 32 = 196,608$ multiply-adds per window [35].

$Z = X W^T + b$ $(4,3) \times (3,2) \rightarrow (4,2)$, b broadcasts over rows
 $A = \text{phi}(Z)$ $(4,2)$ elementwise, shape unchanged

Read $X W^T$ as: for each of the 4 rows of X , take the dot product with each of the 2 rows of W . That is $4 \times 2 = 8$ dot products, each of length 3, which is exactly what a GPU does in one fused operation. If you ever see a shape error in deep-learning code, write the three shapes down in this form and the mismatch becomes visible immediately.

2.6 Worked example: a derivative you can check by hand

Take $J(w) = (w - 3)^2$, the simplest possible loss. Its derivative is $dJ/dw = 2(w - 3)$. Start at $w = 0$ with a learning rate of 0.1 and turn the crank:

FROM ZERO — the idea explained step by step

You can always check a derivative with a small experiment: change the input by a tiny amount, recompute, and divide the change of the output by the change of the input. If this matches the formula, the formula is right.

IN OUR PROJECT — used

This check was used to validate the C backward pass. Example with our softmax: scores (2, 1, 0, 0, 0, 0), true class 0. The probabilities are (0.524, 0.193, 0.071, 0.071, 0.071, 0.071); the formula says the derivative of the loss with respect to the first score is $0.524 - 1 = -0.476$. Raising that score by 0.001 lowers the loss by 0.000476 -- the same [35].

Step t	w	J(w)	dJ/dw	New w = w - 0.1 * dJ/dw
0	0.000	9.000	-6.000	0.600
1	0.600	5.760	-4.800	1.080
2	1.080	3.686	-3.840	1.464
3	1.464	2.359	-3.072	1.771
10	2.678	0.104	-0.644	2.742
30	2.996	0.000	-0.007	2.997

Three lessons live in that table, and all three carry over unchanged to a billion-parameter network. The steps are large when the gradient is large and shrink automatically as you approach the minimum. The loss falls quickly at first and then slowly - which is why loss curves are shaped the way they are. And the process never quite arrives: it converges geometrically towards $w = 3$, which is why 'train until it stops improving' is a practical rule rather than a mathematical one.

WHAT HAPPENS IF THE LEARNING RATE IS WRONG

With $\eta = 0.1$ the update is $w \leftarrow w - 0.2(w - 3)$, so the distance to the optimum is multiplied by 0.8 each step - smooth convergence. With $\eta = 0.5$ the factor is 0, and you land exactly on the optimum in one step. With $\eta = 0.9$ the factor is -0.8: you overshoot and oscillate, but still converge. With $\eta = 1.1$ the factor is -1.2, and the distance GROWS every step - divergence. For this loss the exact threshold is $\eta < 1$, which is $2/L$ with curvature $L = 2$. That is the general rule from Chapter 17 in miniature.

2.7 Worked example: Bayes' rule as counting

Probability is easier when you count people instead of manipulating fractions. Take 100,000 people, a disease affecting 1 in 1,000, and a test with 99% sensitivity and 99% specificity:

FROM ZERO — the idea explained step by step

Bayes' rule turns "how likely is the evidence if the answer is A" into "how likely is A given the evidence". It needs the **prior**: how common A is before you look at the evidence. With a rare A, even strong evidence can leave a lot of doubt.

IN OUR PROJECT — used

With our class counts: LAYING is 0.1906 of the training windows. A test that fires for 99% of LAYING windows and 1% of the others gives $P(\text{LAYING} \mid \text{test fires}) = 0.99 \times 0.1906 / (0.99 \times 0.1906 + 0.01 \times 0.8094) = 0.959$. The same test on data with only 1% LAYING would give 0.5.

```

100,000 people
|
+-- 100 ill      --> 99 test positive   (true positives)
|                1 tests negative     (false negative)
|
+-- 99,900 healthy --> 999 test positive (false positives)
|                98,901 test negative (true negatives)
positives = 99 + 999 = 1,098
P(ill | positive) = 99 / 1,098 = 9.0%
  
```

Figure 2.2 - Bayes' rule done by counting people rather than by algebra.

The result is identical to the algebra in the box above, but the counting version makes the cause visible: there are simply far more healthy people than ill ones, so even a small false-positive RATE produces a large false-positive COUNT. Keep this picture in mind for Chapter 13, where the same arithmetic explains why a 99%-accurate fraud detector can still be wrong nine times out of ten.

2.8 Optimisation: moving downhill

FROM ZERO — the idea explained step by step

Gradient descent is walking downhill in fog: you cannot see the valley, but you can feel which way the ground slopes under your feet. Take a small step downhill, feel again, repeat.

The step size is the **learning rate** (eta). Too small: very slow. Too big: you jump over the valley. **Stochastic** or **mini-batch** gradient descent feels the slope using only a small group of examples each time, which is faster and slightly noisy.

IN OUR PROJECT — used

Our project uses mini-batch stochastic gradient descent with momentum: batches of 64 windows, eta = 0.01, momentum 0.9 (src/optimizer/Sgd.c) [35]. On the board the batch is processed one window at a time and the gradients are summed ("micro-batch 1 with accumulation") to save memory -- the maths is the same.

Gradient descent, stated precisely

```
repeat:  theta <- theta - eta * grad J(theta)
```

Convergence depends on the learning rate eta. For a convex quadratic with largest Hessian eigenvalue L, gradient descent converges only if $\text{eta} < 2/L$. Too small and you crawl; too large and you oscillate or diverge. This one hyperparameter is still, in 2020s practice, the most important knob in deep learning.

Figure 2.1 - The effect of the learning rate on the same loss surface.

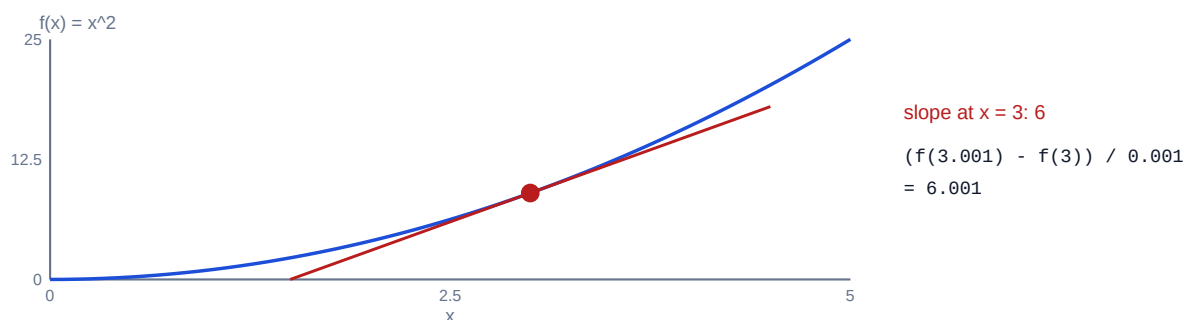
Batch, stochastic and mini-batch

Variant	Gradient uses	Cost per step	Behaviour
Batch GD	all n samples	$O(n)$	Smooth, exact, far too slow for large n
SGD	1 sample	$O(1)$	Very noisy, can escape shallow minima
Mini-batch	B samples (32-8192)	$O(B)$	The universal default: parallel-friendly and stable

IN OUR PROJECT — Batch, stochastic and mini-batch

We use mini-batches of 64. With 6617 windows that is 103 full batches per epoch; the remaining 25 windows are skipped (drop_last) so every batch has the same size [35].

Mini-batching wins for a hardware reason as much as a statistical one: a GPU computes a 256-sample batch in barely more wall-clock time than a single sample, because the bottleneck is memory movement, not arithmetic.



Graph 2.2 — A derivative is the slope of the curve at one point. For $f(x) = x$ squared at $x = 3$ the slope is 6: nudge x by 0.001 and f changes by about 0.006. That is all a gradient is -- one such slope per weight.

Convexity, and why non-convexity is survivable

A function is **convex** if the line segment between any two points on the graph lies above the graph; then any local minimum is global. Linear regression, ridge, logistic regression and linear SVMs are convex - you can trust the optimum. Neural networks are decidedly non-convex, yet gradient descent finds excellent solutions. The current understanding: in very high dimensions, most critical points are saddles rather than poor local minima, and over-parameterised networks have wide, connected basins of good solutions.

IN OUR PROJECT — Convexity, and why non-convexity is survivable

Our network's loss is not convex (it has many valleys). That is why different random seeds give slightly different final models: 10 seeds give accuracy 0.8953 +/- 0.0050 [35].

Constrained optimisation and Lagrange multipliers

To minimise $f(x)$ subject to $g(x) = 0$, form $L(x, \lambda) = f(x) + \lambda g(x)$ and set all partial derivatives to zero. This machinery yields the dual formulation of SVMs (Chapter 10), the eigenvalue problem in PCA (Chapter 12), and the equivalence between constrained-norm and penalised-norm regularisation (Chapter 7).

NUMERICAL CARE THAT SAVES REAL EXPERIMENTS

Never compute $\log(\text{softmax}(z))$ directly - subtract $\max(z)$ first and use the log-sum-exp trick, or call `log_softmax`. Never compute $\log(\text{sigmoid}(z))$ - use `logsigmoid` or a loss that takes raw logits (`BCEWithLogitsLoss`). Float32 has about 7 decimal digits of precision and float16 about 3, with a maximum of 65,504 - which is why mixed precision training needs loss scaling (Chapter 20). Catastrophic cancellation when subtracting nearly equal numbers is the source of most mysterious NaNs.

Exercises

1 Show that for $f(w) = \|Xw - y\|^2$ the gradient is $2 X^T (Xw - y)$. Do it componentwise first, then in matrix form. 2 Derive cross-entropy from the Bernoulli likelihood, then differentiate it with respect to the logit z where $p =$

$\text{sigmoid}(z)$. You should get the famously clean result $p = y$. 3 Compute the condition number of $\begin{bmatrix} 1 & 0 \\ 0 & 100 \end{bmatrix}$ and sketch what gradient descent does on the quadratic it defines. 4 Implement log-sum-exp in NumPy and compare it against a naive implementation on the input $[1000, 1001, 1002]$.

3. The Learning Problem: Generalization, Bias and Variance

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

This chapter explains how to know whether a model will work on new data. For the project this is the most important chapter of Part I, because the product must work for people who were never in the training data.

Training error is easy to drive to zero - store the training set in a lookup table. The entire difficulty of machine learning is **test** error. This chapter gives you the formal statement of that difficulty and the practical tools for managing it. If you read only one chapter of Part I, read this one; nearly every real-world failure traces back to something here.

3.1 The formal setup

Assume samples (x, y) are drawn independently from an unknown distribution D . The quantity you care about is the **risk**, the expected loss on a fresh sample:

$$\begin{aligned} R(f) &= E_{(x,y) \sim D} [L(f(x), y)] && \leftarrow \text{what you want} \\ R_{\text{emp}}(f) &= (1/n) \sum_i L(f(x_i), y_i) && \leftarrow \text{what you can measure} \end{aligned}$$

FROM ZERO — the idea explained step by step

The formal setup says: the examples come from some unknown source (a "distribution"). We see only a sample of it. We fit the model on that sample and hope it works on new examples from the same source. The error on new examples is the one that matters; the error on the training examples is only a guide.

IN OUR PROJECT — used

For us the "source" that matters is "windows from people the model has never seen". The training sample has 21 people, the test sample 9 other people (0 in common) [9]. So our test accuracy measures exactly what a product would face.

You minimise R_{emp} and hope R is close to it. The gap $R - R_{\text{emp}}$ is the **generalisation gap**. Statistical learning theory bounds it, roughly, by a term that grows with the capacity of your model class and shrinks with the square root of the number of samples:

$$R(f) \leq R_{\text{emp}}(f) + O(\sqrt{\text{capacity} / n})$$

The classical capacity measures are **VC dimension** (the largest set of points the class can label in all possible ways) and **Rademacher complexity** (how well the class can fit pure noise). These bounds are loose for deep networks - a ResNet can memorise random labels, so its VC dimension is enormous, yet it still generalises. Explaining that is an active research area (implicit regularisation of SGD, flat minima, margin-based and compression-based bounds). The bounds are still worth knowing because their **shape** is right: more capacity hurts, more data helps, and the trade-off is what you tune.

3.2 Overfitting and underfitting, seen on a curve



Figure 3.1 - The classical capacity/error picture.

FROM ZERO — the idea explained step by step

Underfitting: the model is too simple -- like describing a curvy road with a straight line; it is wrong everywhere.

Overfitting: the model is too flexible -- it follows every bump of the training examples, including noise, and becomes wrong on new examples.

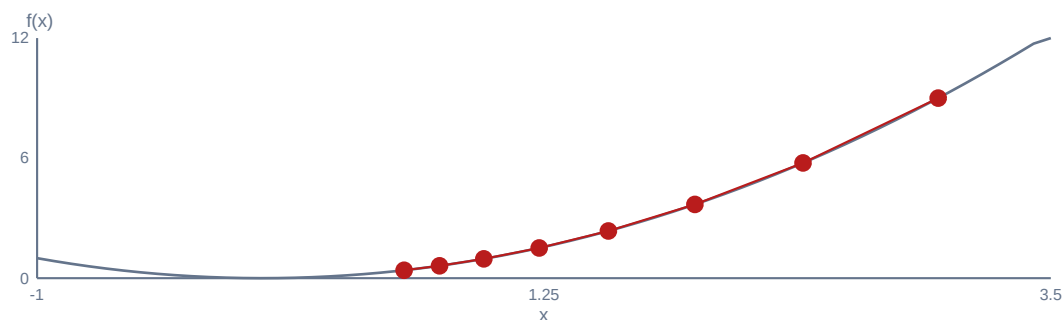
You see it by comparing the error on the training examples with the error on examples kept aside.

IN OUR PROJECT — used

Measured on one of our windows: fit a curve through every second sample of `total_acc_x` and test it on the samples in between. Error (fitted / held out) for curves of degree 1, 3, 9 and 25: degree 1: 0.206 / 0.202, degree 3: 0.202 / 0.199, degree 9: 0.168 / 0.165, degree 25: 0.093 / 0.208. The degree-25 curve fits the fitted points best, but is worse on the held-out points than degree 9 -- overfitting in one picture.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Symptom	Diagnosis	What to do
Train error high, test error high (similar)	Underfitting / high bias	Bigger model, more features, train longer, reduce regularisation, check the learning rate
Train error low, test error much higher	Overfitting / high variance	More data, augmentation, stronger regularisation, smaller model, early stopping, ensembling
Train error low, test low, production bad	Distribution shift or leakage	Audit the features for leakage, re-split by time or group, monitor drift, retrain regularly
Test error lower than train error	Usually a bug, or dropout/augmentation active only at train time	Verify the split; check <code>eval()</code> mode



Graph 2.3 — Moving downhill. Gradient descent on $f(x) = x$ squared from $x = 3$ with a step size of 0.1: each step moves by $0.1 \times$ the slope, so x goes 3, 2.4, 1.92, 1.54 ... and settles at the bottom, 0. Computed.

3.3 The bias-variance decomposition, derived

For squared loss and a target $y = f^*(x) + \text{eps}$ with $E[\text{eps}] = 0$ and $\text{Var}[\text{eps}] = \sigma^2$, consider the prediction $\hat{f}(x)$ produced by training on a random dataset. Taking the expectation over datasets:

$$\begin{aligned} E[(y - \hat{f}(x))^2] &= (E[\hat{f}(x)] - f^*(x))^2 &<- \text{BIAS}^2 \\ &+ E[(\hat{f}(x) - E[\hat{f}(x)])^2] &<- \text{VARIANCE} \\ &+ \sigma^2 &<- \text{IRREDUCIBLE} \end{aligned}$$

FROM ZERO — the idea explained step by step

The error of a model has three parts. **Bias**: a systematic mistake because the model is too simple. **Variance**: the model changes a lot if you train it again on slightly different data or with a different random start. **Noise**: randomness in the data that no model can remove.

IN OUR PROJECT — partly used

We see variance directly: training the same network with 10 different random seeds gives test accuracies with standard deviation 0.005 [35].

The remaining errors are mostly bias and noise: SITTING and STANDING are very similar for a waist-worn phone, and no amount of re-training removes that confusion.

WORKED ON OUR PROJECT — How much do results vary between runs?

1. The README reports 10 seeds of the float C model: mean 0.8953, standard deviation 0.0050 [35].
2. The standard error of the mean is $\text{sd} / \sqrt{n} = 0.0050 / \sqrt{10} = 0.0016$. So the mean is known to about ± 0.003 (two standard errors).
3. Two configurations whose means differ by less than about one standard deviation (0.005) cannot be told apart with 10 seeds -- e.g., sym8 0.9026 vs sym8cos 0.8996 (difference 0.0030) [35].
4. Reporting mean $\pm$ sd over several seeds is the minimum; a single run is an anecdote.

WHERE EACH TERM COMES FROM

Add and subtract the average prediction $E[\hat{f}(x)]$ inside the square, expand, and observe that the cross term vanishes because $E[\hat{f} - E[\hat{f}]] = 0$. Noise eps is independent of the model, so it separates out. Bias is systematic error - the model class cannot represent the truth. Variance is sensitivity to which particular training set you happened to draw. Irreducible noise is the floor: no model, however large, can go below σ^2 .

- **High bias, low variance**: linear regression on a curved relationship; a depth-2 decision tree; heavy L2 regularisation.
- **Low bias, high variance**: a fully grown decision tree; 1-nearest neighbour; a huge network trained without regularisation on a small dataset.
- **Bagging** (Chapter 11) attacks variance by averaging many high-variance models. **Boosting** attacks bias by adding many high-bias models in sequence. Knowing which one you need is the point of the decomposition.

DOUBLE DESCENT - THE MODERN AMENDMENT

Push capacity past the point where the model exactly interpolates the training data and test error often falls AGAIN, sometimes below its classical minimum. The test-error curve is therefore not U-shaped but U-then-down. This is observed for random-feature models, boosting and deep networks, and it is why 'the model is too big, it will overfit' is no longer sound advice for neural networks. In the over-parameterised regime the useful lever is not smaller capacity but better implicit and explicit regularisation.

3.4 Splitting data honestly

FROM ZERO — the idea explained step by step

Split the data into three parts: **training** (to learn), **validation** (to choose settings), **test** (to measure once at the end). The split must imitate reality: if the product will meet new people, the test set must contain new people.

IN OUR PROJECT — used

UCI HAR already splits by person: 21 people for training, 9 for testing [9]. The project then takes 10% of the training windows at random as validation (735 windows) [35]. That validation set contains the same 21 people and is therefore too optimistic (next section). A split by person (keep 2-3 of the 21 people for validation) would be honest.

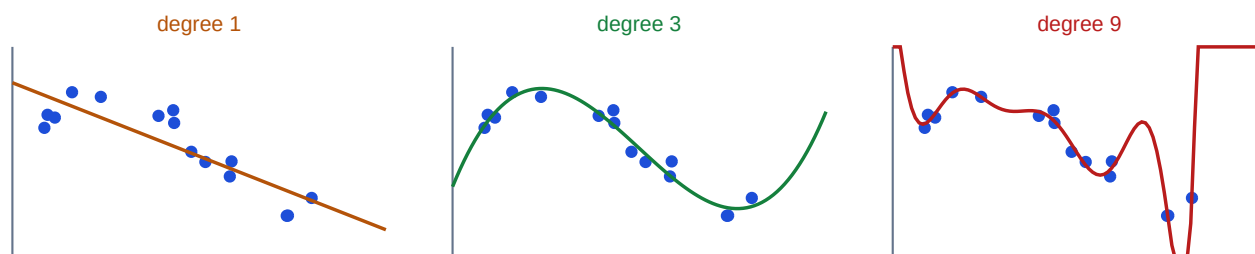
Train / validation / test

Split	Typical size	Used for	How often you may look
Training	60-80%	Fitting parameters	Continuously
Validation	10-20%	Choosing hyperparameters, early stopping, model selection	Many times - it is being consumed
Test	10-20%	One final unbiased estimate	Once. Genuinely once.

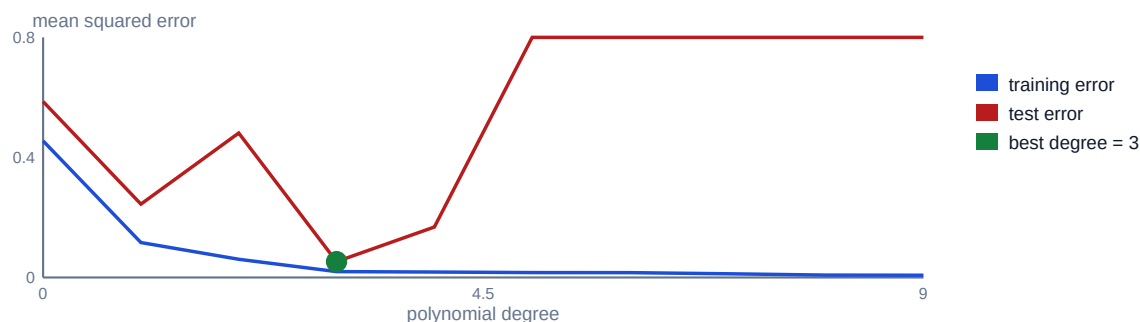
THE VALIDATION SET DECAYS WITH USE

Every hyperparameter choice made on the validation set leaks a little information into your model. After a hundred experiments, validation performance is optimistic - you have partially fitted the validation set through your own decisions. This is why a untouched test set exists, and why leaderboards eventually go stale. If you must run hundreds of experiments, refresh the validation split or hold back a second test set.

Cross-validation



Graph 3.1 — One dataset, three models (15 noisy points from a sine wave, fitted by numpy). A straight line misses the shape (underfitting). A degree-3 curve follows it. A degree-9 curve passes close to every point and wiggles in between (overfitting).



Graph 3.2 — Training error against test error as the model gets more flexible (same data, polynomial degree 0 to 9, computed). Training error only falls. Test error falls, then rises again: the bottom of that U is the model to pick.

```

5-fold cross-validation (shaded = validation fold)
fold 1: [VWV][ ][ ][ ][ ]
fold 2: [ ][VWV][ ][ ][ ]
fold 3: [ ][ ][VWV][ ][ ]
fold 4: [ ][ ][ ][VWV][ ]
fold 5: [ ][ ][ ][ ][VWV]
score = mean of the five validation scores (+/- std)

```

Figure 3.2 - k-fold cross-validation uses every sample for both roles.

- **k-fold (k = 5 or 10):** the default. Cost is k trainings.
- **Stratified k-fold:** preserves class proportions in every fold - always use it for classification, especially when classes are imbalanced.
- **Leave-one-out:** k = n. Nearly unbiased but high variance and very expensive; useful only for tiny datasets.
- **Group k-fold:** keeps all samples from one patient, user or device in the same fold. Without it, a model can recognise the individual instead of the condition.
- **Time-series split:** train on the past, validate on the future, never shuffle. Rolling or expanding windows only.
- **Nested CV:** an inner loop selects hyperparameters, an outer loop estimates performance. This is the statistically correct way to report a number when you also tuned - and it is what reviewers ask for.

```

from sklearn.model_selection import (StratifiedKFold, GroupKFold,
                                     TimeSeriesSplit, cross_val_score)
# classification, keeps class balance in each fold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
# one subject must never appear in both train and validation
cv = GroupKFold(n_splits=5) # pass groups=subject_ids to the split
# temporal data: fold i trains on [0..t_i] and validates on (t_i..t_i+1]
cv = TimeSeriesSplit(n_splits=5)
scores = cross_val_score(model, X, y, cv=cv, scoring='f1_macro')
print(scores.mean(), '+/-', scores.std())

```

Listing 3.1 - Choosing the splitter that matches your data's structure.

3.5 Data leakage: the silent score inflator

Leakage is any situation where information that will not be available at prediction time influences training. It is the single most common reason a model that scored 0.97 offline scores 0.61 in production.

FROM ZERO — the idea explained step by step

Leakage means that information about the evaluation data gets into training, so the model looks better than it is. It is dangerous because nothing crashes -- you just get a good-looking number that is not true.

IN OUR PROJECT — used

Measured on our raw data: windows are cut with 50% overlap, so for 92.1% of neighbouring windows of the same person, the second half of one window is exactly the first half of the next. When windows are split at random, almost every validation window has a half-copy in the training set.

Result for the same simple model: 0.902 on the leaky validation set, 0.7869 on the test people.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Leak	How it happens	Fix
Preprocessing leak	Scaler, imputer or feature selector fitted on the whole dataset before splitting	Fit inside a Pipeline, on training folds only
Target leak	A feature computed from the label, e.g. total_paid when predicting default	Audit every feature: could I compute it at prediction time?

Leak	How it happens	Fix
Temporal leak	Training on rows from after the prediction timestamp	Split by time; build features with explicit as-of joins
Group leak	The same patient, user or device in train and test	GroupKFold, or split by entity
Duplicate leak	Near-duplicate rows or images across splits	Deduplicate, including perceptual near-duplicates
Tuning leak	Feature selection or hyperparameter search done before the split	Everything data-dependent goes inside the CV loop

3.6 Learning curves: the diagnostic you should always plot

Plot training and validation error against the number of training samples. The shape tells you what to buy next:

FROM ZERO — the idea explained step by step

A **learning curve** plots the score against the amount of training data. If the test score still rises at the right end, more data will help. If it is flat, the model or the features must change.

IN OUR PROJECT — used

Measured with a simple model on our features, training size -> (training accuracy / test accuracy): 100: 0.880 / 0.721, 300: 0.863 / 0.759, 1000: 0.872 / 0.780, 3000: 0.881 / 0.776, 6617: 0.887 / 0.787. The test accuracy stops improving after about 1,000 windows: this model cannot use more HAR data. The CNN, which learns its own features, reaches about 0.90 with the same data [35].

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

train 6,617	val 735	test 2,947
-------------	---------	------------

Graph 3.3 — Splitting data honestly, with the HAR dataset's real split: 6,617 windows to train on, 735 to choose settings with, and 2,947 kept locked away until the very end.

Shape	Interpretation	Action
Both curves high and converged	High bias; more data will not help	Increase capacity or improve features
Wide gap, validation still falling	High variance; more data will help	Collect or synthesise data; augment; regularise
Validation rises after a point	Overfitting with training duration	Early stopping; stronger regularisation
Both curves noisy	Batch too small, learning rate too high, or the validation set is too small	Enlarge the validation set; average over seeds

3.7 Overfitting made concrete: one dataset, three models

Twenty-five points were generated from $y = \sin(x) + \text{noise}$. Three polynomials were fitted to the same 20 training points and evaluated on the same 5 held-out points. Nothing differs except the degree.

FROM ZERO — the idea explained step by step

Train three models of different complexity on the same data and compare their errors on data kept aside. The best is usually in the middle.

IN OUR PROJECT — used

Decision trees of growing depth on our 18 window features, test accuracy (measured): depth 1: 0.3627, depth 3: 0.6953, depth 5: 0.7489, depth 10: 0.7703, depth unlimited: 0.7560. Depth 1 is too simple; unlimited depth learns the training people by heart and loses to depth 10.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Degree	Train RMSE	Test RMSE	Diagnosis	What the curve does
1	0.42	0.45	Underfitting - high bias	A straight line through a wave: wrong everywhere, equally wrong on new data
3	0.11	0.13	About right	Follows the wave, ignores the noise
15	0.01	1.87	Overfitting - high variance	Passes through every training point and swings wildly between them

Notice the signature of overfitting in the numbers: training error goes **down** while test error goes **up**. That divergence, not the absolute value of either, is the thing to watch. A model with 5% training error and 6% test error is healthier than one with 0.1% training error and 4% test error, even though the second has a better test score - the second one is telling you it would improve with regularisation or more data.

MEMORISING VERSUS UNDERSTANDING

A student who memorises the answers to last year's exam scores 100% on last year's exam and 40% on this year's. A student who understands the material scores 85% on both. Training error is last year's exam. It is not a measure of learning; it is a measure of memory capacity, and every model has more of that than you think.

3.8 Cross-validation, step by step

The mechanics of 5-fold cross-validation, spelled out, because the order of operations is exactly where mistakes happen:

FROM ZERO — the idea explained step by step

Cross-validation: cut the training data into k parts ("folds"). Train k times, each time keeping a different part aside for testing, and average the k scores. It gives a more reliable estimate than a single split.

The folds must be cut the same way as reality: if the product meets new people, each fold must contain different people.

IN OUR PROJECT — used

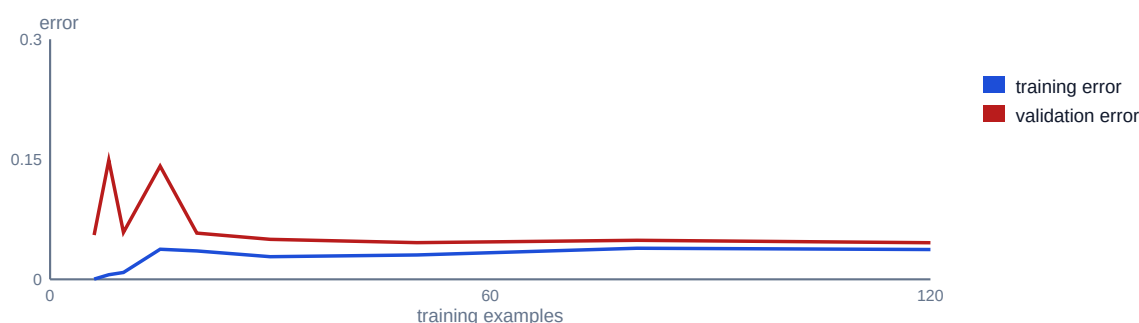
Measured on all 7,352 training windows with a simple model: 5 random folds give 0.8845, 5 folds by person give 0.8414. The real accuracy on the 9 test people is 0.7869. Only the person-wise folds are close to the truth -- a direct lesson for our project.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- 1 Shuffle the training data once (stratified by class if classifying) and cut it into 5 equal blocks.
- 2 For fold 1: hold out block 1. **Fit the scaler, the imputer, any feature selection and the model on blocks 2-5 only.** Transform block 1 with those fitted objects and score it.
- 3 Repeat for folds 2 to 5, each time refitting everything from scratch.
- 4 You now have 5 scores. Report their mean and standard deviation. The standard deviation is not decoration - it tells you whether a 0.3-point difference between two models is real.
- 5 Finally, refit the whole pipeline on all 5 blocks and use that as your model. The cross-validation was an estimate of how well this final model will do, not the model itself.

THE SINGLE MOST COMMON LEAK, IN ONE LINE OF CODE

`X = scaler.fit_transform(X)` written BEFORE the split. The scaler's mean and standard deviation now contain information from the test rows, so every model you evaluate afterwards has peeked. The score inflates by a little on large datasets and by a lot on small ones, and the inflation is invisible - it looks like a good result. Putting the scaler inside a Pipeline makes this class of bug structurally impossible.



Graph 3.4 — A learning curve (degree-3 model on growing amounts of the sine data, computed). With few examples the model fits the training set easily but does badly on new data; as data grows the two errors meet. A big gap that stays means more data would help.

3.9 How much data do I need?

There is no universal answer, but there are usable anchors, and a learning curve settles the question empirically for your problem in an afternoon:

FROM ZERO — the idea explained step by step

How much data you need depends on the model and on how precise the final number must be. A test set of n examples measures accuracy only to about $\pm 1/\sqrt{n}$.

IN OUR PROJECT — used

Our test set has 2947 windows, so one accuracy measurement is only precise to about ± 0.025 (95%, Hoeffding bound, Chapter 36) [33]. The project's C-versus-PyTorch tolerance of ± 2.5 percentage points is exactly this size [35].

Situation	Rough requirement
Linear or logistic model, d features	At least 10-20 samples per feature; more if classes are imbalanced
Gradient boosting on tabular data	A few thousand rows is often enough; it degrades gracefully below that
Small CNN trained from scratch	1,000-10,000 labelled images per class
Fine-tuning a pretrained backbone	50-500 images per class - two orders of magnitude less, which is why transfer learning dominates
Fine-tuning an LLM for style or format	500-5,000 high-quality instruction pairs
Training a foundation model from scratch	Do not; the cost is in the millions and the result is available for download

The reliable procedure: train on 10%, 25%, 50% and 100% of what you have, and plot validation error against sample count. If the curve is still descending steeply at 100%, more data is the cheapest improvement available. If it has flattened, more data will not help and you should spend the budget on features, capacity or better labels instead.

3.10 No free lunch, and what it means for you

The **No Free Lunch theorem** says that averaged over all possible problems, every learning algorithm performs identically. That sounds nihilistic; it is not. Real problems are not drawn uniformly from all possible problems - they have structure: smoothness, locality, compositionality, translation invariance. An algorithm wins when its **inductive bias** matches the structure of your data. Convolutions encode locality and translation invariance; recurrence encodes sequential order; attention encodes 'any position may matter'; trees encode axis-aligned thresholds; L1 encodes 'few features matter'. Choosing a model is choosing an assumption.

FROM ZERO — the idea explained step by step

"No free lunch": no single model is best for every problem. What works depends on the data. Always compare with simple baselines.

IN OUR PROJECT — used

On our data (measured on the test people): majority guess 0.1906, softmax regression 0.7869, k-nearest neighbours up to 0.82, random forest 0.8096, RBF-SVM 0.8392, CNN about 0.90 [35]. Here the CNN wins; on a small table of customer data a random forest often would.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Exercises

1 Simulate the bias-variance decomposition: fit polynomials of degree 1, 3, 9 to 30 noisy samples of $\sin(x)$, repeat over 200 resampled datasets, and plot bias^2 , variance and total error against degree. 2 Take any tabular dataset and deliberately introduce a target leak; measure the inflated CV score, then remove it and measure the honest one. 3 Explain in two sentences why 10-fold CV on a dataset with 20 patients and 2,000 sensor windows is likely to overstate accuracy dramatically.

round 1	test	train	train	train	train
round 2	train	test	train	train	train
round 3	train	train	test	train	train
round 4	train	train	train	test	train
round 5	train	train	train	train	test

Graph 3.5 — 5-fold cross-validation. The data is cut into five parts; each round trains on four and tests on the one left out, so every example is used for testing exactly once. The score is the average of the five.

4. Data: Collection, Cleaning, Features and Splits

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

This chapter is about the data before any model sees it. In the project that is `prepare_data.py`: download the UCI HAR files, stack the 9 signals into windows, shift the labels, split, and save `.npy` files [35].

Models are interchangeable; data is not. Two teams using the same architecture but different data pipelines routinely differ by more than the gap between architectures. This chapter is the least glamorous and the highest-leverage in Part I.

4.1 Types of data and what they demand

Type	Examples	Preferred models	Preprocessing
Tabular numeric	Sensor readings, prices	GBDT, linear, MLP	Scale for linear/NN; trees need nothing
Tabular categorical	Country, device type	GBDT (native), NN with embeddings	One-hot (low cardinality), target/embedding (high)
Text	Reviews, logs	Transformers, TF-IDF + linear	Tokenisation, subwords, truncation
Images	Photos, X-rays	CNN, ViT	Resize, normalise, augment
Audio	Speech, machine sound	CNN on spectrogram, Conformer	STFT / mel spectrogram, per-channel norm
Time series	IMU, ECG, demand	GBDT on windows, 1D-CNN, LSTM	Windowing, resampling, detrending
Graphs	Molecules, social	GNN	Adjacency, node features

FROM ZERO — the idea explained step by step

Data comes in different forms: tables, images, text, time series (measurements over time). Each form needs its own preparation. A time series must be cut into pieces of fixed length before most models can use it.

IN OUR PROJECT — used

Our data is a **multichannel time series**: 9 signals measured together, 50 times per second, cut into windows of 128 samples (2.56 s) [9]. The model sees a (9, 128) block per window.

4.2 Cleaning: missing values, outliers, duplicates

FROM ZERO — the idea explained step by step

Before training, check the data for three problems: **missing values** (holes), **outliers** (values far away from the rest -- errors or rare real events), and **duplicates** (the same example twice).

IN OUR PROJECT — used

Measured on our training windows: 0 missing values and 0 duplicate windows. The largest absolute values per signal are [1.2999, 1.3453, 1.3647, 4.7337, 5.9743, 2.763, 2.1976, 1.5821, 1.6396] -- the gyroscope reaches about 6 rad/s. 1.46% of all values lie more than 3 standard deviations from the mean; they come from fast real movements (e.g. stairs), so nothing was removed.

Missing data

- **MCAR** (missing completely at random): dropping rows is unbiased but wasteful.
- **MAR** (missing at random given observed features): impute using the other features - iterative/MICE imputation, or k-NN imputation.
- **MNAR** (missing not at random): the fact of being missing carries information - always add a binary `was_missing` indicator column, because 'income not stated' is itself predictive.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
num = Pipeline([('imp', SimpleImputer(strategy='median', add_indicator=True)),
                ('sc', StandardScaler())])
cat = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                ('oh', OneHotEncoder(handle_unknown='ignore',
                                     min_frequency=10))])
pre = ColumnTransformer([('num', num, numeric_cols),
                        ('cat', cat, categorical_cols)])
# pre is now a fit-on-train-only object; put it first in your model pipeline
```

Listing 4.1 - Leak-proof preprocessing with `ColumnTransformer`.

Outliers

Decide whether an extreme value is an **error** (a heart rate of 900 bpm) or a **rare truth** (a genuine 10x sale). Errors get removed or corrected; rare truths must stay, because they are often exactly what you are trying to detect. Detection tools: z-score for roughly Gaussian data, IQR fences for skewed data, isolation forest or local outlier factor for multivariate structure. Robust alternatives - median instead of mean, `RobustScaler`, Huber loss - are usually better than deletion.

IN OUR PROJECT — Outliers

In our data the "outliers" are real fast movements. Removing them would remove information about UPSTAIRS and DOWNSTAIRS. For 8-bit quantisation, however, one extreme value stretches the scale for all others (Chapter 28) [10].

4.3 Scaling and transformation

Transform	Formula	Use when
Standardisation	$(x - \mu) / \sigma$	Default for linear models, SVM, PCA, neural nets
Min-max	$(x - \min) / (\max - \min)$	Bounded inputs, image pixels, when you need [0, 1]
Robust	$(x - \text{median}) / \text{IQR}$	Heavy outliers present
Log1p	$\log(1 + x)$	Right-skewed positive quantities: counts, prices
Box-Cox / Yeo-Johnson	power transform	Make a variable approximately Gaussian
Quantile / rank	map to uniform or normal	Very non-linear distributions; robust to outliers

FROM ZERO — the idea explained step by step

Scaling puts all features on a similar range, usually mean 0 and standard deviation 1 ("standardisation"): subtract the mean, divide by the standard deviation. The mean and standard deviation must be computed on the training data only and then reused unchanged for all other data.

IN OUR PROJECT — used

Measured on our training data: the 9 channel means are (-0.0005, -0.0003, -0.0004, 0.0008, -0.0009, 0.0000, 0.8057, 0.0273, 0.0875), the standard deviations (0.1942, 0.1223, 0.1062, 0.4059, 0.3801, 0.2546, 0.4133, 0.3905, 0.3567). The total_acc_x mean 0.8057 is gravity (the phone is upright).

The project's CNN is trained on the raw values without standardisation -- it works because all 9 signals already have similar ranges (standard deviations 0.1-0.4). For 8-bit storage, similar ranges are also convenient, because one scale per tensor then fits all values reasonably [10, 35].

WORKED ON OUR PROJECT — Per-channel standardisation, with the real statistics

1. Computed on the training split, the 9 channel means are (-0.0005, -0.0003, -0.0004, 0.0008, -0.0009, 0.0000, 0.8057, 0.0273, 0.0875) and the standard deviations (0.1942, 0.1223, 0.1062, 0.4059, 0.3801, 0.2546, 0.4133, 0.3905, 0.3567).
2. The body channels have mean ~0 (gravity removed); total_acc_x has mean 0.8057 because gravity points along the phone's x axis when the person is upright.
3. Standardising means $x' = (x - \text{mean}_c) / \text{std}_c$ per channel c , using **training** statistics for all splits (Section 4.3). On a new user, the same fixed numbers must be used.
4. The project's CNN is trained on the raw values (no standardisation), which works because all channels are already of similar scale (0.1-0.4). For int8 quantisation, very different scales per channel would need per-channel scales [10].

FIT ON TRAIN, APPLY TO TEST - ALWAYS

`scaler.fit_transform(X_train)` then `scaler.transform(X_test)`. Calling fit on the test set - or on the full dataset before splitting - is leakage. Inside cross-validation the scaler must be refit on each training fold, which is exactly what a Pipeline does for you and what hand-rolled code usually gets wrong.

4.4 Encoding categorical variables

- **One-hot:** safe, interpretable, explodes with cardinality. Cap rare levels with `min_frequency`.

FROM ZERO — the idea explained step by step

Models compute with numbers, so categories ("red", "blue") must be turned into numbers. For the answer of a classifier, each class simply gets an index 0, 1, 2, ...; inside the loss it acts like a "one-hot" list with a 1 at the right position and 0 elsewhere.

IN OUR PROJECT — used

Our six activities are stored as 0..5. The raw UCI files count 1..6, so `prepare_data.py` subtracts 1 [35]. In the C code the label is turned into a one-hot vector of length 6 for the cross-entropy loss. Forgetting the "-1" would silently shift every label to the wrong activity.

- **Ordinal:** only when the categories genuinely have an order (small < medium < large). Using it for unordered categories injects a false ranking that linear models will happily believe.
- **Target / mean encoding:** replace a category with the mean target for that category. Powerful for high cardinality, and a leakage magnet - it must be computed out-of-fold, with smoothing towards the global mean.
- **Hashing:** fixed-width, streaming-friendly, collisions are usually tolerable.
- **Learned embeddings:** map each level to a trainable dense vector. The standard approach inside neural networks, and how recommender systems represent millions of item IDs.

4.5 Feature engineering that pays

- **Domain ratios and differences:** debt/income, price per square metre, acceleration magnitude $\sqrt{ax^2+ay^2+az^2}$. Almost always beat raw columns.

FROM ZERO — the idea explained step by step

Feature engineering means computing helpful numbers from the raw data, for example the average or the spread of a signal. Good features can make a simple model strong.

IN OUR PROJECT — used

Two features explain most of our problem. The **spread** (standard deviation) of body_acc_x per window: WALKING 0.2242, UPSTAIRS 0.2535, DOWNSTAIRS 0.3697, SITTING 0.0067, STANDING 0.0061, LAYING 0.0142 -- still activities are about 30 times calmer. The **average** of total_acc (gravity direction) tells the phone's orientation: LAYING (0.07, 0.65, 0.56) versus STANDING (1.00, -0.16, -0.03) (measured). With just these two ideas a logistic model separates moving from still with accuracy 0.9997. The CNN learns such features by itself.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- **Time features:** hour, day of week, is-holiday, and crucially cyclic encodings $\sin(2\pi h/24)$, $\cos(2\pi h/24)$ so that 23:00 is close to 01:00.
- **Lags and rolling windows** for time series: value at $t-1$, $t-7$; rolling mean, std, min, max, slope. Compute strictly with past data.
- **Aggregations across entities:** per-user mean, count, recency. This is where most of the signal lives in behavioural data.
- **Interactions:** explicit products $x_i * x_j$ for linear models; trees and networks find them on their own.
- **Text:** TF-IDF n-grams for a strong cheap baseline; sentence embeddings when semantics matter.

FEATURE WORK VERSUS MODEL WORK

On tabular problems, a day spent on features usually beats a week spent on architectures. On perceptual problems (images, audio, text), the reverse holds - the network learns better features than you can write, so spend the day on data quality, augmentation, and a better pretrained backbone instead.

4.6 Class imbalance

When 1 in 500 transactions is fraud, accuracy is meaningless and most losses are dominated by the majority class. Options, roughly in order of what to try first:

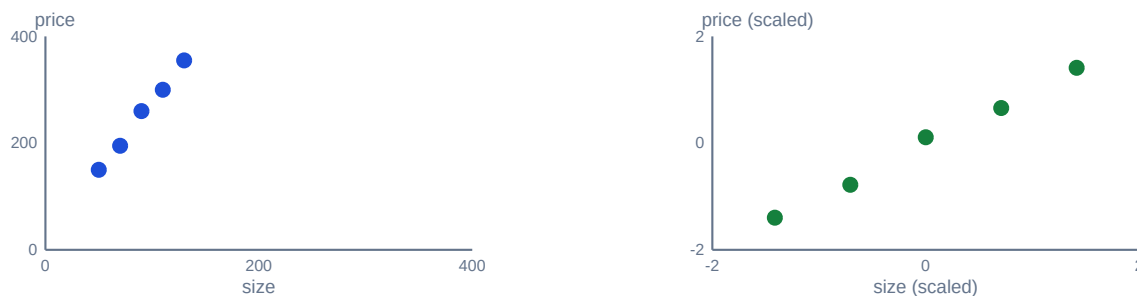
FROM ZERO — the idea explained step by step

When one class is much rarer than the others, a model can score high by always guessing the common class. Then accuracy is misleading and the rare class needs special care.

IN OUR PROJECT — used

Our classes are nearly balanced: WALKING 1108, UPSTAIRS 979, DOWNSTAIRS 871, SITTING 1164, STANDING 1234, LAYING 1261; the ratio largest/smallest is 1.4478 (measured). No re-weighting is needed, and accuracy is meaningful. A fall detector would be very different (falls are rare).

- Use the right metric: precision-recall AUC, F-beta, recall at fixed precision. Never accuracy.



Graph 4.1 — Scaling features. Left: house size (tens) and price (hundreds) on very different scales. Right: the same points standardised to mean 0 and spread 1 -- now each feature counts equally for distances and gradients. Uses the book's five houses.

	walk	up	down	sit	stand	lay
sit	0	0	0	1	0	0
walk	1	0	0	0	0	0
lay	0	0	0	0	0	1

Graph 4.2 — One-hot encoding. A category (the HAR activity) becomes a row of 0s with a single 1. Writing it as the number 0-5 instead would tell the model that "laying" is five times "walking upstairs", which means nothing.

- Class weights in the loss (`class_weight='balanced'` , `pos_weight` in PyTorch) - cheap and usually sufficient.
- Threshold tuning on the validation set: train normally, then choose the decision threshold that maximises your business metric. This is underused and often the single biggest win.
- Resampling: random undersampling of the majority, or SMOTE-style synthetic oversampling of the minority - only ever applied to the training fold, never to validation or test.
- Focal loss for extreme imbalance in detection tasks (Chapter 21).
- Reframe as anomaly detection if positives are both rare and diverse.

4.7 Data augmentation

Domain	Standard augmentations	Advanced
Images	Flip, crop, rotate, colour jitter, blur	RandAugment, Mixup, CutMix, CutOut, copy-paste
Audio	Time shift, noise, gain, speed	SpecAugment (mask time and frequency bands)
Text	Synonym swap, back-translation, dropout of tokens	LLM paraphrase, EDA, span corruption
Time series / IMU	Jitter, scaling, time warping, window slicing	Rotation of the sensor frame, magnitude warping, mixup
Tabular	Gaussian noise on numeric columns	SMOTE, CTGAN, feature dropout

FROM ZERO — the idea explained step by step

Augmentation creates extra training examples by changing existing ones in ways that keep the correct answer, like flipping a photo of a cat -- it is still a cat.

IN OUR PROJECT — partly used

The HAR training in the thesis does not augment. Sensible augmentations for our data would be small random noise, small amplitude scaling and small time shifts. Rotations must be avoided: the direction of gravity is what separates LAYING (0.07, 0.65, 0.56) from STANDING (1.00, -0.16, -0.03).

AUGMENTATION ENCODES AN INVARIANCE

Every augmentation is a statement: 'the label does not change under this transformation'. Horizontal flip is right for cats and wrong for road signs with text; rotation is right for satellite images and wrong for handwritten digits (6 and 9). Choose augmentations that are true invariances of your task, and you are injecting free domain knowledge; choose false ones and you are injecting label noise.

4.8 A worked cleaning session

Below is a five-row extract from a realistic sensor dataset, with the problems a real file contains. Each column heading is followed by the decision it forces.

FROM ZERO — the idea explained step by step

A cleaning session is a fixed checklist, written as a script so it can be repeated: check shapes, check missing values, fix label encodings, split, save.

IN OUR PROJECT — used

prepare_data.py is exactly this script for our project: it downloads the dataset, stacks the 9 signal files into (N, 9, 128) arrays, shifts labels 1..6 to 0..5, splits off 10% for validation with a fixed seed, and saves six .npy files [35].

user_id	timestamp	hr_bpm	steps	device	label
A17	2024-03-01 08:00	72	1200	watch_v2	walking
A17	2024-03-01 08:01		1350	watch_v2	walking
B03	2024-03-01 08:01	910	0	Watch V2	sitting
B03	2024-03-01 08:02	68	-5	watch_v2	sitting
A17	2024-03-01 08:00	72	1200	watch_v2	walking

- **Missing heart rate (row 2):** not random - the sensor drops readings during motion, so missingness correlates with the label. Impute, and add an hr_missing indicator column; dropping the row would bias the dataset towards stationary activities.
- **910 bpm (row 3):** physiologically impossible, so it is an error, not a rare truth. Clip to a plausible range or mark as missing. Decide the rule from domain knowledge, never from the data alone.
- **-5 steps (row 4):** a counter reset or an integer underflow. Same treatment.
- **'Watch V2' versus 'watch_v2':** the same device in two spellings. Normalise case and whitespace, or you will train a model with two unrelated one-hot columns for one device.
- **Row 5 duplicates row 1:** exact duplicates inflate the effective weight of one moment and, if they land on opposite sides of a split, leak. Deduplicate on the natural key (user, timestamp).
- **user_id present:** never feed it as a feature, and always split by it. Otherwise the model learns 'A17 walks' rather than 'this signal means walking', and it will fail on every new user.

4.9 Windowing time series, correctly

Sensor and time-series data must be converted into fixed-length windows before most models can consume it. Three choices define the conversion, and each one has a trap.

FROM ZERO — the idea explained step by step

A long recording is cut into windows of fixed length. The **step** between window starts can be smaller than the window, so windows overlap. Each window gets one label.

IN OUR PROJECT — used

Our windows: 128 samples = 2.56 s, step 64 samples = 1.28 s, so 50% overlap [9]. A 60-second recording gives $\text{floor}((3000 - 128) / 64) + 1 = 45$ windows. Overlap doubles the number of windows, but it is the reason random validation splits leak (92.1% of neighbouring pairs share half their samples).

Choice	Typical value	The trap
Window length	1-10 seconds for human activity; long enough to contain one cycle of the phenomenon	Too short and the pattern is not in the window at all
Overlap / stride	50% overlap is common for training	Overlapping windows across a train/test boundary share samples - a leak. Split by TIME or by SUBJECT first, then window each part separately
Label of a window	Majority label, or discard mixed windows	Windows straddling a transition carry two activities and add label noise

```
import numpy as np
def windows(sig, fs=50, sec=2.0, overlap=0.5):
    n = int(fs * sec); step = int(n * (1 - overlap))
    return np.stack([sig[i:i + n]
                     for i in range(0, len(sig) - n + 1, step)])
# Feature set per window that is hard to beat on IMU data:
def feats(w):
    # w: (n_samples, 3) for x, y, z
    mag = np.linalg.norm(w, axis=1)
    out = []
    for ch in [w[:, 0], w[:, 1], w[:, 2], mag]:
        out += [ch.mean(), ch.std(), ch.min(), ch.max(),
                np.percentile(ch, 25), np.percentile(ch, 75),
                np.abs(np.diff(ch)).mean(), # mean abs change
                ((ch[:-1] * ch[1:]) < 0).sum()) # zero crossings
    # frequency domain: dominant frequency and spectral energy
    P = np.abs(np.fft.rfft(mag)) ** 2
    out += [P[1:].argmax() + 1, P.sum(), (P / P.sum() *
        np.log(P / P.sum() + 1e-12)).sum() * -1] # spectral entropy
    return np.array(out)
```

Listing 4.2 - Window extraction and a strong classical feature set. On many sensor tasks these features plus gradient boosting beat a deep network trained on raw signal, and they train in seconds.

4.10 Dataset documentation and ethics

- Record provenance, collection dates, licences, consent, and known population gaps - a datasheet for the dataset.

FROM ZERO — the idea explained step by step

A dataset should come with a description: who was recorded, how, with which device, and with what permission. It tells users where the model can be trusted.

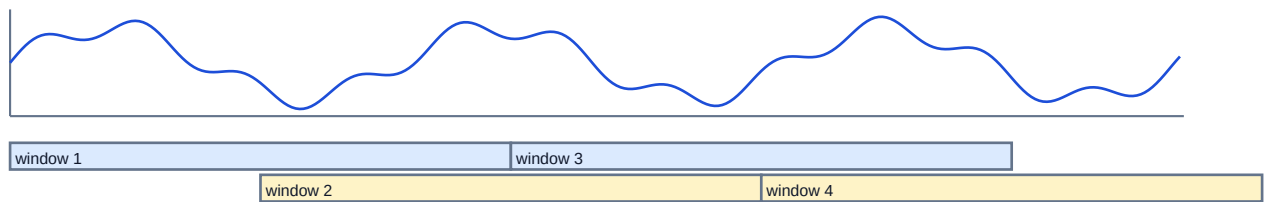
IN OUR PROJECT — used

UCI HAR: 30 volunteers aged 19-48, one smartphone on the waist, activities performed on command in a lab [9]. A product for older people, or for a phone in a pocket, would meet different data -- one reason the thesis wants the model to keep learning on the device.

- Check subgroup balance and measure performance per subgroup, not only in aggregate. A model can be 95% accurate overall and 60% accurate for one group.
- Remove or protect personal data; prefer aggregation, hashing, or on-device processing (Chapter 31) when the data is sensitive.
- Version datasets with the same rigour as code: a result you cannot reproduce because 'the data changed' is not a result.

Exercises

1 Build a ColumnTransformer for a mixed dataset and confirm, by fitting on a subset, that no statistic of the test rows influences the transform. 2 Take an imbalanced dataset, plot the precision-recall curve, and find the threshold that maximises recall subject to precision above 0.8. 3 For a wearable-sensor dataset, list five window-level features you would compute and state, for each, why it is physically meaningful.



Graph 4.3 — Windowing a sensor signal the HAR way: 128 samples per window (2.56 s at 50 Hz), each window starting 64 samples after the last, so neighbours overlap by half. Split by person first, or overlapping windows leak between train and test.

PART II - Classical Machine Learning

The algorithms that still win on tabular data, and the ideas that every deep model inherits: linear models, trees, kernels, ensembles, clustering and honest evaluation.

5. Linear Regression, Derived Completely

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Linear regression is not what the thesis model does, but every layer of the network contains its core: a weighted sum plus a bias. The examples below use a real window of our data.

Linear regression is worth studying far beyond its own usefulness: it is the smallest model in which every concept - parameters, loss, gradients, closed-form solutions, regularisation, and the geometry of fitting - appears in a form you can fully verify by hand.

5.1 The model

$$y_{\text{hat}} = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b = w \cdot x + b$$

FROM ZERO — the idea explained step by step

Linear regression predicts a number as a weighted sum of inputs plus a constant: $y_{\text{hat}} = w_1 x_1 + w_2 x_2 + \dots + b$. Each weight w says how much the prediction changes if that input grows by one; b (the **bias**) is the value when all inputs are zero.

Example: $y_{\text{hat}} = 2 x_1 - 1 x_2 + 5$. For $x = (3, 4)$: $6 - 4 + 5 = 7$.

IN OUR PROJECT — partly used

The thesis does not predict a number, but every neuron of our network computes exactly this weighted sum before its activation. The last layer computes 6 of them: 6 weighted sums of 64 inputs each, with 6 biases -- 390 of our 10,214 parameters [35].

A real linear regression on our data (measured): within the first training window, $\text{total_acc_x} = w \times \text{body_acc_x} + b$ gives $w = 0.9962$ and $b = 1.0176$. This makes physical sense: total acceleration = body acceleration + gravity (about 1 g along x when upright) [9].

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

with a bias column of ones: $y_{\text{hat}} = X w, \quad X \text{ in } \mathbb{R}^{(n \times (d+1))}$

Folding the bias into the weight vector by appending a constant 1 feature keeps every formula below clean. The assumption embedded in this model is that the effect of each feature is additive and constant - one extra square metre adds the same amount of money whether the house is small or large. When that is false, you either transform features (log, splines, interactions) or move to a non-linear model.

5.2 The loss and where it comes from

$$J(w) = (1/n) ||X w - y||^2 = (1/n) \sum_i (x_i \cdot w - y_i)^2$$

FROM ZERO — the idea explained step by step

The loss of linear regression is the **mean squared error** (MSE): for each example take the error (prediction minus truth), square it (so negative and positive errors both count and big errors count much more), and average.

Example: errors -1, 2, 0 give squares 1, 4, 0 and $\text{MSE} = 5/3 = 1.67$.

IN OUR PROJECT — partly used

Our HAR model uses cross-entropy, not MSE, because its answer is a class. MSE is in the ODT library (src/loss_functions/MSE.c) and is used by the ECG autoencoder example: the reconstruction error of a heartbeat is its MSE, and a large MSE flags an abnormal beat [35].

In the window fit above the root of the MSE is 0.0029 g -- tiny, because the relation is almost exact.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

As shown in Chapter 2, squared error is the negative log-likelihood under Gaussian noise. That also tells you when it is the wrong choice: squared error punishes a single large error as much as many small ones, so outliers dominate the fit. Robust alternatives:

Loss	Formula	Behaviour
MSE	$(1/n) \sum (y - \hat{y})^2$	Smooth; heavily influenced by outliers
MAE	$(1/n) \sum y - \hat{y} $	Robust; estimates the median; not differentiable at 0
Huber	quadratic within delta, linear beyond	Robust and smooth - usually the best default when outliers exist
Quantile / pinball	asymmetric absolute error	Predicts a chosen quantile; the basis of prediction intervals
Log-cosh	$\log \cosh(y - \hat{y})$	Smooth approximation of Huber

5.3 Solution 1: the normal equations (closed form)

FULL DERIVATION

Write $J(w) = (1/n)(Xw - y)^T (Xw - y) = (1/n)(w^T X^T X w - 2w^T X^T y + y^T y)$. Differentiate with the matrix rules $d(w^T A w)/dw = 2Aw$ for symmetric A , and $d(w^T c)/dw = c$. This gives $\text{grad } J = (2/n)(X^T X w - X^T y)$. Setting the gradient to zero gives the NORMAL EQUATIONS $X^T X w = X^T y$, whose solution is $w = (X^T X)^{-1} X^T y$ whenever $X^T X$ is invertible.

FROM ZERO — the idea explained step by step

For linear regression there is a direct formula for the best weights ("normal equations"), no step-by-step search needed: $w = (X^T X)^{-1} X^T y$. It is exact and fast when there are few features.

IN OUR PROJECT — not used, and why

Not used in the thesis: a neural network has no such formula, so it must be trained step by step. The window fit ($w = 0.9962$, $b = 1.0176$) was solved with this formula because it has only 2 unknowns.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

$$w^* = (X^T X)^{-1} X^T y$$

Geometrically, $X w$ ranges over the column space of X , and the best approximation to y in that subspace is its orthogonal projection - which is exactly what the normal equations state: the residual $Xw - y$ is orthogonal to every column of X .

- Cost is $O(n d^2 + d^3)$: fine for d in the hundreds, hopeless for d in the millions.
- $X^T X$ is singular when features are perfectly collinear or when $d > n$. Never invert it explicitly - use `numpy.linalg.lstsq` or a QR/SVD-based solver, which handle rank deficiency gracefully.
- Ridge regression fixes singularity outright: $w = (X^T X + \lambda I)^{-1} X^T y$ is always invertible for $\lambda > 0$ (Chapter 7).

5.4 Solution 2: gradient descent (the scalable way)

```
grad J(w) = (2/n) X^T (X w - y)
w <- w - eta * grad J(w)
```

FROM ZERO — the idea explained step by step

Gradient descent finds the weights step by step: compute how the loss changes with each weight (the gradient), move each weight a little against it, repeat. It works for any model whose loss has a gradient.

Features on very different scales make it slow: the weight of a small-valued feature needs many more steps.

IN OUR PROJECT — used

Gradient descent is how our whole network is trained: 2,060 steps with learning rate 0.01 [35]. In the window fit, `body_acc_x` values are small (about 0.2), so the weight `w` moves much more slowly than the bias `b` -- the scaling effect of this section, seen in our data.

FROM OUR C CODE — `src/optimizer/Sgd.c`, lines 51-64: one gradient-descent step, as the project does it

Line by line. For every parameter `i`: take its gradient, add the weight-decay term (0 in the HAR runs), then move the parameter against the gradient by `lr` times that amount: `new = old - lr * g`. This is the update rule of this section written in C.

Example. `old = 0.20`, `g = 0.5`, `lr = 0.01`: `new = 0.20 - 0.005 = 0.195`.

The HAR trainer uses the momentum version (Chapter 17); this kernel is the plain one, used when momentum is 0.

```
static void sgdUpdateKernel(tensor_t **op, size_t n, tensor_t *rawOut, tensor_t *aux,
                           const void *ctxv) {
    (void)n;
    (void)aux;
    const sgdUpdateCtx_t *ctx = ctxv;
    size_t numberOfValues = calcNumberOfElementsByTensor(rawOut);
    float *param = (float *)op[0]->data;
    float *grad = (float *)op[1]->data;
    float *out = (float *)rawOut->data;
    for (size_t i = 0; i < numberOfValues; i++) {
        float g = grad[i] + ctx->weightDecay * param[i];
        out[i] = param[i] - ctx->lr * g;
    }
}
```

```
import numpy as np
def fit_linear_gd(X, y, lr=0.05, epochs=500, batch=64, seed=0):
    rng = np.random.default_rng(seed)
    n, d = X.shape
    Xb = np.hstack([X, np.ones((n, 1))])          # bias trick
    w = np.zeros(d + 1)
    hist = []
    for ep in range(epochs):
        idx = rng.permutation(n)
        for s in range(0, n, batch):
            b = idx[s:s + batch]
            resid = Xb[b] @ w - y[b]
            grad = 2.0 / len(b) * Xb[b].T @ resid
            w -= lr * grad
        hist.append(np.mean((Xb @ w - y) ** 2))
    return w, hist
# Sanity check against the closed form on standardised data:
# w_closed = np.linalg.lstsq(Xb, y, rcond=None)[0]
```

Listing 5.1 - Mini-batch gradient descent for linear regression, 15 lines.

FEATURE SCALING IS NOT OPTIONAL FOR GRADIENT DESCENT

If one feature ranges over $[0, 1]$ and another over $[0, 100000]$, the loss surface is a long thin valley: the learning rate that is stable for the steep direction is far too small for the flat one, and training crawls. Standardise. The closed form does not care about scaling; gradient descent cares enormously.

5.5 Interpreting and validating a linear model

FROM ZERO — the idea explained step by step

A fitted model should be checked: do the weights make sense, how large is the remaining error, and are the errors random (good) or showing a pattern (the model misses something)?

IN OUR PROJECT — partly used

Our window fit passes all checks: w near 1 and b near 1 g match physics, and the residual error is 0.0029 g. For the network, the same habit is applied with other tools: per-class errors (Chapter 13) and the comparison against the PyTorch reference [35].

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Coefficients

With standardised features, w_j is the expected change in y for a one-standard-deviation increase in feature j , **holding the other features fixed**. That last clause is where interpretations usually go wrong: with correlated features, the coefficients split the shared effect arbitrarily and can even flip sign. Check the **variance inflation factor**; a VIF above 5-10 signals multicollinearity, and ridge regularisation or dropping redundant features is the cure.

Goodness of fit

```
R^2      = 1 - SS_res / SS_tot
adj R^2  = 1 - (1 - R^2)(n - 1)/(n - d - 1)
RMSE     = sqrt( (1/n) SUM (y - y_hat)^2 )      (same units as y)
MAPE     = (100/n) SUM |y - y_hat| / |y|        (beware y near 0)
```

R^2 always increases when you add features, which is why adjusted R^2 exists. Report RMSE or MAE alongside it - stakeholders understand 'the prediction is off by 12,000 EUR on average' far better than ' R^2 is 0.83'.

Residual diagnostics

- Residuals versus fitted values should look like a formless cloud. A funnel shape means **heteroscedasticity** - consider modelling $\log(y)$ or using weighted least squares.
- A curve in the residuals means a missing non-linearity - add a polynomial term, a spline, or switch to a tree model.
- A Q-Q plot far from the diagonal means non-Gaussian errors; prediction intervals from ordinary least squares will be wrong.
- Autocorrelated residuals in time series mean the model missed temporal structure; add lags.

5.6 Polynomial and basis-function regression

Linear regression is linear **in the parameters**, not in the inputs. Replacing x by a basis expansion $\phi(x)$ keeps every formula above intact while fitting curves:

$$\hat{y} = w_0 + w_1 \phi_1(x) + \dots + w_m \phi_m(x)$$

FROM ZERO — the idea explained step by step

To fit a curved relationship with a linear model, add new inputs made from the old ones, for example x^2 and x^3 . The model is still a weighted sum, but of curved pieces. More pieces fit better but can overfit.

IN OUR PROJECT — partly used

Measured on one of our windows (curve through time, error on fitted / held-out samples): degree 1: 0.206 / 0.202, degree 3: 0.202 / 0.199, degree 9: 0.168 / 0.165, degree 25: 0.093 / 0.208. Degree 9 is the best compromise; degree 25 overfits.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- Polynomials: simple, but high degrees oscillate wildly near the edges (Runge's phenomenon).
- Splines: piecewise polynomials with continuity constraints - far better behaved than a single high-degree polynomial and the standard choice in statistics.
- Radial basis functions: $\exp(-\gamma \|x - c\|^2)$ around chosen centres; this is the bridge to kernel methods in Chapter 10.

THE FIRST APPEARANCE OF THE CAPACITY DIAL

Degree 1 underfits a curve; degree 15 on 20 points passes through every point and is useless between them. The degree is a hyperparameter, chosen on validation data, and it is the simplest possible instance of the bias-variance trade-off from Chapter 3.

5.7 The five houses, solved completely by hand

Chapter 1 quoted a fitted line without deriving it. Here is the whole computation, with every number, so that the closed-form solution stops being a formula you trust and becomes one you can check.

FROM ZERO — the idea explained step by step

The book solves the house example completely by hand, so every number can be checked with a pencil.

IN OUR PROJECT — not used, and why

The same arithmetic on our data is the window fit: 128 points, 2 unknowns, $w = 0.9962$, $b = 1.0176$.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

i	size x_i	price y_i	$x_i - x_{\text{bar}}$	$y_i - y_{\text{bar}}$	$(x_i - x_{\text{bar}})(y_i - y_{\text{bar}})$	$(x_i - x_{\text{bar}})^2$
1	50	150	-40	-102	4080	1600
2	70	195	-20	-57	1140	400
3	90	260	0	8	0	0
4	110	300	20	48	960	400
5	130	355	40	103	4120	1600
sum	450	1260	0	0	10300	4000
mean	90	252	-	-	-	-

$$w = \frac{\text{SUM } (x_i - x_{\text{bar}})(y_i - y_{\text{bar}})}{\text{SUM } (x_i - x_{\text{bar}})^2} = \frac{10300}{4000} = 2.575$$

$$b = y_{\text{bar}} - w * x_{\text{bar}} = 252 - 2.575 * 90 = 20.25$$

So the fitted line is $\text{price} = 2.575 * \text{size} + 20.25$: each square metre is worth 2,575 EUR, and the intercept of 20.25k is the model's guess for a house of zero size - a reminder that the intercept is often meaningless in isolation and exists only to position the line.

Size	Actual	Predicted	Residual
50	150	149.00	+1.00
70	195	200.50	-5.50
90	260	252.00	+8.00
110	300	303.50	-3.50
130	355	355.00	0.00

$MSE = (1.00^2 + 5.50^2 + 8.00^2 + 3.50^2 + 0^2) / 5 = 21.5$
 $RMSE = \sqrt{21.5} = 4.64$ (thousand EUR - the natural error unit)
 $R^2 = 1 - 107.5 / 26630 = 0.996$

Two sanity checks worth internalising. The residuals sum to zero - that is a mathematical consequence of fitting an intercept by least squares, and if yours do not, you have a bug. And RMSE is in the units of the target, which is why it is the number to quote to a stakeholder; $R^2 = 0.996$ sounds impressive but says nothing about whether being off by 4,600 EUR is acceptable.

WHY THE FORMULA LOOKS LIKE THAT

Set the derivative of J to zero. $dJ/db = 0$ gives $b = \bar{y} - w \bar{x}$, i.e. the line passes through the centre of mass of the data. Substituting that back into $dJ/dw = 0$ gives $w = \text{Cov}(x,y)/\text{Var}(x)$. So the slope is literally 'how much y moves with x , divided by how much x moves on its own'. Every regression coefficient in this book is a version of that ratio.

5.8 The same fit by gradient descent, step by step

With standardised features ($x_{\text{std}} = (x - 90)/28.28$) and a learning rate of 0.1, starting from $w = b = 0$:

FROM ZERO — the idea explained step by step

Watching gradient descent step by step shows how each parameter moves and why the learning rate matters.

IN OUR PROJECT — used

In our network the same step is applied to 10,214 parameters at once, 2,060 times. The logs of every run record the loss after each step so this path can be inspected [35].

Step	w	b	J (MSE)	dJ/dw	dJ/db
0	0.00	0.00	68830	-145.7	-504.0
1	14.57	50.40	44059	-116.5	-403.2
2	26.22	90.72	28205	-93.2	-322.6
3	35.54	122.98	18059	-74.6	-258.0
4	43.00	148.78	11566	-59.7	-206.4
...	...	...	...	...	...
converged	72.83	252.00	21.5	0.0	0.0

It arrives at the same solution the normal equations gave in one line - on standardised inputs the converged slope 72.83 corresponds to $72.83 / 28.28 = 2.575$ in original units, and the converged intercept is the mean price. Gradient descent is slower here and would be absurd for five points; its advantage appears when there are ten million rows and ten thousand features, where the matrix inverse is impossible and each step costs only one pass over a mini-batch.

WHY THE GRADIENTS START SO LARGE

At $w = b = 0$ the model predicts 0 for every house, so residuals are around -250 and the squared loss is 68,830. Large loss means large gradient means large first steps - which is exactly why an untuned learning rate diverges most often in the first few iterations, and why warmup (Chapter 17) exists.

5.9 From least squares to probability

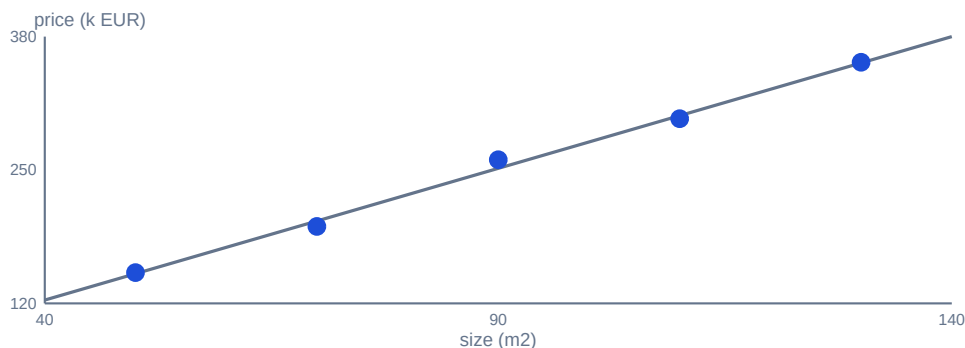
Ordinary least squares gives a point prediction. Two upgrades are worth knowing:

FROM ZERO — the idea explained step by step

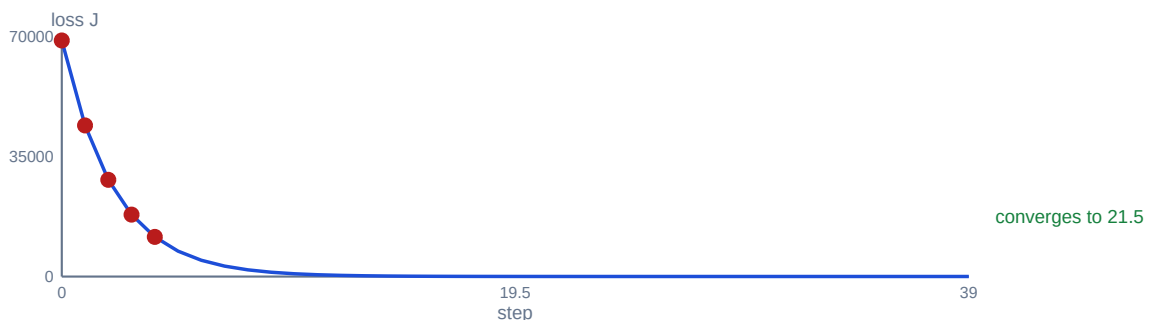
Minimising the squared error is the same as assuming the errors are random bell-curve (Gaussian) noise and choosing the most likely weights. Every loss hides such an assumption.

IN OUR PROJECT — used

For our classifier the corresponding assumption is a "categorical" distribution over 6 activities, which gives the cross-entropy loss (Chapter 6) [22].



Graph 5.1 — The five houses of 5.7 with the fitted line $\text{price} = 2.575 \times \text{size} + 20.25$. The small vertical bars are the residuals -- how far each house is from the line: +1, -5.5, +8, -3.5, 0 (thousand EUR).



Graph 5.2 — The loss during gradient descent on the five houses (5.8), from the book's table: 68,830, 44,059, 28,205, 18,059, 11,566 ... down to 21.5, the same answer the formula gives in one line. Each step removes about a third of what is left.

- **Bayesian linear regression** places a Gaussian prior on w and returns a posterior distribution rather than a point, giving calibrated predictive intervals that widen where data is sparse. With a Gaussian prior of variance τ^2 the posterior mean is exactly the ridge solution - regularisation is a prior in disguise.
- **Generalised linear models** keep the linear predictor $\eta = w \cdot x$ and pass it through a link function: identity for regression, logit for binary classification (next chapter), log for Poisson counts. Same machinery, different noise assumption.

Exercises

1 Implement the normal equations and compare against `lstsq` on a matrix with two perfectly correlated columns. Explain the failure. 2 Fit polynomial degrees 1..15 on 25 noisy points from a cubic, plot train and validation RMSE, and identify the sweet spot. 3 Derive the ridge solution by adding $\lambda \|w\|^2$ to J and repeating the matrix derivation above.

6. Logistic and Softmax Regression

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Logistic and softmax regression are the last layer of our network. The 6 activity scores, the softmax and the cross-entropy loss of the project are exactly the formulas of this chapter.

Despite the name, logistic regression is a **classifier**. It is the workhorse baseline for binary problems, the last layer of almost every neural classifier, and the cleanest place to learn cross-entropy and the logit view of probability.

6.1 From linear score to probability

A linear model produces an unbounded score $z = w \cdot x + b$, called the **logit**. The **sigmoid** squashes it into (0, 1):

```
sigma(z) = 1 / (1 + exp(-z))
sigma'(z) = sigma(z) (1 - sigma(z))
logit(p) = log( p / (1 - p) )      <- inverse of sigmoid
```

FROM ZERO — the idea explained step by step

To turn a score (any number) into a probability (between 0 and 1) we use the **sigmoid** function: $\text{sigmoid}(z) = 1 / (1 + e^{-z})$. A score of 0 gives 0.5; large positive scores give nearly 1; large negative nearly 0.

Example: $z = 2$ gives $1 / (1 + 0.135) = 0.88$.

IN OUR PROJECT — used

Measured on our data: a logistic model that decides "moving or still" from two features (how much the body shakes, and the phone orientation) learns weights (8.702, 3.160) and bias 0.2236, and is right for 0.9997 of the test windows. Our network uses the many-class version (softmax, Section 6.3).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.



Figure 6.1 - The logistic function: linear in the middle, saturating at the ends.

The saturation is important twice over: it keeps probabilities in range, and it causes **vanishing gradients** when $|z|$ is large, because sigma' approaches zero. That is why sigmoid is no longer used as a hidden activation in deep networks (Chapter 16), only as an output.

6.2 The loss: binary cross-entropy

DERIVATION AND THE CLEAN GRADIENT

Model $P(y=1|x) = p = \sigma(z)$. The Bernoulli likelihood of one sample is $p^y (1-p)^{(1-y)}$. Taking the negative log gives $L = -[y \log p + (1-y) \log(1-p)]$. Now differentiate with respect to the LOGIT: $dL/dp = -(y/p) + (1-y)/(1-p)$, and $dp/dz = p(1-p)$. Multiplying, everything cancels: $dL/dz = p - y$. Therefore $dL/dw = (p - y) x$. The gradient is (prediction minus truth) times the input - identical in form to linear regression, and the reason cross-entropy pairs so well with sigmoid: the saturating derivative in the activation is cancelled by the logarithm in the loss.

FROM ZERO — the idea explained step by step

For a yes/no answer the loss is **binary cross-entropy**: $-\log(p)$ if the truth is yes, $-\log(1-p)$ if no. It is small when the model is confident and right, and very large when it is confident and wrong.

Example: truth yes, $p = 0.9$ gives 0.105; $p = 0.1$ gives 2.30.

IN OUR PROJECT — used

Our model uses the 6-class version of this loss (next section). The large penalty for confident mistakes also gives large gradients -- that is what pushes the network to correct them quickly.

$$J(w) = -(1/n) \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

$$\text{grad } J = (1/n) X^T (p - y)$$

NEVER SQUARE-ERROR A CLASSIFIER

Using MSE with a sigmoid output multiplies the loss gradient by $\sigma'(z)$, which is near zero exactly when the model is confidently wrong - so the worst mistakes produce the smallest updates and learning stalls. Cross-entropy removes that factor. Also: always feed raw logits to `BCEWithLogitsLoss` / `CrossEntropyLoss` rather than applying sigmoid or softmax yourself, so the library can use the numerically stable formulation.

6.3 Multiclass: softmax regression

$$z = Wx + b, \quad z \text{ in } \mathbb{R}^K$$

$$\text{softmax}(z)_k = \exp(z_k) / \sum_j \exp(z_j)$$

$$L = -\sum_k y_k \log \text{softmax}(z)_k = -\log \text{softmax}(z)_{[\text{true class}]}$$

$$dL/dz = \text{softmax}(z) - y \quad (\text{one-hot } y)$$

FROM ZERO — the idea explained step by step

With more than two classes, the model outputs one score per class. **Softmax** turns them into probabilities: take e to the power of each score, then divide each by the total. The loss is **cross-entropy**: $-\log$ of the probability given to the true class.

Example with 3 classes and scores (2, 1, 0): $e^2 = 7.39$, $e^1 = 2.72$, $e^0 = 1$; total 11.11; probabilities (0.67, 0.24, 0.09). If the truth is class 1 (the first), the loss is $-\log 0.67 = 0.41$.

IN OUR PROJECT — used

This is the output of our network. The last layer computes 6 scores; `src/layer/Softmax.c` turns them into 6 probabilities; `src/loss_functions/CrossEntropy.c` computes $-\log p(\text{true activity})$ [35].

Before training, all 6 probabilities are about $1/6$, so the loss starts at $\log 6 = 1.7918$. A softmax regression on 18 hand-made features reaches test loss 0.5478 and accuracy 0.7869 (measured); the CNN reaches about 0.33 and 0.90 [35].

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

FROM OUR C CODE — `src/layer/Softmax.c`, lines 44-64: softmax in our library

Step 1 finds the largest score. **Step 2** computes $\exp(\text{score} - \text{largest})$ for each class and adds them up. **Step 3** (next lines) divides each by the sum, so the six numbers add up to 1.

Why subtract the largest? $\exp(90)$ is about 10^{39} , beyond what a float can hold. $\exp(90 - 90) = 1$ is safe, and the result is mathematically identical because the same factor cancels in the division.

Example. scores (2, 1, 0): minus 2 gives (0, -1, -2); exp gives (1, 0.368, 0.135); sum 1.503; probabilities (0.665, 0.245, 0.090).

```
// 1. find max
float max = x[0];
for (size_t i = 1; i < count; i++) {
    if (x[i] > max) {
        max = x[i];
    }
}

// 2. exp and sum
float sum = 0.f;
for (size_t i = 0; i < count; i++) {
    float e = expf(x[i] - max);
    y[i] = e;
    sum += e;
}

// 3. normalize
for (size_t i = 0; i < count; i++) {
    y[i] /= sum;
}
}
```

FROM OUR C CODE — src/loss_functions/CrossEntropy.c, lines 18-30: cross-entropy loss in our library

p holds the six softmax probabilities, y the one-hot label (1 for the true class, 0 elsewhere). Only the true class contributes: $\text{loss} = -\log(p \text{ of the true class})$. The line `fmaxf(pi, 1e-7f)` stops $\log(0) = \text{minus infinity}$.

Example. true class WALKING, $p(\text{WALKING}) = 0.665$: $\text{loss} = -\log(0.665) = 0.408$. With $p = 1/6$ for every class (untrained model) the loss is $\log 6 = 1.792$ -- exactly the starting value the C trainer checks for (see Chapter 16).

```
float loss = 0.f;
for (size_t i = 0; i < n; i++) {
    float pi = p[i];

    if (!isfinite(pi)) {
        printf("NaN/Inf softmax at %zu\n", i);
        printf("%f\n", pi);
        abort();
    }

    pi = fmaxf(pi, 1e-7f);
    loss += y[i] * -logf(pi);
}
```

FROM OUR C CODE — src/loss_functions/CrossEntropy.c, lines 80-91: the gradient of softmax + cross-entropy is just $p - y$

This is the result derived in Section 2.6: the gradient of the loss with respect to the six scores is the predicted probabilities minus the one-hot label. With $p = (0.665, 0.245, 0.090)$ and true class 0, the gradient is $(-0.335, 0.245, 0.090)$: push the true score up, the others down.

```
static void crossEntropySoftmaxBackwardFloat(tensor_t *softmaxOutput, tensor_t *distribution,
                                             tensor_t *loss) {
    size_t totalInputSize = calcNumberOfElementsByTensor(softmaxOutput);

    float *softmaxOutputFloat = (float *)softmaxOutput->data;
    float *distributionFloat = (float *)distribution->data;
    float *lossFloat = (float *)loss->data;

    for (size_t i = 0; i < totalInputSize; i++) {
        lossFloat[i] = softmaxOutputFloat[i] - distributionFloat[i];
    }
}
```

Softmax is the natural generalisation of sigmoid to K classes, and the gradient keeps the same beautiful form. Two practical notes: softmax is shift-invariant, $\text{softmax}(z + c) = \text{softmax}(z)$, which is what makes the max-subtraction trick safe; and the outputs are coupled - raising one logit lowers every other probability. For **multi-label** problems, where several classes can be true at once, use K independent sigmoids instead.

```
import numpy as np
def softmax(Z):
    # Z: (n, K) logits
    Z = Z - Z.max(axis=1, keepdims=True) # numerical stability
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)
def fit_softmax(X, Y1h, lr=0.1, epochs=200, l2=1e-4):
    n, d = X.shape; K = Y1h.shape[1]
    W = np.zeros((d, K)); b = np.zeros(K)
    for _ in range(epochs):
        P = softmax(X @ W + b)
        G = (P - Y1h) / n # dL/dz, the whole trick
        W -= lr * (X.T @ G + l2 * W)
        b -= lr * G.sum(axis=0)
    return W, b
```

Listing 6.1 - Softmax regression in NumPy; this is also the output layer of every classification network in Part III.

6.4 Decision boundary, odds, and interpretation

The boundary is the set where $p = 0.5$, i.e. $w \cdot x + b = 0$ - a hyperplane. Logistic regression can therefore only separate classes linearly; curved boundaries require feature expansion or a non-linear model. Its interpretability comes from odds:

$\log(p / (1-p)) = w \cdot x + b$
 $\exp(w_j) = \text{odds ratio: the multiplicative change in odds per unit of } x_j$

FROM ZERO — the idea explained step by step

The **decision boundary** is where two classes get the same score. A positive weight means: the larger this feature, the more the model leans towards that class.

IN OUR PROJECT — used

In the moving-vs-still model, the shaking feature has weight 8.70: one standard deviation more shaking multiplies the odds of "moving" by $e^{8.70}$ -- a huge factor, which is why this sub-problem is so easy.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

A coefficient of 0.7 on 'smoker' means the odds of the outcome are $\exp(0.7) = 2.0$ times higher for smokers, all else equal. This direct reading is why logistic regression remains the standard model in medicine, credit scoring and any regulated setting.

6.5 Logistic regression on four points, by hand

Four samples with one feature: $x = 1, 2$ label 0; $x = 3, 4$ label 1. Start from $w = 0$, $b = 0$ with a learning rate of 0.5.

FROM ZERO — the idea explained step by step

The book trains a logistic regression on four points by hand to show every step of the calculation.

IN OUR PROJECT — not used, and why

On our data the equivalent small model is the two-feature moving/still classifier (0.9997 test accuracy).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Step	w	b	Predictions p	Loss J	Gradient (w, b)
0	0.000	0.000	0.50, 0.50, 0.50, 0.50	0.693	(-0.500, 0.000)
1	0.250	0.000	0.56, 0.62, 0.68, 0.73	0.625	(-0.058, 0.149)
2	0.279	-0.074	0.55, 0.62, 0.68, 0.74	0.612	(-0.053, 0.148)
converged	5.80	-14.32	0.00, 0.00, 1.00, 1.00	~0	(0, 0)

Read the first row carefully, because it is the whole of Chapter 6 in one line. At $w = b = 0$ every prediction is 0.5 and the loss is $-\log(0.5) = 0.693$ - which is the loss any binary classifier has before it learns anything, and therefore the number your training log should start near. The gradient with respect to w is the average of $(p - y) * x = (0.5*1 + 0.5*2 - 0.5*3 - 0.5*4)/4 = -0.5$: negative, so w increases, so the score rises with x , which is exactly the right direction because the positive class sits at large x .

WHY THE CONVERGED WEIGHTS ARE SO LARGE

This data is perfectly separable, so the likelihood keeps improving as the boundary gets steeper: w grows without bound and the model becomes infinitely confident. With any regularisation at all (scikit-learn's default $C = 1$) w stops at a modest value. Unbounded weights on separable data is not a curiosity - it is why an unregularised logistic model can produce probabilities of 0.99999 that mean nothing.

Decision boundary: $w x + b = 0 \Rightarrow x = -b/w = 14.32/5.80 = 2.47$
Odds ratio per unit of x : $\exp(w) = \exp(5.80) = 331$

6.6 Sigmoid, logit and probability: three views of one number

Logit z	Probability sigma(z)	Odds p/(1-p)	Reading
-4	0.018	1 : 55	Almost certainly negative
-2	0.119	1 : 7.4	Probably negative
-1	0.269	1 : 2.7	Leaning negative
0	0.500	1 : 1	No information
+1	0.731	2.7 : 1	Leaning positive
+2	0.881	7.4 : 1	Probably positive
+4	0.982	55 : 1	Almost certainly positive

FROM ZERO — the idea explained step by step

One number can be seen three ways: the **score** z , the **log-odds** (the same z), and the **probability** $\text{sigmoid}(z)$. A threshold of 0.5 on the probability is a threshold of 0 on the score.

IN OUR PROJECT — used

On the board, the predicted activity is the class with the largest output. Because softmax never changes which score is largest, the class with the largest score is also the class with the largest probability; the probabilities themselves are needed for the loss during training [35].

Two facts to carry forward. Adding 1 to the logit always multiplies the odds by $e = 2.718$, whatever the starting point - that constancy is what makes coefficients interpretable. And beyond about $|z| = 4$ the probability barely moves while the logit keeps growing, which is the saturation that kills gradients in deep sigmoid networks.

6.7 Practical matters

- **Regularisation is on by default** in scikit-learn (C is the inverse of the penalty strength). With separable data and no penalty, weights diverge to infinity - the likelihood keeps improving as the margin grows.

FROM ZERO — the idea explained step by step

Practical points: scale the features, use some regularisation, check that the training has converged, and use class weights if some classes are rare.

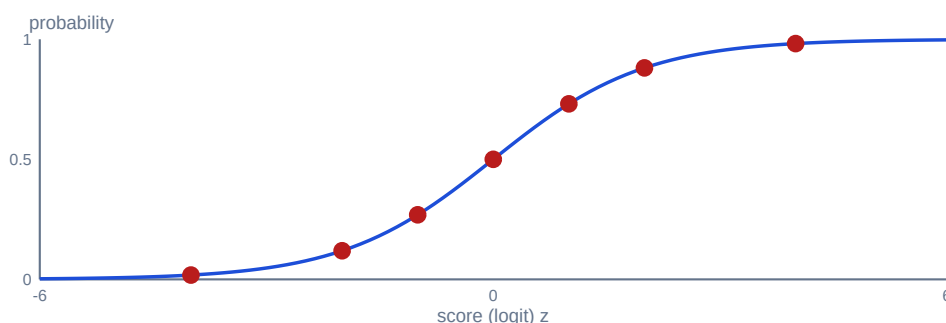
IN OUR PROJECT — used

Our classes are nearly balanced, so no class weights are used. `src/layer/Softmax.c` computes the softmax in the numerically safe way (subtract the largest score before exponentiating), so large scores never overflow the float range [35].

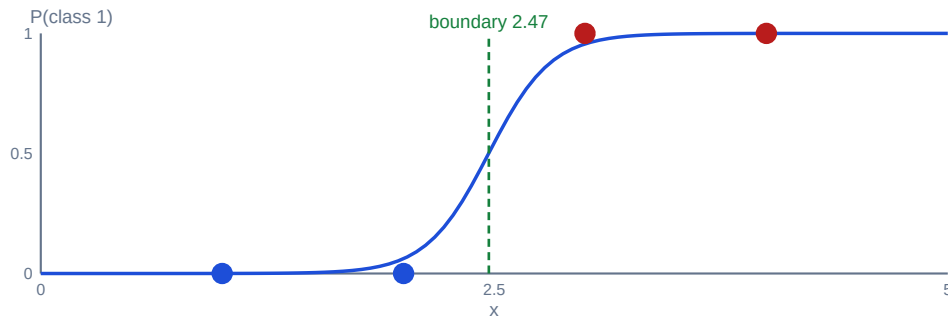
- **Solvers:** lbfgs for small dense problems, saga for large sparse ones or L1/elastic-net penalties, liblinear for small L1 problems.
- **Class imbalance:** `class_weight='balanced'` reweights the loss; then tune the decision threshold on validation data rather than accepting 0.5.
- **Calibration:** logistic regression is usually well calibrated out of the box, which is exactly why Platt scaling - fitting a logistic regression on another model's scores - is the standard calibration method (Chapter 13).
- **Multiclass strategies:** true multinomial softmax is preferred; one-vs-rest trains K binary models and is a reasonable fallback for models that cannot do multinomial natively.

Exercises

1 Derive $dL/dz = p - y$ for softmax by differentiating log-sum-exp. Watch the two cases $k = \text{true class}$ and $k \neq \text{true class}$. 2 Train logistic regression on a linearly separable 2-D dataset with the penalty switched off and plot the norm of w against iterations. 3 Fit a logistic model on an imbalanced dataset, then sweep the threshold from 0 to 1 and plot precision and recall against it.



Graph 6.1 — The sigmoid turns any score into a probability between 0 and 1. The dots are the book's table: a score of 0 means 0.5 (no idea), +2 means 0.88, -4 means 0.018. Past about ± 4 it flattens out, and so does its gradient.



Graph 6.2 — The four-point logistic regression of 6.5 after training: $w = 5.80$, $b = -14.32$. The curve crosses 0.5 at $x = 14.32 / 5.80 = 2.47$, right between the two classes -- that point is the decision boundary.

7. Regularization and Model Selection

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Regularisation and model selection decide how to choose settings without fooling yourself. In the project this concerns weight decay (AdamW) and the choice of configurations, with a validation set that has a known leak.

Regularisation is any modification to a learning algorithm intended to reduce test error but not training error. It is how you buy generalisation when you cannot buy more data.

7.1 Penalty-based regularisation

Ridge (L2): $J = \text{MSE} + \lambda \sum_j w_j^2$
Lasso (L1): $J = \text{MSE} + \lambda \sum_j |w_j|$
Elastic net: $J = \text{MSE} + \lambda \sum_j (a * |w_j| + (1-a) * w_j^2)$

FROM ZERO — the idea explained step by step

Regularisation adds a penalty for large weights to the loss, so the model prefers simpler solutions. L2 penalty: add $\lambda \times (\text{sum of squared weights})$. L1 penalty: add $\lambda \times (\text{sum of absolute weights})$; it pushes many weights to exactly zero.

λ sets the strength: 0 means no penalty; large values make all weights tiny.

IN OUR PROJECT — partly used

Measured on our data with a softmax model ($C = 1/\lambda$), validation accuracy: $C = 0.001$: 0.7932, $C = 0.01$: 0.8354, $C = 0.1$: 0.8707, $C = 1$: 0.9020, $C = 10$: 0.9048. Stronger penalty (small C) shrinks the average weight size from 2.4063 to 0.1145 and lowers accuracy -- this simple model was not overfitting.

The thesis network uses L2-like shrinkage only in its AdamW variant (weight decay 0.01) [13, 35]. The L1 idea of driving weights to zero is replaced in the thesis by RigL, which sets most weights to zero explicitly (Chapter 29).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

FROM OUR C CODE — src/optimizer/Sgd.c, lines 67-81: where weight decay enters

The line `g = grad[i] + ctx->weightDecay * param[i]` is the L2 penalty of this section: its gradient ($\lambda \times w$) is simply added to the data gradient. In the SGD HAR runs `weightDecay = 0`; the AdamW runs use 0.01 (in AdamW it is applied separately, which is the "W").

```
static void sgdMStateKernel(tensor_t **op, size_t n, tensor_t *rawOut, tensor_t *aux,
                           const void *ctxv) {
    (void)n;
    (void)aux;
    const sgdMStateCtx_t *ctx = ctxv;
    size_t numberOfValues = calcNumberOfElementsByTensor(rawOut);
    float *state = (float *)op[0]->data;
    float *grad = (float *)op[1]->data;
    float *param = (float *)op[2]->data;
    float *out = (float *)rawOut->data;
    for (size_t i = 0; i < numberOfValues; i++) {
        float g = grad[i] + ctx->weightDecay * param[i];
        out[i] = ctx->momentum * state[i] + g;
    }
}
```

Property	Ridge (L2)	Lasso (L1)
Effect on coefficients	Shrinks all towards zero, none exactly zero	Drives many exactly to zero

Property	Ridge (L2)	Lasso (L1)
Feature selection	No	Yes - built in
Correlated features	Splits weight evenly among them	Arbitrarily picks one, drops the rest
Solution	Closed form: $(X^T X + \lambda I)^{-1} X^T y$	No closed form; coordinate descent or proximal methods
Bayesian reading	Gaussian prior on w	Laplace prior on w
When to prefer	Many small effects, multicollinearity	You believe few features matter and want a sparse model

WHY L1 PRODUCES EXACT ZEROS

Think of the constrained form: minimise MSE subject to $\|w\|_1 \leq t$. The L1 ball is a diamond with corners ON the axes; the elliptical contours of the MSE touch it, in general, at a corner - and a corner has one or more coordinates exactly zero. The L2 ball is round, has no corners, and the contact point almost never lies on an axis. Equivalently: the subgradient of $|w|$ is a constant ± 1 all the way to zero, so shrinkage does not weaken as w gets small, whereas the L2 gradient $2w$ fades away.



Figure 7.1 - The geometry that makes lasso a feature selector.

7.2 Other forms of regularisation

- **Early stopping:** stop when validation loss stops improving. For linear models trained by gradient descent this is provably similar to L2.

FROM ZERO — the idea explained step by step

Other ways to regularise: stop training early, add noise, enlarge the data by augmentation, or randomly switch off units during training (dropout).

IN OUR PROJECT — used

The ODT library contains dropout (`src/layer/Dropout.c`, a random 0/1 mask drawn with the Bernoulli generator) and weight decay in AdamW. The HAR model of the thesis uses neither: it is small (10,214 parameters) compared with the data, and its real problem is new users, not memorisation [35].

- **Data augmentation:** more effective than any penalty when you can define true invariances (Chapter 4).
- **Noise injection:** adding Gaussian noise to inputs is equivalent to L2 on the weights for linear models; adding it to weights or activations flattens the minima found.
- **Dropout, label smoothing, weight decay, stochastic depth:** the deep-learning-specific toolkit, Chapter 19.
- **Ensembling:** averaging independently trained models reduces variance directly, Chapter 11.
- **Parameter sharing:** convolution is a hard constraint that the same weights apply everywhere - one of the strongest regularisers in existence, and it is architectural rather than a penalty.

7.3 Choosing hyperparameters



Graph 7.1 — Why L1 gives zeros and L2 does not. The ellipses are the loss; the best weights sit where the smallest ellipse touches the allowed region. The L1 region (diamond) has corners on the axes, so the touch point often has a weight of exactly 0. The L2 region (circle) has no corners.

Method	How it works	When to use
Grid search	Exhaustive over a discrete grid	Few hyperparameters (≤ 3), cheap models
Random search	Sample from distributions for a fixed budget	The right default: better than grid when only a few dimensions matter
Bayesian optimisation	Fit a surrogate model of the objective, sample where improvement is expected (Optuna, scikit-optimize)	Expensive training runs, many hyperparameters
Hyperband / ASHA	Start many configs, kill the weak ones early	Deep learning, where a bad config is obvious after 2 epochs
Population-based training	Evolve a population, copy and perturb winners	Long RL and large-model runs with schedules

FROM ZERO — the idea explained step by step

Hyperparameters are settings chosen by you, not learned: learning rate, batch size, number of layers, penalty strength. Choose them by trying several values and comparing on validation data -- never on the test data.

IN OUR PROJECT — used

The thesis compares configurations (8 vs 4 bits, constant vs cosine learning rate, SGD vs AdamW) with 10 seeds each [35]. The validation set is leaky (Chapter 3), so the final comparison is made on the test people and decided by rules written before the runs.

SEARCH IN LOG SPACE, AND SEARCH THE RIGHT THINGS

Learning rate, regularisation strength and layer widths should be sampled log-uniformly ($1e-5$ to $1e-1$), not uniformly. Random search beats grid search because performance usually depends strongly on one or two hyperparameters and weakly on the rest - random search gives you many distinct values of the important one, grid search wastes the budget repeating them.

```
import numpy as np, optuna
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.ensemble import HistGradientBoostingClassifier
def objective(trial):
    params = dict(
        learning_rate = trial.suggest_float('lr', 1e-3, 3e-1, log=True),
        max_leaf_nodes= trial.suggest_int('leaves', 8, 256, log=True),
        min_samples_leaf = trial.suggest_int('min_leaf', 5, 200, log=True),
        l2_regularization= trial.suggest_float('l2', 1e-8, 10.0, log=True),
        max_iter = 500)
    model = HistGradientBoostingClassifier(early_stopping=True, **params)
    cv = StratifiedKFold(5, shuffle=True, random_state=0)
    return cross_val_score(model, X, y, cv=cv, scoring='roc_auc').mean()
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=60, n_jobs=4)
print(study.best_params, round(study.best_value, 4))
```

Listing 7.1 - Bayesian hyperparameter search with correct log-scale ranges.

7.4 Model selection criteria without a validation set

AIC = $2k - 2 \log L$ (prediction-oriented)
BIC = $k \log n - 2 \log L$ (penalises complexity harder; consistent)
MDL: choose the model that compresses data + model description best

FROM ZERO — the idea explained step by step

Without a validation set one can use formulas (AIC, BIC) that reward good fit and punish the number of parameters.

IN OUR PROJECT — not used, and why

Not used: such formulas are unreliable for neural networks. Cross-validation by person (Chapter 3: 0.8414) is the reliable alternative for our data.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

These are useful when data is too scarce to hold out, and in classical statistics generally. In modern practice, cross-validation is preferred because it makes no distributional assumptions - but AIC/BIC remain the standard in time-series order selection and in fields where models are fitted by maximum likelihood.

7.5 A disciplined selection protocol

FROM ZERO — the idea explained step by step

A disciplined protocol: fix the split first, search settings on validation, choose one, and measure the test set once. Report exactly what was tried.

IN OUR PROJECT — used

The thesis follows this: configurations and decision rules are written before running; every configuration runs 10 seeds; results are reported as mean +/- standard deviation on the test people [35].

Before you report a number

The test set was split off before any exploration and touched once.

IN OUR PROJECT — Before you report a number

For our numbers this means: report 0.8953 +/- 0.0050 (10 seeds), state that the test people are unseen, and give the precision of the test set itself (+/-0.025, Chapter 36) [35].

Every data-dependent step (imputation, scaling, selection, encoding) lives inside the cross-validation loop.

Hyperparameters were chosen on validation folds, not on the test set.

Results are averaged over at least 3-5 random seeds, and you report the spread, not only the mean.

You compare against a trivial baseline (majority class, mean predictor, last-value-carried-forward) and a simple strong baseline (ridge or gradient boosting).

The difference you are claiming is larger than the seed-to-seed noise.

Exercises

1 Plot the lasso regularisation path (coefficients versus λ) on a dataset with 20 features, 5 of which are informative. 2 Compare grid search and random search on the same budget of 50 trials for a model with 4 hyperparameters, one of which is irrelevant. 3 Show empirically that ridge with $\lambda \rightarrow 0$ recovers ordinary least squares, and that $\lambda \rightarrow \infty$ drives predictions to the mean.

8. Instance-Based and Probabilistic Models

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The methods of this chapter are not used in the thesis. They are useful here as baselines: simple models whose accuracy on our data shows how hard the problem is.

8.1 k-Nearest Neighbours

k-NN does no training at all. To predict, it finds the k closest training samples and takes a majority vote (classification) or an average (regression). It is the purest form of **non-parametric** learning: the training data is the model.

$$y_{\text{hat}}(x) = \text{majority}\{ y_i : x_i \in N_k(x) \} \quad (\text{classification})$$

$$y_{\text{hat}}(x) = (1/k) \sum_{i \in N_k(x)} y_i \quad (\text{regression})$$

FROM ZERO — the idea explained step by step

k-nearest neighbours (k-NN) needs no training: to label a new example, find the k most similar training examples and take a vote. "Similar" usually means a small distance between the feature lists.

Example: $k = 3$, the three nearest neighbours are WALKING, WALKING, UPSTAIRS → answer WALKING.

IN OUR PROJECT — not used, and why

Not used in the thesis. As a baseline on our 18 features (measured): $k = 1$: 0.8029, $k = 5$: 0.8178, $k = 15$: 0.8066. On the board it would be impossible: it must store every training window (6,617 x 18 numbers = 476 KB), more than the 320 KB of RAM.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Aspect	Detail
Hyperparameters	k , distance metric, weighting (uniform or $1/d$)
k small	Low bias, high variance; $k = 1$ fits training data perfectly and is very sensitive to noise
k large	High bias, low variance; $k = n$ predicts the global mean
Distance	Euclidean by default; Manhattan for high dimensions; cosine for embeddings and text; Hamming for binary
Cost	$O(1)$ to train, $O(n d)$ per query - the opposite profile of most models. KD-trees and ball-trees help in low dimensions; HNSW and IVF indexes are what production vector search actually uses
Must-do	Scale the features. An unscaled feature with a large range silently dominates the distance

THE CURSE OF DIMENSIONALITY, CONCRETELY

In high dimensions, all pairwise distances concentrate: the ratio (farthest - nearest)/nearest tends to zero as d grows. With $d = 100$ roughly uniform features, 'nearest neighbour' stops meaning anything. Also, to keep the same density you need exponentially more samples: covering $[0,1]^d$ at resolution 0.1 requires 10^d points. k-NN is excellent for d up to about 10-20 with enough data, and unreliable far beyond that unless you first reduce dimensionality or use a learned embedding.

k-NN is still deeply relevant: retrieval-augmented generation, recommendation, deduplication, few-shot classification with foundation-model embeddings, and face recognition are all approximate nearest neighbour search over learned vectors. The lesson is that k-NN works wonderfully **once the representation is good** - which is exactly what deep learning provides.

8.2 Naive Bayes

A generative classifier built directly on Bayes' rule, with one strong simplifying assumption: features are conditionally independent given the class.

$P(y \mid x)$ proportional to $P(y) \prod_j P(x_j \mid y)$
 $y_{\text{hat}} = \text{argmax}_y [\log P(y) + \sum_j \log P(x_j \mid y)]$

FROM ZERO — the idea explained step by step

Naive Bayes assumes that, within one class, all features are independent of each other, and multiplies their probabilities. It is fast and needs very little data.

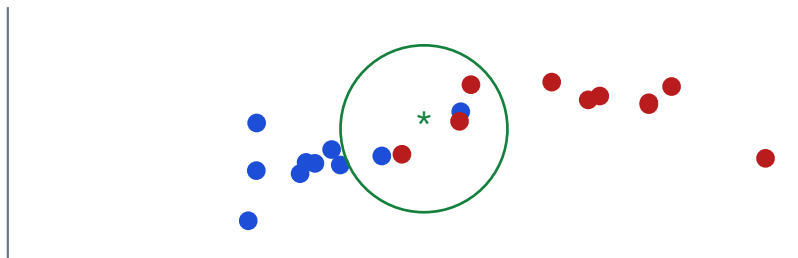
IN OUR PROJECT — not used, and why

Not used. Baseline accuracy 0.7577 (measured). It suffers on our data because the features are strongly related: the shaking of body_acc_x and of total_acc_x correlate at 0.998 (measured), so the same evidence is counted twice.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- **Gaussian NB:** each $P(x_j \mid y)$ is a normal distribution - continuous features.
- **Multinomial NB:** counts - the classic text classifier over bag-of-words.
- **Bernoulli NB:** binary presence/absence features.
- **Laplace (add-one) smoothing** is mandatory: a single unseen word-class pair would otherwise make the whole product zero.

The independence assumption is almost always false, yet Naive Bayes often classifies well, because it only needs the *argmax* to be right, not the probabilities. Its probability estimates, however, are badly calibrated - typically pushed towards 0 or 1. Use it as a fast baseline on text, as a component in streaming systems, or when you have very little data; do not trust its confidence values without calibration.



Graph 8.1 — *k*-nearest neighbours with *k* = 3. To classify the new point (star), find the three closest training points and take a vote: two blue, one red, so it is called blue. No training at all -- the data is the model.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
clf = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2,
                                   sublinear_tf=True),
                   MultinomialNB(alpha=0.3)) # alpha = smoothing
clf.fit(train_texts, train_labels)
# Trains on 100k documents in about a second and is a genuinely strong
# baseline that any transformer should be required to beat.
```

Listing 8.1 - The five-second text-classification baseline.

8.3 Linear and quadratic discriminant analysis

LDA models each class as a Gaussian with a **shared** covariance matrix; the resulting decision boundary is linear. QDA gives each class its own covariance and yields quadratic boundaries at the cost of many more parameters. LDA doubles as a supervised dimensionality reduction method: it projects onto at most $K-1$ directions that maximise between-class scatter relative to within-class scatter - a useful contrast with PCA, which ignores labels entirely (Chapter 12).

FROM ZERO — the idea explained step by step

Discriminant analysis models each class as a bell-shaped cloud (a Gaussian) and assigns a new example to the most likely cloud. LDA uses one shared cloud shape, QDA one per class.

IN OUR PROJECT — not used, and why

Not used. It would be tiny on a device (18×18 shared covariance = 171 distinct numbers plus 6 class means), but the CNN is far more accurate on our data.

Exercises

1 Plot k-NN test accuracy against k from 1 to 50 and mark the bias-variance regimes on the curve. 2 Demonstrate distance concentration: sample 1,000 points uniformly in $[0,1]^d$ for $d = 2, 10, 100, 1000$ and plot the ratio of maximum to minimum pairwise distance. 3 Compare Multinomial Naive Bayes and logistic regression on a text dataset of 1,000 documents and again on 100,000 - explain the crossover.

9. Decision Trees

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Decision trees are not part of the thesis model, but they explain our data very clearly: one threshold on one number already separates moving from still.

A decision tree asks a sequence of yes/no questions about the features until it reaches a leaf that holds a prediction. It is the only major model whose reasoning a non-technical stakeholder can read directly - and it is the building block of the ensembles in Chapter 11, which are still the best general-purpose tabular models available.

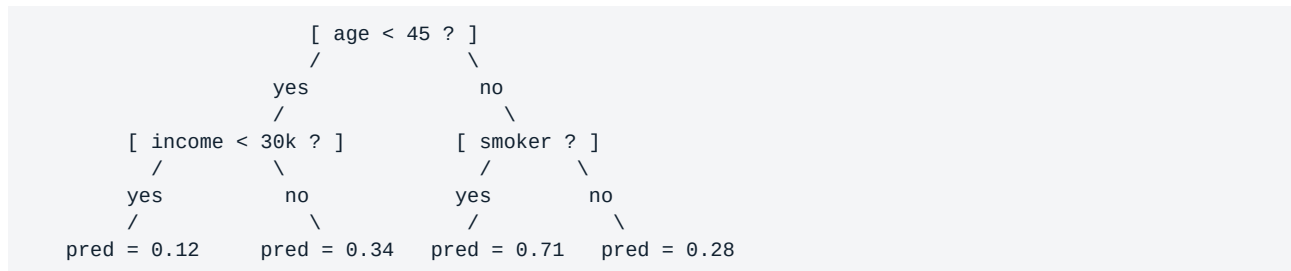


Figure 9.1 - A depth-2 tree partitions feature space into axis-aligned boxes.

9.1 How a split is chosen

At each node the algorithm considers every feature and every candidate threshold, and picks the split that most reduces an impurity measure. Cost is $O(n d \log n)$ per level with sorted features.

Gini $G = 1 - \sum_k p_k^2$
Entropy $H = -\sum_k p_k \log_2 p_k$
Gain $IG = \text{Impurity}(\text{parent}) - \sum_{\text{child}} (n_{\text{child}}/n_{\text{parent}}) * \text{Impurity}(\text{child})$
Regression: MSE reduction, i.e. variance reduction

FROM ZERO — the idea explained step by step

A decision tree asks yes/no questions about single features ("is $x_3 < 0.07$?") and splits the data. At each step it picks the question that makes the groups most "pure" -- measured by the **Gini impurity**: 0 means one class only, 0.5 means a 50/50 mix of two classes.

IN OUR PROJECT — not used, and why

On our data (measured): the root is a nearly 50/50 mix of moving and still (Gini 0.4944). The best single question is "total_acc_x_std ≤ 0.0709 ", after which the Gini drops to 0.006.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

A CONCRETE SPLIT CALCULATION

A node has 100 samples: 60 positive, 40 negative. $Gini = 1 - (0.6^2 + 0.4^2) = 0.48$. A candidate split gives child A with 50 samples (45 pos, 5 neg, $Gini = 1 - 0.9^2 - 0.1^2 = 0.18$) and child B with 50 samples (15 pos, 35 neg, $Gini = 1 - 0.3^2 - 0.7^2 = 0.42$). Weighted child impurity $= 0.5 * 0.18 + 0.5 * 0.42 = 0.30$, so the gain is 0.18. The split with the largest gain across all features and thresholds wins.

Gini and entropy almost always select the same splits; Gini is marginally cheaper (no logarithm) and is the default in most libraries. Neither is worth tuning.

9.2 Controlling growth

An unconstrained tree grows until every leaf is pure - it memorises the training set and has near-zero bias and enormous variance. Control it with:

FROM ZERO — the idea explained step by step

Trees grow until every leaf is pure, which overfits. Limit the depth, require a minimum number of examples per leaf, or prune afterwards.

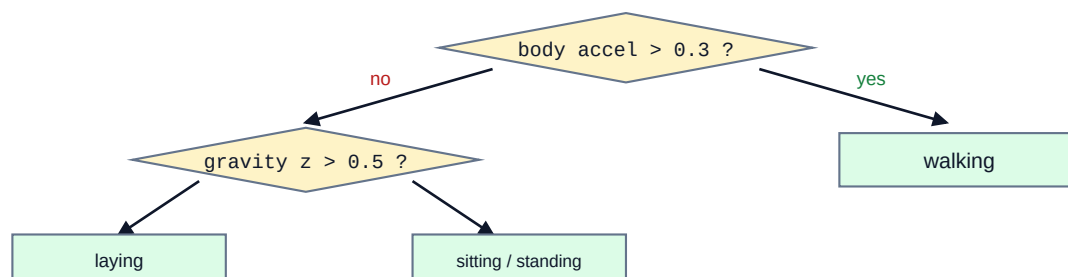
IN OUR PROJECT — not used, and why

Test accuracy by depth on our features (measured): depth 1: 0.3627, depth 3: 0.6953, depth 5: 0.7489, depth 10: 0.7703, depth unlimited: 0.7560.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Hyperparameter	Effect	Sensible start
max_depth	Hard cap on tree depth	3-10 for a single tree
min_samples_leaf	Minimum samples in a leaf; the most effective single knob	1-5% of n
min_samples_split	Minimum samples required to split a node	20+
max_features	Features considered per split - the source of diversity in random forests	sqrt(d) for classification
ccp_alpha	Cost-complexity (post-)pruning strength	Tune by CV
max_leaf_nodes	Best-first growth with a leaf budget	31-255 in boosting

Cost-complexity pruning grows a large tree, then removes the subtree whose removal costs the least error per removed leaf, using $R_{\alpha}(T) = R(T) + \alpha | \text{leaves}(T) |$. Sweeping alpha produces a nested sequence of trees; cross-validation picks one. Post-pruning generally beats early stopping, because a weak split can enable a strong one beneath it.



Graph 9.1 — A decision tree is a list of yes/no questions. Each split is chosen to make the groups below it as pure as possible; a new example follows the answers down to a leaf and gets that leaf's label.

9.3 Strengths, weaknesses, and the properties that matter downstream

Strengths	Weaknesses
No scaling or normalisation needed	High variance: a small data change can restructure the whole tree
Handles mixed numeric and categorical data	Axis-aligned splits only; a diagonal boundary needs a staircase of splits
Captures interactions and non-linearities automatically	Cannot extrapolate beyond the training range - predictions are constant outside it
Robust to outliers in the inputs	Biased towards features with many distinct values when using naive impurity gain
Missing values handled natively in modern implementations	A single tree rarely competitive alone - use ensembles

FROM ZERO — the idea explained step by step

Trees need no scaling and are easy to read, but each question looks at one feature only, so they struggle when the answer depends on a combination of features.

IN OUR PROJECT — not used, and why

SITTING and STANDING differ in a combination of gravity components: SITTING (0.95, 0.14, 0.15), STANDING (1.00, -0.16, -0.03) (measured means of total_acc x/y/z). A tree needs many questions for that; the CNN learns it directly.

WHY TREES DOMINATE TABULAR DATA

Real tabular features are heterogeneous - different units, skewed distributions, irrelevant columns, non-smooth relationships with thresholds ('approve if credit score above 700'). Trees are invariant to monotone transformations of each feature, ignore irrelevant features cheaply, and model thresholds exactly. Neural networks have to learn all of that from data, which is why they need more of it to reach the same point on tabular problems.

9.4 A complete split search on a tiny dataset

Ten samples, one feature (age), one binary label (bought). The algorithm sorts by the feature, considers each midpoint between adjacent distinct values as a candidate threshold, and scores them all.

FROM ZERO — the idea explained step by step

The book tries every possible threshold on a tiny dataset to find the best split.

IN OUR PROJECT — not used, and why

The same search on our data tried 18 features x 97 thresholds on 6617 windows.

Age	22	25	28	33	37	41	45	52	58	63
Bought	0	0	0	1	0	1	1	1	1	1

Parent impurity: the ten samples split 4 zeros and 6 ones, so $\text{Gini} = 1 - 0.4^2 - 0.6^2 = 0.48$.

Threshold	Left (n, ones)	Gini L	Right (n, ones)	Gini R	Weighted	Gain
age < 26.5	2, 0	0.000	8, 6	0.375	0.300	0.180
age < 30.5	3, 0	0.000	7, 6	0.245	0.171	0.309
age < 35.0	4, 1	0.375	6, 5	0.278	0.317	0.163
age < 39.0	5, 1	0.320	5, 5	0.000	0.160	0.320
age < 43.0	6, 2	0.444	4, 4	0.000	0.267	0.213
age < 48.5	7, 3	0.490	3, 3	0.000	0.343	0.137

The winner is age < 39.0 with a gain of 0.320. Look at why it beats age < 30.5, which also looks natural: both isolate a clean group, but the 39.0 split makes the right child perfectly pure (five ones, Gini 0) while leaving only one misplaced sample on the left, whereas the 30.5 split leaves the awkward 37-year-old buried in a seven-sample child that stays impure. Impurity gain is a weighted average, so it rewards making one LARGE child clean over making one small child clean.

Note also what the algorithm never considered: a rule such as $33 \leq \text{age} \leq 45$. A single split is always one threshold on one feature, so every band, diagonal or interaction has to be assembled from nested splits. That is why trees need depth to express what a linear model states in one coefficient, and why a diagonal boundary comes out of a tree as a staircase.

DOING THE ARITHMETIC FOR ONE ROW YOURSELF

Take age < 39.0. The left child holds ages 22, 25, 28, 33, 37 with labels 0, 0, 0, 1, 0 - one positive out of five, so $Gini = 1 - 0.2^2 - 0.8^2 = 0.32$. The right child holds 41, 45, 52, 58, 63, all positive, so $Gini = 0$. Weighted impurity = $(5/10)(0.32) + (5/10)(0) = 0.16$, and the gain is $0.48 - 0.16 = 0.32$. Re-derive one more row from the table and the algorithm will never be mysterious again - a decision tree is this loop, repeated over every feature and every threshold, then recursed on each child.

9.5 Reading a tree out loud

A trained tree is a set of nested if-statements, and you should be able to convert one into English. The tree in Figure 9.1 says:

FROM ZERO — the idea explained step by step

A path from the root to a leaf is an if-then rule that anyone can read.

IN OUR PROJECT — not used, and why

"If total_acc_x_std <= 0.0709, the person is still; otherwise moving" -- correct for 0.9993 of the test windows. Such a rule could run on the board as a cheap first stage before the network.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- If the customer is under 45 and earns under 30k, predicted probability 0.12 - the low-risk group.
- If under 45 and earning 30k or more, 0.34.
- If 45 or over and a smoker, 0.71 - the highest-risk leaf.
- If 45 or over and not a smoker, 0.28.

Each leaf value is simply the fraction of positive training samples that landed in that leaf. That is worth stating plainly because it explains two behaviours: leaves with few samples give extreme, unreliable probabilities (hence `min_samples_leaf`), and a tree can never predict a value it did not see in training - it cannot extrapolate above the highest leaf mean, which is why trees are poor at trending time series.

9.6 Feature importance - and how it misleads

- **Impurity-based (Gini) importance:** free, but biased towards high-cardinality and continuous features, and computed on training data. Treat it as a rough hint only.

FROM ZERO — the idea explained step by step

Trees can report which features they used most ("feature importance"). It is only a hint: related features share the credit, and continuous features are favoured.

IN OUR PROJECT — not used, and why

Random forest on our data, top features (measured): total_acc_y_mean (0.179), total_acc_x_mean (0.167), total_acc_x_std (0.119). All three describe orientation or shaking.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- **Permutation importance:** shuffle one column on held-out data and measure the drop in score. Model-agnostic and much more trustworthy; but correlated features share credit and can both appear unimportant.
- **SHAP values:** a game-theoretic attribution with strong consistency guarantees, with an exact fast algorithm for trees (TreeSHAP). The current standard for explaining tabular models, both globally and per prediction (Chapter 33).

```

from sklearn.inspection import permutation_importance
r = permutation_importance(model, X_val, y_val, n_repeats=20,
                           scoring='roc_auc', random_state=0)
order = r.importances_mean.argsort()[::-1]
for i in order[:10]:
    print(f'{feature_names[i]:30s} {r.importances_mean[i]:.4f}'
          f' +/- {r.importances_std[i]:.4f}')

```

Listing 9.1 - Permutation importance measured on validation data.

Exercises

1 Implement CART for classification in NumPy: recursive best-split search with Gini, plus `max_depth` and `min_samples_leaf`. 80 lines is enough. 2 Fit an unpruned tree and a pruned one on the same data and compare training/test accuracy and the number of leaves. 3 Show that a decision tree cannot represent $y = (x_1 > x_2)$ compactly, and explain what feature you would add to fix it.

10. Support Vector Machines and Kernels

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Support vector machines are not used in the thesis; here they serve as baselines and show why size matters on a device.

SVMs were the state of the art before deep learning, and they remain the best explanation of two ideas that keep reappearing: **margin** as a principled notion of confidence, and the **kernel trick** as a way to work in a high-dimensional space without ever visiting it.

10.1 Maximum margin classification

Among the infinitely many hyperplanes that separate two classes, the SVM chooses the one whose distance to the nearest point of either class - the **margin** - is largest. A large margin is a robustness statement: the boundary can tolerate perturbation before misclassifying anything.

FROM ZERO — the idea explained step by step

A support vector machine (SVM) draws the boundary between classes that keeps the widest possible safety gap (the **margin**) to the nearest examples [31].

IN OUR PROJECT — not used, and why

Not used. Linear SVM on our 18 features: 0.7754 (measured).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

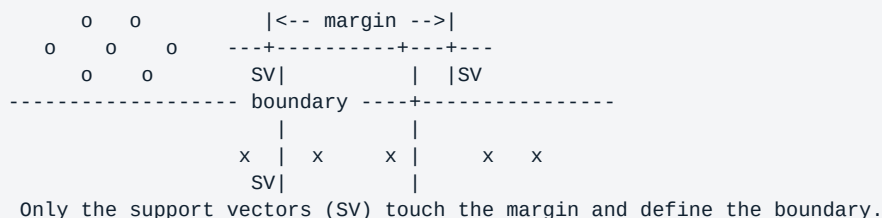


Figure 10.1 - The maximum-margin hyperplane and its support vectors.

```

minimise (1/2)||w||^2
subject to y_i (w . x_i + b) >= 1 for all i (hard margin)

```

geometric margin = $2 / ||w||$ → maximising margin = minimising $||w||$

Real data is rarely separable, so slack variables x_i allow violations, penalised by C:

```

minimise (1/2)||w||^2 + C SUM_i x_i, y_i(w.x_i + b) >= 1 - x_i, x_i >= 0

```

equivalently $\min (1/2)||w||^2 + C \sum_i \max(0, 1 - y_i f(x_i))$

That second form shows the SVM is just a linear model with **hinge loss** and L2 regularisation. Large C means little regularisation (fit the training data hard); small C means a wide, soft margin. C is the hyperparameter that matters most.

10.2 The dual problem and the kernel trick

WHY THE DUAL MATTERS

Introducing Lagrange multipliers α_i for the margin constraints and eliminating w and b gives the dual: maximise $\sum \alpha_i - (1/2) \sum_{ij} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$, subject to $0 \leq \alpha_i \leq C$ and $\sum \alpha_i y_i = 0$. The data appears ONLY through inner products $x_i \cdot x_j$. So if you replace that inner product with any function $K(x_i, x_j)$ that equals an inner product in some (possibly infinite-dimensional) feature space, you get a non-linear classifier at no extra representational cost. Also, α_i is non-zero only for support vectors - the solution is sparse in the training set.

FROM ZERO — the idea explained step by step

The **kernel trick** lets an SVM draw curved boundaries by comparing examples with a similarity function instead of raw features.

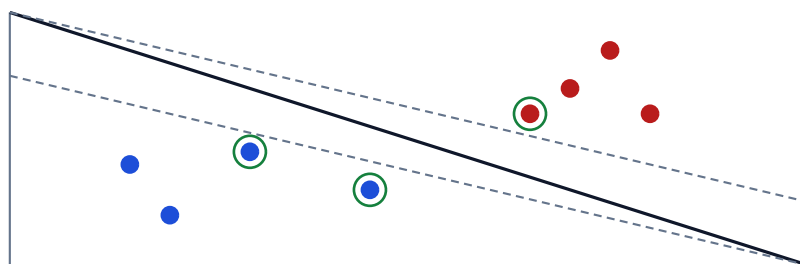
IN OUR PROJECT — not used, and why

RBF-kernel SVM on our features: 0.8392 (measured) -- 6 points better than linear.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

$$f(x) = \sum_i \alpha_i y_i K(x_i, x) + b$$

Kernel	$K(a, b)$	Character
Linear	$a \cdot b$	Text, very high-dimensional sparse data
Polynomial	$(\gamma a \cdot b + r)^p$	Explicit interactions up to degree p
RBF / Gaussian	$\exp(-\gamma \ a - b\ ^2)$	The default; infinite-dimensional feature space, local influence
Sigmoid	$\tanh(\gamma a \cdot b + r)$	Rarely used; not always a valid kernel
String / graph kernels	domain-specific	Structured inputs without vectorisation



Graph 10.1 — A support vector machine picks the line with the widest empty band between the two classes. Only the points touching the band (circled) decide where it goes; move any other point and nothing changes.

A valid kernel must produce a positive semi-definite Gram matrix (Mercer's condition). For the RBF kernel, γ sets the radius of influence of each support vector: large γ means very local, wiggly boundaries and overfitting; small γ approaches a linear model. Tune **C and gamma jointly on a log grid** - they interact strongly.

10.3 Practical guidance

- **Always scale features.** RBF distances are meaningless otherwise.

FROM ZERO — the idea explained step by step

Scale the features, tune two settings (C and the kernel width), and watch how many training examples the model keeps ("support vectors").

IN OUR PROJECT — not used, and why

An RBF-SVM keeps thousands of our windows as support vectors and compares every new window with all of them -- too big and too slow for the board. The CNN is both smaller (10,214 numbers) and more accurate.

- Training cost is between $O(n^2)$ and $O(n^3)$: SVMs are excellent up to roughly 10^4 - 10^5 samples and impractical beyond. For large n use `LinearSVC`, SGD with hinge loss, or random-feature approximations (Nystroem, Random Fourier Features) which approximate the kernel map explicitly and restore linear cost.
- **SVR** (support vector regression) uses an epsilon-insensitive tube: errors smaller than epsilon cost nothing, which yields sparse, robust regression.
- **One-class SVM** learns the support of a distribution and is a classical anomaly detector.
- SVMs output distances, not probabilities. `probability=True` fits Platt scaling internally, which requires an extra internal cross-validation - it is slow and often better done explicitly (Chapter 13).

HINGE LOSS VERSUS CROSS-ENTROPY

Hinge loss is exactly zero once a point is correctly classified with margin at least 1, so well-classified points stop contributing - hence sparsity in the support vectors. Cross-entropy never reaches zero and keeps pushing confident points further, which is why logistic models produce calibrated probabilities while SVMs produce good boundaries. Choose by what you need: a decision or a probability.

Exercises

1 On a 2-D toy dataset, plot the RBF-SVM decision boundary for γ in $\{0.01, 0.1, 1, 10\}$ and C in $\{0.1, 1, 100\}$. Identify overfitting visually. 2 Verify the dual sparsity claim: count support vectors as C decreases. 3 Approximate an RBF kernel with Nystroem plus `LinearSVC` and compare accuracy and training time against the exact SVC on 20,000 samples.

11. Ensembles: Bagging, Random Forests and Boosting

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Ensembles are not part of the deployed thesis model (one model must fit on the board), but the thesis uses the ensemble idea for measurement: every configuration is trained with 10 random seeds.

An ensemble combines many models into one predictor that is better than any member. There are two fundamentally different recipes, and the bias-variance decomposition of Chapter 3 tells you exactly what each one is for.

Famil y	Members are	Trained	Attacks	Examples
Baggi ng	Strong, low-bias, high-variance	In parallel, independently	Variance	Random forest, extra trees
Boosti ng	Weak, high-bias	Sequentially, each fixing the last	Bias (and variance, via shrinkage)	AdaBoost, GBM, XGBoost, LightGBM
Stacki ng	Diverse model types	Level-0 in parallel, level-1 on out-of-fold predictions	Both	Any blend + a meta-learner

11.1 Why averaging works

THE VARIANCE REDUCTION FORMULA

Average M models each with variance s^2 and pairwise correlation ρ . The variance of the average is $\rho s^2 + (1 - \rho) s^2 / M$. As M grows the second term vanishes, but the first does not: the floor is set by how CORRELATED the members are. This single formula explains the entire design of random forests - every mechanism in them exists to lower ρ .

FROM ZERO — the idea explained step by step

If several models make independent mistakes, their average is more reliable than any one of them, like asking several people to estimate a weight and taking the mean.

IN OUR PROJECT — partly used

The thesis does not average models on the board (memory). But it averages **results**: each configuration is trained with 10 seeds and reported as mean +/- standard deviation (e.g. 0.8953 +/- 0.0050) [35].

WORKED ON OUR PROJECT — An ensemble of seeds

1. The project trains the same CNN with 10 different seeds [35]. Averaging the 10 models' softmax outputs is a deep ensemble (Section 19.5).
2. Individual models vary with sd 0.005; an ensemble usually adds 0.5-1 point and reduces that variance.
3. On the board an ensemble multiplies memory and compute by 10 -- the reason the thesis keeps a single model.

11.2 Bagging and random forests

- **Bootstrap sample**: draw n samples with replacement; about 63.2% of the rows appear, the rest are **out-of-bag** and give a free validation estimate.

FROM ZERO — the idea explained step by step

Bagging: train many models on random re-samples of the data and average them. A **random forest** is bagging with decision trees that also see random subsets of the features [29].

IN OUR PROJECT — not used, and why

Not used. As a baseline on our 18 features: 0.8096 test accuracy (200 trees, measured).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- **Feature subsampling at each split** (`max_features = sqrt(d)` for classification, `d/3` for regression) is what separates a random forest from plain bagged trees; it decorrelates the trees strongly.
- **Extra trees** go further: thresholds are drawn at random rather than optimised. Higher bias, much lower variance and much faster.
- Trees are grown deep and unpruned - variance is handled by the average, not by pruning each member.
- More trees never hurt accuracy, only time; 300-1,000 is typical, and the curve flattens.

Random forests are the best 'no-thought' baseline in existence: they need no scaling, tolerate irrelevant features, rarely overfit catastrophically, and their default hyperparameters are usually within a few percent of tuned performance.

11.3 Boosting, from AdaBoost to gradient boosting

FROM ZERO — the idea explained step by step

Boosting builds models one after another, each new one concentrating on the examples the previous ones got wrong. The final answer is a weighted sum of all of them [30].

IN OUR PROJECT — not used, and why

Not used. Gradient boosting on our features: 0.794 (measured).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

AdaBoost, the original idea

Train a weak learner; increase the weights of the samples it got wrong; train the next learner on the reweighted data; repeat; combine with weights based on each learner's accuracy. AdaBoost was later shown to be gradient descent on the **exponential loss** in function space - which opened the door to the general framework.

Gradient boosting, stated properly

Build an additive model $F_M(x) = \sum_m \nu * h_m(x)$. At each stage, fit the next weak learner to the **negative gradient of the loss with respect to the current predictions** - the pseudo-residuals:

```
r_im = - [ dL(y_i, F(x_i)) / dF(x_i) ]_(F = F_(m-1))
h_m = argmin_h SUM_i ( r_im - h(x_i) )^2           (fit a tree to residuals)
F_m = F_(m-1) + nu * gamma_m * h_m              (nu = learning rate)
```

With squared loss the pseudo-residuals are the ordinary residuals, which is the intuition most people learn first: each new tree predicts what the current ensemble is still getting wrong. Because the recipe only needs a gradient, the same algorithm works for logistic loss, Poisson loss, quantile loss, ranking losses and custom business losses.

SHRINKAGE AND THE NUMBER OF TREES TRADE OFF

The learning rate ν (shrinkage) and the number of trees M are coupled: halving ν roughly doubles the M you need. Small ν (0.01-0.1) with many trees and early stopping on a validation set generalises better than large ν with few trees. This pair, plus tree depth, is 80% of gradient boosting tuning.

11.4 Modern implementations

Library	Distinctive ideas	Best for
XGBoost	Second-order (Newton) boosting with an explicit regularisation term, sparsity-aware split finding, weighted quantile sketch	Strong all-rounder, huge ecosystem
LightGBM	Histogram binning, leaf-wise growth, GOSS sampling, EFB feature bundling	Very large datasets; usually the fastest
CatBoost	Ordered boosting to remove target leakage, native categorical handling with ordered target statistics, oblivious trees	Many categorical features; least tuning needed
scikit-learn HistGBDT	LightGBM-style histograms in scikit-learn	Zero extra dependencies

FROM ZERO — the idea explained step by step

Modern boosting libraries (XGBoost, LightGBM) are fast and strong on tables of features.

IN OUR PROJECT — not used, and why

Not used; scikit-learn's histogram gradient boosting trains on our 6,617 x 18 table in seconds.

```
import lightgbm as lgb
params = dict(objective='binary', metric='auc',
              learning_rate=0.03,      # small + many rounds + early stop
              num_leaves=63,          # capacity; 2^depth is the cap
              min_data_in_leaf=50,     # main overfitting control
              feature_fraction=0.8,    # column subsampling per tree
              bagging_fraction=0.8,    # row subsampling
              lambda_l2=1.0, verbose=-1)
dtr = lgb.Dataset(X_tr, y_tr)
dva = lgb.Dataset(X_va, y_va, reference=dtr)
model = lgb.train(params, dtr, num_boost_round=5000,
                  valid_sets=[dva],
                  callbacks=[lgb.early_stopping(200),
                             lgb.log_evaluation(200)])
print('best iteration:', model.best_iteration)
```

Listing 11.1 - A gradient-boosting configuration that is hard to beat on tabular data.

11.5 Gradient boosting on six points, three rounds

Target values 10, 12, 14, 20, 22, 30. The first model is the mean, 18. Each round fits a depth-1 tree (a single split) to the current residuals, and adds it with a learning rate of 0.5.

FROM ZERO — the idea explained step by step

The book runs three boosting rounds by hand on six points.

IN OUR PROJECT — not used, and why

On our data boosting (0.794) and the random forest (0.8096) end close to each other (measured).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

WORKED ON OUR PROJECT — Random forest and gradient boosting as baselines

1. Random forest (200 trees) [29]: test accuracy 0.8096. Histogram gradient boosting [30]: 0.794.
2. Both beat a single tree (about 0.75) by averaging many trees -- exactly the variance reduction of Section 11.1.
3. The most important features of the forest: total_acc_y_mean (0.179), total_acc_x_mean (0.167), total_acc_x_std (0.119). Orientation (gravity components) and total-acceleration variability carry most of the signal.
4. The CNN (~0.90) still wins: it learns frequency and shape features that 18 summary statistics cannot express.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Round	Residuals fed to the tree	Tree's split and outputs	Ensemble prediction after the round	MSE
0	-	constant 18	18, 18, 18, 18, 18, 18	46.67
1	-8, -6, -4, +2, +4, +12	split after 3rd: -6 / +6	15, 15, 15, 21, 21, 21	19.67
2	-5, -3, -1, -1, +1, +9	split after 5th: -1.8 / +9	14.1, 14.1, 14.1, 20.1, 20.1, 25.5	7.52
3	-4.1, -2.1, -0.1, -0.1, +1.9, +4.5	split after 4th: -1.6 / +3.2	13.3, 13.3, 13.3, 19.3, 21.7, 27.1	3.68



Graph 11.1 — Bagging against boosting. Bagging trains many models side by side on resampled data and averages them -- it calms down a jumpy model. Boosting trains them one after another, each fixing the mistakes of the ones before -- it sharpens a weak model.

The error falls 46.67 -> 19.67 -> 7.52 -> 3.68. Each tree is individually useless - it is a single threshold - but each one attacks precisely what the ensemble still gets wrong, and the errors shrink geometrically. That is the whole of boosting; XGBoost and LightGBM differ from this table only in how they find the split, how they regularise the leaf values, and how fast they do it.

WHY THE LEARNING RATE IS 0.5 AND NOT 1.0

With $\text{nu} = 1.0$ the first tree would remove the residuals entirely on the training data and the ensemble would immediately be fitting noise. Shrinkage forces each tree to take only part of the credit, so the correction is spread over many trees and no single one dominates. This is the same principle as a small learning rate in gradient descent, and it is the main reason boosting generalises rather than merely memorising.

11.6 Bagging versus boosting, side by side

Question	Random forest (bagging)	Gradient boosting
What is each tree trained on?	A bootstrap resample, with a random subset of features per split	The residuals of everything built so far
How deep are the trees?	Deep, often unlimited	Shallow - depth 3-8, or 31-255 leaves

Question	Random forest (bagging)	Gradient boosting
Can it be parallelised?	Yes, trees are independent	No across trees (each needs the previous); yes within a tree
What happens with more trees?	Error flattens; more trees never hurt accuracy	Error keeps falling then RISES - you must early-stop
Main risk	Underfitting if trees are too shallow	Overfitting if the learning rate is high or you boost too long
Tuning effort	Very low - defaults are close to optimal	Moderate - learning rate, depth, and rounds interact
Typical winner on tabular data	Strong baseline	Usually the best, by a small but consistent margin

FROM ZERO — the idea explained step by step

Bagging mainly reduces variance (instability); boosting mainly reduces bias (being too simple).

IN OUR PROJECT — not used, and why

Both stay 5-10 points below the CNN on our data: the limit is the 18 hand-made features, not the ensemble.

11.7 Stacking and blending

Train diverse level-0 models (a GBDT, a linear model, a k-NN, a small neural net), collect their **out-of-fold** predictions as new features, and train a simple level-1 model - usually regularised logistic or linear regression - on those. The out-of-fold requirement is absolute: using in-fold predictions leaks and produces a meta-model that trusts an overfitted base model.

FROM ZERO — the idea explained step by step

Stacking trains a small model that combines the outputs of several different models.

IN OUR PROJECT — not used, and why

Not used in the thesis.

PRACTICAL ENSEMBLING THAT IS WORTH THE COMPLEXITY

In order of value per unit of effort: (1) average several seeds of the same model - almost free, reliably worth a few tenths of a percent; (2) average a GBDT with a neural network - their errors are genuinely different; (3) full stacking with a meta-learner. In production, weigh the gain against the latency and maintenance cost of running five models.

Exercises

1 Implement gradient boosting with depth-2 trees and squared loss in 60 lines, and verify that its predictions approach the target as M grows. 2 Show the effect of `max_features` on random forest accuracy and on the average correlation between tree predictions. 3 Take a tuned LightGBM model and add a small MLP to a simple average. Measure whether the blend beats both members, and check the correlation of their errors to explain why.

12. Unsupervised Learning and Dimensionality Reduction

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Unsupervised methods are not used to train the HAR classifier, but PCA is central to one project example: the continual-learning version of HAR stores old data as a compact PCA description.

12.1 Clustering

FROM ZERO — the idea explained step by step

Clustering groups examples without labels, so that examples in one group are similar. **k-means** places k centre points, assigns every example to its nearest centre, moves each centre to the middle of its examples, and repeats.

IN OUR PROJECT — partly used

Not used for training. On our data (measured): k-means with 2 groups separates moving and still windows with purity 0.9986; with 6 groups the purity with respect to the 6 activities is only 0.619. Coarse behaviour is visible without labels; fine activities are not.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

k-means

Partition n points into k clusters minimising within-cluster squared distance. Lloyd's algorithm alternates two steps until assignments stop changing:

```
Assign:  $c_i = \operatorname{argmin}_k ||x_i - \mu_k||^2$ 
Update:  $\mu_k = \text{mean of the points assigned to cluster } k$ 
Objective (inertia):  $J = \sum_i ||x_i - \mu_{c_i}||^2$ 
```

IN OUR PROJECT — k-means

$k = 2$ on our 18 features: purity 0.9986 for moving vs still (measured).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- Each step never increases J , so the algorithm converges - to a **local** optimum that depends on initialisation. Use **k-means++** seeding and several restarts.
- Assumes clusters are roughly spherical, similar in size and density; it fails on elongated or nested shapes.
- Choosing k : the **elbow** of the inertia curve, the **silhouette score** (the more reliable of the two), gap statistic, or a downstream metric if the clusters feed another system.
- Scale features first; k-means is Euclidean and unit-sensitive. MiniBatchKMeans scales to millions of points.

Gaussian mixture models and EM

A GMM is the soft, probabilistic generalisation: data is assumed drawn from a mixture of K Gaussians, each with its own mean, covariance and weight. **Expectation-Maximisation** alternates computing the posterior responsibility of each component for each point (E-step) with re-fitting the components weighted by those responsibilities (M-step). Each iteration provably does not decrease the likelihood.

```
E-step:  $\gamma_{ik} = \pi_k N(x_i | \mu_k, S_k) / \sum_j \pi_j N(x_i | \mu_j, S_j)$ 
M-step:  $\mu_k = \sum_i \gamma_{ik} x_i / \sum_i \gamma_{ik}$  (and similarly  $S_k, \pi_k$ )
```

GMMs give soft assignments, elliptical clusters, a proper likelihood (so BIC can select K), and a density model usable for anomaly detection. k-means is the limiting case with spherical, equal, vanishing-variance components.

Density and hierarchical methods

- **DBSCAN:** clusters are dense regions separated by sparse ones. Finds arbitrary shapes, labels outliers as noise, needs no k - but needs eps and min_samples, and struggles with varying density.
- **HDBSCAN:** builds a hierarchy over eps and extracts the most stable clusters. The best modern default for exploratory clustering.
- **Agglomerative:** merge the closest pair repeatedly; the dendrogram shows structure at every scale. Linkage choice (ward, average, complete) changes the results substantially.
- **Spectral clustering:** cluster the eigenvectors of a similarity graph Laplacian - excellent for manifold-shaped data, $O(n^3)$ without approximation.

CLUSTERING ALWAYS RETURNS CLUSTERS

Every algorithm partitions whatever you give it, including pure noise. Before believing a clustering: check stability across seeds and subsamples, check that silhouette is meaningfully above zero, and, most importantly, validate against something external - a downstream metric or a domain expert's reading of the clusters.

12.2 Dimensionality reduction

FROM ZERO — the idea explained step by step

Dimensionality reduction describes each example with fewer numbers while keeping most of the information. **PCA** (principal component analysis) finds the directions in which the data varies most; keeping only the first few directions keeps most of the variation.

Example: points lying almost on a line in 2-D can be described by 1 number (the position along the line).

IN OUR PROJECT — used

Used in the continual-learning example of the project. When the HAR model learns 5 groups of people one after another, it must not forget the earlier ones. Instead of storing old windows (4,608 bytes each), src/continual_learning/PpcaReplay.c keeps a rank-8 PCA description per class and generates similar windows from it for replay. With rank 8 the memory is 41,592 bytes -- about the same as storing just 9 raw windows (41,472 bytes) [35].

On our 18 features (measured): the first 1..5 components explain 0.501, 0.607, 0.699, 0.772, 0.823 of the variance.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

PCA, derived

PCA finds the orthogonal directions of maximum variance. Centre X, then either eigendecompose the covariance $C = X^T X / (n-1)$ or - better numerically - take the SVD of X directly.

```
x = U S V^T  =>  principal directions are the columns of V
explained variance ratio of component j = s_j^2 / SUM_k s_k^2
projection to k dims:  Z = X V_k           reconstruction: X_hat = Z V_k^T
```

IN OUR PROJECT — PCA, derived

The eigen-vectors that PCA needs are computed on the device by src/arithmetic/JacobiEig.c (Jacobi method) and updated incrementally (CCIPCA) in PpcaReplay.c as new data arrive [35].

TWO EQUIVALENT CHARACTERISATIONS

Maximising projected variance and minimising squared reconstruction error give the SAME subspace. That equivalence is why PCA is simultaneously a compression method and a de-noising method, and why a linear autoencoder with squared loss learns exactly the PCA subspace (Chapter 25).

- **Scale first** unless all features share units - otherwise the largest-variance unit dominates.
- Choose k by cumulative explained variance (90-99%) or by an elbow in the scree plot.
- PCA is linear and unsupervised: it may discard exactly the low-variance direction that carries your label. Check downstream, not just variance.
- **Whitening** rescales components to unit variance - useful before some downstream models, harmful when it amplifies noise directions.
- Variants: randomised/truncated SVD for large matrices, kernel PCA for non-linear structure, sparse PCA for interpretable loadings, incremental PCA for streaming.

Manifold learning: t-SNE and UMAP

Aspect	t-SNE	UMAP
Objective	Match pairwise neighbour probabilities with KL divergence	Match fuzzy topological structure with cross-entropy
Speed	Slow, $O(n \log n)$ with Barnes-Hut	Much faster; scales to millions
Global structure	Poorly preserved - inter-cluster distances are not meaningful	Better preserved, still not metric
Key knob	perplexity (5-50)	<code>n_neighbors</code> , <code>min_dist</code>
New points	No natural transform	Has a transform method

HOW TO MISREAD A T-SNE PLOT

Cluster sizes carry no meaning. Distances between clusters carry almost no meaning. Random seeds change the picture. Different perplexities can invent or dissolve clusters. Use these plots to generate hypotheses and to sanity-check embeddings - never as evidence for a claim about geometry, and never to choose k for clustering.

Other reductions worth knowing

- **LDA** (supervised, $K-1$ dimensions, maximises class separation).
- **NMF** for non-negative data: parts-based, interpretable topics.
- **Autoencoders** for non-linear compression, and **VAEs** for a probabilistic latent space (Chapter 25).
- **Random projection** : the Johnson-Lindenstrauss lemma guarantees that a random linear map into $O(\log n / \epsilon^2)$ dimensions preserves pairwise distances to within ϵ . Astonishingly cheap and useful for very high-dimensional sparse data.

12.3 k-means, iteration by iteration

Six one-dimensional points - 1, 2, 4, 7, 8, 10 - with $k = 2$ and the unlucky initial centroids 1 and 10:

FROM ZERO — the idea explained step by step

The book runs k-means iteration by iteration so each move of the centres can be followed.

IN OUR PROJECT — partly used

On our features the 2-cluster solution converges to "moving" and "still" (purity 0.9986, measured).

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

WORKED ON OUR PROJECT — k-means rediscovers moving versus still without labels

1. Run k-means with $k = 2$ on the 18 standardised features of the training windows, ignoring the labels.
2. Purity with respect to moving/still: 0.9986. The two clusters are almost exactly the two groups.
3. With $k = 6$ the purity with respect to the six activities is only 0.619: the fine distinctions are not visible as separate clusters in this feature space.
4. Unsupervised structure is strong for coarse behaviour and weak for fine classes -- a reason self-supervised pretraining (Chapter 27) needs a good representation, not just clustering.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Iteration	Assignment	New centroids	Inertia at the start of the step
1	{1, 2, 4} -> c1; {7, 8, 10} -> c2	c1 = 2.33, c2 = 8.33	23.00
2	{1, 2, 4} -> c1; {7, 8, 10} -> c2	c1 = 2.33, c2 = 8.33 (unchanged)	9.33
3	no change - converged	-	9.33

Two iterations, and the inertia drops from 23.0 to 9.33 - the value the algorithm reports. Now try initialising at 1 and 2 instead: the first assignment puts {1} in one cluster and {2, 4, 7, 8, 10} in the other, and the algorithm converges to a visibly worse partition with higher inertia. Same data, same k , different answer. That is why you run `n_init` restarts and keep the lowest inertia, and why k-means++ seeding (choose the next centroid far from the existing ones, with probability proportional to squared distance) is the default.

12.4 PCA on ten points, computed in full

Ten two-dimensional points with a strong positive correlation. Centre them, form the covariance matrix, and take its eigenvectors:

```
mean = (1.81, 1.91)
```

FROM ZERO — the idea explained step by step

PCA in full: centre the data, compute the covariance matrix, find its eigen-vectors (directions) and eigen-values (amount of variation along each), keep the largest ones.

IN OUR PROJECT — used

The same steps run in C in the project's continual-learning code (JacobiEig.c for the eigen-vectors) [35]. On our features the first component alone explains about half of the variance -- it is essentially "how much the person moves".

WORKED ON OUR PROJECT — PCA on the 18 features

1. Cumulative explained variance of the first 1..5 principal components: 0.501, 0.607, 0.699, 0.772, 0.823.
2. One component already carries half of the variance -- it is essentially "how much is the person moving".
3. Keeping 5 components keeps 82% of the variance with 5 instead of 18 numbers per window.
4. For the raw (9, 128) input the CNN learns its own reduction; PCA is mainly a diagnostic here [21, Ch. 14].

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

```
covariance = [ 0.6166  0.6154 ]
              [ 0.6154  0.7166 ]
```

```
eigenvalues = 1.284 and 0.049
PC1 = ( 0.678, 0.735) explains 1.284/1.333 = 96.3% of variance
PC2 = (-0.735, 0.678) explains 3.7%
```

PC1 points along the diagonal, which is where the data actually varies; PC2 is perpendicular to it, as it must be, and captures almost nothing. Projecting onto PC1 alone turns each 2-D point into one number - a 50% compression - and reconstructing from that single number recovers the original points with an RMSE of 0.15, small compared with the spread of the data. That is the entire method: rotate so the axes line up with the variance, then drop the axes that barely move.

PCA AS CHOOSING A CAMERA ANGLE

Imagine a flat, disc-shaped galaxy of points floating in 3-D. Photographed face-on you see its full structure; photographed edge-on it collapses to a line and you lose everything. PCA finds the face-on angle automatically, by maximising the spread of the shadow. The eigenvalues tell you how much structure each angle preserves, and the explained-variance ratio is just those numbers normalised to sum to one.

THE DIRECTION WITH THE MOST VARIANCE IS NOT ALWAYS THE ONE YOU WANT

PCA is unsupervised - it never looks at the labels. If your classes differ along a low-variance direction (a small but consistent offset) PCA may discard exactly that direction while faithfully preserving an irrelevant high-variance one such as overall brightness. Always check downstream accuracy after reducing, and consider LDA when you have labels and separation is the goal.

12.5 Anomaly detection

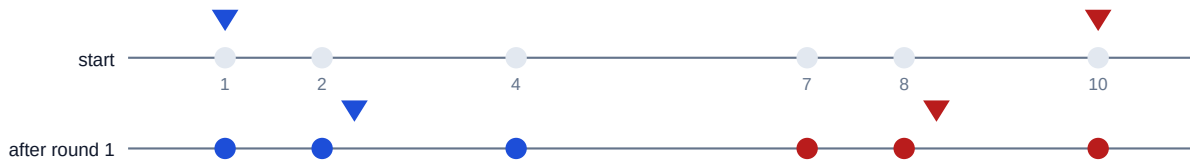
Method	Idea	Notes
Z-score / IQR	Univariate thresholds	Trivial baseline; ignores correlations
Mahalanobis distance	Distance under the covariance	Assumes one Gaussian blob
Isolation Forest	Random splits isolate outliers in fewer steps	Fast, few assumptions - a strong default
Local Outlier Factor	Compares local density to neighbours' density	Catches local anomalies a global method misses
One-class SVM	Learns a boundary around the data	Sensitive to nu and gamma
Autoencoder reconstruction error	Anomalies reconstruct badly	Good for images and signals; needs clean training data

FROM ZERO — the idea explained step by step

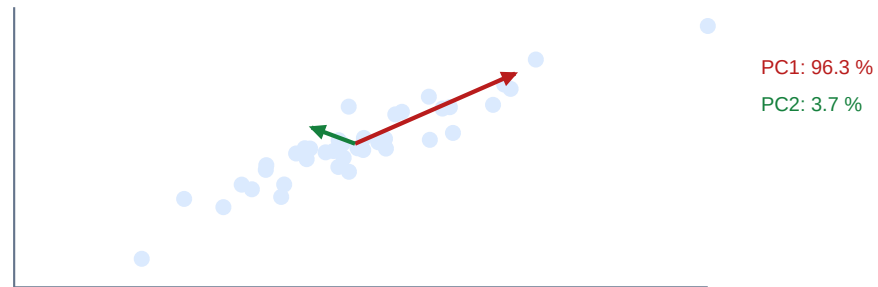
Anomaly detection learns what normal data looks like and flags examples that do not fit.

IN OUR PROJECT — partly used

Used in the ECG example of the project: an autoencoder is trained only on normal heartbeats; a beat it cannot reconstruct well (high MSE) is flagged as abnormal, with the threshold taken from the training normals [35]. In HAR terms, a window far outside the usual shaking levels (a fall) would be such an anomaly.



Graph 12.1 — *k*-means on the book's six points 1, 2, 4, 7, 8, 10 with $k = 2$, starting from centres 1 and 10. Round 1 splits them {1, 2, 4} and {7, 8, 10}; the centres move to 2.33 and 8.33; round 2 changes nothing, so it has converged.



Graph 12.2 — PCA finds the direction the data spreads along most. The arrows are the book's principal components $PC1 = (0.678, 0.735)$, 96.3 % of the variance, and $PC2 = (-0.735, 0.678)$, 3.7 %. The points are drawn from the book's covariance matrix (illustration).

Exercises

1 Implement *k*-means and *k*-means++ initialisation; compare final inertia over 50 random restarts of each. 2 Run PCA on a face or digit dataset and display the first 16 components as images; then reconstruct with $k = 5, 20, 100$ and compare. 3 Cluster the same dataset with *k*-means, GMM, DBSCAN and HDBSCAN and compare silhouette scores and the number of points labelled noise.

13. Evaluation: Metrics, Calibration and Imbalance

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

This chapter decides how the thesis reports its results: accuracy on unseen people, per-class errors, and the uncertainty of every number.

A model is only as good as the number you use to judge it. Choosing that number is a modelling decision at least as important as choosing the architecture, and it is where most projects quietly fail.

13.1 Classification: the confusion matrix and everything derived from it

		PREDICTED		
		positive	negative	
A C T U A L	positive	TP	FN	<- recall = $TP/(TP+FN)$
	negative	FP	TN	
		$\wedge$		
		precision = $TP/(TP+FP)$		<- specificity = $TN/(TN+FP)$

Figure 13.1 - Every classification metric is a ratio taken from this table.

FROM ZERO — the idea explained step by step

A **confusion matrix** is a table: rows are the true classes, columns the predicted classes, and each cell counts the examples. The diagonal holds the correct answers.

Precision of a class: of all examples predicted as this class, how many really are. **Recall**: of all examples that really are this class, how many were found. **F1** combines both.

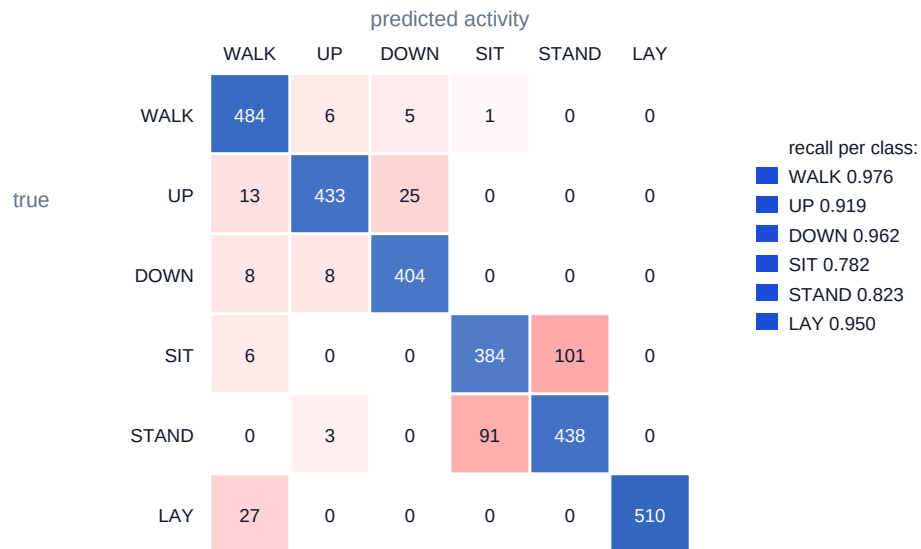
IN OUR PROJECT — used

Our CNN's confusion matrix on the 2,947 test windows of the 9 new people is drawn below. It gets 2653 windows right (accuracy 0.9002) and 294 wrong.

Precision per class: WALKING 0.900, UPSTAIRS 0.962, DOWNSTAIRS 0.931, SITTING 0.807, STANDING 0.813, LAYING 1.000. **Recall**: WALKING 0.976, UPSTAIRS 0.919, DOWNSTAIRS 0.962, SITTING 0.782, STANDING 0.823, LAYING 0.950.

How to read it. The biggest mistakes: 101 SITTING windows predicted as STANDING and 91 STANDING as SITTING -- from a phone at the waist the two postures look almost the same. The three walking activities are confused only a little (e.g. 25 UPSTAIRS as DOWNSTAIRS). A surprise: 27 LAYING windows are predicted as WALKING, so LAYING has precision 1.000 (every "LAYING" answer is right) but recall 0.950 (some LAYING windows are missed).

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.



Graph (from the C run) — The confusion matrix of **our CNN**, from the C trainer (train_c.c, seed 42, 20 epochs) on the 2,947 test windows of the 9 unseen people. Rows = true activity, columns = predicted. Darker = more windows. The diagonal holds the 2653 correct answers; the two big off-diagonal cells are SITTING/STANDING mix-ups.

Metric	Formula	Optimise it when
Accuracy	$(TP+TN)/n$	Classes are balanced and errors cost the same
Precision	$TP/(TP+FP)$	False alarms are expensive (spam filter, arrest, expensive follow-up)
Recall / sensitivity	$TP/(TP+FN)$	Misses are expensive (cancer screening, fraud, safety)
F1	$2PR/(P+R)$	You need a single number balancing the two
F-beta	$(1+b^2)PR/(b^2 P + R)$	You can state how much more recall matters than precision
Specificity	$TN/(TN+FP)$	The true-negative rate matters explicitly
Balanced accuracy	$(\text{recall} + \text{specificity})/2$	Imbalanced data, single number
MCC	correlation of predictions and truth	Imbalanced data; the most informative single scalar
Cohen kappa	agreement above chance	Comparing against annotator agreement

13.2 Threshold-free metrics: ROC and PR curves

- **ROC curve:** true positive rate against false positive rate as the threshold sweeps. **AUC-ROC** is the probability that a random positive scores above a random negative. It is invariant to class balance - which is a strength when comparing across datasets and a serious weakness when positives are rare, because a huge number of false positives barely moves FPR.

FROM ZERO — the idea explained step by step

Many models output a probability; a **threshold** turns it into a yes/no decision. ROC and PR curves show the quality for all thresholds at once, so you can compare models before choosing a threshold.

IN OUR PROJECT — used

Take our CNN's answers and merge the classes into two: "moving" (the three walking activities) and "still" (the other three). Then only 37 of 2,947 windows are wrong (36 still windows called moving, 1 moving called still): accuracy 0.9874. The hard part of HAR is not moving vs still, it is the fine distinctions inside each group. A ROC curve for "moving vs still" would therefore hug the top-left corner.

Source: the project's C trainer train_c.c (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in rebuild/c_runs.

- **Precision-recall curve:** precision against recall. **AUC-PR** (or average precision) is the right summary under heavy imbalance, because its baseline is the positive rate itself, not 0.5.
- **Rule:** with a positive rate below roughly 10%, report PR-AUC. Report ROC-AUC too if you like, but do not make decisions with it.

WHY AUC-ROC FLATTERS A RARE-EVENT MODEL

Take 1,000,000 negatives and 1,000 positives. A model that flags 20,000 negatives as positive has FPR = 2%, which looks excellent on an ROC curve. But if it also catches 800 positives, precision is $800/20,800 = 3.8\%$ - 96% of the alerts are wrong, and the operations team will abandon the system in a week. The PR curve shows this immediately; the ROC curve hides it.

13.3 Choosing the operating threshold

Training produces scores; the threshold is a **separate decision** that should be made with the costs of the application in hand. If a false negative costs C_{FN} and a false positive costs C_{FP} , the expected-cost-minimising threshold on a calibrated probability is:

$$t^* = C_{FP} / (C_{FP} + C_{FN})$$

FROM ZERO — the idea explained step by step

The threshold should be chosen from the costs of the two kinds of mistakes.

IN OUR PROJECT — partly used

On a device one could report an activity only if its softmax probability is above, say, 0.8, and otherwise say "unsure". This trades coverage for reliability; the thesis does not use such a threshold.

So if a miss is nine times more expensive than a false alarm, threshold at 0.1, not 0.5. Choose the threshold on validation data, never on test, and re-check it after any retraining, because score distributions drift.

13.4 Regression metrics

Metric	Formula	Character
MSE / RMSE	mean of squared errors (root)	Penalises large errors hard; RMSE is in the units of y
MAE	mean absolute error	Robust; the median-optimal predictor
MAPE	mean of $ err / y $ in percent	Interpretable, explodes near $y = 0$, asymmetric
sMAPE / WAPE	scaled variants	Fixes some MAPE pathologies
R^2	$1 - SS_{res}/SS_{tot}$	Relative to predicting the mean; can be negative
Pinball loss	asymmetric absolute	Evaluates quantile forecasts
MASE	error relative to a naive forecast	Time series: is the model beating last-value-carried-forward?

FROM ZERO — the idea explained step by step

Regression metrics (MAE, RMSE) measure errors in the units of the target.

IN OUR PROJECT — partly used

Used in the ECG autoencoder example: the reconstruction MSE per heartbeat is the anomaly score [35].

13.5 Probability calibration

A model is **calibrated** when among all samples it scores 0.7, about 70% are truly positive. Ranking quality (AUC) and calibration are independent: a model can rank perfectly and still be systematically overconfident. If the score feeds a threshold rule, an expected-cost calculation, or a human decision, calibration is not optional.

FROM ZERO — the idea explained step by step

A model is **calibrated** if, among all examples it predicts with 80% confidence, about 80% are right. ECE (expected calibration error) measures the average gap.

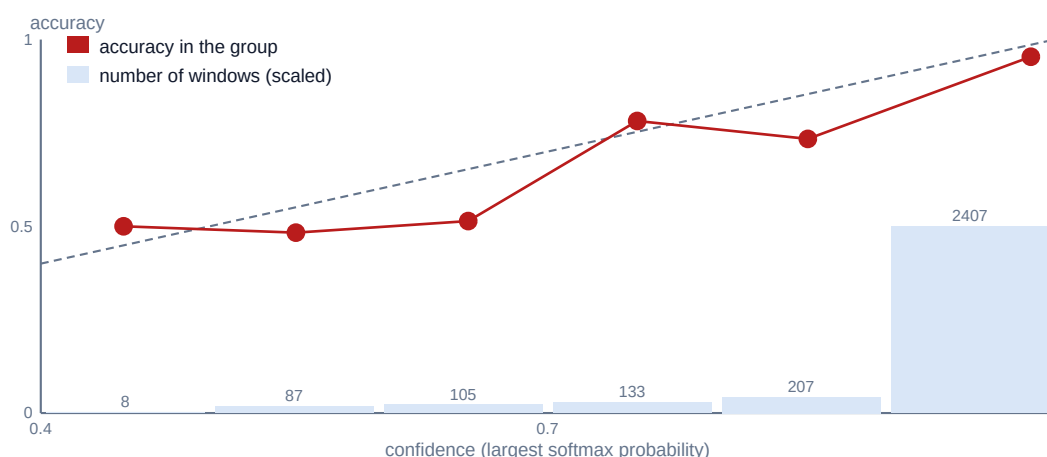
IN OUR PROJECT — used

For our CNN the largest softmax probability is its confidence. Measured on the test people: when the answer is right the average confidence is 0.956, when it is wrong 0.794. 81.7% of the windows get a confidence above 0.9, and 95.43% of those are right. Below 0.6 (3.2% of the windows) only 48.4% are right.

Expected calibration error (10 groups): **0.0424**. The model is slightly over-confident (in the graph the dots lie a little below the diagonal) -- typical for a network trained with cross-entropy [1].

Check: the average $-\log(\text{probability of the true class})$ computed from these saved probabilities is 0.3623, identical to the test loss the C program reports -- the probabilities are the C model's own.

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.



Graph (from the C run) — Reliability of our CNN's softmax (C run, test people). Windows are grouped by the confidence the model gave (bars = how many windows); the dots show how often it was actually right in each group. On the diagonal = well calibrated. Expected calibration error 0.042.

- **Diagnose** with a reliability diagram (predicted probability against observed frequency in bins) and summarise with **expected calibration error (ECE)** or Brier score.
- **Platt scaling**: fit a 1-D logistic regression on the scores. Few parameters, works well with little validation data, assumes a sigmoidal distortion.
- **Isotonic regression**: fit a monotone step function. More flexible, needs more data (about 1,000+ validation samples), can overfit.
- **Temperature scaling**: divide the logits by a single learned scalar T . The standard fix for modern neural networks, which are famously overconfident; it preserves accuracy exactly since it is monotone.
- Fit calibration on a **held-out** set, never on the training data.

```
import torch, torch.nn.functional as F
def fit_temperature(logits, labels, iters=200):
    # logits: (n, K) from a frozen trained model on a VALIDATION set
    logT = torch.zeros(1, requires_grad=True)
    opt = torch.optim.LBFGS([logT], lr=0.1, max_iter=iters)
    def closure():
        opt.zero_grad()
        loss = F.cross_entropy(logits / logT.exp(), labels)
        loss.backward()
        return loss
    opt.step(closure)
    return logT.exp().item()      # T > 1 softens an overconfident model
```

Listing 13.1 - Temperature scaling: one parameter, large calibration gains.

13.6 One confusion matrix, every metric computed

A fraud model is evaluated on 1,000 transactions, of which 100 are genuinely fraudulent. It flags 140, of which 80 are correct.

FROM ZERO — the idea explained step by step

All metrics can be computed from one confusion matrix.

IN OUR PROJECT — used

All metrics from our CNN's one confusion matrix: accuracy 0.9002; macro-F1 0.9015 (the average of the six per-class F1 scores 0.936, 0.940, 0.946, 0.794, 0.818, 0.974).

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.

		PREDICTED		total
		fraud	legit	
A	fraud	80	20	100
C				
T	legit	60	840	900
U				
A	total	140	860	1000

Figure 13.2 - The worked example used throughout this section.

Metric	Computation	Value	What it tells you
Accuracy	$(80+840)/1000$	0.920	Looks excellent - but always predicting 'legit' scores 0.900, so the model has bought you 2 points
Precision	$80/140$	0.571	43% of the alerts are wrong; this is the analyst's wasted time
Recall	$80/100$	0.800	One fraud in five still gets through
F1	$2(0.571)(0.8)/(0.571+0.8)$	0.667	The balance of the two
Specificity	$840/900$	0.933	Most legitimate customers are left alone
Balanced accuracy	$(0.800+0.933)/2$	0.867	Accuracy that is not fooled by the class ratio
MCC	correlation form	0.634	The single most informative scalar here
False positive rate	$60/900$	0.067	The x-axis of the ROC curve

NOW CONVERT THE METRICS INTO MONEY

Suppose an investigated alert costs 20 EUR of analyst time and a missed fraud costs 500 EUR. This model costs $140 \times 20 + 20 \times 500 = 12,800$ EUR. Lower the threshold until recall reaches 0.95: perhaps 300 alerts and 5 misses, costing $300 \times 20 + 5 \times 500 = 8,500$ EUR - materially better, with WORSE precision and WORSE accuracy. This is why Chapter 13 insists that the threshold is a business decision. Compute the cost curve, then pick the operating point; never accept 0.5 by default.

13.7 Reading a ROC curve and a PR curve of the same model

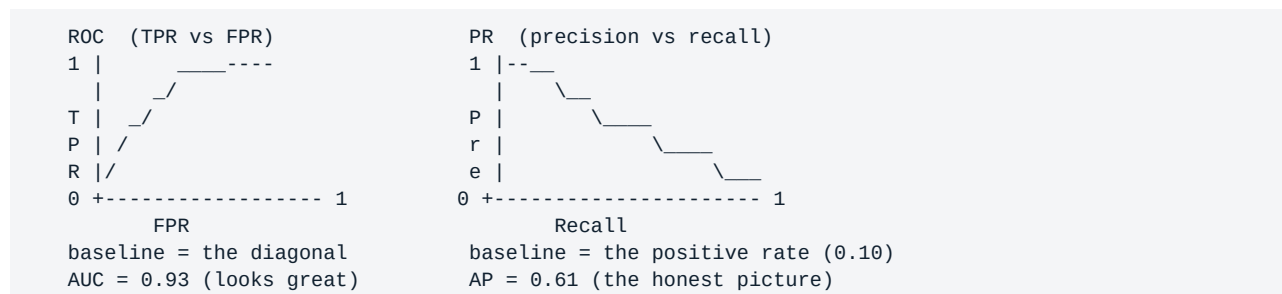


Figure 13.3 - The same predictions, two curves, two impressions.

FROM ZERO — the idea explained step by step

ROC is good when classes are balanced; PR curves are more informative when positives are rare.

IN OUR PROJECT — not used, and why

Our classes are balanced, so both tell the same story; the thesis does not use them.

The ROC curve's baseline is the diagonal regardless of class balance, so a rare-positive problem always looks flattering. The PR curve's baseline is the positive rate - here 0.10 - so an average precision of 0.61 is correctly read as 'six times better than guessing', not as 'nearly perfect'. When positives are rare, quote average precision and show the curve.

13.8 Statistical significance and reporting

- Report mean and standard deviation over at least 3-5 seeds. A single run is an anecdote.

FROM ZERO — the idea explained step by step

Every measured number is uncertain. Report the spread over repeated runs, and only call a difference real if it is clearly larger than that spread.

IN OUR PROJECT — used

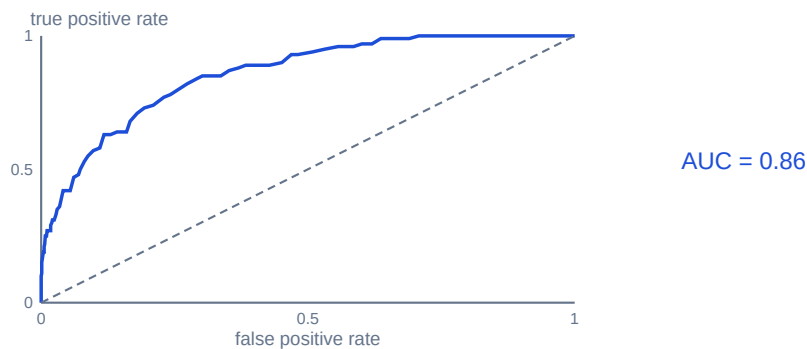
The thesis sweeps are reported this way (10 seeds): float 0.8953 +/- 0.0050 vs AdamW 0.8988 +/- 0.0038 -- the difference 0.0035 is smaller than the spread, so it is not a result [35].

WORKED ON OUR PROJECT — How wide is the parity tolerance in samples?

- The parity harness accepts $|\text{acc_C} - \text{acc_PyTorch}| \leq 0.025$ [35]. With 2947 test windows, $0.025 \times 2947 = 73.7$ windows.
- So the two implementations may disagree on up to 73 windows and still "match".
- Chapter 36 shows that 0.025 is also the Hoeffding 95% half-width for $n = 2947$ -- the tolerance equals the statistical uncertainty of one accuracy measurement.

	predicted fraud	predicted legit	
really fraud	TP 80	FN 20	accuracy 0.920 precision 0.571 recall 0.800 F1 0.667
really legit	FP 60	TN 840	

Graph 13.1 — The fraud example of 13.6 as a picture: 80 caught (TP), 20 missed (FN), 60 false alarms (FP), 840 correctly left alone (TN). Accuracy looks great (0.92) while one alert in two is wrong (precision 0.571).



Graph 13.2 — A ROC curve, computed from 1,000 made-up scores (100 positive). Each point is one threshold: how many positives are caught (up) against how many negatives are wrongly flagged (right). The diagonal is guessing; the area under the curve is the chance a random positive outscores a random negative.

- Use bootstrap confidence intervals on the test set: resample it with replacement 1,000 times and take the 2.5th and 97.5th percentiles of the metric.
- For paired model comparison on the same test set use McNemar's test (classification) or a paired bootstrap; for multiple datasets use the Wilcoxon signed-rank test.
- Correct for multiple comparisons when you test many models (Bonferroni is crude but honest).
- Always report the trivial baseline. 'We reached 94% accuracy' means nothing if the majority class is 93%.

GOODHART'S LAW IN MACHINE LEARNING

When a measure becomes a target, it ceases to be a good measure. A model tuned relentlessly against one offline metric will find the shortcuts that metric permits - background artefacts in X-rays, timestamp leakage in fraud data, annotation quirks in benchmarks. Defend with: multiple metrics, slice-based evaluation across subgroups, a hand-audited error sample every cycle, and an online test before you believe anything.

13.9 Slice-based evaluation

Aggregate metrics hide the failures that matter. Always evaluate per slice: by class, by subgroup (age, sex, device, region, language), by difficulty, by data source, and by time period. A useful discipline is to define the slices **before** training and to treat a large per-slice drop as a release blocker, exactly like a failing test.

FROM ZERO — the idea explained step by step

Report scores for sub-groups (classes, users, devices) -- averages hide where a model fails.

IN OUR PROJECT — used

Sliced by class, our CNN's recall ranges from 0.782 (SITTING) to 0.976 (WALKING). A single average of 0.9002 hides that more than one SITTING window in five is misread. Slicing by person would be the next step for the thesis, since on-device training targets exactly the people for whom the model is weakest.

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.

Exercises

1 Construct a dataset with a 1% positive rate and a model with ROC-AUC 0.95; compute its precision at the 0.5 threshold and explain the gap. 2 Draw a reliability diagram for a random forest and for a neural network on the same data; apply isotonic regression and temperature scaling respectively and re-draw. 3 Compute a bootstrap 95% confidence interval for the F1 of your best model and decide whether it is genuinely better than the runner-up.

PART III - Deep Learning Core

Neurons, backpropagation derived by hand, activations, optimisers, normalisation, regularisation, and a practical playbook for making training actually work.

14. From a Single Neuron to a Multilayer Network

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

From here on the book describes exactly the kind of model the thesis trains. Every term -- neuron, layer, parameter count -- has a direct counterpart in the 10,214-parameter HAR network.

14.1 The artificial neuron

$z = w \cdot x + b$ (a linear score - identical to Chapter 5)
 $a = \text{phi}(z)$ (a non-linearity)

FROM ZERO — the idea explained step by step

An artificial **neuron** takes several numbers, multiplies each by a weight, adds them up, adds a bias, and passes the result through a simple function (the **activation**, e.g. ReLU = keep positives, turn negatives into 0).

Example: inputs (1, 2), weights (0.5, -1), bias 0.3: $0.5 \cdot 2 + 0.3 = -1.2$; ReLU gives 0.

IN OUR PROJECT — used

Our network is built from such neurons. Each of the 6 output neurons has 64 weights and 1 bias. In the convolution layers every neuron looks only at a short piece of the window (Chapter 21) [35].

That is the entire unit. Its power comes from two things: stacking many of them in a layer, and stacking layers. Without the non-linearity phi, stacking is pointless - the composition of linear maps is a linear map, so a 50-layer linear network has exactly the expressive power of one linear layer. **The non-linearity is what makes depth mean anything.**

```

x1 --w1--\
x2 --w2---+--> [ sum ] --z--> [ phi ] --a-->
x3 --w3--/      ^
                |
                b
A LAYER of m such units, for a batch of n samples:
Z = X W^T + b      X:(n,d)  W:(m,d)  b:(m,)  Z:(n,m)
A = phi(Z)

```

Figure 14.1 - A neuron, and the matrix form that a GPU actually executes.

14.2 The multilayer perceptron

$a^{(0)} = x$
 $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$
 $a^{(l)} = \text{phi}(z^{(l)})$ for $l = 1 \dots L-1$
 $y_{\text{hat}} = \text{output_activation}(z^{(L)})$

FROM ZERO — the idea explained step by step

A **multilayer perceptron** (MLP) stacks layers of neurons: the outputs of one layer are the inputs of the next. Every neuron of a "dense" layer is connected to every input.

IN OUR PROJECT — used

The ODT library has dense layers (src/layer/Linear.c); the MNIST MLP example uses them for the whole model [35]. For HAR a dense first layer would be huge: 1,152 inputs x 64 neurons = 73,728 weights, seven times our whole CNN. That is why the HAR model uses convolutions and only one dense layer at the end.

Task	Output units	Output activation	Loss
Regression	1	none (identity)	MSE / Huber
Binary classification	1	sigmoid (or none + BCEWithLogits)	Binary cross-entropy
Multiclass (one label)	K	softmax (or none + CrossEntropyLoss)	Categorical cross-entropy
Multi-label	K	K independent sigmoids	Sum of binary cross-entropies
Count	1	exp / softplus	Poisson NLL
Quantiles	Q	none	Pinball loss per quantile

14.3 The universal approximation theorem, honestly stated

WHAT IT DOES AND DOES NOT PROMISE

A feedforward network with ONE hidden layer and a non-polynomial activation can approximate any continuous function on a compact set to arbitrary accuracy, given ENOUGH hidden units. What the theorem does NOT say: how many units (it can be exponential in the input dimension), that gradient descent will find those weights, or that the result will generalise. It establishes that depth is not required for expressiveness - and practice establishes that depth is required for EFFICIENCY.

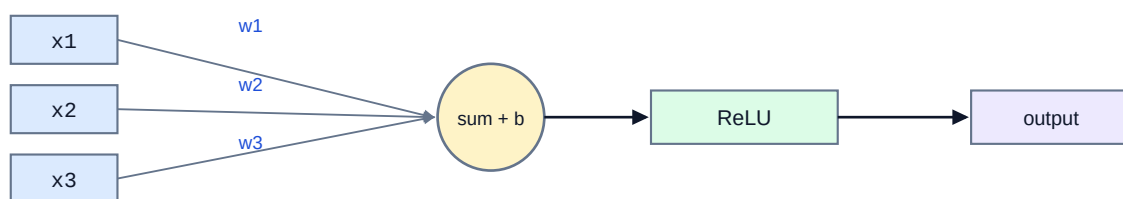
FROM ZERO — the idea explained step by step

The **universal approximation theorem** says a big enough network can imitate almost any function. It does not say that training will find it, or how big "enough" is.

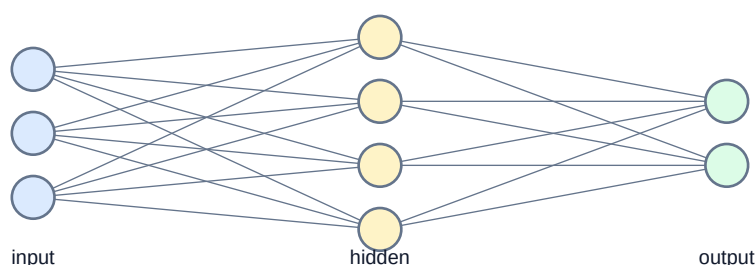
IN OUR PROJECT — used

Our network is small (10,214 parameters) but well matched to the data: short filters over time, pooling, one dense layer. Good structure matters more than size, especially on a 320 KB device.

Depth buys exponential efficiency for compositional functions. A function built from repeated composition - edges to shapes to parts to objects, characters to words to phrases to meaning - is represented by a deep network with a number of units linear in depth, and may need exponentially many units in a shallow one. Real perceptual data is compositional, which is why depth wins in practice.



Graph 14.1 — One artificial neuron: multiply each input by its weight, add them up with a bias, and pass the total through an activation such as ReLU. A network is thousands of these wired together.



Graph 14.2 — A small multilayer network: 3 inputs, 4 hidden neurons, 2 outputs. Every line is one weight: $3 \times 4 + 4 \times 2 = 20$ weights, plus $4 + 2 = 6$ biases.

14.4 Counting parameters and cost

Params of a dense layer: $m * d + m$ (weights + biases)
 MLP 784 -> 256 -> 128 -> 10:
 $784 * 256 + 256 = 200,960$
 $256 * 128 + 128 = 32,896$
 $128 * 10 + 10 = 1,290$ TOTAL = 235,146 parameters
 Memory in float32 = 235,146 * 4 B = 0.94 MB (weights only)

FROM ZERO — the idea explained step by step

Count the **parameters** (numbers to learn) and the **multiply-accumulates** (MACs, the work per example). A layer from n inputs to m outputs has $n \times m$ weights + m biases.

IN OUR PROJECT — used

Our network: conv1 1,024 + conv2 2,592 + conv3 6,208 + linear 390 = 10,214 parameters = 40,856 bytes as float32. Work per window: conv1 129,024 + conv2 163,840 + conv3 196,608 + linear 384 = 489,856 MACs [35].

WORKED ON OUR PROJECT — Counting the parameters of the project's network

- conv1: 9 -> 16 channels, kernel 7: $16 \times 9 \times 9 \times 7 = 1,008$ weights + 16 biases = 1,024.
- conv2: 16 -> 32, kernel 5: $2,560 + 32 = 2,592$. conv3: 32 -> 64, kernel 3: $6,144 + 64 = 6,208$. Linear: 64 -> 6: $384 + 6 = 390$.
- Total $1,024 + 2,592 + 6,208 + 390 = 10,214$ parameters = 40,856 bytes as float32 [35].
- Compared with a fully connected MLP on the flattened input (1152 inputs, one hidden layer of 64 units): $1152 \times 64 = 73,728$ weights for the first layer alone -- 7x more than the whole CNN. Weight sharing is why CNNs fit on microcontrollers [25].

Training memory is far larger than the weights: you also store activations for the backward pass (batch size times all layer outputs), gradients (one per parameter), and optimiser state (two more per parameter for Adam). A useful rule for Adam in float32: **about 16 bytes per parameter** before activations. This arithmetic is the entry point to Part V, where reducing it is the whole game.

14.5 Your first network, twice

```

import torch, torch.nn as nn
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(256, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(128, 10), # raw logits, no softmax here
)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
lossf = nn.CrossEntropyLoss(label_smoothing=0.05)
sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=3e-3,
                                             total_steps=epochs*len(train_dl))

for epoch in range(epochs):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(dev), yb.to(dev)
        opt.zero_grad(set_to_none=True)
        loss = lossf(model(xb), yb)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step(); sched.step()
    model.eval()
    with torch.no_grad():
        acc = sum((model(x.to(dev)).argmax(1).cpu() == y).sum().item()
                  for x, y in val_dl) / len(val_dl.dataset)
    print(epoch, round(acc, 4))

```

Listing 14.1 - A complete, modern training loop. Every ingredient - BatchNorm, dropout, AdamW, warmup schedule, gradient

FROM ZERO — the idea explained step by step

Implementing a network twice -- once by hand, once with a framework -- and comparing the numbers is the best way to find bugs.

IN OUR PROJECT — used

The whole project is built this way: PyTorch is the reference, the C library the implementation, and a parity check requires the same test accuracy within 2.5 percentage points and the same loss within 0.15 [35].

clipping, label smoothing - is explained in Chapters 16-20.

TRAIN() AND EVAL() ARE NOT DECORATION

Dropout must be off and BatchNorm must use running statistics at evaluation time. Forgetting `model.eval()` is the single most common PyTorch bug and produces mysteriously poor, noisy validation numbers. Equally, forgetting `torch.no_grad()` at evaluation wastes memory building a graph you never use.

Exercises

1 Prove that a network with identity activations and L layers computes a single linear map, and give the resulting weight matrix. 2 Count the parameters and estimate the Adam training memory of an MLP 1024-1024-1024-10. 3 Train the network above on MNIST or Fashion-MNIST; then remove all non-linearities and compare - the gap is the value of depth.

15. Backpropagation, Derived and Implemented

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Backpropagation is the heart of the thesis: training on the device means running these equations on the STM32, in C, with 8-bit numbers. Each section below points to the matching part of the ODT library.

Backpropagation is reverse-mode automatic differentiation applied to a neural network. It is not a learning algorithm - gradient descent is - but it is what makes gradient descent affordable: the cost of computing gradients for **all** parameters is about the same as one forward pass, regardless of how many parameters there are.

15.1 The four equations

Define the error signal at layer l as $\delta^l = dJ / dz^l$. Then:

- (1) $\delta^L = \text{grad}_a J * \phi'(z^L)$ [output layer]
- (2) $\delta^l = (W^{l+1})^T \delta^{l+1} * \phi'(z^l)$ [recursion]
- (3) $dJ/dW^l = \delta^l (a^{l-1})^T$
- (4) $dJ/db^l = \delta^l$

FROM ZERO — the idea explained step by step

Backpropagation computes the gradient of the loss for every weight, going backwards from the output. Four formulas are enough: (1) the error at the output, (2) how the error passes to the previous layer (multiply by the weights, then by the slope of the activation), (3) the gradient of each weight = its input x the error of its neuron, (4) the gradient of each bias = the error.

IN OUR PROJECT — used

For our last layer: the error is $p - \text{onehot}(y)$ (6 numbers); the weight gradients are error x input (a 6 x 64 table); the bias gradients are the error; the error passed back is $W^T \times \text{error}$ (64 numbers) [1, Ch. 6].

Every layer file of the ODT library (Conv1d.c, Linear.c, Relu.c, MaxPool1d.c, ...) has a backward function that applies these formulas [35].

FROM OUR C CODE — src/layer/Linear.c, lines 89-113: the backpropagation equations in our Linear layer

linearCalcWeightGradsFloat32: weight gradient = (error of the outputs)^T x (inputs). For our last layer: a 6 x 1 error times a 1 x 64 input = the 6 x 64 table of weight gradients (equation 3 of this section).

linearCalcBiasGradsFloat32: bias gradient = the error, summed over the examples of the batch (equation 4). The trainer uses one window at a time, so the sum has one term.

linearCalcPropLossFloat32 (next lines): the error passed back to the previous layer = error x W (equation 2).

```

void linearCalcWeightGradsFloat32(tensor_t *forwardInput, tensor_t *loss, tensor_t *weightGrads) {
    transposeTensor(loss, 0, 1);
    matmulFloat32Tensors(loss, forwardInput, weightGrads);
    transposeTensor(loss, 0, 1);
}

void linearCalcBiasGradsFloat32(tensor_t *loss, tensor_t *biasGrad) {
    /* Raw emit: per-feature batch sums of loss values; the executeOp epilogue adds. */
    size_t numFeatures = calcNumberOfElementsByTensor(biasGrad);
    size_t numLoss = calcNumberOfElementsByTensor(loss);
    size_t batch = (numFeatures == 0) ? 0 : numLoss / numFeatures;
    float *bg = (float *)biasGrad->data;
    float *l = (float *)loss->data;
    for (size_t f = 0; f < numFeatures; f++) {
        float sum = 0.0f;
        for (size_t n = 0; n < batch; n++) {
            sum += l[n * numFeatures + f];
        }
        bg[f] = sum;
    }
}

void linearCalcPropLossFloat32(tensor_t *loss, tensor_t *weights, tensor_t *propLoss) {
    matmulFloat32Tensors(loss, weights, propLoss);
}

```

WORKED ON OUR PROJECT — Backpropagation through the last layer, with shapes

1. Forward: h (64 values after pooling) $\rightarrow z = W h + b$ with W of shape (6, 64) $\rightarrow p = \text{softmax}(z) \rightarrow L = -\log p_y$.
2. Backward: $\text{delta} = dL/dz = p - \text{onehot}(y)$, shape (6). Then $dL/dW = \text{delta } h^T$ (outer product, shape (6, 64)), $dL/db = \text{delta}$, $dL/dh = W^T \text{delta}$ (shape 64) [1, Ch. 6].
3. Each weight's gradient is the product of an error (delta_i) and an input (h_j): $g = x * \text{delta}$. A weight whose input is always small gets small gradients.
4. With a batch of 64 the gradients of the 64 windows are averaged before the step -- the "mean" in the loss. A complete C implementation of these equations, like the project's, is walked through in [23].

(* denotes elementwise multiplication)

DERIVING EQUATION (2), THE ONLY STEP THAT MATTERS

$z^{(l+1)} = W^{(l+1)} a^{(l)} + b^{(l+1)}$ and $a^{(l)} = \text{phi}(z^{(l)})$. By the chain rule, $dJ/dz^{(l)}_j = \text{SUM}_k (dJ/dz^{(l+1)}_k)(dz^{(l+1)}_k/da^{(l)}_j)(da^{(l)}_j/dz^{(l)}_j) = \text{SUM}_k \text{delta}^{(l+1)}_k W^{(l+1)}_{kj} \text{phi}'(z^{(l)}_j)$. Collecting over j gives exactly $(W^{(l+1)})^T \text{delta}^{(l+1)} * \text{phi}'(z^{(l)})$. Notice what this says: the error is propagated backwards through the TRANSPOSE of the same weight matrix used in the forward pass, and it is gated by the local derivative of the activation.

```

FORWARD    x --> [W1] --> z1 --> phi --> a1 --> [W2] --> z2 --> loss
                                     |
BACKWARD    dw1 <-- d1 <-- *phi' <-- W2^T d2 <----- d2 <--+
Each layer needs, from the forward pass: its input a^(l-1) and z^(l).
That is why activations are stored - and why training memory scales
with batch size times depth.

```

Figure 15.1 - Forward stores activations; backward consumes them.

15.2 Why reverse mode, and what it costs

Mode	Cost	Efficient when
Forward-mode AD	One pass per INPUT variable	Few inputs, many outputs
Reverse-mode AD (backprop)	One pass per OUTPUT variable	Many inputs, one output - exactly the ML case: millions of parameters, one scalar loss

Mode	Cost	Efficient when
Numerical differences	Two evaluations per parameter	Never for training; only for checking a hand-written gradient

FROM ZERO — the idea explained step by step

Going backwards gives all gradients in one pass, which is why it is used. The price: every intermediate result of the forward pass must be kept in memory until the backward pass has used it.

IN OUR PROJECT — used

This price is the main memory problem of on-device training. On the board, the stored activations are part of the 209,680 bytes of training state [35]. Lin et al. reduce it by updating only some layers [5].

15.3 A complete NumPy implementation

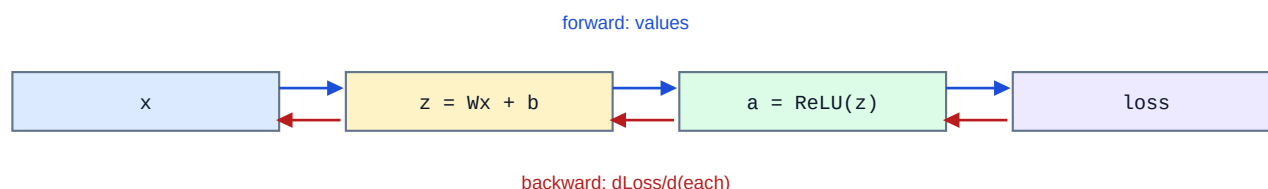
```
import numpy as np
def init(sizes, rng):
    # He initialisation, correct for ReLU (Chapter 16)
    return [ (rng.normal(0, np.sqrt(2.0/a), (b, a)), np.zeros(b))
             for a, b in zip(sizes[:-1], sizes[1:])] ]
def forward(params, X):
    cache = [X]
    A = X
    for i, (W, b) in enumerate(params):
        Z = A @ W.T + b
        A = Z if i == len(params) - 1 else np.maximum(0.0, Z) # ReLU
        cache.append((Z, A))
    return A, cache # A = logits at the last layer
def softmax_xent(logits, Y1h):
    Z = logits - logits.max(1, keepdims=True)
    P = np.exp(Z); P /= P.sum(1, keepdims=True)
    n = len(Y1h)
    loss = -np.sum(Y1h * np.log(P + 1e-12)) / n
    return loss, (P - Y1h) / n # dL/dlogits: the clean form
def backward(params, cache, dZ):
    grads = [None] * len(params)
    for l in reversed(range(len(params))):
        A_prev = cache[0] if l == 0 else cache[l][1]
        dW = dZ.T @ A_prev # eq. (3)
        db = dZ.sum(axis=0) # eq. (4)
        grads[l] = (dW, db)
        if l > 0:
            dA = dZ @ params[l][0] # W^T delta
            dZ = dA * (cache[l][0] > 0) # * phi'(z), ReLU derivative
    return grads
def sgd_step(params, grads, lr):
    return [ (W - lr*dW, b - lr*db)
             for (W, b), (dW, db) in zip(params, grads) ]
```

FROM ZERO — the idea explained step by step

A full NumPy implementation fits in a few dozen lines: forward, loss, backward, update.

IN OUR PROJECT — used

In the project the "few lines" are the PyTorch reference (train_pytorch.py) and the C training loop [35].



Graph 15.1 — Backpropagation. The forward pass (blue) goes left to right and computes the loss. The backward pass (red) goes right to left and, at each step, multiplies by the local slope -- the chain rule -- to get every weight's gradient in one sweep.

Listing 15.1 - Backpropagation for an MLP in 35 lines. Write this once from scratch and deep learning stops being magic.

15.4 Gradient checking

Before trusting a hand-written gradient, compare it against a central finite difference. Use double precision, a step of about $1e-5$, and the relative error criterion below; anything under $1e-7$ is right, above $1e-4$ is a bug.

```
numeric = ( J(theta + eps) - J(theta - eps) ) / ( 2 eps)
rel_err = |numeric - analytic| / max( |numeric| , |analytic| , 1e-8 )
```

FROM ZERO — the idea explained step by step

Gradient checking: change one weight by a tiny amount, measure the change of the loss, and compare with the computed gradient. They must agree.

IN OUR PROJECT — used

The project checks this for every layer, and a continuous-integration test (BIT_PARITY) loads the PyTorch weights into C and requires the same forward and backward results [35].

CHECK GRADIENTS WITH DROPOUT AND BATCH NORM DISABLED

Any source of randomness or batch-dependence makes $J(\theta + \epsilon)$ and $J(\theta - \epsilon)$ evaluate different functions, and the check fails for reasons that have nothing to do with your derivation. Fix the seed, turn off dropout, use eval-mode normalisation, and check on a small batch.

15.5 Backpropagation on real numbers, end to end

Everything in this chapter becomes concrete once you push one sample through a network by hand. Here is a 2-2-1 network with ReLU hidden units and a sigmoid output, one training sample, and every number written out. Verify each line with a calculator; it takes ten minutes and permanently removes the mystery.

FROM ZERO — the idea explained step by step

The book runs one forward and one backward pass with every number written out.

IN OUR PROJECT — used

With our real magnitudes: a weight whose input is 0.2 and whose neuron error is 0.05 gets gradient $0.2 \times 0.05 = 0.01$. With learning rate 0.01 its update is 0.0001 -- smaller than half an int8 step of conv3 (0.001575). Stored in 8 bits with ordinary rounding, this update would be lost (the dead zone, Chapter 28) [3, 11].

WORKED ON OUR PROJECT — When the gradient is too small to move an int8 weight

1. In int8 each weight lives on a grid with step s . For conv3 with range ± 0.4 , $s = 0.4 / 127 = 0.00315$ [35].
2. SGD changes a weight by $\eta \times g = 0.01 \times g$. With round-to-nearest, the change survives only if it is at least $s/2 = 0.001575$, i.e. $|g| \geq 0.1575$.
3. Typical gradients in late training are far smaller, so most updates round to zero: the **dead zone** [3, 11].
4. Stochastic rounding keeps these updates on average: a change of 0.3 of a step happens with probability 0.3 [11]. This is the central mechanism the thesis studies together with RigL.

```

x1=1.0 ---.
      \   W1 = [ 0.5  0.3 ]   b1 = [ 0.1 ]
      >-----[ 0.2  0.8 ]       [-0.1 ] -> h1, h2
x2=2.0 ---'
h1, h2 --> W2 = [1.0, -1.0], b2 = 0.5 --> z2 --> sigmoid --> p
true label y = 1      loss = binary cross-entropy
(note: W1 row 2 is [-0.2, 0.8] in the arithmetic below)

```

Figure 15.2 - The network used for the hand computation.

Forward pass

```

z1_1 = 0.5(1.0) + 0.3(2.0) + 0.1 = 1.2      a1_1 = ReLU(1.2) = 1.2
z1_2 = -0.2(1.0) + 0.8(2.0) - 0.1 = 1.3      a1_2 = ReLU(1.3) = 1.3

```

IN OUR PROJECT — Forward pass

The forward pass of our model: conv -> ReLU -> maxpool (x2), conv -> ReLU, average pool, linear, softmax.

```

z2 = 1.0(1.2) + (-1.0)(1.3) + 0.5 = 0.4
p = sigmoid(0.4) = 1/(1 + e^-0.4) = 0.5987
L = -log(0.5987) = 0.5130

```

Backward pass

Start at the output and walk backwards, applying the four equations from the previous section. Every quantity below is a number you can check.

```
delta2 = p - y = 0.5987 - 1 = -0.4013      <- the clean identity
```

```

dL/dW2 = delta2 * a1 = -0.4013 * [1.2, 1.3] = [-0.4816, -0.5217]
dL/db2 = delta2 = -0.4013

```

```

dL/da1 = W2^T * delta2 = [1.0, -1.0] * -0.4013 = [-0.4013, +0.4013]
delta1 = dL/da1 * ReLU'(z1) = [-0.4013, 0.4013] * [1, 1] = [-0.4013, 0.4013]

```

```

dL/dW1 = delta1 (x)^T = [ -0.4013*1.0   -0.4013*2.0 ]
                  [ +0.4013*1.0   +0.4013*2.0 ]
                  = [ -0.4013   -0.8026 ]
                  [ +0.4013   +0.8026 ]
dL/db1 = delta1 = [-0.4013, +0.4013]

```

The update, and proof that it worked

```

With eta = 0.1:
W1 <- [ 0.5401   0.3803 ]      W2 <- [1.0482, -0.9478]
      [-0.2401   0.7197 ]      b2 <- 0.5401

```

Re-running the forward pass with the new weights:

$z_2 = 1.0464$ $p = 0.7401$ $L = 0.3010$

The loss fell from 0.5130 to 0.3010 in one step, and the predicted probability moved from 0.60 towards the true label 1. That is learning, in its entirety.

CONFIRMING THE GRADIENT NUMERICALLY

Perturb $W1[0,0]$ by $\epsilon = 1e-6$ in each direction and recompute the loss: $(L(+\epsilon) - L(-\epsilon)) / (2 \epsilon) = -0.401312$, which matches the analytic -0.401312 to six decimals. Do this for all four entries of $W1$ and you have written your own gradient checker - the tool that distinguishes a derivation error from a training-hyperparameter problem.

READING THE NUMBERS AS A STORY

$\delta_2 = -0.4013$ says the output was too LOW by about 0.4 (in probability terms). The gradient on $W2$ is negative for both hidden units, so both connections strengthen - the network decides to listen more to h_1 and less negatively to h_2 . The gradient on $W1$ row 2 is positive, so those weights DECREASE, which lowers h_2 , which raises z_2 because $W2$'s second entry is negative. Every sign in the computation has a plain-language reason, and tracing them is the best debugging skill you can develop.

15.6 What ReLU's gate does to the gradient

In the example above both hidden pre-activations were positive, so both gates were open and gradient flowed to every weight. Change x to (1, -2) and z_{1_1} becomes $0.5 - 0.6 + 0.1 = 0.0$ while z_{1_2} becomes $-0.2 - 1.6 - 0.1 = -1.9$. The second unit's ReLU derivative is then 0, so:

$\delta_{a1} = dL/da_1 * [1, 0] = [-0.4013, 0]$
 $dL/dW1 \text{ row } 2 = [0, 0]$ <- this unit learns NOTHING from this sample

FROM ZERO — the idea explained step by step

ReLU passes the gradient only where its input was positive; where the input was negative, the gradient is 0 and the weights behind it do not learn from that example.

IN OUR PROJECT — used

Our network uses ReLU after each convolution (src/layer/Relu.c); its backward pass is exactly this gate [35].

FROM OUR C CODE — src/layer/Relu.c, lines 48-62: ReLU's gate, in our library

Where the forward input was 0 or negative, the gradient is set to 0; elsewhere it passes unchanged. That is the whole backward pass of ReLU -- one comparison per value.

Example. inputs (-0.3, 0.8, 0.0, 1.2), incoming gradients (0.5, -0.2, 0.7, 0.1): outgoing (0, -0.2, 0, 0.1).

```

void reluBackwardFloat(tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss) {
    size_t numberOfElements = calcNumberOfElementsByTensor(forwardInput);

    float *inputArray = (float *)forwardInput->data;
    float *gradOutArray = (float *)loss->data;
    float *gradInArray = (float *)propLoss->data;

    for (size_t i = 0; i < numberOfElements; i++) {
        if (inputArray[i] <= 0) {
            gradInArray[i] = 0;
        } else {
            gradInArray[i] = gradOutArray[i];
        }
    }
}

```

That is the mechanism behind three things you will meet repeatedly: ReLU networks are sparse (typically half the units are inactive on any given input, which is a form of free regularisation); a unit whose pre-activation is negative for **every** sample receives zero gradient forever and is dead; and gradient can vanish not only by shrinking but by being gated off entirely. Leaky ReLU exists precisely to leave the gate slightly ajar.

15.7 Vanishing and exploding gradients

Equation (2) is a repeated matrix product. Over L layers the error signal is multiplied by L factors of the form W^T and ϕ' . If the typical singular value of that product is below 1, the gradient decays exponentially with depth; above 1 and it explodes.

FROM ZERO — the idea explained step by step

In deep networks the gradient is a product of many factors. If they are mostly below 1 it shrinks to nothing (**vanishing**); if mostly above 1 it grows without limit (**exploding**).

IN OUR PROJECT — used

Our network has only 3 convolution layers, so neither problem appears. In 8-bit training a related problem does: small gradients can become exactly 0 after rounding (Chapter 28).

Problem	Symptom	Remedies
Vanishing	Early layers barely change; loss plateaus early; deep net does no better than a shallow one	ReLU-family activations, He/Glorot init, residual connections, normalisation layers, LSTM/GRU gates for sequences
Exploding	Loss becomes NaN or oscillates violently; gradient norms in the thousands	Gradient clipping (norm 1.0 is a good default), lower learning rate, normalisation, careful init, gradient accumulation instead of huge steps

RESIDUAL CONNECTIONS IN ONE LINE OF INTUITION

If a block computes $y = x + F(x)$, then $dy/dx = I + dF/dx$. The identity term guarantees that gradient flows to earlier layers even when dF/dx is tiny. That is why ResNets made 100+ layer networks trainable, and it is why essentially every modern architecture, Transformers included, is built out of residual blocks.

15.8 Automatic differentiation in practice

- Frameworks build a **computation graph** during the forward pass (dynamic in PyTorch, traced/compiled in JAX and TF), then walk it backwards applying each operation's vector-Jacobian product.

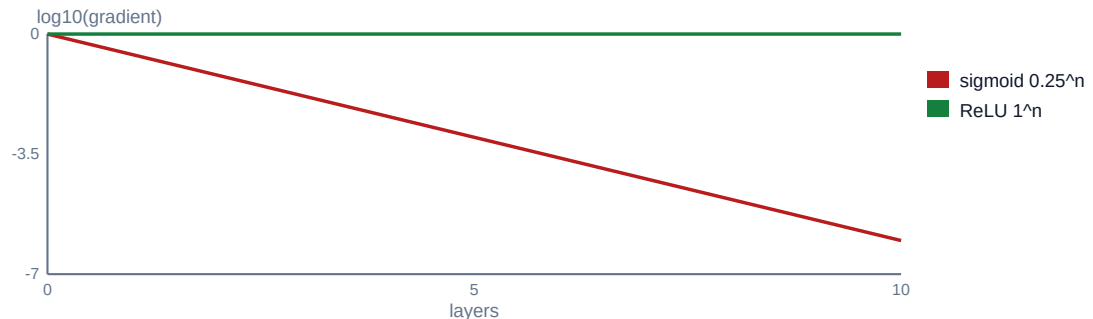
FROM ZERO — the idea explained step by step

Frameworks like PyTorch record every operation and compute the backward pass automatically ("autograd").

IN OUR PROJECT — partly used

The PyTorch reference uses autograd. The C library cannot: each layer's backward is written by hand and checked against PyTorch -- a large part of the thesis work [35].

- **Gradient accumulation:** call backward on several small batches before stepping, to simulate a large batch on limited memory. Remember to scale the loss by $1/\text{accumulation_steps}$.



Graph 15.2 — Vanishing gradients, computed. A sigmoid's slope is at most 0.25, so a gradient passed back through n sigmoid layers can shrink like 0.25^n : after 10 layers, below one millionth. ReLU's slope is 1 where it is active, so the gradient survives.

- **Gradient checkpointing:** discard activations during the forward pass and recompute them during the backward pass - trades roughly 30% extra compute for a large memory saving, and is what makes very deep or very long-context models fit.
- **detach() / stop_gradient** cuts the graph: essential for target networks in RL, teacher outputs in distillation, and any quantity you want treated as a constant.
- The **straight-through estimator** replaces a non-differentiable step (rounding, sign, top-k) with the identity in the backward pass. It is the trick that makes quantization-aware training possible (Chapter 28).

Exercises

1 Implement Listing 15.1, train it on MNIST to above 97% test accuracy, and verify every gradient against finite differences first. 2 Replace ReLU with sigmoid in a 10-layer network and plot the norm of dJ/dW per layer. Then add residual connections and re-plot. 3 Derive the backward pass of a batch-normalisation layer. It is the most instructive non-trivial derivation in deep learning.

16. Activations, Initialization and Loss Functions

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Activations, initialisation and loss functions are small choices with large effects. The project uses ReLU, a standard initialisation and cross-entropy, and the ODT library implements the alternatives too.

16.1 Activation functions

Name	Definition	Range	Notes
Sigmoid	$1/(1+e^{-z})$	(0,1)	Output layer for binary tasks only. Saturates; not zero-centred
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	(-1,1)	Zero-centred sigmoid; still saturates. Used inside LSTM gates
ReLU	$\max(0, z)$	[0,inf)	The default for 20 years of CNNs: cheap, no saturation for $z > 0$. Can die
Leaky ReLU	$\max(0.01z, z)$	R	Fixes dying ReLU with a small negative slope
PReLU	$\max(az, z)$, a learned	R	Leaky with a learned slope
ELU	z if $z > 0$ else $a(e^z - 1)$	(-a,inf)	Smooth, mean activations near zero, costlier
GELU	$z * \Phi(z)$	R	Smooth, probabilistic gating. Standard in Transformers
SiLU / Swish	$z * \text{sigmoid}(z)$	R	Smooth, often slightly better than ReLU in vision
Mish	$z * \tanh(\text{softplus}(z))$	R	Smooth alternative, more compute
Softplus	$\log(1+e^z)$	(0,inf)	Smooth ReLU; used to output positive parameters
GLU / SwiGLU	$(Wx) * \text{sigma}(Vx)$	R	Gated variants; SwiGLU is the standard feedforward in modern LLMs
Softmax	$e^{z_k} / \sum e^{z_j}$	simplex	Output layer for multiclass

FROM ZERO — the idea explained step by step

An **activation function** bends the straight weighted sum, so that the network can learn curved relationships. ReLU ($\max(0, x)$) is the most common: simple and fast. Sigmoid and tanh squash values into a range.

IN OUR PROJECT — used

Our network uses ReLU (`src/layer/Relu.c`). On a microcontroller this matters: ReLU is one comparison per value, also in 8-bit integers, while sigmoid/tanh need exponentials [35].

THE DYING RELU PROBLEM

If a unit's pre-activation is negative for every sample, its gradient is exactly zero forever and the unit is dead. This usually follows a learning rate that was too high early in training, and can silently kill 30-50% of a layer. Diagnose by counting units with zero activation over a validation batch; fix with a lower learning rate, He initialisation, Leaky ReLU or GELU, and normalisation layers.

WHAT TO ACTUALLY USE

Hidden layers of a convnet or MLP: ReLU, or GELU/SiLU if you want the last half percent. Transformers: GELU or SwiGLU. LSTM gates: keep sigmoid and tanh - the gates need bounded outputs. Output layer: chosen by the task, per the table in Chapter 14. Do not spend a week tuning activations; spend it on data.

16.2 Weight initialisation

Initialisation controls the scale of activations and gradients at step zero. Get it wrong and signals vanish or explode before learning begins. The principle is to keep the variance of activations roughly constant across layers.

```
Xavier/Glorot (tanh, sigmoid, linear):
    Var(W) = 2 / (fan_in + fan_out)
He/Kaiming (ReLU family) - accounts for half the outputs being zeroed:
    Var(W) = 2 / fan_in
LeCun (SELU):    Var(W) = 1 / fan_in
Orthogonal:      W = an orthogonal matrix, good for RNNs and deep stacks
```

FROM ZERO — the idea explained step by step

Before training, the weights are set to small random numbers. Their size must be right: too large and signals explode layer by layer; too small and they fade away. **Kaiming** (He) and **Xavier** (Glorot) rules choose the size from the number of inputs ("fan-in") of the layer [14, 24].

IN OUR PROJECT — used

Both rules are implemented in the ODT library (src/arithmetic/Distributions.c: kaimingNormal, kaimingUniform, xavierNormal, xavierUniform) [35]. PyTorch's default for our layers draws uniformly within +/- 1/sqrt(fan-in): conv3 has fan-in 32 x 3 = 96, so +/-0.102; conv1 +/-0.126; the linear layer +/-0.125.

For exact parity the C code can instead load the PyTorch starting weights (BIT_PARITY mode).

FROM OUR C CODE — src/arithmetic/Distributions.c, lines 60-78: Kaiming and Xavier initialisation in our library

kaimingUniform: draw uniformly between -limit and +limit with limit = gain x sqrt(3 / fan). **xavier**: the fan is the average of inputs and outputs. The size shrinks as the fan grows, so each neuron's sum stays about the same size whatever the number of inputs.

Example (conv3). fan-in = 32 channels x 3 taps = 96. With gain 1: limit = sqrt(3/96) = 0.177; with the ReLU gain sqrt(2): 0.250.

```
float kaimingNormal(float gain, size_t fanMode) {
    float std = gain / sqrtf((float)fanMode);
    return randomNormal(0.0f, std);
}

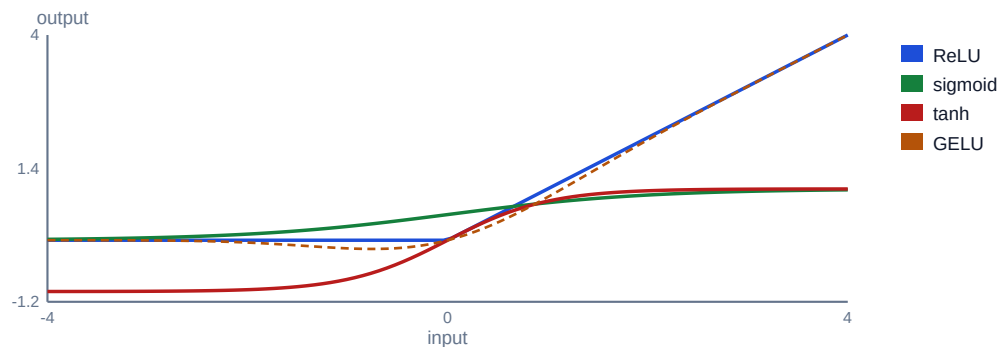
float kaimingUniform(float gain, size_t fanMode) {
    float limit = gain * sqrtf(3.0f / (float)fanMode);
    return randomUniform(-limit, limit);
}

float xavierNormal(float gain, size_t fanIn, size_t fanOut) {
    float std = gain * sqrtf(2.0f / (float)(fanIn + fanOut));
    return randomNormal(0.0f, std);
}

float xavierUniform(float gain, size_t fanIn, size_t fanOut) {
    float limit = gain * sqrtf(6.0f / (float)(fanIn + fanOut));
    return randomUniform(-limit, limit);
}
```

WORKED ON OUR PROJECT — What the project's initialisation produces

1. PyTorch initialises Conv1d/Linear weights uniformly in $\pm 1/\sqrt{\text{fan_in}}$ (Kaiming-uniform with $a = \sqrt{5}$) [14, 35].
2. conv3 has $\text{fan_in} = 32 \times 3 = 96$, so the bound is $1/\sqrt{96} = 0.102$. conv1: $\text{fan_in} = 63 \rightarrow 0.126$. Linear: $\text{fan_in} = 64 \rightarrow 0.125$.
3. He initialisation for ReLU would use $\text{std} \sqrt{2/96} = 0.144$ for conv3 [14]; Glorot initialisation, designed for tanh, would use $\sqrt{2/(96 + 192)} = 0.083$ [24]; the PyTorch default lies in between.
4. The C code must draw from the same distribution with the same RNG order for bit-level parity; the BIT_PARITY mode instead copies the PyTorch weights exactly [35].



Graph 16.1 — The common activation functions, plotted. ReLU: 0 for negatives, the input for positives. Sigmoid: squeezes into 0..1. Tanh: into -1..1. GELU: a smooth ReLU used in Transformers.

- **Never initialise all weights to zero** - every unit in a layer would compute the same thing and receive the same gradient forever. Symmetry must be broken randomly.
- **Biases** start at zero (a small positive value such as 0.01 was once recommended for ReLU; it makes little difference in practice).
- **Residual branches** are often initialised so the block starts as an identity (zero-init the last layer of each block, or use LayerScale). This lets very deep networks train stably from step one.
- Modern transformer stacks scale initialisation by $1/\sqrt{2L}$ on residual projections to keep the variance of the residual stream bounded with depth.

16.3 Loss functions, and what each one assumes

Loss	Task	Assumption / behaviour
MSE	Regression	Gaussian noise; punishes outliers heavily
MAE	Regression	Laplace noise; estimates the conditional median
Huber / smooth L1	Regression, detection	Quadratic near zero, linear far away - robust and smooth
Cross-entropy	Classification	Correct probabilistic loss; pairs with softmax/sigmoid
Focal loss	Detection, heavy imbalance	Down-weights easy examples by $(1-p)^\gamma$
Label-smoothed CE	Classification	Targets $1-\epsilon$ instead of 1; reduces overconfidence and improves calibration
KL divergence	Distillation, VAE	Matches a full distribution rather than a label
Contrastive / InfoNCE	Self-supervision, retrieval	Pull positives together, push negatives apart
Triplet loss	Metric learning	Anchor closer to positive than to negative by a margin
Dice / IoU loss	Segmentation	Directly optimises overlap; robust to class imbalance in masks
CTC	Speech, handwriting	Aligns unsegmented sequences
Pinball	Quantile regression	Asymmetric; yields prediction intervals

FROM ZERO — the idea explained step by step

The loss must fit the kind of answer: **cross-entropy** for classes, **mean squared error** for numbers or reconstructions.

IN OUR PROJECT — used

Both losses are in the library: CrossEntropy.c (HAR, KWS, MNIST) and MSE.c (ECG autoencoder) [35].

For HAR, a model that guesses gives each class 1/6 and a loss of $\log 6 = 1.792$. The 8-bit trainer checks this before training: its measured starting validation loss is 1.7972, and it refuses to train if the value is more than 0.25 away from $\log 6$. In the float run, the average training loss of the first epoch is 1.7605 and of the last 0.1149; on the new people the final test loss is 0.3623 (graph below).

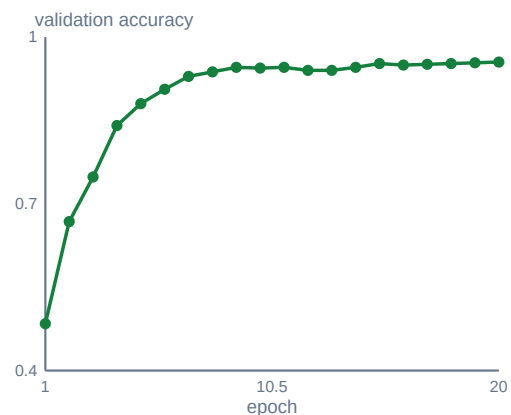
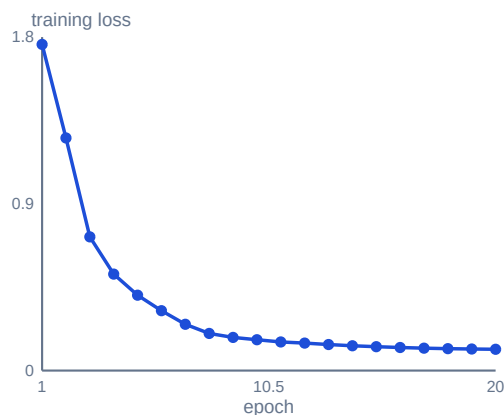
Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.

FROM OUR C CODE — `src/loss_functions/MSE.c`, lines 76-85: mean squared error, used by the ECG autoencoder

The gradient of $(\text{output} - \text{label})^2$ is $2 \times (\text{output} - \text{label})$ -- exactly the line in the loop. If the autoencoder outputs 0.9 where the heartbeat was 1.0, the gradient is -0.2: raise the output.

```
void mseLossBackwardFloat(tensor_t *modelOutput, tensor_t *label, tensor_t *result) {
    size_t numberOfElements = calcNumberOfElementsByTensor(modelOutput);
    float *modelOutputArray = (float *)modelOutput->data;
    float *labelArray = (float *)label->data;
    float *resultArray = (float *)result->data;

    for (size_t i = 0; i < numberOfElements; i++) {
        resultArray[i] = 2.0f * (modelOutputArray[i] - labelArray[i]);
    }
}
```



Graph (from the C run) — Training our CNN in C (`train_c.c`, float, SGD lr 0.01 momentum 0.9, seed 42): average training loss per epoch (blue) starts at 1.761, close to $\log 6 = 1.792$ of a model that guesses, and ends at 0.115; validation accuracy (green) rises from 0.48 to 0.96. Test on new people: loss 0.3623, accuracy 0.9002.

WORKED ON OUR PROJECT — Why cross-entropy for six classes

1. The label is a category, so the model outputs a probability distribution and the loss is the negative log probability of the correct class [1, Ch. 6].
2. Uniform guessing costs $\log 6 = 1.792$ nats; the trained CNN reaches test loss about 0.33 (float, 10 seeds) [35].
3. Mean squared error on one-hot targets would give much smaller gradients when the model is confidently wrong, and train more slowly.

Focal loss: $\text{FL}(p_t) = -\alpha (1 - p_t)^\gamma \log(p_t)$, $\gamma \sim 2$
Label smoothing: $y_{\text{smooth}} = (1 - \epsilon) y_{\text{onehot}} + \epsilon / K$, $\epsilon \sim 0.1$

MULTI-TASK LOSS WEIGHTING

When you sum several losses, their scales decide the effective learning rate of each task. Options: normalise each loss by a running estimate of its magnitude; use uncertainty weighting, which learns a per-task log-variance s_i and optimises $\text{SUM} (L_i / (2 \exp(s_i)) + s_i / 2)$; or use gradient-based balancing such as GradNorm. Fixed hand-tuned weights work but become a maintenance burden as tasks are added.

Exercises

1 Plot ReLU, GELU, SiLU and their derivatives on $[-5, 5]$ and explain the smoothness argument in terms of the gradient at zero. 2 Initialise a 20-layer ReLU MLP with $\text{Var}(W)=1/\text{fan_in}$, $2/\text{fan_in}$ and $4/\text{fan_in}$, and plot the standard deviation of activations per layer at initialisation. 3 Train a classifier with and without label smoothing $\text{eps}=0.1$ and compare accuracy, expected calibration error, and the histogram of maximum softmax probabilities.

17. Optimization Algorithms and Learning-Rate Schedules

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The optimiser is what changes the weights. The project uses SGD with momentum and AdamW and a cosine schedule -- and in 8-bit training the size of each update decides whether a weight can move at all.

Everything in this chapter is a variation on one line: `theta <- theta - eta * (something derived from the gradient)`. The variations differ in how they estimate direction and how they set an effective per-parameter step size.

17.1 The algorithms, in the order they were invented

FROM ZERO — the idea explained step by step

SGD: move each weight against its gradient times the learning rate. **Momentum**: keep a running "velocity" that adds up past gradients, so the steps get smoother and faster in a steady direction. **Adam**: give each weight its own step size, based on how large and how noisy its gradients have been. **AdamW**: Adam with weight decay applied separately [12, 13].

IN OUR PROJECT — used

The project implements SGD with momentum (`src/optimizer/Sgd.c`) and AdamW (`src/optimizer/AdamW.c`). Measured over 10 seeds: SGD (eta 0.01, momentum 0.9) 0.8953 +/- 0.0050; AdamW (eta 0.001, weight decay 0.01) 0.8988 +/- 0.0038 -- the same within noise [35].

FROM OUR C CODE — `src/optimizer/Sgd.c`, lines 67-96: momentum, as the HAR trainer runs it

Two steps. First the velocity: `state = momentum x state + g` (momentum 0.9). Then the weight moves by the velocity: `param = param - lr x state` (`sgdMParamKernel`, just below in the file).

Example. Constant gradient $g = 1$, $lr = 0.01$: velocity 1, 1.9, 2.71, 3.44, ... approaching 10. The step grows from 0.01 to almost 0.1.

```

static void sgdMStateKernel(tensor_t **op, size_t n, tensor_t *rawOut, tensor_t *aux,
                           const void *ctxv) {
    (void)n;
    (void)aux;
    const sgdMStateCtx_t *ctx = ctxv;
    size_t numberOfValues = calcNumberOfElementsByTensor(rawOut);
    float *state = (float *)op[0]->data;
    float *grad = (float *)op[1]->data;
    float *param = (float *)op[2]->data;
    float *out = (float *)rawOut->data;
    for (size_t i = 0; i < numberOfValues; i++) {
        float g = grad[i] + ctx->weightDecay * param[i];
        out[i] = ctx->momentum * state[i] + g;
    }
}

/* operands {param, state} -> rawOut = param - lr*state */
static void sgdMParamKernel(tensor_t **op, size_t n, tensor_t *rawOut, tensor_t *aux,
                             const void *ctxv) {
    (void)n;
    (void)aux;
    const sgdMParamCtx_t *ctx = ctxv;
    size_t numberOfValues = calcNumberOfElementsByTensor(rawOut);
    float *param = (float *)op[0]->data;
    float *state = (float *)op[1]->data;
    float *out = (float *)rawOut->data;
    for (size_t i = 0; i < numberOfValues; i++) {
        out[i] = param[i] - ctx->lr * state[i];
    }
}

```

WORKED ON OUR PROJECT — SGD versus AdamW, measured

1. AdamW keeps two moments m and v per parameter: 81,712 bytes versus 40,856 for SGD momentum, measured in the project [12, 13, 35].
2. 10-seed results: SGD 0.8953 +/- 0.0050, AdamW 0.8988 +/- 0.0038; test loss 0.33 vs 0.42 [35].
3. Accuracy is equal within noise; AdamW doubles the optimiser memory. On a 320 KB device that trade favours SGD.

SGD

```
theta <- theta - eta * g,          g = gradient on the current mini-batch
```

IN OUR PROJECT — SGD

Our settings: $\eta = 0.01$, batch 64, 2,060 steps (src/optimizer/Sgd.c) [35].

Momentum and Nesterov

```

v <- beta v + g          theta <- theta - eta v          (momentum)
Nesterov: evaluate the gradient at the look-ahead point theta - eta beta v

```

IN OUR PROJECT — Momentum and Nesterov

The library's SGD has momentum but no Nesterov variant. Momentum 0.9 in our project. For a steady gradient g the velocity approaches $g / (1 - 0.9) = 10 g$, so steps become 10 times larger. Small gradients add up over steps -- one way a weight can escape the 8-bit dead zone.

Momentum accumulates a velocity in directions of consistent gradient and cancels oscillation across a narrow valley. With $\beta = 0.9$ the effective step is about $1/(1-\beta) = 10$ times larger along a consistent direction. It is not a nicety: SGD with momentum, well tuned, still produces the best final accuracy on many vision benchmarks.

Adaptive methods

```
AdaGrad:  s <- s + g^2,          theta <- theta - eta g / (sqrt(s) + e)
RMSProp:  s <- rho s + (1-rho) g^2,  theta <- theta - eta g/(sqrt(s)+e)
```

IN OUR PROJECT — Adaptive methods

AdamW is in the library. It keeps two extra numbers per weight: 81,712 bytes of optimiser state instead of 40,856 for SGD momentum (measured) [35].

```
Adam:  m <- b1 m + (1-b1) g          (first moment)
       v <- b2 v + (1-b2) g^2        (second moment)
       m_hat = m/(1-b1^t), v_hat = v/(1-b2^t)    (bias correction)
       theta <- theta - eta * m_hat / ( sqrt(v_hat) + eps )
```

AdaGrad's accumulated sum only grows, so the effective learning rate decays monotonically to zero - fine for convex sparse problems, fatal for long deep-learning runs. RMSProp replaces the sum with an exponential moving average, fixing that. Adam adds momentum and bias correction, and it is the default for a good reason: it works without tuning on an enormous range of problems.

ADAMW: DECOUPLED WEIGHT DECAY

Adding $\lambda \|w\|^2$ to the loss is NOT the same as decaying weights when the optimiser rescales the gradient per parameter - the penalty gets divided by $\sqrt{v}$ too, so parameters with large gradients are barely regularised. AdamW decouples it: $\theta \leftarrow \theta - \eta (m_hat / (\sqrt{v_hat} + \epsilon) + \lambda \theta)$. This one change reliably improves generalisation, and AdamW is now the default for Transformers and essentially all large-model training.

Optimiser	Typical LR	When to choose it
SGD + momentum 0.9	0.1 with cosine decay (batch 256)	CNNs on vision benchmarks; best final accuracy with a good schedule
Adam / AdamW	1e-3 (small nets), 1e-4 to 3e-4 (Transformers)	Default for NLP, Transformers, GANs, RL, anything sparse or ill-conditioned
RMSProp	1e-3	RNNs, some RL algorithms
LAMB / LARS	layer-wise scaled	Very large batch training (32k+) where plain scaling diverges
Lion	3-10x smaller than Adam	Memory-lean alternative: sign-based update, one state tensor instead of two
Shampoo / K-FAC / Sophia	problem-specific	Second-order-ish methods; strong on large-scale pretraining, more complex
L-BFGS	line search	Small full-batch deterministic problems only

17.2 Learning-rate schedules

The learning rate should usually be large early (fast progress, escaping poor regions) and small late (fine convergence). The schedule matters at least as much as the optimiser.

FROM ZERO — the idea explained step by step

A **learning-rate schedule** changes the step size during training, usually large at the start and small at the end. **Cosine** decay lowers it smoothly along half a cosine curve.

IN OUR PROJECT — used

`src/optimizer/LrScheduler.c` implements step, exponential and cosine-annealing schedules [35]. Measured (10 seeds): at 8 bits cosine and constant give the same accuracy (0.8996 vs 0.9026); at 4 bits cosine is higher (0.8928 vs 0.8761) and halves the run-to-run spread [35].

FROM OUR C CODE — src/optimizer/LrScheduler.c, lines 23-29: the cosine schedule in our library

$lr = \eta_{\min} + (\text{base} - \eta_{\min}) \times (1 + \cos(\pi \times \text{epoch} / t_{\text{Max}})) / 2$. At epoch 0 the cosine is 1: $lr = \text{base}$. Half way it is 0: $lr = \text{the average}$. At the end it is -1: $lr = \eta_{\min}$.

Example. base 0.01, $\eta_{\min}$ 0, t_{Max} 50: epoch 25 gives 0.005; epoch 45 gives 0.00024.

```
static float cosineAnnealingLrComputeLr(const lrScheduler_t *sched) {
    double etaMin = (double)sched->params.cosineAnnealingLr.etaMin;
    return (float)(etaMin + ((double)sched->baseLr - etaMin) *
                    (1.0 + cos(M_PI * (double)sched->lastEpoch /
                    (double)sched->params.cosineAnnealingLr.tMax)) /
                    2.0);
}
```

WORKED ON OUR PROJECT — Cosine learning-rate schedule and int8

1. Cosine decay lowers eta from its start value towards LR_MIN over training [35].
2. At 8 bits the README measures 0.9026 (constant) vs 0.8996 (cosine): no difference. At 4 bits 0.8761 vs 0.8928, with the spread cut from 0.0210 to 0.0114 [35].
3. Late in a cosine schedule, updates fall below one quantisation step; they survive only with stochastic rounding [11].

Schedule	Shape	Use
Step decay	x0.1 at fixed epochs	Classic ResNet recipes
Cosine annealing	smooth decay to ~0 over the run	The modern default; often with restarts (SGDR)
Linear warmup + decay	rise for k steps, then decay	Mandatory for Transformers and large batches
OneCycle	up then down, momentum inversely	Fast convergence in few epochs; strong for fine-tuning
Exponential	$\eta \times \gamma^{\text{epoch}}$	Simple, needs tuning of gamma
ReduceLROnPlateau	cut when validation stalls	Robust when you cannot predict run length
Constant	flat	Debugging, or very short fine-tunes

WHY WARMUP IS NEEDED

At initialisation Adam's second-moment estimate v is based on very few samples, so its variance is huge and early steps can be wildly mis-scaled; simultaneously, in a deep residual stack the output distribution is far from its trained state, so early large steps do lasting damage. A linear warmup over 1-10k steps (or 1-5% of training) lets both stabilise. The larger the batch and the deeper the model, the more warmup you need.

17.3 Batch size, learning rate, and their interaction

- **Linear scaling rule:** multiply the batch size by k and multiply the learning rate by k , with warmup. Holds well up to batch sizes of a few thousand, then breaks down.

FROM ZERO — the idea explained step by step

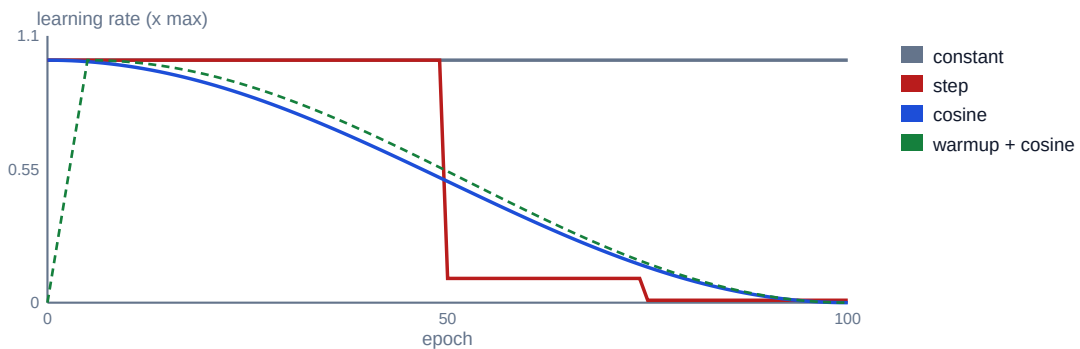
The **batch size** is how many examples are used for one gradient. Larger batches give smoother gradients but need more memory; the learning rate is usually chosen together with it.

IN OUR PROJECT — used

The batch is 64 windows (BATCH 64 in train_c.c). But the C training loop pushes only **one** window at a time through forward and backward (MICRO_BATCH 1) and adds up the gradients of the 64 windows before each update ("gradient accumulation"). The maths is the same as a batch of 64, but only one window's activations are in memory at a time -- essential on a 320 KB chip [35].

- **Square-root scaling** is sometimes better for Adam, since Adam already normalises by gradient magnitude.
- Small batches inject gradient noise, which acts as a regulariser and often improves generalisation; very large batches converge to sharper minima unless compensated with warmup, LARS/LAMB and more epochs.
- Batch size is chosen mostly by hardware: the largest that fits, rounded to a multiple of 8 (or 64) so tensor cores are used efficiently.
- **Gradient accumulation** simulates a larger batch on small memory; **gradient checkpointing** frees memory to enlarge the real batch.

17.4 Finding the learning rate



Graph 17.1 — Four learning-rate schedules over 100 epochs. Constant; step (divide by 10 at epochs 50 and 75); cosine (a smooth slide to 0); warmup then cosine (start small for 5 epochs, which protects the first, largest updates).

```
# LR range test (Smith): sweep the LR exponentially over one epoch and
# plot loss against LR. Pick roughly one order of magnitude below the
# point of steepest descent - NOT the minimum, which is already unstable.
lrs, losses = [], []
lr = 1e-7
for xb, yb in train_dl:
    for g in opt.param_groups: g['lr'] = lr
    opt.zero_grad(set_to_none=True)
    loss = lossf(model(xb.to(dev)), yb.to(dev))
    loss.backward(); opt.step()
    lrs.append(lr); losses.append(loss.item())
    lr *= 1.1
    if loss.item() > 4 * min(losses): break      # diverged; stop
```

Listing 17.1 - The learning-rate range test: five minutes that saves days.

FROM ZERO — the idea explained step by step

Find a good learning rate by trying values from small to large and watching the loss.

IN OUR PROJECT — used

The project's defaults: SGD at 0.01 and AdamW at 0.001 (run_matrix.py). train_pytorch.py has a single LR constant, so it must be changed to 0.001 by hand to mirror the AdamW binary [35].

17.5 One Adam update, arithmetic included

Adam's formula has four moving parts, and seeing them evaluate on a constant gradient makes the design obvious. Take $g = 0.5$ at every step, with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.001$, $\epsilon = 1e-8$.

FROM ZERO — the idea explained step by step

One Adam step computed by hand, including the running averages m and v and their bias correction.

IN OUR PROJECT — used

The C implementation mirrors PyTorch's AdamW operation by operation, verified with the parity checks [35].

Step t	m (first moment)	v (second moment)	m_hat	v_hat	Update size
1	0.05000	0.000250	0.50000	0.250000	0.001000
2	0.09500	0.000500	0.50000	0.250000	0.001000
3	0.13550	0.000749	0.50000	0.250000	0.001000

THREE THINGS THIS TABLE PROVES

(1) BIAS CORRECTION MATTERS: the raw m at step 1 is 0.05, ten times smaller than the true gradient 0.5, because the moving average starts from zero. Dividing by $(1 - \beta_1^t)$ restores it exactly. Without correction, Adam would take absurdly small steps for the first hundred iterations. (2) THE UPDATE IS SCALE-FREE: it comes out at 0.001 = the learning rate, and it would still be 0.001 if the gradient were 0.5, 50, or 5,000, because $m_hat/\sqrt{v_hat}$ cancels the magnitude. That is why Adam needs so little tuning across wildly different layers and losses. (3) THE UNITS ARE INTERPRETABLE: with Adam, the learning rate IS roughly the per-step change in each parameter - which is why $1e-3$ and $3e-4$ are such durable defaults.

The flip side of scale invariance is that Adam ignores information SGD uses: a parameter with a genuinely tiny gradient still gets a full-sized step. That is one reason well-tuned SGD with momentum still edges out Adam on some vision benchmarks, and why the gap closes when Adam is paired with decoupled weight decay.

17.6 Choosing a learning rate: what each failure looks like

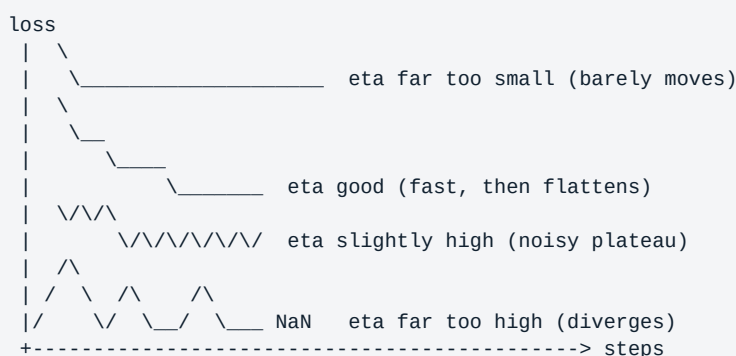


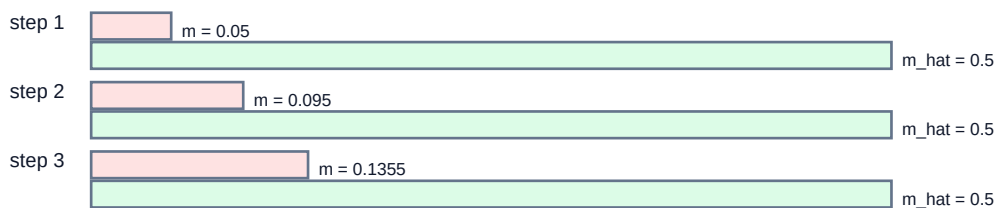
Figure 17.1 - Four learning rates, four distinctive loss-curve shapes.

FROM ZERO — the idea explained step by step

Too large a learning rate: the loss jumps or explodes. Too small: the loss decreases very slowly.

IN OUR PROJECT — used

In 8-bit training "too small" has a sharper meaning: an update smaller than half a quantisation step is rounded away completely. For conv3 at eta 0.01 this happens for every gradient below 0.1575 -- unless stochastic rounding is used [3, 11].



Graph 17.2 — One Adam update, from the book's table (gradient 0.5 every step). The raw average m starts at 0.05 -- ten times too small because it starts from 0. Bias correction divides by $(1 - 0.9^t)$ and gives the true 0.5 from the first step.

What you see	Diagnosis	Action
Loss barely changes over an epoch	Learning rate 10-100x too small	Multiply by 10 and retry; run the LR range test
Smooth fall then a flat floor	Healthy	Add a decay schedule to squeeze the last few percent
Falls then plateaus with visible noise	Slightly too high for the final phase	Cosine or step decay; this is exactly what a schedule fixes
Spikes upward, then NaN	Far too high, or exploding gradients	Reduce by 10x, add gradient clipping, check for bad inputs
Falls, then rises steadily	Not an LR problem - this is overfitting	Regularise; early stop

17.7 Momentum, seen as a ball on a surface

WHY MOMENTUM IS NOT JUST 'BIGGER STEPS'

Picture a narrow valley whose floor slopes gently towards the minimum while its walls are steep. Plain gradient descent is dominated by the steep direction: it bounces from wall to wall and creeps along the floor. Momentum accumulates a velocity, and because the wall-bouncing components alternate in sign they CANCEL, while the floor component always points the same way and ACCUMULATES. With $\beta = 0.9$ the consistent direction is amplified by roughly $1/(1 - 0.9) = 10\times$, and the oscillation is damped. That is why momentum both stabilises and accelerates, which sounds contradictory until you see the cancellation.

FROM ZERO — the idea explained step by step

Momentum is like a ball rolling downhill: it keeps moving in the direction it was going and is not stopped by every small bump.

IN OUR PROJECT — used

Our momentum 0.9 remembers about the last 10 gradients $(1 / (1 - 0.9))$ [35].

WORKED ON OUR PROJECT — One SGD-with-momentum step

1. Update: $v \leftarrow 0.9 v + g$; $\theta \leftarrow \theta - \eta v$, with $\eta = 0.01$ (the project's settings) [35].
2. For a constant gradient g the velocity approaches $g / (1 - 0.9) = 10 g$, so the effective step becomes $10 \times 0.01 = 0.1$ times the gradient.
3. Momentum also accumulates tiny gradients over steps -- one of the two escape routes from the int8 dead zone (the RigL document, VI.4).
4. Memory: one extra float per parameter, 40,856 bytes [35].

17.8 Gradient clipping and stability

```
if ||g|| > c :   g <- c * g / ||g||           (clip by global norm)
```

FROM ZERO — the idea explained step by step

Gradient clipping limits the size of the gradient, so one bad batch cannot throw the weights far away.

IN OUR PROJECT — not used, and why

Not used in the HAR training. In fully quantised training the gradient range is fixed by its bit width (16 bits in the ODT configuration), which acts like a built-in limit [3, 35].

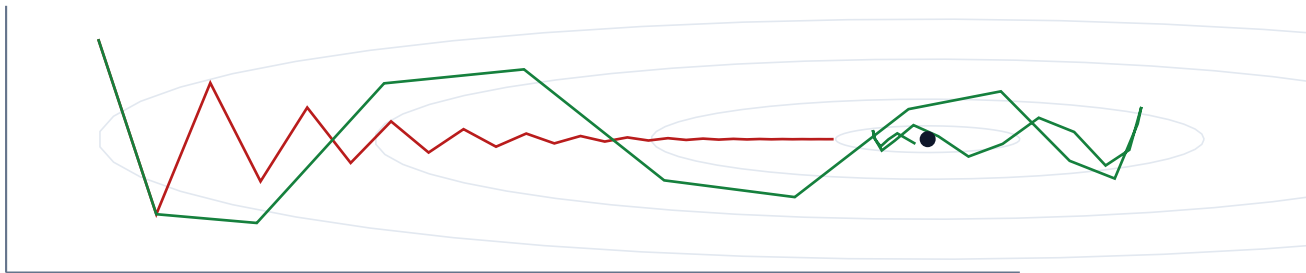
Clip by **global norm** (across all parameters), not per-parameter, so the update direction is preserved. $c = 1.0$ is a standard default for Transformers and RNNs. If you must clip constantly, your learning rate is too high or your data contains pathological samples - clipping is a seatbelt, not a steering wheel.

SHARPNESS, FLATNESS AND SAM

Solutions in flat regions of the loss surface tend to generalise better than those in sharp ones, because a flat minimum is robust to the shift between the training and test distributions. Sharpness-Aware Minimisation (SAM) makes this explicit: it takes an ascent step to the worst point within a small neighbourhood, computes the gradient THERE, and applies it at the original point. It roughly doubles the cost per step and reliably buys a point or two of accuracy on vision benchmarks.

Exercises

1 Implement SGD, momentum, RMSProp and Adam in 20 lines each and race them on the Rosenbrock function, plotting the trajectories. 2 Run the LR range test on a small CNN and compare the picked LR against a manual sweep of 5 values. 3 Train the same model with cosine decay, step decay and a constant LR to the same number of epochs and compare final accuracy.



Graph 17.3 — Plain gradient descent against momentum on a long narrow valley ($f = 0.5x^2 + 12.5y^2$, 30 steps, computed). Plain steps (red) bounce across the valley and crawl along it; momentum (green) builds speed along the valley and gets much closer to the bottom.

18. Normalization Layers

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The HAR model has no normalisation layer, but the ODT library implements LayerNorm and GroupNorm, and the raw-audio keyword example (kws_raw) needs LayerNorm to train reliably. BatchNorm is deliberately absent: it needs a batch, and the C trainer processes one window at a time.

Normalisation layers rescale intermediate activations so that each layer sees inputs with a stable distribution. They made deep networks trainable at practical learning rates, and every modern architecture contains one variety or another.

18.1 Batch normalisation

```
mu_B    = (1/m) SUM_i x_i           (per channel, over the batch)
var_B   = (1/m) SUM_i (x_i - mu_B)^2
x_hat_i = (x_i - mu_B) / sqrt(var_B + eps)
y_i     = gamma * x_hat_i + beta    (learned scale and shift)
```

FROM ZERO — the idea explained step by step

Normalisation rescales the numbers flowing through a network so that they have mean about 0 and spread about 1. Without it, the size of the numbers can drift from layer to layer and training becomes slow or unstable.

Batch normalisation (BatchNorm) computes the mean and spread over the examples of the current batch. Example: four values 2, 4, 6, 8 have mean 5 and standard deviation 2.24; normalised they become -1.34, -0.45, 0.45, 1.34 [1, Ch. 8].

IN OUR PROJECT — not used, and why

Not used, and not in the ODT library. BatchNorm needs many examples at once, but the C trainer runs one window at a time (MICRO_BATCH 1) to save memory, so a batch mean does not exist [35]. With one example the "batch" statistics would be the example itself, and every value would become 0.

WORKED ON OUR PROJECT — What batch normalisation would cost on the board

1. Adding BN after conv1-3 needs 2 learnable and 2 running numbers per channel: $(16 + 32 + 64) \times 4 = 448$ numbers [15].
2. On the device the project trains with micro-batch $B = 1$ and accumulates gradients [35]. With $B = 1$ the batch mean equals the sample itself, and BN normalises every channel to zero -- it breaks.
3. Group or layer normalisation computes statistics per sample and works with $B = 1$; or BN can be frozen (running statistics fixed) during on-device fine-tuning [5].

gamma and beta preserve expressiveness - the layer can undo the normalisation if that is what the loss prefers. At inference time the batch statistics are replaced by **running averages** collected during training, which is exactly what `model.eval()` switches on.

Benefit	Mechanism
Higher learning rates are stable	Rescaling makes the loss surface smoother and bounds the effect of a large step
Reduced sensitivity to initialisation	The layer re-centres and re-scales whatever arrives
A mild regularising effect	Each sample's normalisation depends on the random batch it landed in - noise that acts like dropout
Faster convergence	Typically 2-5x fewer epochs on deep convnets

BATCH NORM'S FAILURE MODES

(1) Small batches: with batch size 2-8 the statistics are noisy and performance collapses - a real problem for detection and segmentation, where images are large. (2) Train/test mismatch: running statistics differ from batch statistics, causing a gap that appears only at evaluation. (3) Sequence models: statistics vary with position and padding. (4) Distributed training needs SyncBatchNorm to pool statistics across GPUs. (5) It leaks information between samples in a batch, which breaks some privacy and contrastive setups.

18.2 The alternatives, and how they differ

Tensor (N, C, H, W). Shaded = the elements averaged together.

BatchNorm	: normalise over (N, H, W) for each channel C
LayerNorm	: normalise over (C, H, W) for each sample N
InstanceNorm	: normalise over (H, W) for each (N, C)
GroupNorm	: normalise over (C/g, H, W) for each sample and group
RMSNorm	: LayerNorm without mean subtraction; divide by RMS only

Figure 18.1 - The normalisation family differs only in which axes are pooled.

FROM ZERO — the idea explained step by step

LayerNorm normalises over the numbers of *one* example (all channels and time steps of it). **GroupNorm** splits the channels into groups and normalises each group of one example. Neither needs a batch, so both work with one example at a time.

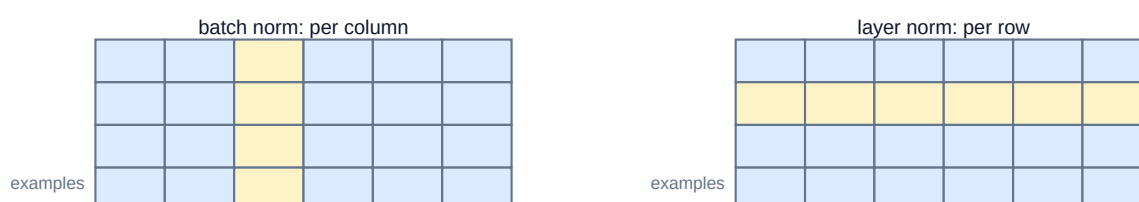
IN OUR PROJECT — partly used

Both are in the library: `src/layer/LayerNorm.c` and `src/layer/GroupNorm.c` [35].

LayerNorm is used in the `kws_raw` example (keyword spotting from raw audio): after each of its 3 convolutions comes a LayerNorm over the whole feature map. Its source records the reason: with only one LayerNorm at the end, a 10-seed sweep collapsed on about 40% of the seeds; with one per convolution, training converges every time [35].

The HAR model trains well without normalisation because its inputs are already small and similar in size (the standard deviation of every channel is between 0.11 and 0.41, measured) and it is only 3 layers deep.

Layer	Depends on batch?	Standard use
BatchNorm	Yes	CNNs with batch ≥ 32
LayerNorm	No	Transformers, RNNs, any variable-length input
RMSNorm	No	Modern LLMs - cheaper than LayerNorm, equally effective
GroupNorm	No	Detection/segmentation with small batches ($g = 32$ is a good default)
InstanceNorm	No	Style transfer, image generation
Weight norm / spectral norm	No	Reparameterise or constrain the weights themselves; spectral norm stabilises GAN discriminators



Graph 18.1 — Batch norm against layer norm on a grid of 4 examples x 6 features. Batch norm standardises each feature across the batch (a column); layer norm standardises each example across its features (a row) -- which is why it works with a batch of 1.

18.3 Placement: pre-norm versus post-norm

Post-norm (original Transformer): `x <- Norm( x + Sublayer(x) )`
Pre-norm (modern default): `x <- x + Sublayer( Norm(x) )`

FROM ZERO — the idea explained step by step

"Pre-norm" puts the normalisation before a block, "post-norm" after it. In deep Transformers pre-norm trains more stably.

IN OUR PROJECT — partly used

kws_raw uses the order Conv1d -> LayerNorm -> ReLU -> MaxPool, i.e. normalisation right after each convolution [35].

Pre-norm keeps a clean identity path through the whole stack, so gradients reach the first layer without passing through a normalisation, and deep models train without a delicate warmup. Post-norm can reach slightly better final quality but is much harder to train deep. Every large language model of the last few years uses pre-norm (usually pre-RMSNorm).

WHAT NORMALISATION IS ACTUALLY DOING

The original 'internal covariate shift' explanation has not held up well under scrutiny. The better-supported account is that normalisation reparameterises the loss surface so that it is smoother (bounded Lipschitz constant of the gradient), which permits larger stable steps. A second effect is scale invariance: with a normalisation layer downstream, scaling a weight matrix by c leaves the output unchanged, so weight decay acts on the effective learning rate rather than on the function - which is why weight decay and normalisation interact in ways that surprise people.

Exercises

1 Train a 20-layer MLP with and without BatchNorm at learning rates 0.001, 0.01, 0.1 and tabulate which combinations converge. 2 Replace BatchNorm with GroupNorm in a small CNN and compare accuracy at batch sizes 64, 8 and 2. 3 Derive the BatchNorm backward pass and verify it numerically.

19. Regularization for Deep Networks

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The HAR model is small (10,214 parameters for 6,617 windows), so it barely overfits and uses no explicit regulariser. Quantisation and sparsity, the two thesis techniques, themselves act like regularisers.

Deep networks can memorise random labels, so they can certainly memorise your training set. Regularisation is how you push them towards solutions that generalise. The techniques below are cumulative - a strong recipe uses several at once.

19.1 Dropout

During training, zero each unit independently with probability p and scale the survivors by $1/(1-p)$ so the expected activation is unchanged (inverted dropout). At evaluation, nothing is dropped.

FROM ZERO — the idea explained step by step

Dropout switches off a random part of the neurons (e.g. 50%) at every training step. The network cannot rely on any single neuron and learns more robust features. At test time all neurons are used [1, Ch. 7].

Example: with $p = 0.5$, the outputs (0.8, 0.2, 0.6, 0.4) might become (1.6, 0, 1.2, 0) -- the kept ones are doubled so that the average stays the same.

IN OUR PROJECT — not used, and why

In the library (src/layer/Dropout.c, using a Bernoulli random mask from src/rng/Bernoulli.c) but not used in any model of the project [35]. The HAR model is too small to overfit much: train and test loss stay close. Sparsity (RigL) has a related effect: 80-90% of the weights are zero, which also stops the network from depending on every connection.

FROM OUR C CODE — src/layer/Dropout.c, lines 51-54: dropout in our library

Each position has a random 0/1 mask entry (drawn by bernoulliFillMask). Kept values are multiplied by $1 / (1 - p)$, dropped ones become 0. With $p = 0.5$ kept values are doubled, so the average is unchanged.

In evaluation mode (!cfg->training) the input is copied unchanged.

```
float scale = dropoutScale(cfg->p);
for (size_t i = 0; i < numberOfElements; i++) {
    out[i] = tensorBoolGet(cfg->mask, i) ? in[i] * scale : 0.0f;
}
```

- **Why it works:** it prevents co-adaptation - no unit can rely on a specific partner being present - and it approximates an ensemble of exponentially many sub-networks that share weights.
- **Typical values:** 0.5 for wide fully connected layers, 0.1-0.3 in Transformers, and often 0 in convnets with BatchNorm, where the normalisation already supplies noise. **Spatial dropout** (drop whole channels) is the correct variant for convolutions.
- **Variants:** DropConnect drops weights instead of units; DropPath / stochastic depth drops entire residual blocks and is standard in modern vision transformers; DropBlock drops contiguous regions of a feature map.
- **Interaction warning:** dropout before BatchNorm changes the variance the normalisation sees, producing a train/test discrepancy. Put dropout after the normalisation, or omit one of the two.

19.2 Weight decay

Weight decay shrinks weights towards zero each step. As Chapter 17 explained, use the **decoupled** form (AdamW). Two practical details are usually ignored and matter:

FROM ZERO — the idea explained step by step

Weight decay shrinks every weight a little at every step, which keeps the weights small (the squared L2 norm of Chapter 2 is the penalty).

IN OUR PROJECT — partly used

Used with the AdamW optimiser: weight decay 0.01 [35]. Not used with the SGD runs. With learning rate 0.001, each step multiplies a weight by $1 - 0.001 \times 0.01 = 0.99999$; over 2,060 steps that is about 2% shrinkage.

- **Do not decay biases or normalisation parameters.** Decaying gamma and beta fights the normalisation layer and costs accuracy. Build two parameter groups.
- Typical values: 1e-4 for convnets with SGD, 0.01-0.1 for Transformers with AdamW. It interacts with the learning rate and the schedule; tune them together.

```
decay, no_decay = [], []
for name, param in model.named_parameters():
    if not param.requires_grad:
        continue
    if param.ndim <= 1 or name.endswith('.bias') or 'norm' in name.lower():
        no_decay.append(param)      # biases, LayerNorm/BatchNorm terms
    else:
        decay.append(param)         # weight matrices and conv kernels
opt = torch.optim.AdamW([
    {'params': decay, 'weight_decay': 0.05},
    {'params': no_decay, 'weight_decay': 0.0},
], lr=3e-4, betas=(0.9, 0.95))
```

Listing 19.1 - Correct parameter grouping for weight decay.

19.3 Early stopping

Monitor a validation metric, keep the best checkpoint, and stop when it has not improved for patience evaluations. Two rules: stop on the metric you actually care about, not on the training loss; and restore the best weights rather than keeping the last ones. Patience of 5-20 evaluations is typical; with a cosine schedule it is often better to train the full schedule and simply keep the best checkpoint.

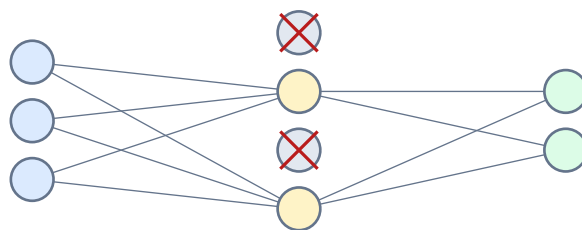
FROM ZERO — the idea explained step by step

Early stopping: keep an eye on the validation loss and stop (or keep the best model) when it starts going up again.

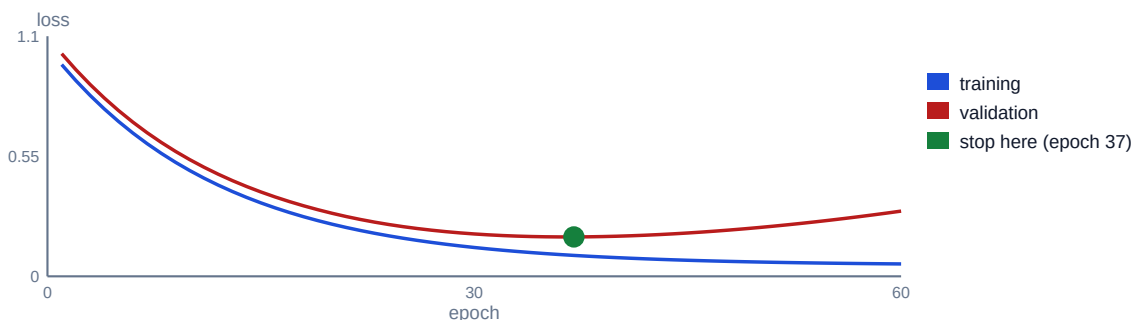
IN OUR PROJECT — partly used

The project keeps 735 validation windows and logs validation accuracy and loss after every epoch [35]. It trains a fixed 20 epochs rather than stopping early, so that all configurations are compared after the same number of steps.

19.4 Augmentation and label-level regularisers



Graph 19.1 — Dropout. During training each hidden neuron is switched off at random (here half of them, crossed out) for each batch, so no neuron can rely on a particular partner. At test time all are on.



Graph 19.2 — Early stopping. Training loss keeps falling; validation loss falls, reaches its lowest point, then climbs as the model starts memorising. Keep the weights from the lowest point (illustrative curves).

Technique	What it does	Effect
Mixup	Train on convex combinations of two samples AND their labels	Smoother decision boundaries, better calibration
CutMix	Paste a patch of one image into another; mix labels by area	Strong for classification; forces use of the whole object
CutOut / random erasing	Blank a random rectangle	Robustness to occlusion
RandAugment / TrivialAugment	Sample augmentation ops randomly with one or two hyperparameters	Near-AutoAugment quality without the search cost
Label smoothing	Targets 1-eps instead of 1	Less overconfidence, better calibration
Noisy student / self-training	Pseudo-label unlabelled data, train a larger student with noise	Large gains when unlabelled data is plentiful

FROM ZERO — the idea explained step by step

Data augmentation creates extra training examples by changing the existing ones in harmless ways (for sensor data: small rotations, scaling, added noise, shifts in time).

IN OUR PROJECT — not used, and why

Not used. For HAR it would be natural (a phone is never worn exactly the same way twice). The C run shows why it could help: the same CNN reaches 0.9551 on validation windows from the training people but 0.9002 on the new people -- a gap of 5.5 points that more varied training data could narrow.

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.

WORKED ON OUR PROJECT — Augmentations that make sense for accelerometer data

1. Jitter (add small noise, e.g. std 0.01 g), scaling (multiply a window by 0.9-1.1), and time shift (start the window a few samples later) all preserve the activity label.
2. Rotation must be used carefully: total_acc contains gravity, and the phone orientation is informative (lying vs standing). Rotating by 90 degrees would turn STANDING into LAYING.
3. Early stopping uses the 735 validation windows; remember they are optimistic (Chapter 3).
4. The project uses no dropout: the model is small (10k parameters) compared with the data (6.6k windows x 1152 values) [16].

Mixup: $x_{mix} = \lambda x_i + (1-\lambda) x_j$, $y_{mix} = \lambda y_i + (1-\lambda) y_j$
 $\lambda \sim \text{Beta}(\alpha, \alpha)$, α in $[0.1, 0.4]$ typically

19.5 Ensembling for deep networks

- **Independent seeds:** train the same architecture 3-5 times and average the softmax outputs. Reliably the largest easy gain, and it also gives a usable uncertainty estimate (deep ensembles, Chapter 33).

FROM ZERO — the idea explained step by step

Averaging several networks improves accuracy but multiplies memory and compute.

IN OUR PROJECT — partly used

Not on the board. The 10-seed averages of the thesis use the same idea only for measurement (Chapter 11).

- **Snapshot ensembles:** use a cyclic learning rate and save a checkpoint at each minimum; you get an ensemble for the price of one run.
- **Stochastic Weight Averaging (SWA):** average the WEIGHTS of checkpoints from the late phase of training with a constant or cyclic learning rate. Costs one model at inference, finds flatter minima, and is essentially free. Remember to recompute BatchNorm statistics afterwards.
- **Exponential moving average (EMA)** of weights: keep a shadow copy updated as $ema \leftarrow d * ema + (1-d) * w$ with d around 0.999 and evaluate with it. Standard practice in diffusion models and semi-supervised learning.

19.6 A regularisation recipe by data size

Situation	Recipe
Small data (<10k), pretrained backbone available	Freeze most layers, strong augmentation, dropout 0.3-0.5, weight decay, early stopping, 5-seed ensemble
Medium data (10k-1M)	Full fine-tune or train from scratch, RandAugment + Mixup, label smoothing 0.1, cosine schedule, EMA
Large data (>10M)	Weak augmentation, little or no dropout, weight decay, large batch with warmup - the data is the regulariser
Noisy labels	Label smoothing, symmetric or generalised cross-entropy, co-teaching, early stopping (networks fit clean data first)

FROM ZERO — the idea explained step by step

How much regularisation is needed depends on how much data there is per parameter.

IN OUR PROJECT — used

We have 6,617 windows x 1,152 numbers for 10,214 parameters. The float model reaches ~90% on new people without any regulariser, which shows it is not badly overfitting [35].

Exercises

1 Train a CNN on 5,000 CIFAR images with no regularisation, then add augmentation, then Mixup, then an ensemble; report the accuracy after each addition. 2 Show that inverted dropout preserves the expected activation, and measure the variance it introduces. 3 Implement SWA over the last 25% of training and compare against the best single checkpoint.

20. Training in Practice: A Debugging Playbook

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Most of this chapter describes what the project actually did: start with a PyTorch reference, check parity, log everything, and repeat with 10 seeds. Chapter 20.5 (reproducibility) has a real project bug as example.

Most of the time lost in deep learning is lost to bugs that produce a plausible-looking but wrong result. This chapter is the ordered checklist that finds them.

20.1 Start correctly

FROM ZERO — the idea explained step by step

Start simple and check each piece: the loss at the very first step should be $\log(\text{number of classes})$; a tiny subset of data should be learnable to 100%.

IN OUR PROJECT — used

For our 6 classes the first loss must be $\log 6 = 1.7918$. The project starts every model as a PyTorch reference and only then writes the C version, checking that both agree [35].

The first hour of any new model

Overfit a single batch of 8 samples to near-zero loss. If you cannot, the bug is in the model, loss or data pipeline - not in the hyperparameters. This is the single most valuable test in deep learning.

IN OUR PROJECT — The first hour of any new model

In the project the "first hour" is the parity run: same data, same starting weights (BIT_PARITY=1), same results in PyTorch and C, checked by the CI [35].

Verify the loss at initialisation: for K balanced classes it should be about $\log(K)$ - 2.30 for 10 classes. A wildly different value means the output layer or the loss is wrong.

Print shapes at every stage, once, and check them against what you intended.

Visualise a batch after augmentation, with its labels. Half of all data bugs are visible immediately - wrong channel order, labels off by one, normalisation applied twice.

Check the label mapping in both directions, and confirm that class indices in the loss match the ones in your evaluation code.

Fix all seeds and record them; log the git commit, the config, and the data version.

20.2 Symptom-to-cause table

Symptom	Likely causes	What to try
Loss is NaN	Learning rate too high; $\log(0)$; division by zero; fp16 overflow; bad input values	Lower LR; use logits-based losses; add eps; enable loss scaling; assert finite inputs
Loss does not move	LR too low; zero_grad missing; frozen parameters; dead ReLUs; broken data pipeline	LR range test; check requires_grad; check gradient norms per layer
Train loss falls, validation does not	Overfitting, or a train/validation mismatch	More regularisation and data; verify that both use identical preprocessing

Symptom	Likely causes	What to try
Validation better than training	Dropout/augmentation active only in training (often normal), or leakage	Compare at eval mode on the same data; audit the split
Metrics good offline, bad in production	Leakage, distribution shift, or skew between training and serving features	Audit features; re-split by time; log serving inputs and compare distributions
Results change wildly between runs	Seed sensitivity, too-small validation set, unstable LR	Average over seeds; enlarge validation; lower LR; add warmup
GPU out of memory	Batch too large; activations retained; memory leak from keeping loss tensors	Reduce batch; use AMP and checkpointing; store <code>loss.item()</code> , not the tensor
Training is slow	Data loading is the bottleneck; small batch; unfused ops; CPU-GPU sync per step	More workers + <code>pin_memory</code> ; profile; AMP; <code>torch.compile</code> ; avoid <code>.item()</code> inside the loop

FROM ZERO — the idea explained step by step

A table from symptoms (loss is NaN, loss stuck, accuracy at chance) to likely causes.

IN OUR PROJECT — used

A real symptom from the project: with 8-bit weights and ordinary rounding, many weights stop changing at all. The cause is the dead zone (updates smaller than half a step round to zero); the fix is stochastic rounding, which is the default in `train_c_sym.c` [35].

Measured in C (graph in Section 28.5): with ordinary rounding only 1.6% of the weight bytes still change per epoch at the end of training; with stochastic rounding 40.9%.

WORKED ON OUR PROJECT — The project's debugging checks, in order

1. Start: the first loss must be near $\log 6 = 1.79$ (Chapter 16). If not, labels or initialisation are wrong.
2. Overfit one batch: 64 windows must reach ~100% training accuracy quickly; otherwise the backward pass is broken.
3. Bit parity: in `BIT_PARITY` mode the C model loads the PyTorch weights, and one forward/backward pass must match PyTorch to float rounding -- a CI gate in the project [35].
4. NaN hunt: a NaN in the loss is traced to the first step where it appears (the C book, 28.6), not to the final number.

20.3 Instrumentation worth having from day one

- Log the training loss, the validation metric, the learning rate, the gradient global norm, and the weight norm - all against step, not epoch.

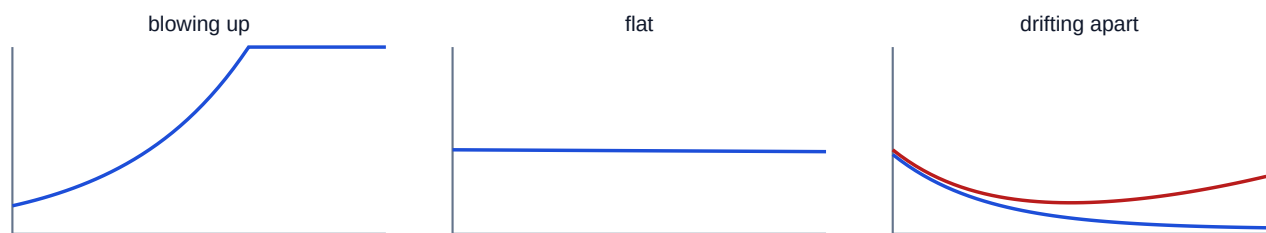
FROM ZERO — the idea explained step by step

Log the loss, accuracy, learning rate and gradient sizes from the first day.

IN OUR PROJECT — used

The C trainers write a JSON log per run (config, per-epoch loss and accuracy, memory report). `train_c_sym.c` also logs how many weight bytes changed per epoch -- a direct measure of the dead zone [35].

- Log per-layer gradient norms occasionally; a layer with a norm 1000x different from its neighbours is a bug.
- Log a few predictions and their inputs every N steps; looking at the data is diagnostic in a way that scalars are not.
- Track throughput (samples/second) and GPU utilisation. Under 80% utilisation almost always means the data pipeline, not the model.



Graph 20.1 — Three loss curves worth recognising (illustrative). *Blowing up*: the learning rate is too high. *Flat from the start*: a bug, dead units or far too small a rate. *Training and validation drifting apart*: overfitting.

- Use an experiment tracker (MLflow, Weights & Biases, TensorBoard, or a CSV plus a config file) so that a result from three weeks ago can be reproduced.

20.4 Mixed precision and throughput

```
scaler = torch.amp.GradScaler('cuda')
for xb, yb in train_dl:
    opt.zero_grad(set_to_none=True)
    with torch.amp.autocast('cuda', dtype=torch.bfloat16):
        loss = lossf(model(xb), yb)          # matmuls run in low precision
    scaler.scale(loss).backward()             # loss scaling avoids fp16
    scaler.unscale_(opt)                      # underflow in the gradients
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(opt); scaler.update()
```

Listing 20.1 - Automatic mixed precision: roughly 2x faster and half the activation memory. bfloat16 has fp32's exponent range

FROM ZERO — the idea explained step by step

Mixed precision: do most arithmetic in 16-bit numbers on a GPU to go faster, keeping a 32-bit copy where precision matters.

IN OUR PROJECT — partly used

The thesis goes further than mixed precision: integers instead of decimals. In `train_c_sym.c` the forward pass is integer arithmetic (SYM_INT32, operands of at most 12 bits) and the weights are stored with 8 or 4 bits [3, 35].

and usually needs no loss scaling; fp16 does.

Lever	Typical speedup	Cost
Mixed precision (bf16/fp16)	1.5-3x	Rare numerical issues
torch.compile / graph capture	1.1-2x	Compile time, occasional graph breaks
Larger batch + scaled LR	up to linear	Memory; needs warmup
Better data pipeline (workers, prefetch, webdataset)	often 2x+	Engineering time
Gradient checkpointing	0.7x speed	Enables much larger models
Multi-GPU DDP	near-linear	Communication; code complexity
FSDP / ZeRO sharding	enables huge models	Communication overhead

20.5 Reproducibility

- Seed Python, NumPy and the framework; set `torch.use_deterministic_algorithms(True)` when you need bit-exactness, and accept the slowdown.

FROM ZERO — the idea explained step by step

Reproducibility: running the same code with the same seed must give the same result. Every random choice (starting weights, shuffling, dropout) must be controlled by a seed.

IN OUR PROJECT — used

A real project example: `train_c.c` sets the random generator with the SEED, but the data loader later resets the *same* generator with SHUFFLE_SEED (`DataLoader.c`). All randomness during training therefore follows SHUFFLE_SEED, and changing SEED alone changes only the starting weights. Found while writing the C book; it matters when the 10-seed sweeps are interpreted [35].

A second property of the same code: the data loader shuffles the windows **once** at start-up; every epoch then visits them in the same order (see the code box below). PyTorch's `DataLoader`, used by the reference, reshuffles every epoch -- one more reason the C and PyTorch runs agree only within a tolerance.

FROM OUR C CODE — `src/data_loader/DataLoader.c`, lines 35-43: where the shuffle happens -- and why it matters for seeds

The data loader shuffles the index list **once**, when it is created, and it first calls `rngSetSeed( shuffleSeed )` on the single global random generator. Two consequences, both visible in this code: (1) the windows are visited in the same order in every epoch (nothing reshuffles later); (2) every random number drawn after this point -- for example stochastic rounding in the 8-bit trainer -- follows SHUFFLE_SEED, not SEED.

```
size_t sizeDataset = getDatasetSize();

for (size_t i = 0; i < sizeDataset; ++i) {
    indices[i] = i;
}

if (shuffle) {
    rngSetSeed(shuffleSeed);
    rngShuffleIndices(indices, sizeDataset);
}
```

WORKED ON OUR PROJECT — Reproducibility: which seed controls what

1. SEED initialises the weights; SHUFFLE_SEED decides the validation split and the batch order [35].
2. Found in the C code: `initDataLoader` reseeds the single global RNG with SHUFFLE_SEED, so stochastic-rounding decisions during training come from the SHUFFLE_SEED stream -- changing SEED alone does not change them (C book 2.7).
3. For a valid 10-seed study both seeds, or separate generators, must be varied.

- Pin library versions and record the CUDA/cuDNN version - kernel changes alter results.
- Version the data, not just the code. A hash of the dataset belongs in the run log.
- Save the full config with every checkpoint. A checkpoint whose hyperparameters are unknown is close to worthless.
- Accept that exact reproducibility across hardware is often impossible; aim instead for statistical reproducibility over seeds.

Exercises

1 Deliberately introduce four bugs (missing `zero_grad`, wrong label order, missing eval mode, unnormalised inputs) and practise finding each from the loss curve alone. 2 Profile a training step and determine whether you are data-bound or compute-bound. 3 Reproduce one of your earlier results from its config and checkpoint only. If you cannot, fix your logging before doing anything else.

PART IV - Architectures

Convolutional networks, recurrent networks, attention and Transformers, large language models, generative models, graph networks and self-supervised learning.

21. Convolutional Neural Networks

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

This is the chapter of our model. The HAR network is a 1-D convolutional network: three convolution layers slide small filters along the 128 time steps of a window. Every formula here can be checked on it.

A dense layer connecting a 224x224x3 image to 1,000 hidden units needs 150 million weights, treats neighbouring pixels as unrelated, and has to relearn every pattern separately at every position. Convolution fixes all three problems with two structural assumptions: **locality** (nearby pixels are related) and **translation equivariance** (a cat is a cat wherever it appears).

21.1 The convolution operation

$$y[i, j] = \text{SUM}_c \text{SUM}_u \text{SUM}_v w[c, u, v] * x[c, i + u, j + v] + b$$

FROM ZERO — the idea explained step by step

A **convolution** slides a small filter (for example 3 numbers) along a signal. At each position it multiplies the filter with the piece of signal under it and adds up the products; the result is one output value.

Example: signal (1, 2, 3, 4, 5), filter (1, 0, -1): positions give 1 - 3 = -2, 2 - 4 = -2, 3 - 5 = -2. This filter measures the slope, which is constant here.

With several input channels the filter has one row per channel, and all products are added together. With several output channels there is one filter per output channel.

IN OUR PROJECT — used

Our three convolutions (src/layer/Conv1d.c): conv1 9 -> 16 channels, kernel 7; conv2 16 -> 32, kernel 5 (2,592 parameters); conv3 32 -> 64, kernel 3 (6,208 parameters). conv1 has 16 x 9 x 7 = 1,008 weights + 16 biases = 1,024 parameters [35].

A conv1 filter looking at 7 samples = 0.14 s of motion can detect, for example, the short sharp rise of the acceleration when a foot hits the ground.

FROM OUR C CODE — src/arithmetic/Conv1dKernel.c, lines 51-66: the convolution loop that runs our three conv layers

For one output channel oc at one position outPos: start from the bias, then for every input channel and every filter tap add input x weight. slice skips taps that would fall outside the signal (the padding).

Count. conv1: 16 output channels x 128 positions x 9 input channels x 7 taps = 129,024 multiply-adds per window -- the line `sum += xv * wv` runs that many times. This loop is where most of the training time goes (measured with a profiler: about half of all time).

```

float sum = bArr ? bArr[oc] : 0.0f;

for (size_t icOffset = 0; icOffset < inChPerGroup; icOffset++) {
    size_t ic = inLo + icOffset;
    for (size_t i = 0; i < slice.validCount; i++) {
        size_t inputIdx = slice.firstValidInputIdx + i * geom.dilation;
        size_t kernelIdx = slice.firstValidKernelOffset + i;

        float xv = xArr[(b * inChannels + ic) * inputLength + inputIdx];
        float wv =
            wArr[(oc * inChPerGroup + icOffset) * kernelSize + kernelIdx];
        sum += xv * wv;
    }
}

yArr[(b * outChannels + oc) * outputLength + outPos] = sum;

```

Output size: $O = \text{floor}((I + 2P - K) / S) + 1$
I input size, **K** kernel, **P** padding, **S** stride
Params of a conv layer: $K * K * C_{in} * C_{out} + C_{out}$
FLOPs (multiply-adds): $K * K * C_{in} * C_{out} * H_{out} * W_{out}$

input 5x5	kernel 3x3	output 3x3 (stride 1, no pad)
+---+---+---+---+	+---+---+---+	+---+---+---+
a b c . .	1 0 -1	y . .
+---+---+---+---+	+---+---+---+	+---+---+---+
d e f . .	1 0 -1	. . .
+---+---+---+---+	+---+---+---+	+---+---+---+
g h i . .	1 0 -1	. . .
+---+---+---+---+	+---+---+---+	+---+---+---+
y = a+d+g - (c+f+i)	<- this kernel detects vertical edges	

Figure 21.1 - A 3x3 convolution slides one small weight matrix over the whole input: parameter sharing in action.

THREE PROPERTIES, ONE AT A TIME

SPARSE CONNECTIVITY: each output depends on a small patch, so cost is linear in image size instead of quadratic. **PARAMETER SHARING:** the same kernel is applied everywhere, so a feature learned in one corner transfers to all others - a massive reduction in parameters and a powerful regulariser. **EQUIVARIANCE:** shift the input and the feature map shifts identically; adding pooling or global average pooling converts equivariance into approximate INVARIANCE, which is what classification wants.

21.2 The standard building blocks

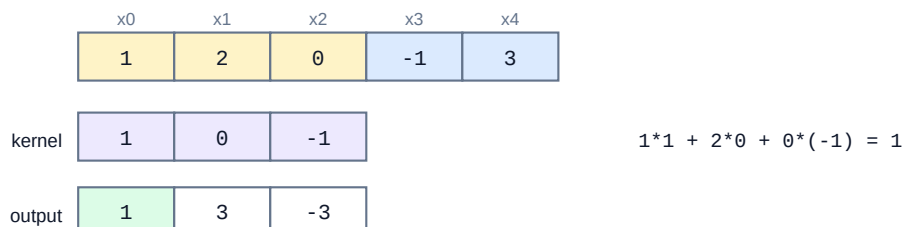
Layer	What it does	Typical use
Conv 3x3	The workhorse; two stacked 3x3 have the receptive field of one 5x5 with fewer parameters and more non-linearity	Everywhere
Conv 1x1	Mixes channels only; changes depth cheaply	Bottlenecks, projections, channel attention
Stride 2 conv	Downsamples while learning	Modern replacement for pooling
Max pool 2x2	Downsamples by taking the maximum	Classic; adds small translation invariance
Global average pool	Averages each channel to one number	Replaces the huge final dense layer
Transposed conv	Learned upsampling	Segmentation decoders, generators
Dilated conv	Inserts gaps in the kernel to enlarge the receptive field without cost	Segmentation, audio (WaveNet)
Depthwise separable	Depthwise spatial conv + pointwise 1x1; about 8-9x fewer FLOPs than a dense 3x3	MobileNet and every efficient on-device model

FROM ZERO — the idea explained step by step

The usual blocks: convolution, activation (ReLU), **pooling** (shrink the length by keeping the maximum or average of small groups), and at the end a dense layer for the decision.

IN OUR PROJECT — used

Our model: conv1 -> ReLU -> max-pool 2 -> conv2 -> ReLU -> max-pool 2 -> conv3 -> ReLU -> average over time -> linear 64 -> 6 -> softmax. The final average (AdaptiveAvgPool1d) turns 64 channels x remaining time steps into 64 numbers -- which is why the linear layer is only 390 parameters [35].



Graph 21.1 — A 1-D convolution done by hand: slide the kernel $[1, 0, -1]$ along $x = [1, 2, 0, -1, 3]$ and at each position multiply and add. Output: $1 - 0 = 1$, $2 + 1 = 3$, $0 - 3 = -3$. The same three weights are used at every position.

Standard 3x3 conv: $3*3*C_{in}*C_{out}$ multiply-adds per position
 Depthwise separable: $3*3*C_{in} + C_{in}*C_{out}$
 Ratio $\sim 1/C_{out} + 1/9$ -> about 8-9x cheaper for $C_{out} = 256$

21.3 Receptive field - the quantity to reason about

$$RF_{out} = RF_{in} + (K - 1) * PROD(previous\ strides)$$

FROM ZERO — the idea explained step by step

The **receptive field** of a neuron is the part of the input it can "see". Each convolution and pooling layer makes it larger.

IN OUR PROJECT — used

Ours grows 7 -> 8 -> 16 -> 18 -> 26 samples through the layers: a conv3 neuron sees 26 samples = 0.52 s. The main walking rhythm has a period of about 29 samples (0.58 s, measured), so a single neuron sees roughly one step; the final average over time combines all steps of the window.

A stack of ten 3x3 convolutions with stride 1 has a receptive field of 21 pixels; if a decision needs context wider than that, the architecture cannot make it, no matter how much data you have. Downsampling, dilation and attention are the three ways to grow the receptive field quickly, and the choice among them defines much of modern architecture design.

21.4 Convolution arithmetic worked through

Three formulas do all the work. Here they are applied to the cases you will actually meet.

FROM ZERO — the idea explained step by step

Output length = (input length + 2 x padding - kernel) / stride + 1. "Same" padding keeps the length unchanged.

IN OUR PROJECT — used

Our layers keep the length with padding, and the pooling halves it: 128 -> 128 -> 64 -> 64 -> 32 -> 32 time steps [35].

WORKED ON OUR PROJECT — Convolution arithmetic in the project's CNN

1. Output length $L_{out} = \text{floor}((L_{in} + 2P - K) / S) + 1$. conv1: K 7, P 3, S 1: $(128 + 6 - 7) / 1 + 1 = 128$ -- "same" padding [2, Ch. 7].
2. Max-pooling by 2 halves the length: 128 -> 64. conv2 (K 5, P 2) keeps 64; pooling -> 32; conv3 (K 3, P 1) keeps 32.
3. Average pooling over all 32 positions gives one value per channel: (64, 32) -> (64, 1) -> flatten to 64 -> linear to 6 classes.
4. Global average pooling makes the classifier independent of where in the window a step happens.

Input	Kernel	Padding	Stride	Output	Comment
32x32	3x3	1	1	32x32	'same' padding - the standard feature-extraction layer
32x32	3x3	0	1	30x30	'valid' - loses a ring of pixels each layer
32x32	3x3	1	2	16x16	Strided downsample, replaces pooling
224x224	7x7	3	2	112x112	The ResNet stem
32x32	1x1	0	1	32x32	Channel mixing only - spatial size untouched

$$O = \text{floor}((I + 2P - K) / S) + 1$$

Check the third row: $(32 + 2 - 3)/2 + 1 = \text{floor}(15.5) + 1 = 16$. OK.

21.5 Cost of one real layer, counted exactly

Take a 3x3 convolution with 64 input channels and 128 output channels, operating on a 32x32 feature map:

$$\begin{aligned} \text{Parameters} &= K * K * C_{in} * C_{out} + C_{out} \\ &= 3 * 3 * 64 * 128 + 128 = 73,728 + 128 = 73,856 \end{aligned}$$

FROM ZERO — the idea explained step by step

Work of a convolution layer = output channels x output length x input channels x kernel size (multiply-adds).

IN OUR PROJECT — used

conv1: $16 \times 128 \times 9 \times 7 = 129,024$; conv2: $32 \times 64 \times 16 \times 5 = 163,840$; conv3: $64 \times 32 \times 32 \times 3 = 196,608$; linear 384. Total 489,856 per window. On a 216 MHz Cortex-M7 at roughly one multiply-add per cycle this is a few milliseconds of forward pass; training (backward) costs about twice as much again [35].

$$\begin{aligned} \text{Multiply-adds} &= K * K * C_{in} * C_{out} * H_{out} * W_{out} \\ &= 3 * 3 * 64 * 128 * 32 * 32 = 75,497,472 \quad (\sim 75 \text{ M per image}) \end{aligned}$$

Now replace it with a depthwise separable block - a 3x3 depthwise convolution followed by a 1x1 pointwise convolution:

$$\begin{aligned} \text{Parameters} &= (3 * 3 * 64 + 64) + (64 * 128 + 128) = 640 + 8,320 = 8,960 \\ \text{Multiply-adds} &= (3 * 3 * 64 + 64 * 128) * 32 * 32 = 8,978,432 \quad (\sim 9 \text{ M}) \end{aligned}$$

Parameters: 8.2x fewer. Compute: 8.4x fewer.

WHERE THE SAVING COMES FROM

A standard convolution does two jobs at once: it mixes across SPACE (the 3x3 neighbourhood) and across CHANNELS (all 64 inputs feeding all 128 outputs), and it pays the product of the two costs. Separating them turns a product into a sum: $3 \times 3 \times 64$ for space plus 64×128 for channels. The saving is roughly $1/C_{out} + 1/K^2$, which for typical values is 8-9x. Every efficient mobile architecture is built on this one observation.

21.6 Receptive field, computed layer by layer

A unit's receptive field is the patch of input that can influence it. Grow it with the recurrence $RF \leftarrow RF + (K - 1) \times (\text{product of all previous strides})$:

FROM ZERO — the idea explained step by step

The receptive field computed layer by layer: each layer adds $(\text{kernel} - 1) \times (\text{product of earlier strides})$.

IN OUR PROJECT — used

conv1 +6 = 7; pool +1 = 8; conv2 +4 x 2 = 16; pool +1 x 2 = 18; conv3 +2 x 4 = 26 samples.

Layer	Kernel	Stride	Cumulative stride	Receptive field
input	-	-	1	1
conv 3x3	3	1	1	3
conv 3x3	3	1	1	5
maxpool 2x2	2	2	2	6
conv 3x3	3	1	2	10
conv 3x3	3	1	2	14
maxpool 2x2	2	2	4	16
conv 3x3	3	1	4	24

Two practical consequences. Downsampling grows the receptive field far faster than depth alone - after the second pool, each extra 3x3 convolution adds 8 pixels of context instead of 2. And if your task needs context wider than the final receptive field - a 200-pixel object seen by units with a 24-pixel view - no amount of training will fix it; the architecture is the constraint. Dilated convolutions, more downsampling, or attention are the three ways out.

21.7 A short history worth knowing

Model	Year	Contribution
LeNet-5	1998	Conv + pool + dense; digits; the template
AlexNet	2012	ReLU, dropout, GPUs, augmentation; won ImageNet by a huge margin and started the era
VGG	2014	Uniform 3x3 stacks; showed depth matters; very heavy
GoogLeNet / Inception	2014	Multi-scale branches; 1x1 bottlenecks
ResNet	2015	Residual connections; 152 layers trainable; the single most influential idea in the list
DenseNet	2016	Concatenate all previous feature maps
MobileNet / ShuffleNet	2017	Depthwise separable convolutions for phones
EfficientNet	2019	Compound scaling of depth, width and resolution together
ConvNeXt	2022	A convnet modernised with Transformer-era training recipes; matches ViT

FROM ZERO — the idea explained step by step

LeNet, AlexNet, VGG, ResNet: the milestones of image CNNs.

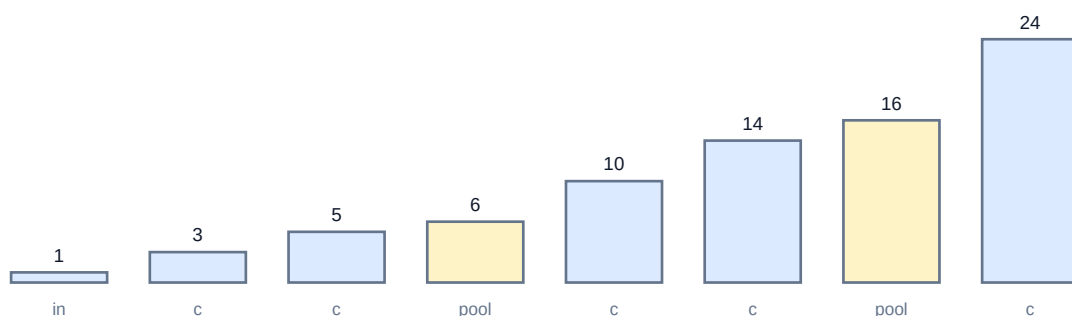
IN OUR PROJECT — partly used

Our model is LeNet-sized: a few small layers, tens of thousands of parameters at most -- the scale a microcontroller can hold.

Residual block: $y = x + F(x)$ (identity path + learned residual)

Bottleneck block: 1x1 reduce -> 3x3 -> 1x1 expand, all with a skip

21.8 Beyond classification



Graph 21.2 — Receptive field layer by layer, from the book's table in 21.6: 1, 3, 5, 6, 10, 14, 16, 24 pixels. Plain 3x3 convolutions add 2 each; after each pooling the same convolution adds twice as much.

- **Object detection:** two-stage (Faster R-CNN: propose regions, then classify) versus one-stage (YOLO, SSD, RetinaNet with focal loss). Metrics: mAP at IoU thresholds. DETR reframes detection as set prediction with a Transformer and removes anchors and NMS.

FROM ZERO — the idea explained step by step

CNNs also detect, segment and reconstruct, not just classify.

IN OUR PROJECT — partly used

The ECG example (ecg_anomaly_ae) uses a convolutional **autoencoder** that reconstructs its input, with Conv1dTransposed layers to make the signal longer again (Chapter 25) [35].

- **Semantic segmentation:** per-pixel classification. U-Net's encoder-decoder with skip connections is still the standard in medical imaging; DeepLab adds dilated convolutions and multi-scale pooling.
- **Instance and panoptic segmentation:** Mask R-CNN adds a mask head to detection; panoptic unifies things and stuff.
- **Video:** 3D convolutions, two-stream (RGB + optical flow), or factorised (2+1)D convolutions; increasingly video Transformers.
- **Non-image uses:** 1D convolutions over time series and audio, and over text characters; convolution is about locality, not about pixels.

21.9 Transfer learning - the default workflow for vision

```
import torch, torchvision as tv, torch.nn as nn
m = tv.models.resnet50(weights=tv.models.ResNet50_Weights.IMAGENET1K_V2)
for p in m.parameters():
    p.requires_grad = False          # stage 1: freeze the backbone
m.fc = nn.Linear(m.fc.in_features, NUM_CLASSES) # new head, trainable
# stage 1: train the head only, LR ~1e-3, a few epochs
# stage 2: unfreeze the last block(s) and fine-tune with a small LR
for p in m.layer4.parameters():
    p.requires_grad = True
opt = torch.optim.AdamW([
    {'params': m.layer4.parameters(), 'lr': 1e-4}, # discriminative LRs:
    {'params': m.fc.parameters(), 'lr': 1e-3},    # deeper = smaller LR
], weight_decay=1e-4)
```

Listing 21.1 - Two-stage fine-tuning with discriminative learning rates.

FROM ZERO — the idea explained step by step

Transfer learning: take a network trained on a big dataset and adapt it to a new task with little data.

IN OUR PROJECT — partly used

This is the thesis scenario in small: the model is pre-trained on the training people, then keeps learning on the device for the new wearer. The continual-learning example adapts to 5 groups of people in turn (Chapter 31) [35].

HOW MUCH TO FINE-TUNE

Small dataset and similar domain: train the head only. Small dataset, different domain: fine-tune the last block or two with a small learning rate. Large dataset: fine-tune everything, still with a lower learning rate for early layers. Always keep the preprocessing (resize, mean/std normalisation) identical to what the backbone was pretrained with - this is a surprisingly common silent failure.

Exercises

1 Compute the receptive field, parameter count and FLOPs of a 5-layer CNN by hand, then verify with a profiler. 2 Implement a residual block and train a 20-layer plain CNN and a 20-layer ResNet on CIFAR-10; compare convergence. 3 Replace every 3x3 convolution in a small CNN with a depthwise separable block; report the accuracy, parameter and latency changes.

22. Sequence Models: RNN, LSTM and GRU

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Recurrent networks are the classic alternative for sequences such as our sensor windows. The project uses a CNN instead; this chapter explains why that is the better choice on a microcontroller.

Sequences - text, speech, sensor streams, prices - have order and variable length. A recurrent network processes them one step at a time, carrying a hidden state that summarises everything seen so far.

```
h_t = tanh( w_hh h_(t-1) + w_xh x_t + b )
y_t = w_hy h_t + b_y
```

The same weights are used at every timestep - parameter sharing across time, exactly analogous to a convolution's sharing across space. Training uses **backpropagation through time** : unroll the network over the sequence and apply ordinary backpropagation, usually truncated to a window of a few hundred steps to bound memory.

22.1 Why plain RNNs fail on long sequences

The gradient over T steps contains a factor $\text{PROD}_t (w_{hh}^T \text{diag}(\tanh'))$. A repeated matrix product either vanishes or explodes exponentially in T . Exploding is easy to fix with clipping; vanishing is the hard one, and it means the network cannot connect an event at step 5 to a consequence at step 500.

FROM ZERO — the idea explained step by step

A **recurrent network** (RNN) reads a sequence one step at a time and keeps a memory ("hidden state") that it updates at every step. Training goes backwards through all steps, and gradients can vanish over long sequences.

IN OUR PROJECT — not used, and why

Not used. For our windows, an RNN would run 128 steps one after another, and training would need to store all 128 hidden states. A CNN processes all positions with the same small filters and is simpler to implement and quantise in C.

22.2 LSTM: a cell state with gates

<code>f_t = sigma(w_f [h_(t-1), x_t] + b_f)</code>	forget gate
<code>i_t = sigma(w_i [h_(t-1), x_t] + b_i)</code>	input gate
<code>g_t = tanh (w_g [h_(t-1), x_t] + b_g)</code>	candidate
<code>o_t = sigma(w_o [h_(t-1), x_t] + b_o)</code>	output gate
<code>c_t = f_t * c_(t-1) + i_t * g_t</code>	CELL STATE (additive!)
<code>h_t = o_t * tanh(c_t)</code>	

FROM ZERO — the idea explained step by step

An **LSTM** adds gates (small switches between 0 and 1) that decide what to keep, forget and output, so that information can survive many steps.

IN OUR PROJECT — not used, and why

Not used. LSTMs are strong on HAR, but their gates use sigmoid and tanh (expensive in integers) and four weight matrices per layer. The project chose 1-D convolutions, which are cheap in 8-bit arithmetic.

WORKED ON OUR PROJECT — An LSTM alternative, sized

1. An LSTM with 9 inputs and 32 hidden units has $4 \times (32 \times (9 + 32) + 32) = 5,376$ parameters [26].
2. It must process the 128 time steps one after another: $128 \times 4 \times 32 \times 41 = 671,744$ MACs per window -- more than the whole CNN (489,856) and strictly sequential.
3. For short fixed windows on a microcontroller the CNN is cheaper and parallel; RNNs pay off for long or streaming sequences.

THE ONE LINE THAT MATTERS

$c_t = f_t * c_{(t-1)} + i_t * g_t$. The cell state is updated ADDITIVELY, not by a matrix multiplication, so the gradient path along c is a product of forget gates rather than of weight matrices. If the forget gate stays near 1, information and gradient flow for hundreds of steps. This is the same trick as a residual connection, invented for time.

GRU merges the forget and input gates into one update gate and drops the separate cell state: about 25% fewer parameters, usually indistinguishable in accuracy, and slightly faster. Try both; there is no reliable winner.

- Initialise the **forget-gate bias to 1** so the cell remembers by default - a small change that historically made LSTMs much easier to train.
- **Bidirectional** RNNs run one pass in each direction and concatenate; only usable when the whole sequence is available (not for streaming or generation).
- Stack 2-4 layers; deeper recurrent stacks rarely help without residual connections between layers.
- **Variational (locked) dropout** applies the same mask at every timestep; ordinary per-step dropout destroys the recurrent signal.
- Pack padded sequences (`pack_padded_sequence`) so the network never consumes padding tokens.

22.3 Sequence-to-sequence and the birth of attention

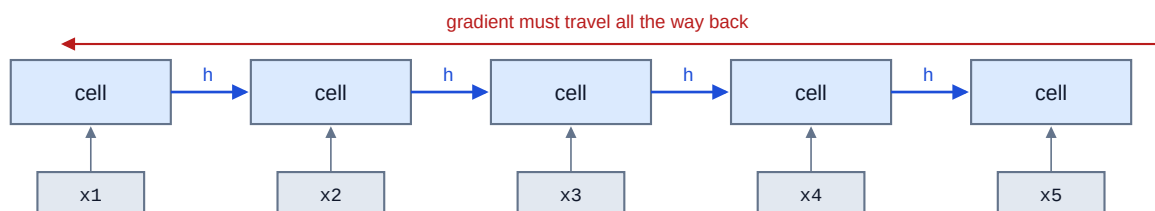
An encoder RNN compresses the input into a single fixed vector; a decoder RNN generates the output from it. The bottleneck is obvious: one vector cannot hold a 50-word sentence. Attention (Bahdanau, 2014) let the decoder look back at **all** encoder states, weighting them per output step:

FROM ZERO — the idea explained step by step

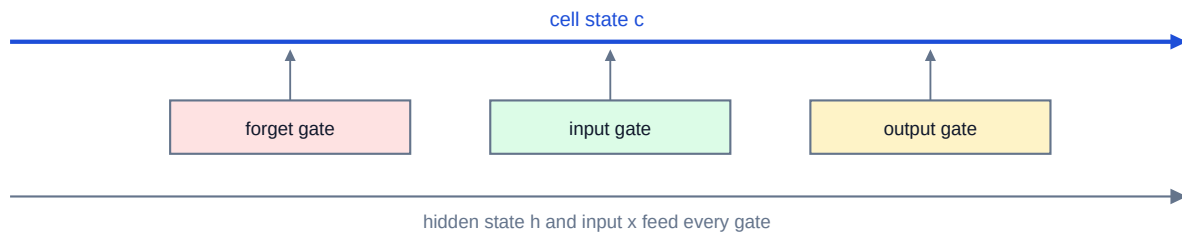
Sequence-to-sequence models turn one sequence into another; attention was invented to help them.

IN OUR PROJECT — not used, and why

Not relevant: our output is one label per window, not a sequence.



Graph 22.1 — An RNN unrolled over time: the same cell is applied at each step, passing a hidden state h along. The gradient for step 1 has to travel back through every later step, which is why plain RNNs forget long-range patterns.



Graph 22.2 — An LSTM cell. A separate cell state (top line) runs straight through with only small changes; three gates decide what to forget, what to add, and what to output. That straight line is what lets gradients survive over long sequences.

```
score(s_t, h_i) -> alpha_ti = softmax_i(score)
context c_t = SUM_i alpha_ti h_i
decoder consumes [s_t ; c_t]
```

This removed the bottleneck, gave interpretable alignments, and led directly to the conclusion of the next chapter: if attention is doing the work, the recurrence can be dropped entirely.

22.4 When to still use an RNN in the 2020s

- Streaming and low-latency inference with strict memory limits - an LSTM has $O(1)$ state per step, while a Transformer's KV cache grows with length.

FROM ZERO — the idea explained step by step

RNNs are still useful for very long streams processed step by step with little memory.

IN OUR PROJECT — partly used

A streaming activity recogniser on a watch is such a case. The thesis instead cuts the stream into 2.56 s windows with 50% overlap, which gives the CNN a fixed input size.

- Small on-device models over sensor streams, where a two-layer GRU of 50k parameters is enough and a Transformer is not affordable.
- Very long sequences where quadratic attention is prohibitive - though **state-space models** (S4, Mamba) are now the stronger option: they keep the recurrent $O(1)$ inference state while training in parallel like a convolution, and are competitive with Transformers on long-context tasks.
- Otherwise, for text and most sequence modelling, the default is a Transformer.

Exercises

1 Implement an LSTM cell from the equations above and check it against `nn.LSTMCell`. 2 Train a plain RNN, an LSTM and a GRU on the copy task (repeat a sequence after a delay of T steps) for $T = 10, 50, 200$ and plot accuracy against T . 3 Add Bahdanau attention to a small seq2seq translation model and visualise the alignment matrix.

23. Attention and the Transformer

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Transformers are not used in the project -- attention costs grow with the square of the sequence length and the model would not fit. But its two ingredients, softmax and dot products, are in our model.

The Transformer replaced recurrence with attention entirely. Its advantage is not only accuracy: every position is processed in parallel during training, so the architecture scales with hardware in a way RNNs never could. Understanding this chapter thoroughly is the highest-value investment in modern deep learning.

23.1 Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

FROM ZERO — the idea explained step by step

Attention lets each position of a sequence look at all other positions and take a weighted average of them. The weights come from a softmax over dot products ("how similar are these two positions?").

IN OUR PROJECT — not used, and why

Not used. Over 128 time steps attention would compute $128 \times 128 = 16,384$ scores per head and layer and store them for training -- more than the whole activation memory of our CNN. The ingredients, however, are ours: the dot product (every neuron) and softmax (the last layer, `src/layer/Softmax.c`).

$Q = X W_Q$	(n, d_k)	queries: what am I looking for?
$K = X W_K$	(n, d_k)	keys: what do I contain?
$V = X W_V$	(n, d_v)	values: what do I pass on?

THE DATABASE ANALOGY

Each token emits a QUERY describing what it needs, and every token offers a KEY describing what it has. The dot product query-dot-key scores the match; softmax turns the scores into weights that sum to one; the output is the weighted average of the VALUES. Unlike a hash lookup, the match is soft - every token contributes something, in proportion to relevance.

WHY DIVIDE BY $\sqrt{D_K}$

If the components of q and k are independent with zero mean and unit variance, then $q.k$ has variance d_k . With $d_k = 64$ the logits have a standard deviation of 8, and softmax of such large values is nearly one-hot with vanishing gradients. Dividing by $\sqrt{d_k}$ restores unit variance and keeps the softmax in its responsive range.

23.2 Multi-head attention

$$\text{head}_i = \text{Attention}(X W_Q^i, X W_K^i, X W_V^i), \quad i = 1..h$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

typically $d_k = d_v = d_{\text{model}} / h$

FROM ZERO — the idea explained step by step

Several attention "heads" run in parallel, each looking for a different kind of relation.

IN OUR PROJECT — not used, and why

Not used in the project.

Splitting the representation into h heads lets the layer attend to different relationships at once - one head tracking syntactic dependency, another coreference, another position. The total compute is the same as one head of full width, so it is free capacity in the useful sense.

```

x --> LayerNorm --> Multi-Head Attention --+--> (+) -->
|                                     | ^
+-----+ residual
|
--> LayerNorm --> FeedForward (d -> 4d -> d) --> (+) -->
|                                     | ^
+-----+ residual
One pre-norm Transformer block. Stack N of these.

```

Figure 23.1 - The block: attention mixes across tokens, the feedforward mixes across channels, residuals carry the signal.

23.3 The other components

- **Position-wise feedforward:** two linear layers with a non-linearity, usually expanding by 4x. It holds most of the parameters and, on current evidence, most of the model's factual knowledge. Modern LLMs use SwiGLU here, which needs three matrices at roughly 2/3 the width.

FROM ZERO — the idea explained step by step

A Transformer block = attention + a small MLP, with residual connections and LayerNorm.

IN OUR PROJECT — not used, and why

Not used. LayerNorm, one of its components, is in the library and used in `kws_raw` (Chapter 18) [35].

- **Residual connections and normalisation** around every sublayer (pre-norm; see Chapter 18).
- **Causal masking** in decoders: set the scores for future positions to $-\infty$ before the softmax so a token can never see its own future. This single mask is what makes parallel training of an autoregressive model possible.
- **Cross-attention** in encoder-decoder models: queries come from the decoder, keys and values from the encoder.

23.4 Positional information

Attention is permutation-equivariant - shuffle the tokens and the outputs shuffle with them. Order must be injected explicitly.

FROM ZERO — the idea explained step by step

Attention itself ignores order, so position information must be added to the inputs.

IN OUR PROJECT — not used, and why

A CNN needs no such trick: the order is built in, because a filter always sees neighbouring samples.

Scheme	How	Properties
Sinusoidal (original)	Add fixed sin/cos of varying frequency	No parameters; some extrapolation to longer sequences
Learned absolute	A trainable embedding per position	Simple; cannot exceed the trained length
Relative (T5, Shaw)	Bias the attention logits by the distance $i-j$	Generalises across lengths; used in encoder models

Scheme	How	Properties
RoPE (rotary)	Rotate q and k by an angle proportional to position	Relative by construction, extrapolates well, the de facto standard in modern LLMs; extendable by frequency scaling (YaRN, NTK)
ALiBi	Add a linear distance penalty to attention logits	Very simple, strong length extrapolation

23.5 Complexity and the efficiency ladder

Time: $O(n^2 d)$ **Memory (naive):** $O(n^2)$
n = sequence length, d = model dimension

FROM ZERO — the idea explained step by step

Attention costs grow with the square of the sequence length.

IN OUR PROJECT — not used, and why

This is the main reason Transformers are rarely trained on microcontrollers; convolution costs grow only linearly.

WORKED ON OUR PROJECT — What self-attention over one window would cost

1. Treat each of the 128 time steps as a token. One attention head computes a 128 x 128 score matrix: 16,384 numbers = 64 KB in float32 [27].
2. That is one fifth of the board's RAM for a single head in the forward pass; training also stores it for the backward pass.
3. Patching reduces it: 8 patches of 16 samples give an 8 x 8 matrix (64 numbers). This is how small transformers for sensor data are built.

Quadratic cost in sequence length is the Transformer's defining limitation, and a decade of work exists to soften it:

Technique	Idea	Status
FlashAttention	Tile the computation in SRAM; never materialise the $n \times n$ matrix. Exact, IO-aware	Universal in practice; the first thing to enable
Multi-query / grouped-query attention	Share K and V across heads	Shrinks the KV cache 8-64x; standard in modern LLMs
Sliding window / local attention	Attend within a window only	Mistral-style; combine with a few global tokens
Sparse patterns (Longformer, BigBird)	Local + global + random	Long documents
Linear attention (Performer, Linformer)	Kernel or low-rank approximation of softmax	$O(n)$; some quality loss
State-space models (S4, Mamba)	Structured recurrence, parallel training, $O(1)$ inference state	Strong for very long context
KV cache quantisation / paging	Store the cache in 8 or 4 bits, in pages	The main memory lever at serving time

23.6 Implementing attention

```

import torch, torch.nn as nn, torch.nn.functional as F
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, causal=True, p=0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=False)
        self.proj = nn.Linear(d_model, d_model, bias=False)
        self.causal, self.p = causal, p
    def forward(self, x):
        # x: (B, T, D)
        B, T, D = x.shape
        q, k, v = self.qkv(x).split(D, dim=2)
        # (B, T, D) -> (B, heads, T, dk)
        q = q.view(B, T, self.h, self.dk).transpose(1, 2)
        k = k.view(B, T, self.h, self.dk).transpose(1, 2)
        v = v.view(B, T, self.h, self.dk).transpose(1, 2)
        # FlashAttention kernel: exact, memory-efficient, fused
        y = F.scaled_dot_product_attention(
            q, k, v, is_causal=self.causal,
            dropout_p=self.p if self.training else 0.0)
        y = y.transpose(1, 2).contiguous().view(B, T, D)
        return self.proj(y)
# The explicit form, for understanding only:
# att = (q @ k.transpose(-2, -1)) / self.dk**0.5
# att = att.masked_fill(mask == 0, float('-inf')).softmax(-1)
# y = att @ v

```

FROM ZERO — the idea explained step by step

A short implementation of attention in NumPy.

IN OUR PROJECT — not used, and why

Not used in the project.

Listing 23.1 - Multi-head self-attention. The commented lines are the equations; the live code is what you should actually run.

23.7 Attention computed on three tokens

Nothing clarifies attention like doing it with small numbers. Three tokens, $d_k = 2$, with queries and keys chosen so the pattern is readable:

```

Q = K = [ 1  0 ]      V = [ 1  0 ]
         [ 0  1 ]      [ 0  2 ]
         [ 1  1 ]      [ 3  1 ]
token 1 points along x, token 2 along y, token 3 along both.

```

FROM ZERO — the idea explained step by step

Attention on three tokens with every number written out.

IN OUR PROJECT — partly used

Step 2 of this example is the same softmax our last layer computes. The ODT Softmax.c subtracts the largest score first (max-shift), exactly as in this example, so that $\exp()$ cannot overflow [35].

Step 1 - scores

```
Q K^T / sqrt(2) = [ 0.707  0.000  0.707 ]
                  [ 0.000  0.707  0.707 ]
                  [ 0.707  0.707  1.414 ]
```

Row 3 is the interesting one: token 3 scores 1.414 against itself and 0.707 against each of the others, because its direction overlaps both. Rows 1 and 2 each score 0 against the orthogonal token - orthogonal means 'unrelated', and the dot product says so numerically.

Step 2 - softmax over each row

```
weights = [ 0.401  0.198  0.401 ]    <- token 1 attends to 1 and 3
          [ 0.198  0.401  0.401 ]    <- token 2 attends to 2 and 3
          [ 0.248  0.248  0.504 ]    <- token 3 attends mostly to 3
```

Step 3 - weighted sum of values

```
output row 1 = 0.401*[1,0] + 0.198*[0,2] + 0.401*[3,1] = [1.604, 0.797]
output row 2 = 0.198*[1,0] + 0.401*[0,2] + 0.401*[3,1] = [1.401, 1.203]
output row 3 = 0.248*[1,0] + 0.248*[0,2] + 0.504*[3,1] = [1.759, 1.000]
```

Each token has been replaced by a blend of all tokens' values, weighted by relevance. That is the entire operation. Everything else in a Transformer - multiple heads, the projections $W_Q/W_K/W_V$, residuals, the feedforward block - is packaging around these three steps.

WATCHING THE SQRT(D_K) DO ITS JOB

Without the scaling, row 1's scores are 1, 0, 1 and the softmax gives 0.422, 0.155, 0.422 - a bit sharper. With $d_k = 2$ the difference is small, but scores grow like $\sqrt{d_k}$: at $d_k = 64$ an unscaled logit gap of 8 instead of 1 turns the softmax nearly one-hot, its gradient to nearly zero, and training stalls at initialisation. The division is not cosmetic; it is what keeps the softmax differentiable at realistic widths.

23.8 Why causal masking makes parallel training possible

Mask for a 4-token causal decoder (1 = allowed, . = -inf before softmax)

	attends to:	t1	t2	t3	t4	
query t1		1	.	.	.	token 1 sees only itself
query t2		1	1	.	.	token 2 sees 1 and 2
query t3		1	1	1	.	
query t4		1	1	1	1	

One forward pass computes the prediction for EVERY position at once, each conditioned only on its own past. An RNN needs 4 sequential steps to do the same thing. This is why Transformers train so much faster.

Figure 23.2 - The causal mask: n training examples from one sequence, in one parallel pass.

FROM ZERO — the idea explained step by step

A causal mask stops a position from seeing the future, so all positions can be trained at once.

IN OUR PROJECT — not used, and why

Not used in the project.

At **inference** the advantage disappears - tokens must still be generated one at a time, each attending to all previous ones. That asymmetry (parallel training, sequential generation) is why the KV cache exists, why generation is memory-bandwidth bound, and why speculative decoding is worth the complexity. Chapter 24 follows the consequences.

23.9 Sizing a Transformer block

For $d_{\text{model}} = 512$ with 8 heads and a 4x feedforward expansion:

FROM ZERO — the idea explained step by step

Counting the parameters and memory of one Transformer block.

IN OUR PROJECT — not used, and why

For comparison: a single block with width 64 has about 50,000 parameters ($4 \times 64 \times 64$ for attention + $8 \times 64 \times 64$ for the MLP) -- five times our entire HAR model.

Component	Shape	Parameters
W_Q, W_K, W_V	$3 \times (512 \times 512)$	786,432
W_O (output projection)	512×512	262,144
Feedforward up	512×2048	1,048,576
Feedforward down	2048×512	1,048,576
2 x LayerNorm	$2 \times 2 \times 512$	2,048
Total per block	-	~3.15 M
12 blocks	-	~37.7 M, plus embeddings

Note the split: attention holds about a third of the parameters and the feedforward block about two thirds. That ratio holds across most model sizes, and it is why pruning and quantization work on the feedforward matrices give the largest returns (Chapters 28-29), and why mixture-of-experts replaces exactly that block.

23.10 The three architectural families

	key 1	key 2	key 3
token 1	0.401	0.198	0.401
token 2	0.198	0.401	0.401
token 3	0.248	0.248	0.504

Graph 23.1 — The attention weights of 23.7 as a heat map (three tokens, from the book). Each row says how much one token looks at every token. Token 3 points along both directions, so it spreads its attention and looks most at itself (0.504).

query t1	1			
query t2	1	1		
query t3	1	1	1	
query t4	1	1	1	1

Graph 23.2 — The causal mask for four tokens. Each token may look at itself and earlier tokens (green) but not later ones (grey). That lets one forward pass train all four next-token predictions at once.

Family	Masking	Trained by	Examples	Best at
Encoder-only	Bidirectional	Masked token prediction	BERT, RoBERTa, DeBERTa, ViT	Classification, retrieval, embeddings

Family	Masking	Trained by	Examples	Best at
Decoder-only	Causal	Next-token prediction	GPT family, Llama, Mistral	Generation, few-shot, chat - the dominant design
Encoder-decoder	Both	Span corruption / seq2seq	T5, BART, Whisper	Translation, summarisation, speech

FROM ZERO — the idea explained step by step

Encoder-only, decoder-only and encoder-decoder Transformers.

IN OUR PROJECT — not used, and why

Not used in the project.

23.11 Transformers outside text

- **Vision Transformer (ViT):** split the image into 16x16 patches, embed each as a token, add positional embeddings, run a standard encoder. Needs large data or strong augmentation and distillation (DeiT) because it lacks the convolutional inductive bias - which is also why it eventually surpasses convnets given enough data.

FROM ZERO — the idea explained step by step

Transformers are also used for images, audio and sensor data.

IN OUR PROJECT — not used, and why

On larger devices, sensor Transformers exist for HAR; on a 320 KB board the CNN remains the practical choice.

- **Swin Transformer:** hierarchical windows with shifting - reintroduces locality and multi-scale structure for detection and segmentation.
- **Audio:** Whisper (encoder-decoder over log-mel spectrograms), Conformer (convolution + attention).
- **Multimodal:** CLIP aligns image and text encoders with a contrastive loss; modern vision-language models feed image patch embeddings into an LLM's token stream.
- **Time series and sensor data:** patch-based Transformers (PatchTST) are now competitive with classical methods, though strong linear baselines remain surprisingly hard to beat.

Exercises

1 Implement attention with explicit einsum and verify it matches `scaled_dot_product_attention` to 1e-5. 2 Remove the `sqrt(d_k)` scaling and plot the entropy of the attention weights during the first 200 steps. 3 Train a 4-layer character-level decoder-only Transformer on a small text corpus and sample from it. This is the single best exercise in this book. 4 Visualise attention maps for a trained model and check whether any head has a recognisable role.

24. Large Language Models

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Large language models are the opposite end of the scale from the thesis: billions of parameters versus our 10,214. A few of their tricks -- low-bit weights, fine-tuning only a small part -- are the same ideas the thesis uses on a microcontroller.

An LLM is a decoder-only Transformer trained to predict the next token on a very large text corpus, then adapted to follow instructions. Everything surprising about them - in-context learning, reasoning, code generation - emerges from that one objective at scale.

24.1 Tokenization

- **Byte-pair encoding (BPE)** and its variants (WordPiece, Unigram, SentencePiece) build a vocabulary by repeatedly merging the most frequent adjacent pair, producing subword units. Common words become one token; rare words split into pieces; nothing is ever out-of-vocabulary.

FROM ZERO — the idea explained step by step

Text is cut into **tokens** (pieces of words) and each token gets a number.

IN OUR PROJECT — not used, and why

Not used. Our "tokens" are sensor samples: 128 time steps x 9 channels, already numbers.

- Typical vocabularies are 32k-256k tokens. English averages roughly 0.75 words per token; code and non-Latin scripts are considerably less efficient, which is a real cost and fairness issue.
- Tokenisation explains several famous failure modes: character counting, arithmetic on long numbers, and reversal tasks are hard because the model never sees characters.
- Byte-level fallbacks guarantee coverage of any input; some recent work removes tokenisation entirely in favour of byte or patch-level models.

24.2 Pretraining

Objective: $\text{maximise } \sum_t \log P(x_t | x_{<t}; \theta)$
Loss: **cross-entropy;** **Perplexity** = $\exp(\text{cross-entropy})$

FROM ZERO — the idea explained step by step

An LLM is first trained to predict the next token on huge amounts of text.

IN OUR PROJECT — not used, and why

Not used. Our pre-training is ordinary supervised training on 6,617 labelled windows.

Ingredient	Current practice
Data	Trillions of tokens: filtered web crawl, code, books, scientific text. Quality filtering, deduplication and decontamination matter more than raw volume
Architecture	Decoder-only, pre-RMSNorm, RoPE, SwiGLU feedforward, grouped-query attention, no biases
Optimiser	AdamW, beta2 about 0.95, cosine decay with warmup, gradient clipping at 1.0, weight decay 0.1
Parallelism	Data + tensor + pipeline + sequence parallelism; ZeRO/FSDP sharding of optimiser state
Precision	bf16 compute with fp32 master weights; selective activation checkpointing

Ingredient	Current practice
Context	4k-1M tokens, usually extended after the main run by adjusting RoPE frequencies and training on long documents

SCALING LAWS

Loss falls as a smooth power law in parameters N , data D and compute C - $L(N) = (N_c/N)^a + \text{irreducible}$. The Chinchilla result showed that for a fixed compute budget the optimum is roughly 20 tokens per parameter, and that most earlier models were badly undertrained on data rather than too small. In practice, inference cost now dominates for deployed models, so the industry deliberately overtrains smaller models far past Chinchilla optimality - a 7B model trained on 10-15T tokens is cheaper to serve forever than a 30B model trained to the same loss.

24.3 Post-training: from predictor to assistant

- **Supervised fine-tuning (SFT):** train on curated instruction-response pairs. A few thousand high-quality examples change behaviour dramatically; quality dominates quantity.

FROM ZERO — the idea explained step by step

Post-training turns a next-token predictor into a helpful assistant (instructions, feedback).

IN OUR PROJECT — not used, and why

Not used in the project.

- **RLHF:** collect human preference comparisons, fit a reward model, then optimise the policy with PPO against that reward plus a KL penalty to stay near the SFT model. Powerful and operationally complex.
- **DPO and friends:** skip the explicit reward model - a closed-form loss on preference pairs directly optimises the same objective. Much simpler, now the common choice; variants include IPO, KTO, ORPO and SimPO.
- **RLAIF / Constitutional AI:** use model-generated critiques and preferences against a written set of principles, reducing the human labelling burden.
- **Reasoning training:** reinforcement learning on verifiable outcomes (maths, code tests) trains models to produce long chains of thought before answering, trading inference compute for accuracy.

24.4 Parameter-efficient fine-tuning

LoRA: $W' = W + (\alpha/r) B A$, $A \in \mathbb{R}^{(r \times d)}$, $B \in \mathbb{R}^{(d \times r)}$, $r \ll d$
 Train A and B only; W stays frozen. Merge $B A$ into W at deployment for zero added latency.

FROM ZERO — the idea explained step by step

Parameter-efficient fine-tuning (e.g. LoRA) changes only a small added part of a big model and freezes the rest, so fine-tuning needs little memory.

IN OUR PROJECT — partly used

The same idea appears in on-device training: Lin et al. update only part of the layers to fit 256 KB [5]. The thesis updates all 10,214 parameters but keeps only 10-20% of them active (sparsity) and stores them in 8 bits.

Method	Trainable share	Notes
Full fine-tune	100%	Best quality, needs ~16 bytes/param of optimiser + gradient memory
LoRA	0.1-1%	The default; $r = 8-64$, applied to attention and often MLP projections

Method	Trainable share	Notes
QLoRA	0.1-1%	Base model quantised to 4-bit NF4, LoRA adapters in bf16 - fine-tunes a 70B model on a single 48 GB GPU
DoRA / rsLoRA	0.1-1%	Refinements on the LoRA parameterisation
Prefix / prompt tuning	<0.1%	Learn virtual tokens; weaker but extremely light
Adapters	1-5%	Small bottleneck modules inserted per layer; add inference latency unless merged

24.5 Inference: what actually costs money

- Generation is **memory-bandwidth bound**, not compute bound: each token requires reading all weights. This is why quantization (Chapter 28) speeds up generation almost proportionally to the bit width.

FROM ZERO — the idea explained step by step

Serving an LLM is limited by memory bandwidth: every weight must be read for every token.

IN OUR PROJECT — not used, and why

The same limit exists on our board at a tiny scale: every forward pass reads all weights from memory. With 8-bit weights that is 10,214 bytes instead of 40,856.

WORKED ON OUR PROJECT — Why an LLM cannot run where this project runs

1. A 1-billion-parameter language model in int8 needs 1 GB just for its weights. The STM32F746 has 1 MB of flash and 320 KB of RAM: a factor of about 1,000 in flash and 3,000 in RAM [8].
2. The project's model is 10,214 parameters = 10 KB in int8 -- 100,000 times smaller than that LLM.
3. The lesson of Chapter 24 that does transfer is parameter-efficient fine-tuning: update only a small part of a model. On the board, updating only the last layer means 390 parameters instead of 10,214 [5].

- **KV cache** size = $2 * \text{layers} * \text{heads_kv} * \text{head_dim} * \text{seq_len} * \text{batch} * \text{bytes}$. For long contexts it exceeds the weights themselves; grouped-query attention, paged attention (vLLM) and cache quantisation are the standard mitigations.
- **Speculative decoding**: a small draft model proposes k tokens, the large model verifies them in one forward pass - a 2-3x speedup with identical output distribution.
- **Batching** (continuous/in-flight batching) is what makes serving economical: it converts a bandwidth-bound workload into a compute-bound one.
- **Sampling controls**: temperature, top-k, top-p (nucleus), repetition and presence penalties. Temperature 0 is deterministic-greedy; higher temperature increases diversity and error rate together.

24.6 What 'next token prediction' actually looks like

The training objective is unglamorous: given a prefix, predict the distribution over the next token. Every capability people find surprising is a side effect of doing this extremely well on a very large corpus.

FROM ZERO — the idea explained step by step

What next-token prediction looks like in practice.

IN OUR PROJECT — not used, and why

Not used in the project.

```

text:      'The capital of France is Paris'
tokens:    [The] [ capital] [ of] [ France] [ is] [ Paris]
training example 1: The                -> ' capital'
training example 2: The capital          -> ' of'
training example 3: The capital of       -> ' France'
training example 4: The capital of France -> ' is'
training example 5: The capital of France is -> ' Paris'
All five are computed in ONE forward pass, thanks to causal masking.

```

Figure 24.1 - One six-token sentence yields five supervised examples.

To predict 'Paris' reliably, a model must store a fact. To predict the closing bracket of a nested expression it must track state. To predict the next line of a proof it must do something that looks like reasoning. None of these were trained for directly; they are what minimising prediction error on a large enough corpus requires.

Loss value	Perplexity	Interpretation
11.0	~60,000	Untrained - uniform over the vocabulary
6.9	~1,000	Learned token frequencies only
4.0	~55	Basic grammar and local coherence
3.0	~20	Fluent text, weak factuality
2.0	~7.4	Strong modern model on general text
1.5	~4.5	Approaching the noise floor of natural text

Perplexity is just $\exp(\text{cross-entropy})$ and reads as 'the model is as uncertain as if it were choosing uniformly among this many tokens'. Watch it during any language-model training run: it should start near the vocabulary size and fall fast.

24.7 The KV cache, sized with real numbers

During generation the model re-reads every previous token's keys and values. Caching them avoids recomputation, but the cache is large:

```

bytes = 2 (K and V) * layers * kv_heads * head_dim * seq_len
        * batch * bytes_per_value
7B-class model, 32 layers, 32 heads, head_dim 128, 8k context, fp16:
2 * 32 * 32 * 128 * 8192 * 1 * 2 = 4.3 GB

```

FROM ZERO — the idea explained step by step

The KV cache stores past attention values so they are not recomputed.

IN OUR PROJECT — not used, and why

Not used. The analogue in our training is the stored forward activations needed for the backward pass.

```

Same model with grouped-query attention, 8 KV heads:
2 * 32 * 8 * 128 * 8192 * 1 * 2 = 1.07 GB    (4x smaller)

```

Compare that with the weights themselves: 7B parameters in fp16 is 14 GB. So a single 8k-context conversation adds nearly a third again on top of the model - and it scales linearly with both context length and batch size, which is why serving many users at long context is a memory problem before it is a compute problem. Grouped-query attention, paged attention and 8-bit KV quantization each attack this directly.

24.8 Prompting, from worst to best

Version	Prompt	Why it behaves better
Bad	'Summarise this.'	No audience, no length, no format - the model guesses all three
Better	'Summarise the text below in 3 bullet points for a non-technical manager.'	Audience, length and format are now constraints rather than guesses
Good	Add: 'Use only information from the text. If a fact is not stated, omit it.'	Reduces fabrication by making abstention an explicit option
Best	Add: 'Return JSON: {summary: string[], omitted_topics: string[]}' plus 2 examples	Machine-parseable, and the examples pin down the style far more precisely than adjectives can

FROM ZERO — the idea explained step by step

How to write good prompts.

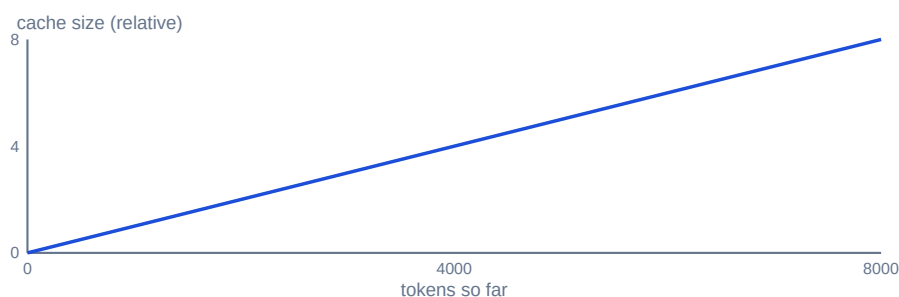
IN OUR PROJECT — not used, and why

Not used in the project.

THE TWO HIGHEST-VALUE PROMPTING HABITS

First, give the model somewhere to put its uncertainty - an 'unknown' field, permission to say the text does not answer the question, a confidence score. A model with no legitimate way to express doubt will invent an answer. Second, show rather than describe: two examples of the exact output you want beat two paragraphs describing it, because the examples constrain format, tone and depth simultaneously.

24.9 Using LLMs well



Graph 24.1 — The KV cache grows with every token generated: for each earlier token the model keeps its keys and values so it need not recompute them. Memory rises in a straight line with the length of the conversation -- the main cost of long contexts (illustrative, per 1,000 tokens).

- **Prompting:** state the role, the task, the constraints, and the output format explicitly; give 2-5 examples for a format-sensitive task; ask for reasoning before the answer when the task needs it; request structured output (JSON schema) when a program will parse it.

FROM ZERO — the idea explained step by step

How to use LLMs well in practice.

IN OUR PROJECT — not used, and why

Not used in the project.

- **Retrieval-augmented generation (RAG):** chunk documents, embed them, retrieve the top-k by vector similarity (plus keyword search - hybrid retrieval beats either alone), rerank, and put the evidence in the prompt with citations. This is the standard way to give a model private, current, verifiable knowledge without training.

- **Tool use / agents:** let the model call functions - search, a calculator, a database, code execution - and loop on the results. Reliability comes from constraining the action space and validating every result, not from longer prompts.
- **Evaluation:** build a fixed test set of real inputs with expected properties; use exact checks where possible, an LLM judge with a rubric where not, and always keep a human-reviewed sample. Track regressions like any other software test.

THE FAILURE MODES TO DESIGN AROUND

Hallucination (confident fabrication) - mitigate with retrieval, citations and abstention instructions. Prompt injection - never let untrusted text carry authority; separate data from instructions and constrain tools. Context-length degradation - relevant facts placed in the middle of a long context are recalled least well. Non-determinism - even at temperature 0, batching and kernel choice can change outputs. Data contamination - benchmark scores may reflect memorised test sets.

Exercises

1 Compute the KV-cache size for a 7B model (32 layers, 32 heads, head_dim 128) at 8k context in fp16, and again with 8 KV heads (GQA). 2 Fine-tune a small open model with LoRA on 500 instruction pairs and measure the change on a held-out set you wrote yourself. 3 Build a minimal RAG pipeline over 200 of your own documents; measure answer accuracy with and without retrieval.

25. Generative Models: Autoencoders, VAEs, GANs and Diffusion

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

One generative model is part of the project: the ECG example trains an autoencoder that learns to rebuild normal heartbeats and flags the ones it cannot rebuild.

A discriminative model learns $P(y|x)$. A generative model learns $P(x)$, or $P(x|condition)$, and can therefore produce new samples. The families below differ in how they make the intractable business of modelling a high-dimensional distribution tractable.

25.1 Autoencoders

$$z = \text{Encoder}(x) \quad x_{\text{hat}} = \text{Decoder}(z) \quad L = ||x - x_{\text{hat}}||^2$$

FROM ZERO — the idea explained step by step

An **autoencoder** has two halves: an **encoder** squeezes the input into a few numbers (the "code"), and a **decoder** tries to rebuild the input from the code. It is trained so that output = input, measured with mean squared error. Because the code is small, the network must learn what is typical of the data.

Example: a heartbeat of 140 samples is squeezed to 16 channels x 7 = 112 numbers and rebuilt to 140 samples.

IN OUR PROJECT — used

Used in the project: examples/ecg_anomaly_ae. Input: one ECG heartbeat of 140 samples (ECG5000 dataset). Encoder: two Conv1d layers with ReLU and pooling down to [16, 7]. Decoder: three Conv1dTransposed layers back to [1, 140]. Loss: MSE (src/loss_functions/MSE.c) [35].

It is trained only on **normal** heartbeats. A new beat that is rebuilt badly (high MSE) does not look like the normal ones -- it is flagged as an anomaly. The threshold is taken from the reconstruction errors of the training normals [35].

WORKED ON OUR PROJECT — An autoencoder for HAR windows, sized

1. Encoder: flatten (9, 128) = 1,152 values -> 64 -> 16; decoder mirrors it. Parameters about $2 \times (1152 \times 64 + 64 \times 16) = 149,504$ -- 15x the classifier.
2. Trained only on normal activities, it reconstructs them well; a window it reconstructs badly (high error) is unusual -- a fall, or a sensor fault. This is anomaly detection without labels [1, Ch. 14].
3. A convolutional autoencoder with the classifier's layers would be much smaller, because convolution shares weights.

A bottleneck forces a compressed representation. A linear autoencoder with squared loss recovers the PCA subspace exactly; non-linear ones learn curved manifolds. Variants: **denoising** (reconstruct a clean input from a corrupted one - the ancestor of both BERT and diffusion), **sparse** (penalise activations), **contractive** (penalise the Jacobian). Plain autoencoders are useful for compression, denoising and anomaly detection, but their latent space has holes - sampling a random z usually decodes to nonsense.

25.2 Variational autoencoders

A VAE makes the latent space a proper probability distribution. The encoder outputs a mean and log-variance; a sample is drawn; the decoder reconstructs. The loss is the **evidence lower bound** :

$$\text{ELBO} = \underbrace{\mathbb{E}_{q(z|x)} [\log p(x|z)]}_{\text{reconstruction}} - \underbrace{\text{KL}(q(z|x) || p(z))}_{\text{regulariser}}$$

FROM ZERO — the idea explained step by step

A **variational** autoencoder makes the code random (a mean and a spread), so that new examples can be generated by picking random codes.

IN OUR PROJECT — not used, and why

Not used. The ECG autoencoder only needs to rebuild, not to generate.

Reparameterisation: $z = \mu + \sigma * \epsilon$, $\epsilon \sim N(0, I)$

WHY THE REPARAMETERISATION TRICK IS NECESSARY

You cannot backpropagate through 'sample from $N(\mu, \sigma)$ ' because the sampling node has no derivative with respect to μ and σ . Writing $z = \mu + \sigma * \epsilon$ moves the randomness into ϵ , which carries no parameters, leaving a deterministic differentiable path to μ and σ . This trick is the reason VAEs train with ordinary gradient descent, and it reappears throughout probabilistic deep learning.

- **Posterior collapse:** with a powerful decoder the KL term can drive $q(z|x)$ to the prior and the latent is ignored. Fix with KL annealing, free bits, or a weaker decoder.
- **beta-VAE** scales the KL term to trade reconstruction quality against disentangled, interpretable latents.
- **VQ-VAE** replaces the continuous latent with a discrete codebook; it underpins modern image and audio tokenisers, which is how images enter and leave a Transformer.
- VAE samples tend to be blurry because the Gaussian likelihood averages over plausible reconstructions.

25.3 Generative adversarial networks

$$\min_G \max_D \mathbb{E}_x [\log D(x)] + \mathbb{E}_z [\log(1 - D(G(z)))]$$

FROM ZERO — the idea explained step by step

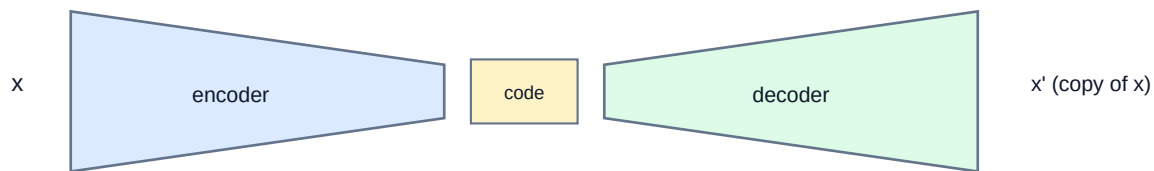
A GAN trains a generator against a discriminator that tries to tell real from fake.

IN OUR PROJECT — not used, and why

Not used in the project.

A generator turns noise into samples; a discriminator tries to tell real from fake; they train against each other. At the theoretical optimum the generator matches the data distribution and the discriminator is at chance everywhere. GANs produce the sharpest images of any family per unit of inference compute, and they are notoriously unstable to train.

Problem	Cause	Standard fixes
Mode collapse	The generator finds a few outputs that fool D and stops exploring	Minibatch discrimination, unrolled GAN, WGAN-GP, diverse conditioning
Vanishing generator gradient	D wins too easily	Non-saturating loss, WGAN with Lipschitz constraint, weaker D
Training oscillation	It is a two-player game, not a minimisation	Two time-scale update rule (different LR), spectral normalisation, EMA of generator weights
No usable likelihood	Implicit model	Evaluate with FID/KID and human study; do not expect a density



Graph 25.1 — An autoencoder squeezes the input through a narrow middle (the code) and has to rebuild it on the other side. What survives the squeeze is the compact description the model learned.

Landmarks: DCGAN (convolutional recipe), WGAN-GP (Wasserstein distance with a gradient penalty), Pix2Pix and CycleGAN (paired and unpaired translation), StyleGAN (style-based generator, still the reference for face synthesis), and SRGAN for super-resolution.

25.4 Diffusion models

Diffusion is now the dominant family for images, audio and video. The idea is disarmingly simple: destroy data with noise in small steps, then learn to reverse each step.

Forward: $q(x_t | x_{(t-1)}) = N(\sqrt{1-b_t} x_{(t-1)}, b_t I)$
Closed form: $x_t = \sqrt{a_{bar}_t} x_0 + \sqrt{1 - a_{bar}_t} * \epsilon$
Training loss (DDPM): $L = E || \epsilon - \epsilon_{\theta}(x_t, t) ||^2$
 i.e. a network that predicts the noise that was added.

FROM ZERO — the idea explained step by step

Diffusion models learn to remove noise step by step, and generate by starting from pure noise.

IN OUR PROJECT — not used, and why

Not used in the project.

```
x0 --noise--> x1 --noise--> ... --noise--> xT ~ N(0, I)      FORWARD
x0 <--denoise-- x1 <--denoise-- ... <--denoise-- xT          REVERSE
    ^ the network eps_theta(x_t, t) predicts the noise at each step
```

Figure 25.1 - Diffusion: a fixed corruption process and a learned reversal.

- **Architecture:** a U-Net with residual blocks, self-attention at low resolutions, and a timestep embedding; increasingly a Transformer (DiT) instead.
- **Latent diffusion** runs the process in the latent space of a VAE rather than in pixels - roughly a 48x reduction in compute, and the reason Stable Diffusion runs on consumer hardware.
- **Conditioning:** cross-attention on text embeddings; **classifier-free guidance** trains with and without the condition and extrapolates at sampling time, $\epsilon = \epsilon_{uncond} + w(\epsilon_{cond} - \epsilon_{uncond})$, trading diversity for prompt fidelity.
- **Samplers:** DDPM needs hundreds of steps; DDIM, DPM-Solver++ and flow matching reduce this to 10-50. Distillation (progressive, consistency, adversarial) reaches 1-4 steps.
- **Flow matching / rectified flow** reframes the same idea as learning a velocity field along straight paths between noise and data - simpler objective, fewer steps, and the current direction of the field.

WHY DIFFUSION BEAT GANS

Diffusion replaces one impossibly hard problem - map noise to a complex distribution in a single shot, judged by an adversary - with a thousand easy, stable regression problems. The training signal is a plain mean-squared error, so there is no minimax game, no mode collapse, and scaling behaves predictably. The cost is inference: many network evaluations per sample, which is exactly what step-distillation research attacks.

25.5 The other families, briefly

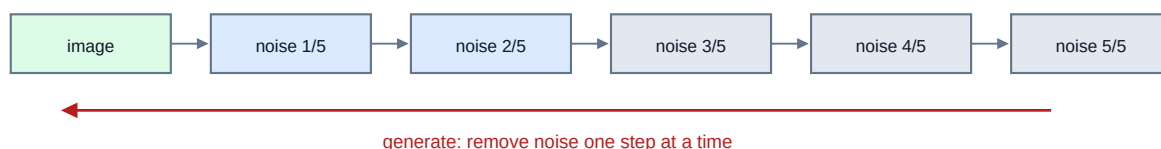
Family	Mechanism	Trade-off
Autoregressive (PixelCNN, LLMs)	Factorise $P(x)$ into a product of conditionals	Exact likelihood, best quality on text; slow sequential sampling
Normalising flows	Invertible transforms with tractable Jacobians	Exact likelihood and fast sampling; architecturally constrained
Energy-based models	Learn an unnormalised energy	Very flexible; sampling requires MCMC
Consistency models	Learn a direct map from any noise level to data	One-to-few step sampling; distilled from diffusion

FROM ZERO — the idea explained step by step

Other families: flows, autoregressive models, energy-based models.

IN OUR PROJECT — partly used

A simple generative model is used in the continual-learning example: PPCA (probabilistic PCA) describes each class with a mean and 8 directions and can generate new windows for replay (PpcaReplay.c) [35].



Graph 25.2 — Diffusion in one line. Training adds noise to real examples step by step until only noise is left (top); the model learns to undo one step. Generating runs it backwards from pure noise (bottom).

25.6 Evaluating generative models

- **FID** compares the mean and covariance of Inception features between real and generated sets - lower is better, but it is sensitive to sample count and preprocessing, and it rewards matching the training distribution rather than quality per se.

FROM ZERO — the idea explained step by step

How to evaluate whether generated data is good.

IN OUR PROJECT — partly used

For the PPCA replay the test is indirect: does replaying generated windows prevent forgetting as well as storing real ones? The project compares it with a 9-window exemplar buffer of the same memory size [35].

- **Precision/recall for generative models** separates fidelity from coverage - useful when FID hides mode dropping.
- **CLIP score** measures prompt adherence for text-to-image.
- **Human evaluation** remains the ground truth; report it with proper sample sizes and blinding.
- For likelihood-based models, report bits-per-dimension; never compare it against a GAN, which has none.

Exercises

1 Train an autoencoder and a VAE on MNIST; interpolate between two latent codes in each and compare the decoded paths. 2 Implement DDPM training on 32x32 images in under 150 lines and sample with both DDPM and DDIM; compare step counts for equal quality. 3 Train a small DCGAN and deliberately induce mode collapse by making the discriminator too strong; then fix it.

26. Graph Neural Networks

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Graph neural networks are not used in the project. The short boxes say why and what is related.

Molecules, road networks, social graphs, program dependency graphs and meshes are not grids or sequences. A GNN generalises convolution to arbitrary graphs by passing messages along edges.

26.1 Message passing

$$\begin{aligned} m_v^{(k)} &= \text{AGGREGATE}(\{ M(h_u^{(k-1)}, h_v^{(k-1)}, e_{uv}) : u \in N(v) \}) \\ h_v^{(k)} &= \text{UPDATE}(h_v^{(k-1)}, m_v^{(k)}) \end{aligned}$$

FROM ZERO — the idea explained step by step

In a graph neural network each node collects "messages" from its neighbours and updates itself.

IN OUR PROJECT — not used, and why

Not used. Our data are regular time series, not graphs; a 1-D convolution is message passing along a chain.

WORKED ON OUR PROJECT — The sensors as a tiny graph

1. Nodes: accelerometer and gyroscope (and, with more devices, wrist, ankle, chest). Edges: which sensors are on the same body part.
2. One message-passing step: each node's feature vector becomes the average of its neighbours' vectors passed through a small shared layer.
3. With only two sensors in one phone, as in UCI HAR [9], a graph adds nothing; it becomes useful for multi-device body networks. The project does not use GNNs.

AGGREGATE must be permutation-invariant: sum, mean, max, or attention.

Model	Aggregation	Character
GCN	Normalised mean: $D^{-1/2} A D^{-1/2} H W$	Simple, strong baseline, spectral motivation
GraphSAGE	Mean/LSTM/max over a sampled neighbourhood	Scales to large graphs by sampling; inductive
GAT	Attention-weighted neighbours	Learns which neighbours matter; multi-head
GIN	Sum with an MLP	Provably as expressive as the Weisfeiler-Lehman test - the most expressive of the simple message-passing schemes
MPNN / SchNet / DimeNet	Physics-aware messages with distances and angles	Molecular property prediction, force fields
Graph Transformer	Full attention plus structural encodings	Avoids over-squashing; needs positional encodings for graphs

26.2 Tasks and readouts

- **Node classification:** predict a label per node (fraud in a transaction graph, role in a social network). Often transductive.

FROM ZERO — the idea explained step by step

Tasks on nodes, edges or whole graphs.

IN OUR PROJECT — not used, and why

Not used in the project.

- **Link prediction:** score whether an edge exists - recommendation, knowledge-graph completion.
- **Graph classification/regression:** pool node embeddings into one vector (sum, mean, max, or a learned pooling) and predict - molecular property prediction is the canonical case.
- **Generation:** produce new graphs, e.g. candidate molecules, often with diffusion over adjacency and node features.

26.3 The characteristic problems

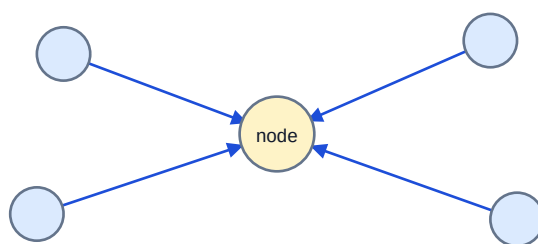
Problem	Description	Mitigation
Over-smoothing	After many layers all node embeddings converge to the same vector	2-4 layers; residual/jumping-knowledge connections; PairNorm; initial-residual (GCNII)
Over-squashing	Information from an exponentially growing neighbourhood is compressed into a fixed vector	Graph rewiring, virtual global nodes, graph Transformers
Scalability	Neighbourhood explosion when sampling k hops	Neighbour sampling (GraphSAGE), cluster-based batching (Cluster-GCN), historical embeddings
Expressiveness limit	Message passing cannot distinguish some non-isomorphic graphs (1-WL bound)	Add structural or positional features, subgraph counts, or higher-order schemes
Heterophily	Connected nodes often have DIFFERENT labels, breaking the smoothing assumption	Separate self and neighbour transformations; signed or higher-order aggregation

FROM ZERO — the idea explained step by step

Typical problems: over-smoothing, limited expressiveness.

IN OUR PROJECT — not used, and why

Not used in the project.



Graph 26.1 — Message passing in a graph network: each node collects messages from its neighbours, combines them with its own state, and updates. After k rounds, a node has heard from everything k steps away.

BEFORE REACHING FOR A GNN

Test a strong tabular baseline with hand-made graph features - degree, neighbour label counts, PageRank, triangle counts, embeddings from node2vec - fed to gradient boosting. On many industrial graph problems this matches or beats a GNN at a fraction of the engineering cost. Use a GNN when the relational structure is genuinely the signal and the features alone are weak.

Exercises

1 Implement a 2-layer GCN in raw PyTorch using a sparse adjacency matrix and train it on Cora. 2 Demonstrate over-smoothing: plot the average pairwise cosine similarity of node embeddings against the number of layers, 1 to 12. 3 Compare a GNN against gradient boosting on hand-crafted graph features for the same node-classification task.

27. Self-Supervised and Transfer Learning

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Self-supervised learning is not used, but transfer learning is the thesis scenario: a model trained on some people keeps learning on the device for a new person.

Labels are expensive; raw data is not. Self-supervised learning invents a supervised task out of unlabelled data, learns a representation from it, and transfers that representation to the task you actually care about with a fraction of the labels.

27.1 Pretext tasks by modality

Modality	Objective	Representative methods
Text	Predict the next token; predict masked tokens; corrupt and reconstruct spans	GPT, BERT, T5, ELECTRA
Images - contrastive	Two augmented views of the same image should match; different images should not	SimCLR, MoCo, CLIP (image-text)
Images - non-contrastive	Match views without negatives, avoiding collapse by architecture	BYOL, SimSiam, DINO, DINOv2
Images - generative	Mask 75% of patches and reconstruct	MAE, BEiT, SimMIM
Audio	Contrastive prediction of masked latent speech units	wav2vec 2.0, HuBERT, BEATS
Video	Predict future frames, ordering, or masked spatio-temporal patches	VideoMAE, V-JEPA
Time series / sensors	Masked reconstruction, contrastive over augmented windows	TS2Vec, TF-C, PatchTST pretraining
Graphs	Mask node or edge attributes; contrast subgraphs	GraphMAE, GRACE

FROM ZERO — the idea explained step by step

Self-supervised learning invents a task from unlabelled data (e.g. predict a hidden part of the signal) so that a network learns useful features without labels.

IN OUR PROJECT — not used, and why

Not used. It would fit the on-device scenario well: a watch records many unlabelled windows but few labelled ones.

WORKED ON OUR PROJECT — Self-supervised pretraining on unlabeled sensor data

1. A phone collects hours of unlabeled signal. Pretext task: take a window, apply two augmentations (jitter, scale), and train the encoder so both views get similar embeddings (contrastive learning).
2. Only a few labeled windows per class are then needed to train the head.
3. The k-means result of Chapter 12 (purity 0.9986 for moving vs still) shows unlabeled structure exists for coarse activities.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

27.2 Contrastive learning in detail

```
InfoNCE: L = -log [ exp(sim(z_i, z_j)/tau) /
                  SUM_k exp(sim(z_i, z_k)/tau) ]
sim = cosine similarity, tau = temperature (0.05-0.2)
```

FROM ZERO — the idea explained step by step

Contrastive learning pulls two views of the same example together and pushes other examples apart.

IN OUR PROJECT — not used, and why

Not used in the project.

- **Augmentation is the whole design.** The representation becomes invariant to exactly the transformations you apply. For SimCLR, random crop plus colour jitter is essential - without colour jitter the network solves the task by matching colour histograms.
- **Negatives matter:** large batches (4k+) or a momentum-updated queue (MoCo) provide enough negatives; otherwise the loss is too easy.
- **Collapse** - all embeddings identical - is the failure mode. Contrastive methods avoid it with negatives; BYOL/SimSiam with a predictor head plus a stop-gradient; DINO with centring and sharpening; Barlow Twins and VICReg with explicit decorrelation terms.
- **Projection head:** contrast in the space AFTER a small MLP, but transfer the representation from BEFORE it - the head discards information useful downstream.

27.3 Transfer learning strategies

Scenario	Strategy
Target task similar, few labels	Freeze the backbone, train a linear probe. Fast, and a strong evaluation of representation quality
Target similar, moderate labels	Fine-tune the last blocks with discriminative learning rates
Target different, many labels	Fine-tune everything, low LR, longer warmup; consider re-initialising the last block
Very few labels (<100/class)	Few-shot with a frozen foundation model plus k-NN or a prototype classifier; or LoRA on a large pretrained model
Domain shift only	Continue self-supervised pretraining on unlabelled target-domain data first (domain-adaptive pretraining), then fine-tune

FROM ZERO — the idea explained step by step

Transfer learning strategies: re-train only the last layer, re-train everything with a small learning rate, or something in between.

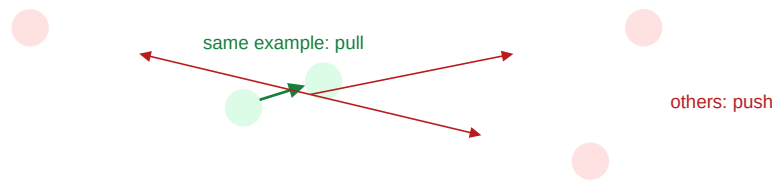
IN OUR PROJECT — used

This is what on-device training does for a new wearer. The C-trained CNN shows the gap it has to close: 0.9551 accuracy on validation windows of the 21 training people, 0.9002 on the 9 new people. Re-training only the last layer would touch 390 of the 10,214 parameters.

Source: the project's C trainer train_c.c (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in rebuild/c_runs.

WORKED ON OUR PROJECT — Fine-tune only the head on the device

1. Pretrain the full CNN on the PC with all 21 training people. On a new user's board, freeze conv1-conv3 and train only the linear layer (390 parameters) [5].
2. Memory for training: the gradients and momentum of 390 parameters are $2 \times 390 \times 4 = 3,120$ bytes instead of $2 \times 40,856 = 81,712$ for the full model.
3. The backward pass stops at the head, so no activation memory of the convolutions is needed for gradients.
4. This is the idea of "sparse update" in [5]: choose which layers to update under a memory budget.



Graph 27.1 — Contrastive learning. Two views of the same example (two crops of one recording) are pulled together in the embedding space; views of different examples are pushed apart. No labels needed.

CATASTROPHIC FORGETTING

Fine-tuning on a narrow task overwrites general capability. Mitigations: lower learning rates, freezing lower layers, replaying a sample of the original data, elastic weight consolidation (penalise movement of parameters important to the old task), or keeping the base model frozen and training adapters/LoRA so the original weights are never touched at all - which is also why adapter-based deployment is operationally attractive.

27.4 Semi-supervised learning

- **Pseudo-labelling:** predict on unlabelled data, keep confident predictions as labels, retrain. Simple and effective; guard against confirmation bias with a high threshold and strong augmentation.

FROM ZERO — the idea explained step by step

Semi-supervised learning combines a few labelled and many unlabelled examples.

IN OUR PROJECT — not used, and why

Not used in the project.

- **Consistency regularisation:** the prediction should not change under augmentation - the core of FixMatch, which combines a weak-augmentation pseudo-label with a strong-augmentation consistency loss and remains a very strong baseline.
- **Mean teacher:** an EMA of the student's weights produces the targets, which stabilises training.
- **Noisy student:** iteratively train a larger student on pseudo-labels with noise (dropout, augmentation), then make it the teacher.

Exercises

1 Train SimCLR on an unlabelled subset of CIFAR-10, then compare a linear probe on 1%, 10% and 100% of the labels against training from scratch. 2 Ablate the augmentations in a contrastive setup one at a time and report linear-probe accuracy - this reproduces the key finding of the SimCLR paper. 3 Take a pretrained sensor or audio encoder and evaluate a k-NN classifier on its frozen embeddings with 5 labelled examples per class.

PART V - Expert Topics

Making models small and fast, training them on the device itself, learning from interaction, trusting the output, and shipping the whole thing to production.

28. Efficient Deep Learning I: Quantization

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

This is one of the two central chapters for the thesis. The project stores the weights as 8-bit (or 4-bit) integers and runs the forward pass in integer arithmetic -- and, unlike the usual case in this chapter, it also **trains** in that form (fully quantised training, FQT [3]). Every section below is shown on the real conv3 layer of the HAR model.

Quantization stores and computes with fewer bits. It is the single highest-leverage efficiency technique available: 8-bit integers cut memory by 4x against float32 and, on hardware with integer units, raise throughput by 2-4x, usually for well under one point of accuracy.

28.1 Why it works and why it pays

- Trained networks are heavily over-parameterised and their weights cluster in a narrow range; the mapping from weights to function is much less precise than float32 suggests.

FROM ZERO — the idea explained step by step

Quantisation stores numbers with fewer bits. A 32-bit decimal number ("float") becomes an 8-bit integer between -127 and 127, together with one shared **scale** that says how much one integer step is worth.

It works because a network does not need 7 decimal digits per weight: small rounding errors behave like a little noise. It pays because 8-bit numbers take 4 times less memory and integer arithmetic is cheaper on a microcontroller [2, 3].

IN OUR PROJECT — used

The thesis quantises the HAR model. 10,214 weights take 40,856 bytes as float32 but 10,214 bytes at 8 bits and about 5,107 bytes at 4 bits [35]. On a board with 320 KB of RAM, this is the difference between fitting and not fitting when optimizer state and activations are added.

- Inference for large models is **memory-bandwidth bound**. Fewer bits per weight means proportionally fewer bytes moved, so latency falls close to linearly with bit width even when the arithmetic is unchanged.
- Integer arithmetic units are smaller and far more energy-efficient than floating-point ones: an int8 multiply-accumulate costs roughly an order of magnitude less energy than an fp32 one, which is decisive on battery power.
- Microcontrollers frequently have no floating-point unit at all - int8 is not an optimisation there, it is the only option.

Format	Bits	Memory for 7B params	Typical use
FP32	32	28 GB	Training master weights, reference accuracy
TF32 / FP16 / BF16	19 / 16 / 16	14 GB	Training compute; BF16 has FP32's exponent range
FP8 (E4M3 / E5M2)	8	7 GB	Training and inference on recent accelerators
INT8	8	7 GB	The standard deployment format; excellent hardware support
INT4 / NF4	4	3.5 GB	LLM weight-only quantization; NF4 is information-theoretically matched to a normal distribution
INT2 / ternary / binary	2 / 1.58 / 1	<= 1.75 GB	Research and extreme edge; needs quantization-aware training from scratch

28.2 The mathematics of affine quantization

Quantize: $q = \text{clamp}(\text{round}(r / s) + z, q_{\min}, q_{\max})$
Dequantize: $r_{\text{hat}} = s * (q - z)$

FROM ZERO — the idea explained step by step

Affine quantisation: $q = \text{round}(x / \text{scale}) + \text{zero_point}$. **Symmetric** quantisation uses $\text{zero_point} = 0$ and $\text{scale} = \max|x| / 127$, so 0 stays exactly 0 and the range is -127..127.

Example: weights with $\max|w| = 0.4$ get scale $0.4 / 127 = 0.0031496$. The weight 0.1 becomes $\text{round}(0.1 / 0.0031496) = \text{round}(31.75) = 32$, which means $32 \times 0.0031496 = 0.10079$ -- an error of 0.00079.

IN OUR PROJECT — used

The project uses **symmetric** quantisation ("SYM" in the code, e.g. `train_c_sym.c`). The scale is the L-infinity norm of the tensor divided by the largest integer -- the norm of Chapter 2 in action [35]. Symmetric is chosen because zero must stay exactly zero: with RigiL, 80-90% of the weights are zero.

FROM OUR C CODE — `src/tensor/TensorConversion.c`, lines 271-300: symmetric quantisation in our library

Line by line. (1) find $\max|x|$; (2) $q_{\max} = 2^{\text{bits}-1} - 1$ (2047 for the 12-bit operands); (3) $\text{scale} = \max|x| / q_{\max}$; (4) every value becomes $\text{round}(x / \text{scale})$, clamped to the integer range. Zero point = 0, so 0.0 is stored exactly as 0.

Example. $x = (0.4, -0.1, 0.0)$ with 8 bits: scale 0.0031496; integers (127, -32, 0).

```
void convertFloatTensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
    size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);

    float absMax = findAbsMaxFloat(inputTensor->data, numberOfElements);

    symInt32QConfig_t *symInt32QC = outputTensor->quantization->qConfig;
    uint8_t qMaxBits = symInt32QC->qMaxBits;

    const float qMax = powf(2, (float)qMaxBits - 1) - 1;
    const float qMin = -powf(2, (float)qMaxBits - 1);

    float scale;
    if (absMax == 0.f) {
        scale = 1.f;
    } else {
        scale = absMax / qMax;
    }

    symInt32QConfig_t *outputSymInt32QC = outputTensor->quantization->qConfig;
    outputSymInt32QC->scale = scale;

    int32_t *outputInt32 = (int32_t *)outputTensor->data;
    float *inputFloat = (float *)inputTensor->data;

    for (size_t i = 0; i < numberOfElements; i++) {
        outputInt32[i] =
            clampInt32(roundByMode(inputFloat[i] / scale, outputSymInt32QC->roundingMode),
                       (int32_t)qMin, (int32_t)qMax);
    }
}
```

$s = (r_{\max} - r_{\min}) / (q_{\max} - q_{\min})$
 $z = q_{\min} - \text{round}(r_{\min} / s)$

scale (a float)
zero-point (an integer)

Symmetric quantization forces $z = 0$ and uses a range centred on zero: $s = \max|r| / (2^{(b-1)} - 1)$. It makes the arithmetic cheaper (no cross-terms with the zero-point) and is the right choice for weights, which are roughly zero-centred. **Asymmetric** quantization keeps z and uses the full range; it is the right choice for activations after ReLU, which are non-negative and would waste half the codes under a symmetric scheme.

WHERE THE ZERO-POINT COMES FROM, AND WHY IT MUST BE EXACT

Real zero must map to an exact integer, because padding, masking and ReLU all produce exact zeros and any error there shows up as a systematic bias across the whole tensor. Solving $r = 0$ in $r = s(q - z)$ gives $q = z$, so the zero-point is precisely the integer code that represents 0.0. This is why the rounding in the definition of z is not optional.

Integer matmul with symmetric weights and asymmetric activations:

```
r_y = SUM_i r_w[i] r_x[i]
      = s_w s_x * SUM_i q_w[i] ( q_x[i] - z_x )
      = s_w s_x * ( SUM_i q_w[i] q_x[i] - z_x SUM_i q_w[i] )
                  \__ int32 accumulation __/   \_ precomputed _/
```

The whole inner loop is integer multiply-accumulate into an int32 accumulator; the float scale is applied once at the end, usually as a fixed-point multiply-and-shift so that no floating-point unit is needed anywhere.

28.3 Granularity and what it costs

Granularity	One scale per	Accuracy	Overhead
Per-tensor	Whole weight tensor	Lowest	Negligible; fastest
Per-channel (per-output)	Output channel / row	Much better - the standard for weights	One scale per channel
Per-group (e.g. 128)	Block of weights within a row	Best for INT4 LLM weights	Scales stored per group; still cheap
Per-token (activations)	Row of the activation matrix	Handles varying dynamic range across tokens	Computed on the fly

FROM ZERO — the idea explained step by step

Granularity: one scale for the whole tensor (per-tensor), or one scale per output channel (per-channel). Per-channel is more accurate but needs more scales and more bookkeeping.

IN OUR PROJECT — used

The project uses one scale per tensor: every weight tensor and every bias tensor has its own scale [35]. This keeps the C kernels simple; the price is that one large weight makes the step coarse for all others in the tensor.

PER-CHANNEL WEIGHT QUANTIZATION IS NEARLY FREE ACCURACY

Different output channels of a convolution or linear layer often have weight ranges that differ by 10-100x. A single per-tensor scale is then set by the widest channel and crushes all the others into a handful of codes. Per-channel scales fix this at the cost of one float per channel, and they are supported by essentially all inference runtimes. Use them by default.

28.4 Post-training quantization (PTQ)

PTQ quantizes a trained model without retraining. It needs only a small **calibration set** - typically 100-1,000 unlabelled samples - to estimate activation ranges.

FROM ZERO — the idea explained step by step

Post-training quantisation (PTQ): train in float, then convert to integers. Simple, but the model never learned to live with the rounding.

IN OUR PROJECT — partly used

The project's SYM trainer starts like this: layers are built in float with Kaiming initialisation and then re-quantised in place to SYM_BITS before training begins (`train_c_sym.c`) [35]. After that, training continues in the quantised form.

- **Dynamic quantization:** weights are quantized offline, activation ranges computed at runtime per batch. Trivial to apply, no calibration data, good for LSTMs and Transformer linear layers where the memory of the weights dominates.
- **Static quantization:** activation ranges are calibrated offline and baked in, so the entire graph runs in integer arithmetic. Faster; needs representative calibration data.
- **Calibration methods:** min-max (simple, outlier-sensitive), percentile (clip at 99.9%), entropy/KL (TensorRT's default, minimises information loss), and MSE-optimal search over clipping thresholds.
- **Cross-layer equalisation:** exploit the positive homogeneity of ReLU to rescale consecutive layers so their per-channel ranges match, improving per-tensor quantization for free.
- **Bias correction:** quantization introduces a systematic shift in the mean activation; measure it on the calibration set and fold the correction into the bias.
- **AdaRound:** learn, per weight, whether to round up or down by minimising the layer output error rather than the weight error. Reliably recovers most of the INT4 gap without labels.
- **GPTQ / AWQ / SmoothQuant** for LLMs: GPTQ solves a layerwise second-order reconstruction problem column by column; AWQ scales salient channels identified by activation magnitude before quantizing; SmoothQuant migrates activation outliers into the weights so both become quantizable.

28.5 Quantization-aware training (QAT)

QAT simulates quantization during training so the network learns weights that are robust to it. Fake-quantize nodes round and clamp in the forward pass while the backward pass uses the straight-through estimator.

```
Forward:  x_q = s * ( clamp( round(x/s) + z, q_min, q_max ) - z )
Backward: dL/dx = dL/dx_q * 1[ q_min <= round(x/s)+z <= q_max ]
          (identity inside the range, zero outside - the STE)
```

FROM ZERO — the idea explained step by step

Quantisation-aware training (QAT) keeps a float copy of the weights and simulates the rounding in the forward pass, so the model learns to cope. **Fully quantised training (FQT)** goes further: there is no float copy -- the weights themselves are stored and updated as integers [3].

IN OUR PROJECT — used

The thesis is on the FQT side, because a float master copy would cost 40,856 bytes and defeat the purpose. In `train_c_sym.c` the weights are stored at SYM_BITS (8 or 4), the forward pass uses integer arithmetic with operands of at most 12 bits, and each update is written back to the integer weights [35].

The consequence is the **dead zone**: if an update is smaller than half a step, ordinary rounding writes back the same integer and the weight never moves. For conv3 (step 0.0031496) and learning rate 0.01 every gradient smaller than 0.1575 in size is lost. The project therefore uses **stochastic rounding** for the write-back by default (`SR_HALF_AWAY` in `src/arithmetic/Rounding.c`): an update of 0.3 steps rounds up with probability 0.3, so on average the weight moves by the right amount [3, 11, 35].

Measured in our C trainer (graph below). Same model, same seed, only the write-back rounding differs. With ordinary rounding the share of weight bytes that change per epoch falls from 73% to about 1.6% -- the weights freeze. With stochastic rounding it stays around 41%. In this single run the final test accuracy is similar (0.8751 with stochastic, 0.8768 with ordinary rounding); the project's 10-seed sweep finds that stochastic rounding recovers 96-97% of the accuracy gap that ordinary rounding leaves [35].

Source: the project's C trainer `train_c_sym.c` (weights stored as 8-bit SYM, SEED 1, 50 epochs, `LOG_CODE_MOVEMENT=1`), once with the default stochastic rounding and once with `SYM_ROUNDING=det`.

FROM OUR C CODE — src/arithmetic/Rounding.c, lines 11-21: stochastic rounding -- the escape from the dead zone

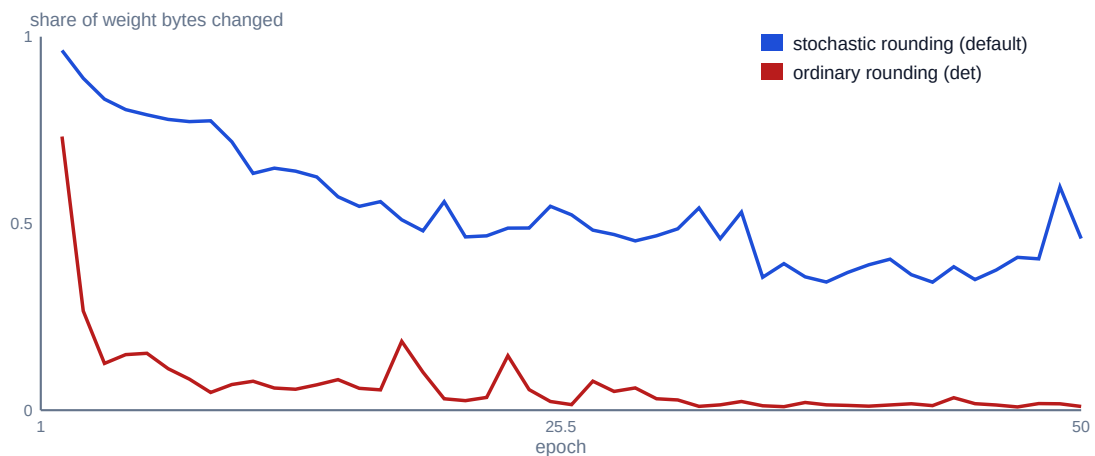
roundHalfAway is ordinary rounding. **roundSRHalfAway** first adds a random number between -0.5 and +0.5, then rounds. A value 0.3 above an integer is rounded up with probability 0.3 and down with probability 0.7 -- on average exactly right.

Example. A weight at integer 32 receives an update worth +0.3 steps: 32.3. Ordinary rounding gives 32 every time -- the update is lost. Stochastic rounding gives 33 in about 3 of 10 steps. If the update of +0.3 steps repeats 10 times, the weight moves by about 3 steps in total -- the same as the sum of the ten updates (10×0.3), which ordinary rounding would have thrown away completely.

```
int32_t roundHalfAway(float input) {
    return round(input);
}

float randfloat() {
    return rngNextFloat();
}

int32_t roundSRHalfAway(const float input) {
    return roundHalfAway(input + randfloat() - 0.5f);
}
```



Graph (from the C run) — The dead zone, **measured in our C trainer** (train_c_sym.c, SYM@8 weights, seed 1, 50 epochs, LOG_CODE_MOVEMENT=1). Share of the stored weight bytes that changed during each epoch. With ordinary rounding (red, SYM_ROUNDING=det) almost all weights freeze: over the last 10 epochs only 1.6% change per epoch. With stochastic rounding (blue, the default) 40.9% still change.

WORKED ON OUR PROJECT — Fully quantised training: the operand and gradient widths

1. FQT keeps weights, activations and gradients in integers during training, not only at inference [3].
2. In the ODT library, operands are quantised to 12 bits and gradients to 16 bits, so that a product of two operands (24 bits) plus a sum of up to 511 products still fits into a 32-bit accumulator [35].
3. Measured in the project (10 seeds): full-SYM wires at 8-bit storage 0.9076 vs float 0.9065 test accuracy -- no degradation, with parameter gradients still in float32 [35].
4. Quantising the gradient path as in [3] is the open research axis of the thesis.

```

import torch, torch.nn as nn
from torch.ao.quantization import QConfig, prepare_qat, convert
from torch.ao.quantization.observer import (MovingAverageMinMaxObserver,
                                             MovingAveragePerChannelMinMaxObserver)

qconfig = QConfig(
    activation=MovingAverageMinMaxObserver.with_args(
        dtype=torch.qint8, qscheme=torch.per_tensor_affine),
    weight=MovingAveragePerChannelMinMaxObserver.with_args(
        dtype=torch.qint8, qscheme=torch.per_channel_symmetric))
model.train()
model.qconfig = qconfig
model_prepared = prepare_qat(model.eval(), inplace=False).train()
# Fine-tune for a few epochs at ~1/10 of the original learning rate.
# Freeze the observers near the end so the ranges stop moving:
# model_prepared.apply(torch.ao.quantization.disable_observer)
# model_prepared.apply(nn.intrinsic.qat.freeze_bn_stats)
model_int8 = convert(model_prepared.eval(), inplace=False)

```

Listing 28.1 - Quantization-aware training in PyTorch. QAT typically recovers most of the INT8 gap and is often the only way to reach INT4.

	PTQ	QAT
Data needed	100-1,000 unlabelled samples	The training set and a training loop
Time	Minutes	Hours to days (a fraction of full training)
INT8 accuracy loss	Typically <1% on CNNs; larger on compact models	Usually within noise of the float model
INT4 accuracy	Needs GPTQ/AWQ-class methods; noticeable loss	The reliable route
When to use	Always try first	When PTQ is not good enough, or below 8 bits

28.6 Quantizing eight real weights, by hand

Take these eight trained weights and quantize them to symmetric INT8 per-tensor. Every number below is computed, not illustrative.

```
w = [-0.82, -0.31, 0.05, 0.11, 0.47, 1.23, -1.05, 0.63]
```

FROM ZERO — the idea explained step by step

Eight real weights quantised by hand, with the error of each.

IN OUR PROJECT — used

Same procedure as in the project: $\text{scale} = \max|w| / 127$, $q = \text{round}(w / \text{scale})$, $\text{error} = w - q \times \text{scale}$. The largest error is always at most half a step (0.0015748 for our conv3 example).

WORKED ON OUR PROJECT — Quantising conv3 to int8, symmetric

1. Symmetric scheme: $q = \text{clamp}(\text{round}(w / s), -127, 127)$, $w_{\text{hat}} = s \cdot q$, with $s = \max|w| / 127$ [10].
2. For a weight range of ± 0.4 : $s = 0.4 / 127 = 0.0031496$. The weight 0.0378 becomes $q = 12$ exactly; -0.2 becomes -64 and reads back as -0.2016 (error $0.0016 < s/2$) [35].
3. Memory: the whole model shrinks from 40,856 to 10,214 bytes.
4. Zero is exactly representable ($q = 0$), which matters for pruned weights: a masked weight stays exactly 0 [35].

```

absmax = 1.23
scale s = absmax / 127 = 1.23 / 127 = 0.009685
q_i = round(w_i / s), clipped to [-127, 127]

```

Original w	w / s	Quantized q	Dequantized q*s	Error
-0.82	-84.67	-85	-0.8232	-0.0032
-0.31	-32.01	-32	-0.3099	+0.0001
0.05	5.16	5	0.0484	-0.0016
0.11	11.36	11	0.1065	-0.0035
0.47	48.53	49	0.4746	+0.0046
1.23	127.00	127	1.2300	0.0000
-1.05	-108.42	-108	-1.0460	+0.0040
0.63	65.05	65	0.6295	-0.0005

Max absolute error = 0.0046 RMSE = 0.0028
 Storage: 8 floats (32 B) -> 8 int8 (8 B) + 1 float scale (4 B) = 12 B

Two observations that generalise. The largest-magnitude weight is represented exactly, because it defines the scale - so the widest weight in a tensor gets perfect treatment while everything else is rounded, which is precisely why a single outlier ruins per-tensor quantization for the other 4,095 weights in its row. And the error is bounded by half a step, $s/2 = 0.0048$, uniformly across the range: quantization error is roughly uniform noise, not proportional error.

28.7 What happens as the bits come off

Bit width	Levels	Step size s	RMSE on the weights above	Rule of thumb
8-bit	255	0.0097	0.0028	Essentially free; PTQ suffices
4-bit	15	0.176	0.0507	18x worse; needs per-group scales or GPTQ/AWQ
2-bit	3	1.23	0.334	120x worse; needs QAT and usually still hurts

FROM ZERO — the idea explained step by step

With fewer bits the steps get coarser: 8 bits give 255 levels, 4 bits only 15 (-7..7). Errors grow and at some point accuracy drops.

IN OUR PROJECT — used

Measured over 10 seeds on HAR: float 0.8953 +/- 0.0050; 8-bit 0.9026 +/- 0.0074 (no loss at all); 4-bit 0.8761 +/- 0.0210 (2 points lower and four times more spread); 4-bit with a cosine learning rate 0.8928 +/- 0.0114 [35]. At 4 bits the conv3 step with $\max|w| = 0.4$ would be $0.4 / 7 = 0.057$ -- 18 times coarser than at 8 bits.

WORKED ON OUR PROJECT — What happens as the bits come off

1. README sweep (10 seeds): sym8 0.9026 +/- 0.0074, sym4 0.8761 +/- 0.0210 [35].
2. Going from 8 to 4 bits multiplies the step s by $127 / 7 = 18$, so many more updates fall into the dead zone and accuracy drops by 2.7 points with three times the spread.
3. A cosine schedule with stochastic rounding recovers most of it at 4 bits: 0.8928 +/- 0.0114 [11, 35].

Error grows roughly as $2^{(-b)}$, so each bit removed doubles the noise. That is the whole trade-off curve, and it explains why INT8 is nearly free while INT4 needs help and INT2 needs a rethink of the training itself.

28.8 Asymmetric quantization for activations, worked

Post-ReLU activations are non-negative, so a symmetric scheme wastes half its codes on values that never occur. With a calibrated range of $[0, 4.0]$ into uint8:

$$s = (4.0 - 0.0) / 255 = 0.015686 \quad z = 0 \quad (\text{since } r_{\min} = 0)$$

FROM ZERO — the idea explained step by step

Asymmetric quantisation (with a zero point) fits ranges that are not centred on 0, such as ReLU outputs (always ≥ 0).

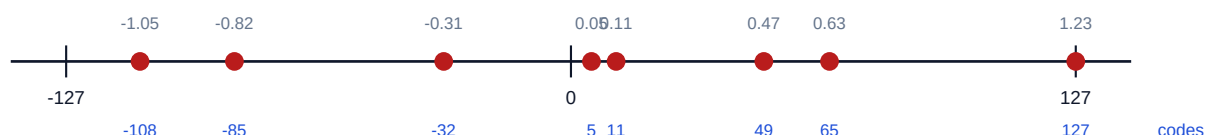
IN OUR PROJECT — partly used

The project uses symmetric quantisation everywhere; for ReLU outputs half of the integer range (the negative half) is then unused. The library also has an asymmetric accumulate primitive (`accumulateFloatIntoAsymTensorRescale`), but the HAR SYM trainer does not use it [35].

```
a = 0.0  -> q = 0    -> 0.0000
a = 0.4  -> q = 26   -> 0.4078
a = 1.2  -> q = 76   -> 1.1922
a = 2.8  -> q = 178  -> 2.7922
a = 3.9  -> q = 249  -> 3.9059
```

WHY EXACT ZERO MUST MAP TO AN EXACT CODE

Here $z = 0$, so the real value 0.0 maps to the integer 0 with no error at all. That is essential: padding, masking and ReLU all produce exact zeros in bulk, and if 0.0 quantized to, say, 0.008 instead, every padded position in every sequence would contribute a small systematic bias that accumulates across layers. The zero-point exists to make this impossible.



Graph 28.1 — The eight weights of 28.6 on the INT8 grid. The scale is $1.23 / 127 = 0.009685$, so the biggest weight, 1.23, lands exactly on 127 and every other weight is rounded to the nearest step. Worst error 0.0046, just under half a step.



Graph 28.2 — Error against bit width, from the book's table in 28.7 for the same eight weights: RMSE 0.0028 at 8 bits, 0.0507 at 4 bits, 0.334 at 2 bits. Each bit removed roughly doubles the noise.

28.9 The whole INT8 layer, arithmetic included

Real: $y = \sum_i w_i x_i$
 Quantized: $w_i = s_w q_{w_i}, \quad x_i = s_x (q_{x_i} - z_x)$

FROM ZERO — the idea explained step by step

An integer layer: multiply int8 x int8, add up in a 32-bit accumulator, then rescale the sum back to 8 bits with the combined scale (input scale x weight scale / output scale).

IN OUR PROJECT — used

This is the forward pass of the SYM trainer: operands of at most 12 bits, products added in int32 (ARITH_SYM_INT32), then rescaled. The operand limit exists because two 16-bit numbers multiplied and summed can overflow 32 bits (docs/conventions/arithmetic-sym.md) [35].

```
y = s_w s_x [ SUM_i q_w_i q_x_i - z_x SUM_i q_w_i ]
              \__ int8 x int8 -> int32 __/ \__ constant __/
```

The bracketed sum is pure integer arithmetic. The second term depends only on the weights, so it is precomputed once and folded into the bias. The float scales multiply the int32 accumulator once at the end - and even that is usually done as a fixed-point multiply-and-shift, so a device with no floating-point unit can run the layer end to end.

This is the reason INT8 inference is fast rather than merely small: the inner loop is integer multiply-accumulate, which is what DSPs, NPU's and microcontroller SIMD units are built for, and which costs roughly an order of magnitude less energy per operation than the float equivalent.

28.10 What breaks, and how to fix it

Failure	Cause	Remedy
Large drop on a depthwise-separable model	Depthwise layers have very different per-channel ranges	Per-channel weights; QAT; cross-layer equalisation
LLM collapses below 8 bits	A few activation channels have outliers 100x larger than the rest	SmoothQuant, AWQ, keeping outlier channels in fp16 (LLM.int8), group-wise scales
Accuracy fine offline, poor on device	The runtime fuses or reorders ops differently, or uses different rounding	Evaluate with the actual runtime, not the simulation
BatchNorm statistics wrong after quantization	BN folded into the conv with float statistics that no longer match	Fold BN before calibration; freeze BN statistics during QAT
Softmax/LayerNorm degrade	Narrow dynamic range in normalisation	Keep sensitive layers in higher precision - mixed precision by sensitivity

FROM ZERO — the idea explained step by step

What breaks: outliers stretch the scale, small updates vanish, accumulated errors grow.

IN OUR PROJECT — used

All three are documented in the project: (1) one large weight makes the per-tensor step coarse; (2) the dead zone, fixed with stochastic rounding; (3) quantised gradient accumulation adds noise that grows with the number of micro-batches -- recorded as an open problem in arithmetic-sym.md [35].

Point (2) measured: see the graph in Section 28.5 (1.6% vs 40.9% of weight bytes still changing).

SENSITIVITY ANALYSIS: THE WORKFLOW THAT ACTUALLY FINDS THE RIGHT CONFIGURATION

Quantize one layer at a time, keeping everything else in float, and record the accuracy drop. This produces a sensitivity ranking, usually showing that the first layer, the last layer and a handful of normalisation-adjacent layers account for most of the damage. Keep those in 8 or 16 bits and push the rest to 4. Automating this - solving for a bit-width assignment under a size or latency budget - is the core of mixed-precision quantization methods such as HAWQ, which use Hessian traces as the sensitivity proxy.

28.11 Reporting quantization results honestly

FROM ZERO — the idea explained step by step

Report quantised results against a float baseline, with seeds, spreads and memory.

IN OUR PROJECT — used

The thesis sweeps do exactly this: 10 seeds per configuration, mean +/- standard deviation, and a memory report from the same run (compare_memory.py) [35].

A complete quantization report

Baseline float accuracy on the same evaluation code and data.

IN OUR PROJECT — A complete quantization report

Project version: float 0.8953 +/- 0.0050 | 8-bit 0.9026 +/- 0.0074 | 4-bit 0.8761 +/- 0.0210 | 4-bit + cosine 0.8928 +/- 0.0114; weights 40,856 / 10,214 / ~5,107 bytes [35].

Bit widths per tensor type (weights, activations, accumulator), and the granularity used for each.

Which layers, if any, were kept in higher precision.

Calibration set size and selection method.

Measured model size on disk and peak RAM at inference, not just the theoretical figure.



Graph 28.3 — Asymmetric uint8 for a ReLU output in $[0, 4]$ (28.8): scale $4 / 255 = 0.0157$ and zero point 0, so real 0.0 is code 0 exactly. The book's five examples: 0 -> 0, 0.4 -> 26, 1.2 -> 76, 2.8 -> 178, 3.9 -> 249.

Latency and energy measured ON THE TARGET DEVICE with the target runtime, over many runs, reporting median and tail.

Accuracy per class or per slice - quantization damage is often concentrated in rare classes.

Exercises

1 Implement affine quantize/dequantize in NumPy and measure the reconstruction error of a trained weight tensor at 8, 6, 4 and 2 bits, per-tensor versus per-channel. 2 Apply dynamic and static PTQ to the same CNN and compare accuracy, size and latency; then run QAT and compare again. 3 Take a small Transformer, plot the per-channel maximum activation magnitude for each layer, and identify the outlier channels that make naive INT8 fail.

29. Efficient Deep Learning II: Pruning and Sparsity

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The second central chapter. The thesis keeps 80-90% of the weights at exactly zero and lets RigL move the remaining connections during training [4]. The numbers below are those of the HAR model.

Pruning removes parameters, channels or whole blocks from a network. Trained networks are massively redundant - 80-95% of the weights in a typical over-parameterised model can be removed with careful procedure and little or no accuracy loss. The difficulty is not deciding what to remove; it is turning removal into actual speed.

29.1 Structured versus unstructured

UNSTRUCTURED	STRUCTURED (channel)
$W = \begin{bmatrix} 0 & a & 0 & b \\ c & 0 & 0 & d \\ 0 & e & f & 0 \end{bmatrix}$	$W = \begin{bmatrix} a & b & 0 & 0 \\ c & d & 0 & 0 \\ e & f & 0 & 0 \end{bmatrix}$
scattered zeros	whole columns removed
90% sparse, same runtime unless the kernel exploits it	25% smaller AND 25% faster on any hardware

Figure 29.1 - Sparsity that a matrix multiply can exploit versus sparsity that it cannot.

FROM ZERO — the idea explained step by step

Pruning removes weights. **Unstructured**: individual weights anywhere become zero (fine-grained, needs a mask). **Structured**: whole filters or channels are removed (the layer simply gets smaller).

IN OUR PROJECT — used

The thesis uses **unstructured** sparsity: each weight has a 1-bit mask entry saying active or not. For our 10,096 weights (10,214 parameters minus 118 biases) the masks take about 1,262 bytes [4, 35].

Type	Granularity	Compression	Real speedup
Unstructured	Individual weights	Very high (90-99%)	Only with sparse kernels or a sparse accelerator; often none on a dense GPU
Semi-structured (2:4)	2 non-zeros in every group of 4	2x on weights	Yes - up to about 1.5-2x on NVIDIA Ampere and later sparse tensor cores
Channel / filter	Whole output channels	Moderate (30-70%)	Yes, on all hardware - the network is simply smaller
Block / head	Attention heads, FFN blocks, layers	Coarse	Yes; the cleanest option for Transformers
Layer dropping	Entire residual blocks	Coarse	Yes; large latency wins, larger accuracy risk

CHOOSE THE GRANULARITY YOUR HARDWARE CAN CASH IN

The most common mistake in pruning is reporting '95% sparsity' with no latency change. If your deployment target is a dense CPU or GPU kernel, prune channels or heads. If it is an Ampere-class GPU, use 2:4 semi-structured sparsity. Use unstructured pruning only when your runtime has genuine sparse kernels, or when you care about model SIZE (compressed storage, over-the-air update) rather than speed.

29.2 What to prune: saliency criteria

Criterion	Score	Comment
Magnitude	$ w $	The strongest simple baseline; still competitive with everything else
L1/L2 norm of a filter	$\ W_c\ $	The channel analogue of magnitude
Gradient-based (SNIP)	$ w * dL/dw $	Estimates the loss change from removal; works at initialisation
Taylor / first-order	$ w * g $ summed over a batch	Standard for structured channel pruning
Second-order (OBD/OBS, SparseGPT)	H-weighted importance	Theoretically better; SparseGPT makes it tractable for LLMs
Activation-based (APoZ)	Fraction of zero activations	Cheap channel criterion for ReLU networks
Wanda (LLMs)	$ w * \text{activation} $	Extremely cheap, no retraining, strong for one-shot LLM pruning
Learned gates	L0 / Gumbel gates trained jointly	Optimises structure and weights together; more complex

FROM ZERO — the idea explained step by step

Which weights to remove? The simplest rule: the smallest in size (magnitude). Other rules also use the gradient.

IN OUR PROJECT — used

RigL uses both: it **drops** the active weights with the smallest $|w|$ and **grows** new connections where the gradient $|g|$ is largest -- the absolute values (L1-style sizes) of Chapter 2 [4].

29.3 When to prune: the four schedules

- **One-shot after training:** train, prune, fine-tune. Simple and often sufficient up to moderate sparsity.

FROM ZERO — the idea explained step by step

When to prune: after training, gradually during training, at the start, or **dynamically** -- keep the number of active weights fixed but move them during training.

IN OUR PROJECT — used

RigL is the dynamic kind. Every $\Delta T = 100$ steps it drops a fraction α of the active weights and grows the same number elsewhere; α starts at 0.30 and falls with a cosine schedule to 0 by the end [4]. With 2,060 training steps that is about 20 rewirings.

- **Iterative magnitude pruning:** repeat (prune a little, fine-tune) many times. Consistently the best accuracy at high sparsity, and the most expensive.
- **Gradual sparsity during training** (Zhu-Gupta): raise the sparsity from 0 to the target with a cubic schedule between step t_0 and t_n . Trains once, reaches high sparsity, and is the standard production recipe.
- **Sparse from the start / dynamic sparse training:** RigL, SET and similar methods keep a fixed sparsity budget throughout, periodically dropping the smallest weights and regrowing new connections where gradients are largest. They never require a dense model in memory, which is the property that makes them relevant to on-device training (Chapter 31).

Zhu-Gupta cubic schedule:

$$s_t = s_f + (s_0 - s_f) * (1 - (t - t_0)/(n - t_0))^3$$

RigL step: drop the $|w|$ -smallest fraction f of active weights, grow the same number where $|dL/dw|$ is largest, f decayed cosine-wise over training.

WHY REGROWTH BY GRADIENT MAGNITUDE IS THE RIGHT RULE

A weight that is currently zero has no effect on the loss, but its GRADIENT still says how much the loss would change if it became non-zero. RigL uses exactly that signal to decide where to spend its sparsity budget next, which is why it beats static random or magnitude-only sparsity at the same parameter count. The cost is one dense gradient computation at each update step - cheap if done every few hundred steps, and the reason the update interval is a key hyperparameter.

29.4 The lottery ticket hypothesis

A randomly initialised dense network contains a sparse subnetwork - a 'winning ticket' - that, when trained **from the original initialisation**, matches the full network's accuracy. The practical recipe that demonstrates it is iterative magnitude pruning with weight rewinding (reset the surviving weights to their values at initialisation, or at an early step, rather than keeping the trained values).

FROM ZERO — the idea explained step by step

The **lottery ticket hypothesis**: a big network contains a small sub-network that, trained from the same start, would do as well.

IN OUR PROJECT — partly used

RigL avoids searching for the ticket: it starts sparse (random mask) and lets the mask improve during training -- the full dense network is never needed in memory [4].

WORKED ON OUR PROJECT — Lottery tickets versus RigL

1. The lottery ticket hypothesis finds a good sparse subnetwork by training dense, pruning, and rewinding [17]. That needs the dense model in memory.
2. RigL is sparse from step 1 and changes the mask during training, so it never needs the dense model [4, 6].
3. For a device that cannot hold a dense training state, only the second kind is possible.

- It reframes pruning as **finding a good architecture and initialisation**, not merely as removing fat.
- For large models, rewinding to an early step (a few percent into training) works where rewinding to step 0 does not.
- It does not yet give a cheap way to find the ticket without training the dense model first, which limits its direct practical use - but it motivates sparse-training methods that try to.

29.5 Pruning in practice

```
import torch, torch.nn.utils.prune as prune
# --- global unstructured magnitude pruning across all conv/linear layers
params = [(m, 'weight') for m in model.modules()]
if isinstance(m, (torch.nn.Conv2d, torch.nn.Linear)):
prune.global_unstructured(params, pruning_method=prune.L1Unstructured,
                        amount=0.8) # 80% of weights zeroed
# --- structured: remove whole output channels by L2 norm
for m in model.modules():
    if isinstance(m, torch.nn.Conv2d):
        prune.ln_structured(m, name='weight', amount=0.3, n=2, dim=0)
# --- fine-tune here with a small learning rate (masks stay applied) ---
for m, n in params:
    prune.remove(m, n) # bake the mask into the weights, drop the mask
# NOTE: prune.remove leaves a DENSE tensor containing zeros. To gain
# speed you must either export to a sparse format your runtime supports,
# or physically rebuild the network with smaller layers (structured).
```

FROM ZERO — the idea explained step by step

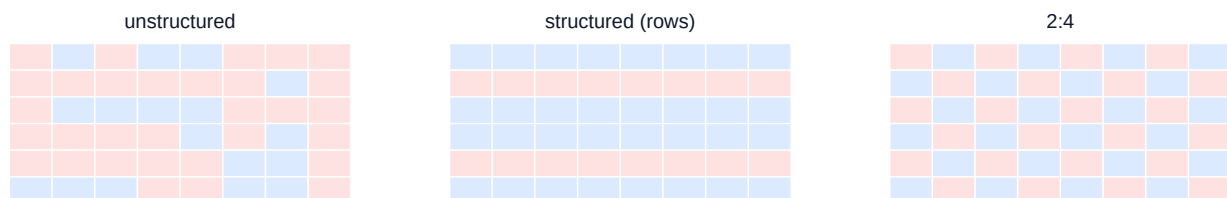
A practical workflow: choose the target sparsity, distribute it over layers, train, check accuracy.

IN OUR PROJECT — used

The thesis distributes sparsity with **ERK** (Erdos-Renyi-Kernel): layers with many weights get sparser, small layers stay denser. At 90% overall the four layers keep 127 / 211 / 394 / 278 active weights = 1,010 in total [4, 35]. The runnable HAR project uses RIGL_SPARSITY 0.80.

WORKED ON OUR PROJECT — ERK distribution of a 90% sparse budget

1. At 90% global sparsity the four layers (10,096 maskable weights) may keep 1,010 [4, 18].
2. ERK density $\sim (\text{fan_in} + \text{fan_out} + \text{kernel}) / n$: conv1 127, conv2 211, conv3 394, linear 278 active weights. Uniform 90% would give 101, 256, 614 and 38 [35].
3. Uniform sparsity would leave the classifier head only 38 weights for 64 x 6 connections; ERK protects small layers.
4. The masks cost 1,262 bytes as packed bits (one bit per weight).



Graph 29.1 — Three kinds of sparsity on a 6 x 8 weight matrix (red = removed). Unstructured: any weight, the most accurate but hard to speed up. Structured: whole rows (channels), easy to speed up. 2:4: two of every four, which some hardware runs twice as fast.

Listing 29.1 - Pruning in PyTorch, including the caveat that most tutorials omit.

Pruning workflow that produces a deployable model

Establish the dense baseline accuracy and latency on the target device.

IN OUR PROJECT — Pruning workflow that produces a deployable model

In the project: ERK mask at the start -> FQT training with rewiring every 100 steps -> 1-bit masks + integer weights on the device [4, 35].

Choose the granularity that your runtime can exploit.

Decide global versus per-layer sparsity. Global allocates the budget automatically but can wipe out a small sensitive layer - always exclude the first and last layers.

Use a gradual schedule with fine-tuning rather than one-shot, if you can afford the training time.

After pruning, physically rebuild or export the compact model, then re-measure latency and memory on the device.

Re-check per-class and per-slice accuracy: pruning damage concentrates in rare classes. Consider combining with quantization - prune first, then quantize, then fine-tune once more.

29.6 Pruning a small layer, weight by weight

Take a 4x4 weight matrix and apply 50% magnitude pruning three ways. The difference between the three is the entire practical content of this chapter.

```
W = [ 0.90  -0.05  0.60  0.02 ]    row norms (L2):
     [ 0.03   0.01 -0.02  0.04 ]        row 1: 1.08
     [-0.70   0.80  0.10 -0.50 ]        row 2: 0.06
     [ 0.20  -0.15  0.05  0.30 ]        row 3: 1.19
                                         row 4: 0.39
```

FROM ZERO — the idea explained step by step

Pruning a small layer by hand: sort the weights by size, zero the smallest.

IN OUR PROJECT — used

RigL does the same at each rewiring on each layer, plus the grow step: the zeroed positions with the largest gradients become active again, starting at weight 0 [4].

WORKED ON OUR PROJECT — One RigL update on conv3

1. Every deltaT = 100 steps RigL drops the k weakest active weights and grows the k inactive weights with the largest gradients [4].
2. $k = \alpha \times 394$ with $\alpha = 0.3 \times 0.5 \times (1 + \cos(\pi \times \text{step} / \text{total}))$. At step 100 of 2,000: $\alpha = 0.298$, $k = 117$.
3. Under int8, many weights share the same magnitude, so several weights can tie at the cut-off (ties); RigL then grows exactly as many as it dropped [35].
4. A grown weight starts at 0. With round-to-nearest it moves only if $|g| \geq 0.1575$ (Chapter 15 example) -- otherwise it is "stillborn". This interaction of RigL and FQT is the thesis question [3, 4, 11].

Scheme	What is removed	Result	Speedup on a dense kernel
Unstructured, 50%	The 8 smallest $ w $ anywhere	Scattered zeros; W keeps its 4x4 shape	None - the kernel still multiplies 16 numbers
2:4 semi-structured	The 2 smallest of every 4 consecutive weights	Exactly 2 non-zeros per group of 4	Up to ~2x on sparse tensor cores
Structured (rows), 50%	The 2 rows with the smallest norm - rows 2 and 4	A genuine 2x4 matrix	2x everywhere, on any hardware

Look at what structured pruning chose. Row 2 has every weight near zero: removing it costs almost nothing, and it removes a whole output channel - the next layer loses an input, the tensor genuinely shrinks, and every runtime on earth is faster as a result. Row 4 is a real sacrifice: its norm is 0.39, so some signal is lost. That is the structured/unstructured trade in miniature - unstructured pruning removes exactly the least useful weights but changes nothing about the computation, while structured pruning removes some useful weights and actually makes the model smaller.

THE REPORTED-SPARSITY TRAP

A paper or a blog post that says '95% of weights removed, 0.3% accuracy lost' and shows no latency measurement has almost certainly measured nothing that a deployment would feel. Ask three questions: which granularity, which runtime, and what was the measured milliseconds-per-inference before and after on the target device. If the answer to the third is missing, the compression is theoretical.

29.7 How far can you actually prune?

Sparsity	Typical accuracy effect (with fine-tuning)	Notes
0-50%	None measurable on an over-parameterised model	Essentially free; the network was carrying redundancy
50-80%	0-1 point	Needs gradual pruning plus fine-tuning
80-95%	1-4 points	Needs iterative pruning; small layers must be protected
>95%	Large and erratic	Only for heavily over-parameterised models; consider a smaller architecture instead

FROM ZERO — the idea explained step by step

How far can one prune? Often 80-90% with little loss on large models; small models are more fragile.

IN OUR PROJECT — used

Our model is small (10,214 parameters), so 90% leaves only about 1,010 weights. The RigL document notes that the spread between different random masks is substantial at this size -- another reason for 10-seed averages [35].

PRUNE THE RIGHT LAYERS

Never prune the first and last layers at the same rate as the middle. The first layer has few parameters but sees the raw input, and the last maps to the classes - damage there is disproportionate, while the savings are trivial because both are small. Global magnitude pruning with no exclusions will happily gut them, which is the most common reason a pruning run collapses.

29.8 Compression stacked, with the numbers

Stage	Size	Latency	Accuracy	What changed
Dense FP32 baseline	14.0 MB	100 ms	94.2%	-
+ 50% channel pruning	7.2 MB	58 ms	93.6%	Genuinely smaller tensors
+ INT8 quantization	1.9 MB	24 ms	92.7%	4x memory, integer kernels
+ distillation from the dense teacher	1.9 MB	24 ms	93.9%	Same model, better weights

FROM ZERO — the idea explained step by step

Compression stacks: pruning x quantisation x encoding.

IN OUR PROJECT — used

For our model: 40,856 bytes (float) -> 10,214 (8-bit) -> about 1,010 active 8-bit weights + 1,262 bytes of masks at 90% sparsity. The masks are then larger than the weights themselves -- a real trade-off for such a small model.

Read the last row carefully: distillation changed nothing about the model's size or speed, and recovered most of the accuracy lost to the first two stages. That is the standard modern recipe - compress aggressively, then use the uncompressed model as a teacher to repair the damage. It is almost always better than compressing less.

29.9 Sparsity in large language models

- **SparseGPT** and **Wanda** prune LLMs in one shot to 50-60% unstructured or 2:4 sparsity using a small calibration set and no retraining.

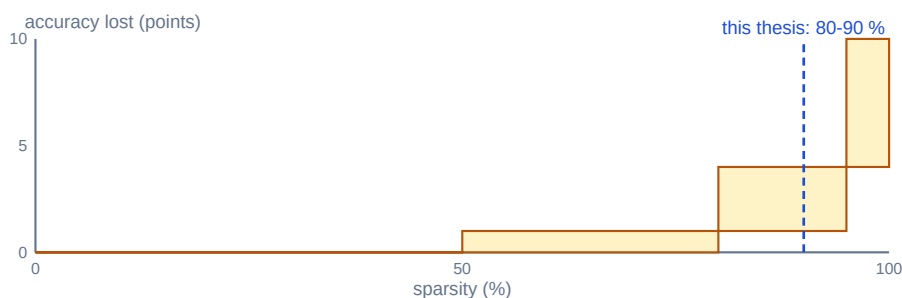
FROM ZERO — the idea explained step by step

Sparsity in large language models.

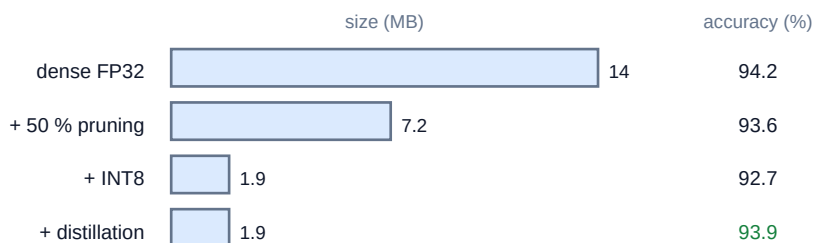
IN OUR PROJECT — not used, and why

Not relevant to the project scale.

- **Structured LLM pruning** (LLM-Pruner, Sheared-Llama) removes attention heads, FFN channels or layers and then continues pretraining briefly to recover - this is what produces genuinely smaller, faster models.
- **Mixture-of-experts** is conditional sparsity by design: only k of N expert FFNs run per token, so parameter count and compute decouple. It is the dominant way large models buy capacity without buying latency, at the cost of memory and routing complexity.
- **Activation sparsity**: ReLU-family models have naturally sparse activations that can be exploited to skip computation at inference (deja-vu-style predictors), and it is a reason some recent models deliberately reintroduce ReLU.



Graph 29.2 — How far pruning can go, from the book's table in 29.7: up to 50 % costs nothing measurable, 50-80 % about 0-1 point, 80-95 % about 1-4 points, beyond 95 % the loss is large and erratic. The shaded band shows the typical range.



Graph 29.3 — Compression stacked, from the book's table in 29.8. Pruning halves the size, INT8 shrinks it four times more, and distillation then wins back most of the accuracy without changing size or speed.

29.10 Combining compression techniques

Order	Rationale
Distill -> prune -> quantize -> fine-tune	Distillation defines a smaller architecture first; pruning trims what remains; quantization is applied last because it is the least forgiving; a final short fine-tune (or QAT) recovers the residual loss
Prune and quantize jointly	Better in principle - the two interact - but harder to tune and to debug
Quantize before pruning	Generally worse: pruning decisions made on quantized weights are noisier

FROM ZERO — the idea explained step by step

Combining techniques: the order and interaction of pruning and quantisation matter.

IN OUR PROJECT — used

The thesis combines FQT and RigL during training. One interaction: RigL grows new weights at value 0, and with symmetric quantisation 0 is exactly representable; the growth then depends on stochastic rounding to leave the dead zone.

Compound compression example (typical, MobileNet-class model):

dense fp32	14.0 MB	100.0%	baseline accuracy
+ 50% channels	7.2 MB	51.4%	-0.6 pt
+ INT8	1.9 MB	13.6%	-0.9 pt
+ distillation	1.9 MB	13.6%	-0.3 pt (recovered by teacher)

Exercises

1 Prune a trained CNN to 50%, 80%, 90%, 95% and 99% with global magnitude pruning and plot accuracy against sparsity, with and without fine-tuning. 2 Implement RigL's drop-and-grow step and compare against static sparse training at the same parameter budget. 3 Structurally prune 30% of channels, rebuild the network with the smaller layers, and measure the actual latency change on your target device. Compare with the latency change from unstructured pruning at the same parameter count.

30. Efficient Deep Learning III: Distillation, NAS and Efficient Design

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Distillation and architecture search are not used, but the HAR model itself is an example of efficient design: small filters, pooling, global averaging instead of a big dense layer.

30.1 Knowledge distillation

A small **student** is trained to imitate a large **teacher**. The teacher's full output distribution carries much more information than a one-hot label - the relative probabilities of the wrong classes encode which categories resemble each other, the 'dark knowledge'.

```
L = (1 - a) * CE( y_true, student )
    + a * T^2 * KL( softmax(z_T / T) || softmax(z_S / T) )
```

FROM ZERO — the idea explained step by step

Knowledge distillation: a small "student" network learns to copy the outputs (probabilities) of a big "teacher" network.

IN OUR PROJECT — not used, and why

Not used. It could pre-train our small model from a larger PC model before deployment.

WORKED ON OUR PROJECT — Distilling the CNN into a smaller student

1. Teacher: the 10,214-parameter CNN. Student: the same design with channels 8/16/32 instead of 16/32/64.
2. Student parameters: conv1 $8 \times 9 \times 7 + 8 = 512$; conv2 $16 \times 8 \times 5 + 16 = 656$; conv3 $32 \times 16 \times 3 + 32 = 1,568$; linear $32 \times 6 + 6 = 198$. Total 2,934 -- 29% of the teacher.
3. The student is trained on the teacher's softened probabilities (temperature $T > 1$) plus the true labels [19].
4. Distillation happens on the PC; the student is what would be deployed and fine-tuned on the board.

T = temperature (2-10); the T^2 factor keeps gradient magnitudes comparable between the two terms.

Variant	What is matched	Notes
Response / logit KD	Output distribution	The classic; simple and strong
Feature / hint KD	Intermediate activations (FitNets)	Needs a projection when widths differ
Attention transfer	Attention maps	Effective for CNNs and Transformers
Relational KD	Pairwise distances between samples in feature space	Transfers structure rather than points
Self-distillation	The model teaches itself (deeper layers teach shallower, or an EMA teacher)	Improves accuracy with no larger teacher
Sequence-level KD	Teacher-generated outputs used as training data	The standard way to make small LLMs; effectively synthetic data
Born-again networks	Student identical in size to the teacher	Often beats the teacher - evidence that the soft targets themselves are the benefit

DISTILLATION IS THE MOST RELIABLE COMPRESSION METHOD

Unlike pruning and quantization, distillation lets you choose the student architecture freely - so you can pick one that is fast on your actual hardware rather than one that is merely smaller. It also composes with everything else: distil into a pruned, quantized student and use the teacher's soft targets during quantization-aware fine-tuning.

30.2 Efficient architecture design

- **Depthwise separable convolutions** (MobileNet) - 8-9x fewer FLOPs per 3x3 layer.

FROM ZERO — the idea explained step by step

Efficient design: small kernels, pooling, depthwise-separable convolutions, global average pooling instead of large dense layers.

IN OUR PROJECT — used

The HAR model follows this: kernels 7/5/3, two max-pools that halve the length, and global averaging that replaces a dense layer of $64 \times 32 \times 6 = 12,288$ weights by one of 384 [35].

- **Inverted residuals with linear bottlenecks** (MobileNetV2) - expand, depthwise, project, with the skip on the narrow tensors to keep memory traffic low.
- **Squeeze-and-excitation** - cheap channel attention, consistently worth its small cost.
- **Group convolution and channel shuffle** (ShuffleNet) - reduce cost while preserving cross-channel mixing.
- **Compound scaling** (EfficientNet) - scale depth, width and resolution together rather than one at a time.
- **Hardware-aware design**: FLOPs are a poor proxy for latency. Memory access, kernel support, and degree of parallelism often dominate; a model with fewer FLOPs can easily be slower. Measure on the device.

30.3 Neural architecture search



Graph 30.1 — Knowledge distillation. A big teacher model's soft probabilities ("mostly walking, a bit walking-upstairs") carry more information than the hard label; a small student learns from both.

Approach	Mechanism	Cost
Reinforcement learning (NASNet)	A controller proposes architectures, reward = validation accuracy	Thousands of GPU-days; historical
Evolutionary	Mutate and select architectures	High but parallelisable
Differentiable (DARTS)	Relax the discrete choice into a weighted mixture and optimise with gradients	A few GPU-days; can collapse to trivial operations
One-shot / supernet (Once-for-All)	Train one weight-sharing supernet, then extract sub-networks per device	Train once, deploy many - the practical modern approach
Zero-cost proxies	Score architectures at initialisation from gradient or Jacobian statistics	Minutes; noisy but useful for filtering

FROM ZERO — the idea explained step by step

Neural architecture search lets an algorithm try many network designs.

IN OUR PROJECT — not used, and why

Not used. The model was designed by hand.

NAS IS RARELY THE RIGHT FIRST MOVE

A well-tuned existing architecture with good training recipes beats a poorly executed NAS almost every time, and NAS results frequently fail to transfer across datasets and hardware. Reach for it when you have a hard, fixed hardware constraint, a stable task, and enough compute to do it properly - typically hardware-aware supernet search against a measured latency table.

30.4 Choosing a compression strategy

Constraint	First move	Then
Model too large for flash/RAM	INT8 quantization (4x)	Structured pruning, then INT4 with QAT
Latency too high	Structured pruning + a hardware-friendly architecture	Quantization; operator fusion; a better runtime
Energy budget	Quantization (integer arithmetic)	Reduce input resolution/sampling rate; early-exit networks
Accuracy must not drop	Distillation into a compact student	QAT rather than PTQ; keep sensitive layers at higher precision
Many target devices	Once-for-All supernet	Extract a sub-network per device from measured latency

FROM ZERO — the idea explained step by step

Choose a compression strategy from the budget: memory, compute, accuracy.

IN OUR PROJECT — used

The thesis budget: 320 KB RAM for everything needed to train. Measured: 209,680 bytes of training state + 27,768 bytes of stack = 237,448 bytes = 72.5% of the RAM [35].

Exercises

1 Distil a ResNet-50 teacher into a ResNet-18 student and compare against training the student from scratch, sweeping T and alpha. 2 Measure FLOPs and on-device latency for MobileNetV2, EfficientNet-B0 and ResNet-18 and show that the FLOPs ranking does not match the latency ranking. 3 Build a small supernet with three width options per layer and extract two sub-networks meeting different latency budgets.

31. On-Device and Federated Training

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

This chapter is the thesis topic itself. Each section below shows where the project stands on it: the memory wall on the STM32F746, the techniques used to climb it, and what is not done (federated learning).

Inference on the edge is now routine. **Training** on the edge is the harder and more interesting problem: it lets a model personalise to its user, adapt to sensor drift, and improve without any data ever leaving the device.

31.1 Why train on the device at all

- **Privacy:** raw sensor data, keystrokes, audio and health signals never leave the hardware. This is often a legal requirement, not a preference.

FROM ZERO — the idea explained step by step

Why train on the device: privacy (data stay on the device), personalisation (each user is different), and no network connection needed.

IN OUR PROJECT — used

For HAR, personalisation is the key reason. The C-trained CNN reaches 0.9551 on windows of people it was trained on but 0.9002 on new people. A watch that keeps learning from its wearer can close that gap without sending movement data anywhere.

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.

- **Personalisation:** a gesture, gait or keyboard model tuned to one person beats a global model by a wide margin.
- **Adaptation:** sensors age, mounting positions change, environments shift. A model that can fine-tune locally survives drift that would otherwise require a recall or an update campaign.
- **Connectivity and cost:** no round trip, no bandwidth bill, no cloud inference cost, and it works offline.

31.2 The resource wall

Inference memory ~ weights + one layer's activations
 Training memory ~ weights + gradients + optimiser state
 + ALL activations kept for the backward pass

FROM ZERO — the idea explained step by step

The **resource wall**: training needs weights + gradients + optimiser state + stored activations, often many times the memory of inference.

IN OUR PROJECT — used

On the STM32F746ZG: 320 KB of RAM, 1 MB flash, 216 MHz. For the HAR model in float the weights (40,856 B) and the SGD momentum state (40,856 B) alone are 80 KB; AdamW would need 81,712 B of state. The full measured training state is 209,680 B + 27,768 B stack = 72.5% of RAM [35].

WORKED ON OUR PROJECT — The memory budget of training on the board

1. Measured for the HAR project: static training state 209,680 bytes + stack peak 27,768 bytes = 237,448 bytes, which is 72.5% of the 327,680 bytes of SRAM; 90,232 bytes remain [35].
2. Weights, gradients and momentum alone are $3 \times 40,856 = 122,568$ bytes in float32.
3. Switching the optimiser to AdamW would add another 40,856 bytes -- the difference between fitting and not fitting with other firmware on the same chip [35].
4. Lin et al. show how to train under 256 KB by combining quantised graphs with sparse layer updates [5]; Deutel et al. train fully quantised on Cortex-M [3].

Adam, fp32: weights 4B + grad 4B + m 4B + v 4B = 16 B / parameter
A 1M-parameter model therefore needs ~16 MB before activations -
more than the total SRAM of most microcontrollers.

Class of device	RAM	Compute	What is feasible
Cloud GPU	40-80 GB	100+ TFLOPs	Anything
Phone / SoC NPU	4-16 GB	1-10 TOPS	Full fine-tuning of small models; LoRA on medium ones
Embedded Linux (Pi-class)	0.5-8 GB	10-100 GFLOPs	Fine-tuning small CNNs; sparse or partial updates
MCU (Cortex-M)	256 KB - 1 MB SRAM	~100 MFLOPs	Last-layer or sparse-subset updates only; int8 arithmetic

31.3 Techniques that make on-device training possible

Technique	What it saves	Cost / caveat
Freeze most layers, train the head	Activations and gradients for frozen layers	Limited adaptation capacity
Bias-only / BitFit updates	Gradients and optimiser state (biases are <1% of parameters)	Surprisingly effective for domain shift
LoRA / adapters	Optimiser state and gradient memory	Still needs backward through the frozen layers
Sparse updates (a subset of channels/layers)	Both memory and compute, tunably	Needs a rule for choosing the subset - this is where RigL-style criteria are used
Quantized training (int8/fp8 forward and backward)	Memory bandwidth and energy	Needs stochastic rounding and careful scaling to avoid gradient underflow
Gradient checkpointing	Activation memory (the dominant term)	About 30% extra compute
Small batch + gradient accumulation	Peak activation memory	More steps; noisier BatchNorm - prefer GroupNorm
SGD or Lion instead of Adam	8 bytes/parameter of optimiser state	May need more careful learning-rate tuning
Forward-only / zeroth-order methods	The entire backward pass	Much slower convergence; viable for tiny adaptation

FROM ZERO — the idea explained step by step

Techniques that break the wall: low-bit numbers, sparse updates, one example at a time, fewer layers trained.

IN OUR PROJECT — used

The project uses three of them: integer weights (FQT [3]), sparsity with RigL [4], and micro-batch 1 with gradient accumulation over 64 windows (train_c.c) [35]. Lin et al. show the fourth, training only part of the layers [5].

SRAM of the STM32F746ZG: 327,680 bytes

training state 209,680	stack 27,768	free 90,232
------------------------	--------------	-------------

Graph 31.1 — The resource wall, with this thesis's numbers. Training the HAR model in float32 needs 209,680 bytes of state plus a 27,768-byte stack; the STM32F746ZG has 327,680 bytes of SRAM. It fits with 90,232 bytes left -- a phone or PC would not notice it.

ACTIVATIONS, NOT WEIGHTS, ARE THE BINDING CONSTRAINT

For a small model, the weights may be a few hundred kilobytes while the activations retained for backpropagation are several megabytes, because they scale with batch size, spatial resolution and depth. The first three things to try are therefore: batch size 1 with accumulation, gradient checkpointing, and freezing the early layers - which removes their stored activations entirely.

```
# Update only the last block and all bias terms - a strong, cheap
# on-device personalisation recipe.
for name, p in model.named_parameters():
    p.requires_grad = name.endswith('.bias') or name.startswith('layer4')
trainable = [p for p in model.parameters() if p.requires_grad]
print('trainable:', sum(p.numel() for p in trainable),
      'of', sum(p.numel() for p in model.parameters()))
opt = torch.optim.SGD(trainable, lr=1e-3, momentum=0.9) # 4 B/param state
# Halve activation memory again at ~30% compute cost:
model.layer4 = torch.utils.checkpoint.checkpoint_wrapper(model.layer4)
```

Listing 31.1 - Partial fine-tuning: the practical basis of on-device personalisation.

31.4 Federated learning

Federated learning trains a shared model across many devices without collecting their data. The canonical algorithm, **FedAvg**, is simple:

FROM ZERO — the idea explained step by step

Federated learning: many devices train locally and send only their weight updates (not their data) to a server, which averages them.

IN OUR PROJECT — not used, and why

Not used. The thesis stays on one device. The continual-learning example is the closest relative: it adapts to 5 groups of people in turn and uses PPCA replay (41,592 B) to avoid forgetting the earlier ones [35].

WORKED ON OUR PROJECT — Federated averaging across phones

1. Each of 10 phones fine-tunes the linear head (390 parameters) on its user's data and sends the new weights.
2. The server averages them, weighted by each phone's number of windows, and sends the average back [20].
3. Upload per round: $390 \times 4 = 1,560$ bytes per phone -- small enough for BLE.
4. Raw sensor windows never leave the phone, which is the privacy argument of Chapter 45.

server		devices
-----		-----
broadcast global weights	----->	1..K selected clients
		each trains E local epochs
aggregate: $w \leftarrow \sum_k (n_k/n) w_k$	<---	each returns its weights
repeat for R rounds		

Figure 31.1 - One round of federated averaging.

Challenge	Why it is hard	Approaches
Non-IID data	Each device sees a biased slice; local models diverge and averaging degrades	FedProx (proximal term), SCAFFOLD (control variates), server momentum, personalised heads
System heterogeneity	Devices differ in speed and availability; stragglers stall a round	Asynchronous or semi-synchronous aggregation, client sampling, deadline-based partial updates
Communication cost	Model updates are large and uplinks are slow	Update quantization and sparsification, top-k updates, low-rank updates, fewer rounds with more local work
Privacy	Weights leak information; gradients can be inverted to reconstruct inputs	Secure aggregation, differential privacy (noise + clipping), trusted execution environments
Evaluation	No central test set	Federated evaluation, held-out clients, careful per-client metrics

Differentially private SGD, per client:

```
clip: g <- g * min(1, C / ||g||)
```

```
noise: g <- g + N(0, sigma^2 C^2 I)
```

```
privacy accounting composes over rounds -> (eps, delta) guarantee
```

WHAT THE PRIVACY GUARANTEE ACTUALLY SAYS

An (eps, delta)-differentially private mechanism guarantees that the output distribution barely changes if any single user's data is removed - so an adversary cannot confidently infer whether you participated. It does NOT guarantee that the model is safe to publish for other reasons, and eps values used in industry (often 5-10) are far weaker than the theoretical ideal of $\epsilon < 1$. Report eps, delta, the clipping norm, the noise multiplier and the unit of privacy (per example or per user) - without all five, a privacy claim is not checkable.

31.5 The deployment stack for edge machine learning

Layer	Options
Training frameworks	PyTorch, JAX, TensorFlow
Export / IR	ONNX, ExecuTorch, TFLite FlatBuffer, TorchScript
Optimisation	Quantization, pruning, operator fusion, constant folding, layout transformation
Runtime	ONNX Runtime, TFLite / LiteRT, ExecuTorch, TVM, CoreML, NNAPI, TensorRT
Bare-metal / MCU	TFLite Micro, CMSIS-NN, microTVM, vendor SDKs
Accelerators	NPU, DSP, GPU, or a fixed-function CNN engine on the SoC

FROM ZERO — the idea explained step by step

The deployment stack: training framework, conversion, runtime, hardware.

IN OUR PROJECT — used

In the project the stack is: PyTorch reference -> the ODT C library (same model, checked for parity) -> the STM32F746 firmware. There is no converter: the C library itself trains [35].

Before shipping a model to a device

Measured latency, peak RAM and energy on the real hardware, not simulated.

IN OUR PROJECT — Before shipping a model to a device

Project checklist in practice: parity with PyTorch passed; memory report under the RAM limit; 10-seed accuracy known for each configuration [35].

Accuracy verified with the target runtime's own arithmetic, not the training framework's.

Thermal behaviour under sustained load checked - sustained throughput can be far below burst throughput.

A fallback path if the accelerator is unavailable or busy.

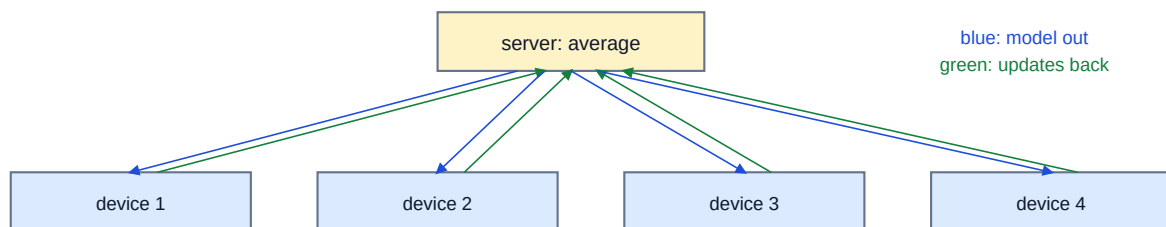
An update mechanism, with model versioning and rollback.

On-device metrics collected (with consent) so drift is detectable.

For on-device training: bounds on how far the local model may drift, and a way to reset to the global model.

Exercises

1 Measure the peak memory of full fine-tuning, head-only training and bias-only training for the same model, and separate the weight, gradient, optimiser and activation components. 2 Implement FedAvg over 20 simulated clients with IID and then pathologically non-IID splits; plot global accuracy against rounds. 3 Add gradient clipping and Gaussian noise to the federated setup and plot the accuracy/privacy trade-off for several noise multipliers.



Graph 31.2 — Federated learning. The server sends the model out; each device trains on its own data, which never leaves it; only the weight updates come back and are averaged into the next model.

32. Reinforcement Learning

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Reinforcement learning is not used in the project; the boxes explain the difference from what we do.

In reinforcement learning an agent learns by acting. There is no dataset of correct answers - only a reward signal that may be sparse, delayed, and affected by everything the agent did earlier.

32.1 The formalism: Markov decision processes

```
MDP = (S, A, P, R, gamma)
Policy    pi(a | s)
Return    G_t = SUM_(k>=0) gamma^k r_(t+k+1)
Value     V^pi(s) = E[ G_t | s_t = s ]
Q-value   Q^pi(s,a) = E[ G_t | s_t = s, a_t = a ]
Advantage A(s,a) = Q(s,a) - V(s)
```

FROM ZERO — the idea explained step by step

In **reinforcement learning** an agent acts in an environment, receives rewards, and learns which actions lead to high total reward. There are no correct labels, only rewards, often delayed.

IN OUR PROJECT — not used, and why

Not used. HAR is supervised: every window has a correct label.

WORKED ON OUR PROJECT — A bandit for choosing the sampling mode

1. The only decision-over-time problem near this project is energy: sample at 50 Hz (accurate, expensive) or 25 Hz (cheap, less accurate).
2. Model it as a two-armed bandit: reward = correct classification - lambda x energy. An epsilon-greedy agent tries both and settles on the better arm [28].
3. This is a simplification of reinforcement learning (no states). The project itself uses supervised learning only.

Bellman optimality: $Q^*(s,a) = E[r + \gamma \max_{a'} Q^*(s', a')]$

The discount factor γ in $[0, 1)$ makes infinite-horizon returns finite and encodes how much the agent cares about the future; 0.99 is a common default, and small changes to it can change behaviour drastically.

32.2 The two families

Family	Learns	Representative algorithms	Character
Value-based	$Q(s,a)$, acts greedily	Q-learning, DQN, Double DQN, Rainbow	Off-policy, sample-efficient, discrete actions
Policy-based	$\pi(a s)$ directly	REINFORCE, TRPO, PPO	Handles continuous actions and stochastic policies; higher variance
Actor-critic	Both	A2C/A3C, DDPG, TD3, SAC	The practical middle ground; SAC and PPO are today's defaults
Model-based	A model of P and R	Dyna, MuZero, Dreamer	Much more sample-efficient; harder to build and to trust

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to reinforcement learning.

IN OUR PROJECT — not used, and why

Not used in the project.

32.3 Q-learning and DQN

Tabular update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to reinforcement learning.

IN OUR PROJECT — not used, and why

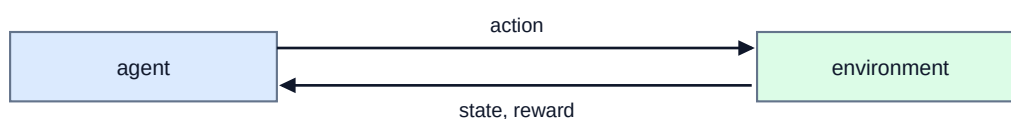
Not used in the project.

DQN loss (neural approximation):

$$L = (r + \gamma \max_{a'} Q_{\text{target}}(s',a') - Q(s,a))^2$$

- **Replay buffer:** store transitions and sample uniformly (or by TD error - prioritised replay). Breaks the correlation between consecutive samples that would otherwise destabilise training.
- **Target network:** a periodically-copied frozen copy of Q supplies the bootstrap target, preventing the network from chasing its own moving predictions.
- **Double DQN:** select the action with the online network and evaluate it with the target network, removing the systematic overestimation of max.
- **Exploration:** epsilon-greedy decayed over training is the baseline; noisy networks and count-based or curiosity bonuses do better on hard-exploration tasks.

32.4 Policy gradients and PPO



Graph 32.1 — Reinforcement learning as a loop: the agent sees a state, picks an action, and the environment answers with a reward and the next state. The agent learns to pick actions with the largest reward over time.

Policy gradient theorem:

$$\text{grad } J = E[\text{grad } \log \pi(a|s) * A(s,a)]$$

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to reinforcement learning.

IN OUR PROJECT — not used, and why

Not used in the project.

PPO clipped objective:

$$L = E[\min(r_t A_t , \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)], \quad r_t = \pi/\pi_{\text{old}}$$

The clipping keeps each update inside a trust region: if the new policy moves too far from the old one on a sample, the objective stops rewarding the move. PPO is the default on-policy algorithm because it is robust, simple to implement, and parallelises well - which is also why it became the workhorse of RLHF for language models.

- **GAE** (generalised advantage estimation) interpolates between high-bias/low-variance TD and low-bias/high-variance Monte Carlo advantage estimates with a parameter lambda around 0.95.
- **SAC** adds an entropy bonus to the objective, producing a stochastic policy that explores by construction; it is the standard for continuous control and is markedly more sample-efficient than PPO.
- **Normalise observations and rewards**, clip the value loss, and anneal the learning rate - RL implementations are notoriously sensitive to these details, and published gains have repeatedly turned out to come from them rather than from the algorithm.

32.5 A gridworld you can solve on paper

Reinforcement learning becomes concrete on a four-state corridor. States S1..S4, actions left and right, reward +1 for reaching S4 and 0 elsewhere, gamma = 0.9.

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to reinforcement learning.

IN OUR PROJECT — not used, and why

Not used in the project.

```
[S1] <---> [S2] <---> [S3] <---> [S4 goal, +1]
Optimal values, working backwards from the goal:
V(S4) = 1.0          (terminal)
V(S3) = 0 + 0.9 * 1.0 = 0.90
V(S2) = 0 + 0.9 * 0.90 = 0.81
V(S1) = 0 + 0.9 * 0.81 = 0.729
```

Figure 32.1 - Value propagates backwards from reward, decaying by gamma per step.

This tiny example shows the three ideas that carry all the way to AlphaZero. Value is **discounted distance to reward**: gamma = 0.9 means a reward three steps away is worth 0.729 now, so the agent prefers shorter paths without being told to. Value **propagates backwards** one step per update, which is why sparse-reward problems need so many episodes - the signal has to crawl back from the goal. And a **policy falls out of the values for free**: in each state, move to the neighbouring state with the higher value.

gamma	Value of a reward 10 steps away	Behaviour
0.5	0.001	Extremely myopic - ignores anything beyond a few steps
0.9	0.35	Balanced; the common default
0.99	0.90	Far-sighted; needed for long tasks, but slows learning and increases variance
1.0	1.00	No discounting - only valid for episodes guaranteed to terminate

32.6 Why reinforcement learning is harder than supervised learning

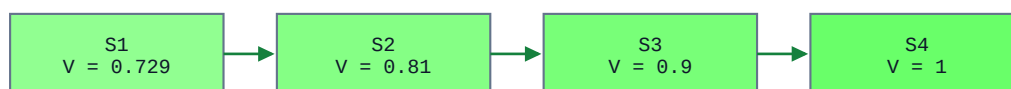
Difficulty	Supervised learning	Reinforcement learning
Where the data comes from	A fixed dataset, independent of the model	The agent's own behaviour - a bad policy collects bad data
Feedback	The correct answer, for every sample	A scalar reward, often delayed by hundreds of steps
Credit assignment	Immediate: this prediction, this loss	Which of the last 200 actions caused the reward?
Stationarity	The target never moves	The target moves - the value function bootstraps from itself
Evaluation	Held-out test set	You must run the policy to know how good it is, and running it may be expensive or unsafe
Failure mode	Overfitting	Collapse, reward hacking, or silently converging to a mediocre policy

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to reinforcement learning.

IN OUR PROJECT — not used, and why

Not used in the project.



Graph 32.2 — The corridor of 32.5: four states, a reward of +1 at S4 and gamma = 0.9. Value flows backwards from the goal, shrinking by 0.9 each step: 1.0, 0.90, 0.81, 0.729. Moving towards the higher value is the best policy.

THE ORDER TO TRY THINGS IN

If you can log what a good decision-maker did, start with behaviour cloning - it is plain supervised learning and it works. If decisions do not affect future state, use a contextual bandit. Only when actions genuinely change the world you observe next is full reinforcement learning the right tool, and even then start with PPO or SAC and a carefully shaped reward rather than an exotic algorithm.

32.7 Where RL is worth it

- Sequential decisions where actions change the future state: robotics, control, inventory, ad auctions, game playing.

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to reinforcement learning.

IN OUR PROJECT — not used, and why

Not used in the project.

- Alignment of language models from preferences (Chapter 24), and training for verifiable outcomes such as passing unit tests.
- Not worth it when a supervised model on logged decisions would do - start with behaviour cloning or contextual bandits, which are far easier to make work and to evaluate.

OFFLINE EVALUATION IS THE HARD PART

You cannot evaluate a new policy from logged data by simply replaying it - the logged actions came from a different policy. Use off-policy evaluation (importance sampling, doubly robust estimators), a simulator you have validated, or a carefully bounded online experiment. Reward misspecification is equally dangerous: an agent optimises exactly what you wrote, not what you meant, and reward hacking is the norm rather than the exception.

Exercises

1 Implement tabular Q-learning on FrozenLake and plot the effect of epsilon decay and gamma. 2 Implement DQN on CartPole in 150 lines with a replay buffer and target network; then remove each of the two and observe the failure. 3 Implement PPO with GAE on a continuous-control task and compare against SAC in sample efficiency.

33. Uncertainty, Robustness and Interpretability

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Uncertainty and robustness matter for a device that meets new people: the test people are, in effect, a shift in the data. Several sections connect directly to the thesis measurements.

33.1 Two kinds of uncertainty

Type	Source	Reducible by more data?	Example
Aleatoric	Noise inherent in the data	No	Two identical X-rays with different outcomes; sensor noise
Epistemic	Ignorance of the right model or parameters	Yes	An input unlike anything in the training set

FROM ZERO — the idea explained step by step

Two kinds of uncertainty: **aleatoric** -- the data themselves are ambiguous (noise); **epistemic** -- the model has not seen enough similar data and could learn more.

IN OUR PROJECT — partly used

Both appear in HAR. SITTING vs STANDING is partly aleatoric: from a phone on the waist the two postures look almost the same (measured gravity values: sitting (0.95, 0.14, 0.15), standing (1.00, -0.16, -0.03)). A new person moving differently is epistemic: the model can learn it -- which is what on-device training is for.

The distinction is operational: high epistemic uncertainty says 'collect data here or abstain'; high aleatoric uncertainty says 'this is as good as it gets - widen the interval'.

33.2 Estimating uncertainty in deep networks

Method	How	Cost	Quality
Softmax probability	Take the max output	Free	Poor - modern networks are overconfident and confident off-distribution
Temperature scaling	One scalar fitted on validation	Free	Fixes calibration in-distribution only
MC dropout	Keep dropout on at inference, average N passes	N forward passes	Cheap approximation to a Bayesian posterior
Deep ensembles	Train 5 models with different seeds	N trainings	The strongest practical baseline, including under shift
Bayesian NN (VI, Laplace, SWAG)	Posterior over weights	Moderate to high	Principled; Laplace approximations are now cheap and practical
Conformal prediction	Calibrate a nonconformity score on held-out data	Negligible	Distribution-free FINITE-SAMPLE coverage guarantee - underused and excellent
Evidential / quantile heads	Predict distribution parameters or quantiles directly	Free	Single-pass; needs the right loss

FROM ZERO — the idea explained step by step

Deep ensembles, Monte Carlo dropout, and similar methods estimate uncertainty.

IN OUR PROJECT — partly used

The thesis reports uncertainty over training runs (10 seeds), not per prediction. Per window, the softmax probability is the only confidence available -- and for our CNN it is informative: right answers average 0.956 confidence, wrong ones 0.794 (Section 13.5).

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.

WORKED ON OUR PROJECT — Calibration of a HAR classifier

1. Softmax regression on the 18 features: accuracy 0.7869, expected calibration error (10 bins) 0.0629 -- its confidence is about 6 points off its real accuracy on average.
2. Calibration matters for the device: a confidence threshold ("only report an activity if $p > 0.8$ ") is only meaningful if p is calibrated.
3. Temperature scaling on the validation set is the usual fix -- but the validation set must come from other people (Chapter 3), or the temperature is tuned on the wrong distribution.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

CONFORMAL PREDICTION IN THREE LINES

Fit any model. On a held-out calibration set compute a nonconformity score for each sample (for classification, $s = 1 - p(\text{true class})$). Take the $\text{ceil}((n+1)(1-\alpha))/n$ empirical quantile q of those scores. At test time, output the SET of classes with $1 - p(\text{class}) \leq q$. That set contains the true label with probability at least $1 - \alpha$, with no assumption about the model or the data distribution beyond exchangeability. It is the cheapest rigorous guarantee available in machine learning.

33.3 Out-of-distribution detection and drift

- **Score-based OOD:** maximum softmax probability, energy score, Mahalanobis distance in feature space, or k-NN distance to the training set. Feature-space methods beat output-space ones.

FROM ZERO — the idea explained step by step

Distribution shift (drift): the data at use time differ from the training data. **Out-of-distribution** detection tries to notice it.

IN OUR PROJECT — used

The test split of HAR is a shift by construction: 9 people who are not among the 21 training people. The C run measures its size: 0.9551 on validation windows (training people) against 0.9002 on the new people. The ECG autoencoder is an out-of-distribution detector: a high reconstruction error means "unlike the training data" [35].

Source: the project's C trainer `train_c.c` (float, SGD lr 0.01, momentum 0.9, SEED 42, 20 epochs), run unchanged on the real HAR data; logs in `rebuild/c_runs`.

validation (known people)



95.5%

test (new people)



90.0%

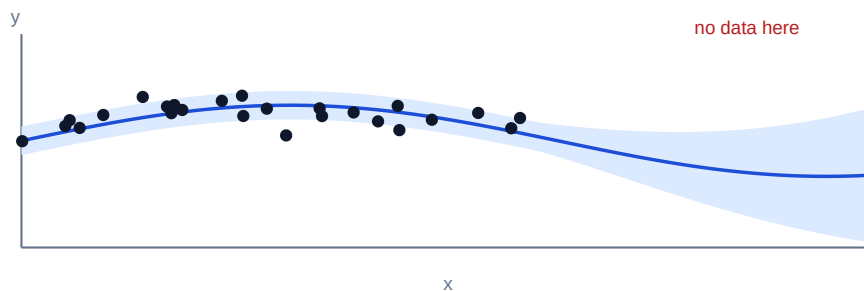
Graph (from the C run) — The same C-trained CNN on people it has seen and on new people. Validation windows come from the 21 training people (and overlap training windows in time); test windows come from 9 other people. The gap is the distribution shift that on-device training is meant to close.

WORKED ON OUR PROJECT — New users are a distribution shift

1. The test people are different from the training people (0 overlap) [9]. Every accuracy in this book's HAR examples is therefore already an out-of-distribution number.
2. The validation-test gap (0.902 vs 0.7869 for the softmax model) is the size of that shift.
3. On-device training is the answer the thesis studies: adapt to the new user with his or her own data [3, 5].

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

- **Input drift:** monitor feature distributions with population stability index, KS tests or MMD. Cheap and catches pipeline breakages.
- **Prediction drift:** monitor the distribution of outputs - it moves before labels arrive.



Graph 33.1 — Two kinds of uncertainty (illustrative). Aleatoric: the data itself is noisy (wide band everywhere). Epistemic: the model has not seen enough data there (the band opens up where there are no training points). More data cures only the second.

- **Concept drift:** the relationship $X \rightarrow y$ changes; only labels reveal it, so invest in a delayed-label pipeline and a small continuously labelled sample.

33.4 Adversarial robustness

```
FGSM: x' = x + eps * sign( grad_x L(f(x), y) )
PGD : iterate x' <- clip_(x, eps)( x' + a * sign(grad_x L) )
Adversarial training: min_theta E[ max_(||d|| <= eps) L(f(x+d), y) ]
```

FROM ZERO — the idea explained step by step

Adversarial examples: tiny, crafted changes to the input that fool a model.

IN OUR PROJECT — not used, and why

Not used in the project.

- Imperceptible perturbations flip predictions on ordinary networks; this is a property of high-dimensional linear-ish decision boundaries, not a bug in a particular model.
- **Adversarial training** (train on PGD examples) is the only defence that has held up broadly. It costs 3-10x training time and typically several points of clean accuracy.
- **Certified defences** (randomised smoothing, interval bound propagation) give provable radii, at further cost.
- Beware evaluation pitfalls: gradient masking makes a defence look strong against weak attacks. Always evaluate with strong adaptive attacks such as AutoAttack.
- For most products, natural robustness (augmentation, distribution coverage, sanity checks on inputs) matters far more than L-infinity adversarial robustness - unless you face a genuine adversary.

33.5 Interpretability

Method	Scope	Notes
Linear/tree coefficients	Global, intrinsic	Use an interpretable model when the stakes require it - often the right answer
Permutation importance	Global, post-hoc	Model-agnostic; correlated features share credit
Partial dependence / ALE	Global	Shows the average shape of a feature's effect; ALE handles correlation better than PDP
SHAP	Local and global	Additive attributions with consistency guarantees; TreeSHAP is exact and fast
LIME	Local	Fits a local surrogate; unstable across runs
Integrated gradients	Local, differentiable models	Attribution along a path from a baseline; needs a meaningful baseline
Grad-CAM	Local, CNNs	Class-discriminative heat maps from the last convolutional layer
Attention maps	Local, Transformers	Attention is NOT explanation on its own; use with attribution methods
Concept-based (TCAV), probing	Global	Tests whether a human-defined concept is encoded
Mechanistic interpretability	Circuit level	Reverse-engineering internal algorithms; sparse autoencoders on activations are the current frontier

FROM ZERO — the idea explained step by step

Interpretability: which inputs made the model decide? Feature importance, saliency maps.

IN OUR PROJECT — partly used

Measured with a random forest on our features: the most important features are `total_acc_y_mean` (0.179), `total_acc_x_mean` (0.167), `total_acc_x_std` (0.119). For the CNN, the receptive field (26 samples) tells what time span each feature summarises.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

EXPLANATIONS CAN BE CONFIDENTLY WRONG

Saliency maps can look identical for a trained and a randomly initialised network. LIME and SHAP make independence assumptions that correlated features violate. A plausible explanation is not evidence that the model reasons that way. Use explanations to generate hypotheses, then TEST them - by intervening on the input, by ablating the feature and retraining, or by constructing counterfactual cases.

33.6 Fairness

- Define the harm before the metric. **Demographic parity** (equal positive rates), **equalised odds** (equal TPR and FPR across groups) and **calibration within groups** are mutually incompatible except in degenerate cases - you must choose, and the choice is a value judgement, not a technical one.

FROM ZERO — the idea explained step by step

Fairness: a model should not be systematically worse for some groups of people.

IN OUR PROJECT — not used, and why

Not studied. A per-person accuracy table would be the first step, since HAR accuracy varies from person to person.

- Mitigations act at three stages: pre-processing (reweighting, resampling), in-processing (constrained optimisation, adversarial debiasing), post-processing (group-specific thresholds - often the most practical, sometimes legally constrained).
- Removing the protected attribute does not remove the bias; proxies (postcode, device, name) carry it. Measuring by group requires having the attribute, which creates its own governance problem.
- Report per-group performance as standard practice, exactly as you report per-class performance.

Exercises

1 Implement conformal prediction for a classifier and empirically verify 90% coverage on a held-out set. 2 Compare MC dropout, a 5-model deep ensemble and temperature scaling on both in-distribution and corrupted test data. 3 Attack a small CNN with FGSM and PGD, then adversarially train it and measure the clean/robust accuracy trade-off.

34. MLOps: Shipping and Keeping Models Alive

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

MLOps at the project scale: version control, a continuous-integration pipeline that checks C against PyTorch, JSON logs for every run. The ideas of this chapter appear in miniature.

A model in a notebook has produced no value. This chapter is the engineering that stands between a good validation score and a system that keeps working for years.

34.1 The lifecycle

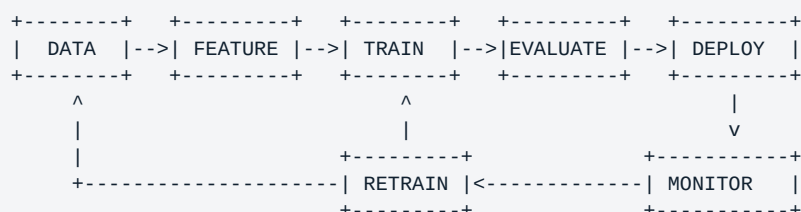


Figure 34.1 - The loop. Most teams build the top row and neglect the bottom one, which is where models die.

FROM ZERO — the idea explained step by step

The lifecycle: data -> training -> deployment -> monitoring -> retraining.

IN OUR PROJECT — partly used

In the thesis the loop is closed on the device: the model is deployed and then keeps training there.

34.2 Versioning everything

Artefact	Tool / practice
Code	Git, with the training entry point and config in the repo
Data	DVC, LakeFS, Delta Lake, or immutable dated snapshots plus a content hash
Features	A feature store, or at minimum shared feature code used by both training and serving
Experiments	MLflow, Weights & Biases, or structured logs - config, metrics, environment, seeds
Models	A model registry with stages (staging, production, archived), lineage back to data and code, and signed artefacts
Environment	Pinned dependencies and a container image digest

FROM ZERO — the idea explained step by step

Version the code, the data and the models, so every result can be reproduced.

IN OUR PROJECT — used

The project uses git for code; the data files are generated by a script (prepare_data.py); every run writes its configuration (seed, learning rate, bits) into its JSON log [35].

WORKED ON OUR PROJECT — Versioning a run so it can be repeated

1. Each project run writes a JSON log with the configuration (epochs, batch, lr, momentum, seed, shuffle_seed), the per-epoch losses and the final metrics [35].
2. Together with the git hash of the code and the .npy data files, this is enough to rerun the experiment.
3. The parity harness is a test in the CI: C and PyTorch must agree within +/-2.5 pp accuracy and +/-0.15 loss before a change is merged [35].

34.3 Serving patterns

Pattern	Latency	Use when
Batch / offline scoring	Hours	Predictions can be precomputed - churn scores, recommendations refreshed nightly
Online / real-time API	10-500 ms	Predictions depend on live input
Streaming	Seconds	Event-driven scoring over Kafka/Flink
Edge / on-device	1-50 ms	Privacy, offline, or cost constraints (Chapter 31)
Embedded in the client	-	Small models shipped inside the app binary

FROM ZERO — the idea explained step by step

Serving patterns: batch, online, streaming, on-device.

IN OUR PROJECT — not used, and why

The project is "on-device" -- the model runs where the data are produced.

- **Training/serving skew** is the number one production bug: the feature computed in the training pipeline differs from the one computed at serving time. The structural fix is to share one implementation, and to log serving features so you can compare distributions directly.
- **Shadow deployment:** run the new model alongside the old one on live traffic without acting on its output. Then **canary** to a small percentage, then ramp.
- **A/B test** against the business metric, not the offline metric. The two disagree more often than people expect.
- Always keep a **rollback** path, and rehearse it.

34.4 Monitoring

FROM ZERO — the idea explained step by step

Monitor inputs, outputs and accuracy after deployment.

IN OUR PROJECT — partly used

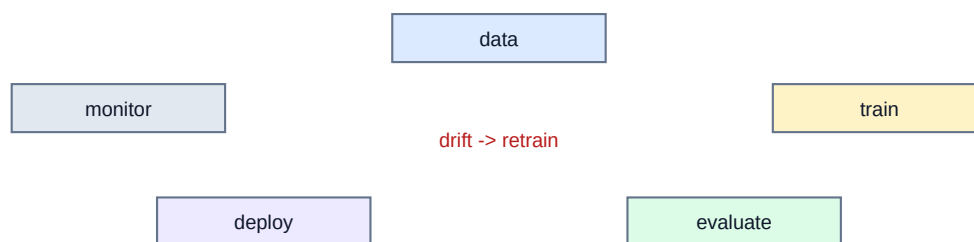
Not implemented on the board. The training logs (loss, accuracy per epoch) are the project's monitoring.

What a production model dashboard must show

Operational: request rate, latency percentiles (p50/p95/p99), error rate, resource use.

IN OUR PROJECT — What a production model dashboard must show

For the device: memory high-water mark, time per training step, current loss.



Graph 34.1 — The machine-learning lifecycle. It is a loop, not a line: once deployed, the model is watched, and drift in the data sends it back to be retrained.

Input health: missing-value rates, out-of-range values, schema violations, cardinality changes.

Drift: PSI or KS per important feature; embedding drift for unstructured inputs; prediction distribution over time.

Performance: metrics on delayed labels, sliced by segment.

Business: the metric the model exists to move.

Alerting with thresholds someone has agreed to, and a documented response for each alert.

34.5 Retraining

- **Scheduled** (weekly/monthly): simple, predictable, and usually adequate.

FROM ZERO — the idea explained step by step

Retrain when the data change.

IN OUR PROJECT — used

On-device training is continuous retraining, done where the new data appear.

- **Triggered** by drift or a performance drop: efficient but needs a trustworthy trigger and guard against thrashing.
- **Continual/online:** rarely necessary and easy to get wrong - feedback loops can make a model train on the consequences of its own predictions.
- Always validate a retrained model against the current production model on the same holdout before promoting it, and never promote automatically without that gate.

34.6 Testing machine learning systems

Test type	Example
Data validation	Schema, ranges, null rates, class balance (Great Expectations, Pandera)
Unit tests on features	A known input produces the expected feature value, including edge cases
Model behavioural tests	Invariance (irrelevant change leaves the prediction alone), directional expectation (income up -> risk down), minimum functionality on a curated set
Regression tests	Performance on a frozen golden set does not drop below a threshold
Slice tests	Per-subgroup metrics stay within bounds
Integration tests	The full pipeline runs end to end on a small sample in CI
Load tests	Latency under peak concurrency, with realistic payloads

FROM ZERO — the idea explained step by step

Test ML code: unit tests, tests with known numbers, and end-to-end checks.

IN OUR PROJECT — used

The ODT library has unit tests per layer and a continuous-integration pipeline (.github/workflows/ci.yml) that runs the HAR, ECG and MNIST examples in BIT_PARITY mode against PyTorch [35].

34.7 Cost and carbon

- Track cost per 1,000 predictions and per training run. Quantization, batching, caching, distillation and a smaller model are the levers, roughly in that order of return.

FROM ZERO — the idea explained step by step

Training costs money and energy.

IN OUR PROJECT — partly used

One HAR training run is 2,060 steps x 489,856 forward MACs plus the backward pass -- small enough to run on a microcontroller, which is the point of the thesis.

- Cache aggressively: identical or near-identical requests are common, and semantic caching works well for LLM workloads.
- Right-size the hardware - many production models run happily on CPU, and a GPU that is idle 90% of the time is pure cost.
- Report energy or estimated emissions for large training runs; it is increasingly expected in papers and in corporate reporting.

Exercises

1 Take a model you have trained and wrap it in a container with a prediction endpoint, input validation and structured logging. 2 Implement PSI-based drift monitoring on a feature and simulate a drift event to verify the alert fires. 3 Write five behavioural tests for a model in your domain: two invariance, two directional, one minimum functionality.

35. Doing Research and Reading the Literature

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

This chapter describes how the thesis itself should be read and written: claims, baselines, seeds, spreads.

35.1 Reading a paper efficiently

- **Pass 1 (5 minutes):** title, abstract, figures, tables, conclusion. Decide whether to continue.

FROM ZERO — the idea explained step by step

Read a paper for its question, method, evidence and limits before its details.

IN OUR PROJECT — used

The thesis rests on a few such papers: FQT [3], RigL [4], on-device training under 256 KB [5].

- **Pass 2 (30 minutes):** introduction, method, and the experimental setup. Write down, in your own words, the one idea and the one experiment that supports it.
- **Pass 3 (hours):** re-derive the equations, check the ablations, and ask what is missing - which baseline is absent, which hyperparameter budget was unequal, which dataset would break it.
- Keep notes in a searchable form with a one-line summary per paper. In a year you will remember the idea and not the title.

Questions to ask of any empirical claim

Is the baseline tuned as carefully as the proposed method?

IN OUR PROJECT — Questions to ask of any empirical claim

Asked of our own results: is 0.9026 (8-bit) vs 0.8953 (float) larger than the spread of 0.0050-0.0074? Barely -- so the honest claim is "8-bit loses nothing", not "8-bit is better" [35].

Are results averaged over seeds, with variance reported?

Is the comparison at equal compute, equal parameters, or equal wall clock - and does the choice flatter the method?

Is there an ablation isolating the claimed contribution? Could the gain come from the training recipe rather than the architecture?

Is the test set possibly contaminated by the training data?

Is the effect size larger than the noise, and is it practically meaningful?

35.2 Running an experiment that survives scrutiny

- Write the hypothesis and the decision rule **before** running: 'if X improves validation F1 by more than 0.5 points across 5 seeds, adopt it'.

FROM ZERO — the idea explained step by step

A robust experiment: fixed baseline, one change at a time, several seeds, report the spread.

IN OUR PROJECT — used

The thesis sweeps follow this: same model and data, one setting changed per configuration, 10 seeds each [35].

WORKED ON OUR PROJECT — Reading the project's own sweep tables like a reviewer

1. Claim: "at 8 bits cosine vs constant LR is a wash". Evidence: 0.8996 vs 0.9026, difference 0.0030, standard deviations 0.0076 and 0.0074 over 10 seeds [35]. The difference is below half a standard deviation: the claim is justified.
2. Claim: "at 4 bits cosine is higher". 0.8928 vs 0.8761, difference 0.0167, sd of sym4 0.0210: less than one sd, so the README correctly says "not clearly outside noise".
3. A paired design (same seeds in both arms, compare per-seed differences) would give a sharper test than comparing means.

- Change one thing at a time. A run that changes three things teaches you nothing when it improves.
- Keep a fixed, tuned baseline and re-run it whenever the codebase changes.
- Budget compute equally across compared methods, and say what the budget was.
- Log everything automatically; a result you cannot reproduce is a rumour.
- Report negative results to yourself honestly - the discipline of recording what failed is what stops you repeating it in six months.

35.3 Where the field is heading

- **Efficiency as the primary axis:** quantization, sparsity, distillation and better architectures are where most practical progress now lands, because inference cost dominates total cost of ownership.

FROM ZERO — the idea explained step by step

Where the field is heading.

IN OUR PROJECT — partly used

TinyML and on-device learning are one of these directions; the thesis contributes to it.

- **Long context and memory:** state-space models, hybrid attention, and retrieval as an architectural component rather than a bolt-on.
- **Multimodality by default:** one model over text, image, audio and video, with tokenisers per modality.
- **Post-training and reasoning:** more of the capability gain now comes from what happens after pretraining - preference optimisation, verifiable-reward RL, and inference-time compute.
- **Agents and tool use:** models that act, with the attendant problems of reliability, evaluation and security.
- **On-device and private learning:** small capable models, personalised locally, with federated and differentially private updates.
- **Science of deep learning:** scaling laws, mechanistic interpretability, and a slowly improving theoretical account of why any of this generalises.

HOW TO STAY CURRENT WITHOUT DROWNING

Follow a small number of sources: two or three researchers' feeds, one curated newsletter, and the proceedings of NeurIPS/ICML/ICLR/CVPR/ACL skimmed once per cycle. Read one paper properly per week rather than twenty abstracts per day. Reimplement one method per quarter - implementation is the only reading comprehension test that works.

PART VI - Applied Domains, Theory and Systems

Why generalization is possible at all, the probabilistic view of learning, the four application domains with their own rules - time series, recommenders, vision beyond classification, language, audio and multimodal - the hardware and distributed systems that make scale possible, causal questions that prediction cannot answer, and the privacy and security obligations that come with shipping.

36. Learning Theory: Why Generalization Is Possible At All

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Learning theory gives a guarantee for our test accuracy: with 2,947 test windows the measured accuracy is close to the true one. The box of 36.2 does this calculation.

Chapter 3 showed empirically that a model can score well on training data and badly on new data. This chapter answers the harder question behind that observation: **why should fitting a finite sample ever tell you anything about the infinite population it was drawn from?** The answer is not obvious - and the theory that provides it also explains why some model classes need thousands of samples and others need millions, and why the classical story breaks in the deep-learning regime.

THE ONE-SENTENCE VERSION

Generalization is possible because the number of genuinely different functions a model class can express on n points grows slowly enough that fitting the sample cannot be a coincidence - and the rate of that growth, not the number of parameters, is the quantity that controls the gap between training error and test error.

36.1 Empirical risk minimization, stated precisely

Assume every example is drawn independently from a fixed but unknown distribution D over pairs (x, y) . This is the *i.i.d. assumption* and it is the load-bearing assumption of the whole field: break it and the results below say nothing.

True risk	$R(h)$	$= E_{(x,y) \sim D} [L(h(x), y)]$
Empirical risk	$R_{\text{emp}}(h)$	$= (1/n) \sum_i L(h(x_i), y_i)$
ERM	$\hat{h}$	$= \operatorname{argmin}_{(h \in H)} R_{\text{emp}}(h)$

You can compute the second line. You care about the first. The gap between them is what theory bounds.

FROM ZERO — the idea explained step by step

Empirical risk = the average loss on the training data. **True risk** = the average loss on all data the model will ever see. Training minimises the first; we care about the second.

IN OUR PROJECT — used

Ours: training loss is the empirical risk on 6,617 windows; the test loss on 2,947 windows of new people estimates the true risk.

Three quantities decompose the failure of the model you actually train:

$R(\hat{h}) - R(h^*)$	$=$	$[R(\hat{h}) - R_{\text{emp}}(\hat{h})]$	generalization gap
		$+ [R_{\text{emp}}(\hat{h}) - R_{\text{emp}}(h_{\text{opt}})]$	optimization gap
		$+ [R_{\text{emp}}(h_{\text{opt}}) - R(h^*)]$	approximation gap

h^ is the best possible predictor of all; h_{opt} is the best in your class H ; $\hat{h}$ is what your optimizer returned.*

Gap	Cause	What reduces it
Approximation	H is too small to contain a good function	A richer model class: more capacity, better inductive bias, pretraining
Optimization	SGD did not reach the best point in H	Better optimizer, schedule, initialization, longer training
Generalization	H is rich enough to fit noise in this sample	More data, regularization, restricting H , better inductive bias

Every disappointing model is one of these three. Diagnosing which one you have is the entire content of Chapter 20's playbook:

train error high -> approximation or optimization; train error low and test error high -> generalization.

36.2 A single hypothesis: Hoeffding's inequality

Fix one hypothesis h before looking at the data, and let the loss be bounded in $[0, 1]$. Then $R_{\text{emp}}(h)$ is an average of n independent bounded random variables whose expectation is $R(h)$. Hoeffding's inequality says such an average concentrates:

$$P(|R_{\text{emp}}(h) - R(h)| > \epsilon) \leq 2 \exp(-2n\epsilon^2)$$

The probability of being wrong by more than ϵ decays exponentially in the number of samples.

FROM ZERO — the idea explained step by step

Hoeffding's inequality: for one fixed model, the accuracy measured on n test examples is within ϵ of the true accuracy with probability at least $1 - \delta$, where $\epsilon = \sqrt{\ln(2/\delta) / (2n)}$.

IN OUR PROJECT — used

For our $n = 2,947$ test windows and $\delta = 0.05$: $\epsilon = \sqrt{\ln 40 / 5,894} = 0.025$. So a measured accuracy of 0.90 means the true accuracy is between 0.875 and 0.925 with 95% certainty -- a worst-case bound; the spread over seeds (about 0.005) is much smaller.

Inverting it: with probability at least $1 - \delta$ over the draw of the sample,

$$|R_{\text{emp}}(h) - R(h)| \leq \sqrt{\ln(2/\delta) / (2n)}$$

With $n = 10,000$ and $\delta = 0.05$ this is about 0.0136 - a validation score of 0.91 means the true accuracy is 0.91 plus or minus roughly one and a half points. **This is the correct way to read a single held-out number**, and it is why a 0.3-point improvement on a 2,000-row test set is not an improvement at all.

WHY THIS IS NOT YET A THEORY OF LEARNING

Hoeffding applies to a hypothesis chosen before the data. The one you report was chosen because it scored well - you searched. Searching over many hypotheses and keeping the best is exactly the setting where the maximum of many noisy numbers is biased upward. The rest of this chapter pays for that search.

36.3 Finite hypothesis classes: the union bound

Suppose H is finite with $|H|$ elements. The bad event is that *some* h in H has a large gap. The union bound says the probability of a union is at most the sum of probabilities:

$$P(\text{exists } h : |R_{\text{emp}}(h) - R(h)| > \epsilon) \leq 2 |H| \exp(-2n\epsilon^2)$$

FROM ZERO — the idea explained step by step

With many candidate models, the bound must be paid for each (union bound).

IN OUR PROJECT — partly used

Comparing 6 configurations on the same test set slightly weakens each claim; this is why the thesis also has a separate validation split (735 windows) for choices.

Setting the right-hand side to δ and solving for ϵ gives the first real generalization bound. With probability $1 - \delta$, *simultaneously for every h in H* :

$$R(h) \leq R_{\text{emp}}(h) + \sqrt{(\ln |H| + \ln(2/\delta)) / (2n)}$$

Uniform convergence: the bound holds for the hypothesis you picked after searching, because it holds for all of them at once.

WORKED NUMBERS

20 Take a class of 20-bit decision rules, so $|H| = 2^{20} = 1,048,576$, with $n = 10,000$ and $\delta = 0.05$. Then $\log|H| = 20 \log 2 = 13.86$, $\log(2/\delta) = 3.69$, the sum is 17.55, divided by 20,000 is $8.8e-4$, and the square root is 0.030. Fitting a million-hypothesis class on ten thousand samples costs about three accuracy points of uncertainty. Halve the data and the penalty grows by $\sqrt{2}$; square the class size and it grows by $\sqrt{2}$ as well - the bound is logarithmic in the size of the class and inverse-square-root in the data. That trade is the central fact of the subject.

Two consequences follow immediately and are worth internalizing:

- **The $\sqrt{1/n}$ rate.** To halve your uncertainty you need four times the data. This is why data collection has diminishing returns, and why learning curves flatten (Chapter 3).
- **The $\log|H|$ price of search.** Trying 1,000 hyperparameter configurations on the validation set is a class of size 1,000; the penalty is $\sqrt{\log 1000 / 2n}$, small but real. Trying 100,000 is only 1.5x worse in the bound - but only if each was evaluated on a fresh sample, which it was not. This is the formal reason for a test set used exactly once.

36.4 Infinite classes: shattering and VC dimension

Linear classifiers form an infinite class, so $\log|H|$ is infinite and the bound above is vacuous. The fix is to count not hypotheses but *behaviours on n points*. Two hypotheses that label every point in your sample identically are indistinguishable from the sample's point of view.

Growth function $\Pi_H(n) = \max \text{ over } n \text{ points of the number of distinct labelings } H \text{ can produce (at most } 2^n \text{)}$

FROM ZERO — the idea explained step by step

VC dimension measures how many points a model class can separate in every possible way.

IN OUR PROJECT — not used, and why

Not used. For neural networks the resulting bounds are far too loose to be useful.

A set of points is *shattered* by H if H realizes all 2^n labelings of it. The **VC dimension** d_{VC} is the size of the largest set that H can shatter.

Hypothesis class	VC dimension	Note
Thresholds on the line, $x > a$	1	Cannot shatter two points labelled (1, 0) left-to-right
Intervals $[a, b]$ on the line	2	
Linear classifiers in $\mathbb{R}^d$ (with bias)	$d + 1$	3 points in the plane, in general position, can be shattered; 4 cannot
Axis-aligned rectangles in $\mathbb{R}^2$	4	
1-nearest neighbour	infinite	Fits any labeling - and yet works; see the caveat below
Neural net, W weights, ReLU	$O(W L \log W)$	L = depth; the bound is loose but shows parameters matter

Sauer's lemma is the combinatorial miracle that makes this useful: once n exceeds d_{VC} , the growth function stops doubling and becomes polynomial.

$$\Pi_H(n) \leq \sum_{i=0}^{d_{VC}} \binom{n}{i} \leq (e n / d_{VC})^{d_{VC}}$$

Exponential below the VC dimension, polynomial above it - a phase transition at $n = d_{VC}$.

Substituting the polynomial growth function for $|H|$ gives the VC bound: with probability $1 - \delta$,

$$R(h) \leq R_{\text{emp}}(h) + O\left(\sqrt{\left(d_{\text{VC}} \log(n/d_{\text{VC}}) + \log(1/\delta)\right) / n}\right)$$

THE PRACTICAL READING OF THE VC BOUND

You need on the order of d_{VC} samples (times a logarithmic factor) before the training error means anything. This is the rigorous version of the folklore '10 to 100 samples per parameter' rule quoted for classical models in Chapter 3 - and the reason that rule is quoted for classical models only.

36.5 Rademacher complexity and margin bounds

VC dimension has two weaknesses: it ignores the data distribution, and it counts only labelings, not confidence. **Rademacher complexity** fixes the first by measuring how well the class can fit random noise *on your actual sample* :

$$R_n(H) = E_{\sigma} \left[\sup_{h \in H} \left(\frac{1}{n} \sum_i \sigma_i h(x_i) \right) \right]$$

$\sigma_i = +1 \text{ or } -1 \text{ with probability } 1/2 \text{ each}$

The expected best correlation between a hypothesis and a coin flip. Zero for a constant class; 1 for a class that can fit anything.

FROM ZERO — the idea explained step by step

Rademacher complexity and margin bounds.

IN OUR PROJECT — not used, and why

Not used in the project.

This is directly measurable: shuffle your labels and train. If the model reaches high training accuracy on randomized labels, its effective complexity on your data is large. That experiment - **the randomization test** - is the single most informative diagnostic in this chapter, and it should be part of any serious project.

$$R(h) \leq R_{\text{emp}}(h) + 2 R_n(H) + 3 \sqrt{\log(2/\delta) / (2n)}$$

Margin bounds fix the second weakness. For classifiers that output a real score, a prediction that is correct by a wide margin is more robust than one that barely crosses the boundary. The margin bound replaces the 0/1 training error with the fraction of points whose margin is below γ , and scales the complexity term by $1/\gamma$:

$$R(h) \leq R_{\gamma\text{-emp}}(h) + O\left(R_n(H) / \gamma\right) + \text{small term}$$

Large margins buy generalization. This is the theory behind SVMs (Chapter 10) and behind why weight decay and normalization

help deep networks: both control the scale-sensitive complexity in the numerator.

36.6 Where the classical story breaks: double descent

Modern networks have far more parameters than samples, can fit random labels perfectly (so their Rademacher complexity is near 1), and yet generalize well on real labels. Uniform-convergence bounds computed for the full class are therefore vacuous - they permit test errors above 1. This is not a flaw in the theorems; it means the bounds are being applied to the wrong class. The class that matters is not 'all networks of this size' but 'the networks SGD actually reaches from this initialization'.

FROM ZERO — the idea explained step by step

Double descent: very large models can generalise better again, against the classical picture.

IN OUR PROJECT — partly used

Our model is in the small, classical regime (10,214 parameters, 6,617 examples).



Double descent. The classical bias-variance curve (*) is the left half; past the interpolation threshold the curve descends a second time (#).

The mechanism, in one paragraph: exactly at the interpolation threshold there is essentially one way to fit the data, and it is a wildly oscillating one, so the variance term explodes. Well past the threshold there are infinitely many interpolating solutions, and gradient descent with small initialization finds one of the *smoothest* of them - implicit regularization. Capacity stops being the relevant axis; the bias of the optimization algorithm takes over. The same effect appears in **epoch-wise** double descent (test error rises then falls again as training continues) and **sample-wise** double descent (more data temporarily hurts near the threshold).

IMPLICIT REGULARIZATION, CONCRETELY

For separable data, gradient descent on logistic loss converges in direction to the maximum-margin separator, even with no explicit penalty - a result of Soudry et al. For linear regression solved by gradient descent from zero, the solution converges to the minimum-norm interpolant. Neither is imposed; both fall out of the trajectory. This is why 'the optimizer is part of the model class' is the correct modern framing, and why a bound that ignores the optimizer cannot be tight.

36.7 Scaling laws: the empirical theory that replaced the bounds

For large models, practitioners predict performance not from VC bounds but from measured power laws. Across many orders of magnitude, loss falls as a power of parameters N , dataset size D , and compute C :

$$\begin{aligned} L(N) &= L_{\text{inf}} + (N_c / N)^{\alpha_N} & \alpha_N &\sim 0.07 \text{ for LLMs} \\ L(D) &= L_{\text{inf}} + (D_c / D)^{\alpha_D} & \alpha_D &\sim 0.10 \\ L(C) &= L_{\text{inf}} + (C_c / C)^{\alpha_C} \\ \text{Compute } C &\sim 6 N D \text{ FLOPs for a dense Transformer} \end{aligned}$$

The 6ND rule: 2ND for the forward pass, 4ND for the backward pass, per token per parameter.

FROM ZERO — the idea explained step by step

Scaling laws: error falls smoothly as data and model size grow.

IN OUR PROJECT — partly used

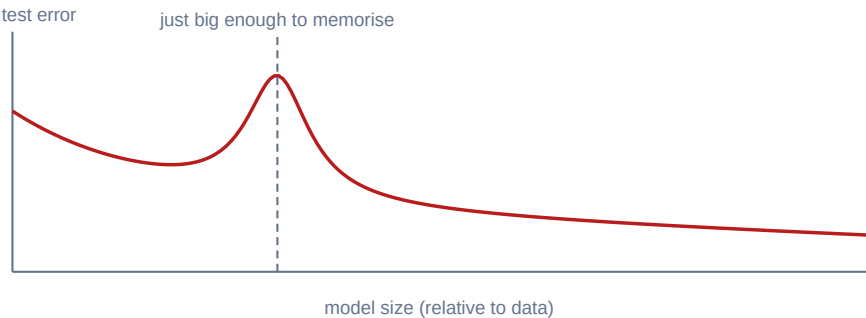
A learning curve on our data shows the same trend at small scale: 100: 0.880 / 0.721, 300: 0.863 / 0.759, 1000: 0.872 / 0.780, 3000: 0.881 / 0.776, 6617: 0.887 / 0.787 (measured). The board fixes the model size, so more data (from the wearer) is the lever that remains.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

The Chinchilla result is the practically important one: for a fixed compute budget, N and D should be scaled in **equal proportion**, giving roughly 20 training tokens per parameter as the compute-optimal ratio. Models trained

before that result were badly undertrained relative to their size, which is why a well-trained 7B model can beat a 175B model from an earlier generation. In deployment the calculus shifts again: if you will serve billions of tokens, training a smaller model on more data than Chinchilla-optimal is cheaper overall, because inference cost scales with N and not with D.

Budget question	What the laws say
I have 10x more compute. What do I change?	About 3.2x parameters and 3.2x tokens - not 10x either one.
My loss curve is above the law's prediction.	Something is broken: data quality, learning rate, or duplication. The law is a ceiling you should nearly reach.
Should I train longer or grow the model?	Below 20 tokens/parameter, train longer. Above it, and if inference cost is not dominant, grow.
Does this apply to my 50k-row tabular problem?	No. Power-law extrapolation is validated for large-scale pretraining, not small supervised datasets.



Graph 36.1 — Double descent (schematic). Classical theory says test error rises once a model is big enough to memorise the data. In practice, growing the model further past that point makes the error fall again.

36.8 What theory gives you, and what it does not

Theory delivers	Theory does not deliver
The $\sqrt{1/n}$ rate, so you can budget data	A usable numeric bound for a deep network
The formal cost of searching over models	Permission to skip a held-out test set
Why margins, norms and smoothness help	Which architecture will win on your data
Why i.i.d. violations are fatal, not cosmetic	Any guarantee at all under distribution shift
The randomization test as a diagnostic	A replacement for measurement

FROM ZERO — the idea explained step by step

Theory gives orders of magnitude and warnings, not exact predictions.

IN OUR PROJECT — used

For the thesis it justifies the test-set size: 2,947 windows are enough to measure accuracy to a few points.

THE THREE THINGS TO CARRY AWAY

- 1. Report held-out numbers with an interval, computed from n.
- 2. Every model you compare on the validation set enlarges the class you are implicitly searching, so keep the test set sacred.
- 3. Run the label-randomization test once on any new architecture: if it memorizes noise as fast as it learns signal, your regularization and your data budget both need attention.

Exercises

- 1 Compute the Hoeffding interval for your own validation set size at $\delta = 0.05$. Compare it with the improvements you have been claiming in the last month.
- 2 Show that the VC dimension of thresholds on the line

is 1 by exhibiting a shattered set of size 1 and proving no set of size 2 is shattered.

10 3 Verify Sauer's lemma numerically for $d_{VC} = 3$ and $n = 10$: compare 2 with the sum of binomial coefficients up to 3. 4 Run the randomization test: shuffle the labels of your training set and train the same model. Record how many epochs it needs to reach 90% training accuracy, and compare with the true-label run. 5 Reproduce epoch-wise double descent on a small network: train far past the point where test error first rises, and plot the whole curve.

37. Probabilistic Machine Learning: Bayes, EM, HMMs and Gaussian Processes

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Bayesian ideas appear twice in the project: as a way to put an interval around the test accuracy, and in the probabilistic PCA of the continual-learning example.

Most of this book optimizes a single set of parameters. The probabilistic view keeps a *distribution* over parameters and over hidden structure instead. It costs more computation and buys three things nothing else gives you: honest uncertainty on small data, a principled way to handle missing and latent variables, and a language in which regularization, clustering, sequence models and active learning turn out to be the same idea seen from different angles.

37.1 The Bayesian recipe in one page

Prior	$p(\theta)$	belief before data
Likelihood	$p(D \theta)$	how the model explains data
Posterior	$p(\theta D) = p(D \theta) p(\theta) / p(D)$	
Evidence	$p(D) = \text{INTEGRAL } p(D \theta) p(\theta) d(\theta)$	
Predictive	$p(y^* x^*, D) = \text{INTEGRAL } p(y^* x^*, \theta) p(\theta D) d(\theta)$	

Learning is conditioning; prediction is averaging over everything you still do not know.

FROM ZERO — the idea explained step by step

The Bayesian recipe: start with a **prior** belief, observe data, and update to a **posterior** belief with Bayes' rule: posterior is proportional to likelihood x prior.

IN OUR PROJECT — partly used

Used below to put an interval around a test accuracy. The class priors of HAR are almost equal (LAYING 0.1906).

Estimator	Objective	Uncertainty	Cost
MLE	$\max p(D \theta)$	None (a point)	One optimization
MAP	$\max p(D \theta) p(\theta)$	None (a point)	One optimization
Full Bayes	the whole posterior	Full	Integration: conjugacy, VI or MCMC

RIDGE REGRESSION IS A GAUSSIAN PRIOR - THE DERIVATION

Take a Gaussian likelihood $y = x.w + \text{noise}$ with noise variance s^2 , and a Gaussian prior $w \sim N(0, t^2 I)$. The log posterior is $-(1/2s^2) \sum (y_i - x_i.w)^2 - (1/2t^2) ||w||^2 + \text{const}$. Maximizing it is exactly minimizing $\sum (y_i - x_i.w)^2 + \lambda ||w||^2$ with $\lambda = s^2 / t^2$. Ridge is MAP under a Gaussian prior; Lasso is MAP under a Laplace prior. Every penalty in Chapter 7 is a prior in disguise, and the strength of the penalty is a ratio of variances.

37.2 Conjugacy, worked on real numbers

A prior is *conjugate* to a likelihood when the posterior stays in the same family, so updating is arithmetic rather than integration. The Beta-Bernoulli pair is the one to know by heart.

Prior	Beta(a, b)
Data	h heads and t tails out of n = h + t Bernoulli trials
Posterior	Beta(a + h, b + t)
Mean	(a + h) / (a + b + n)

The prior acts as a + b pseudo-counts you saw before the experiment.

FROM ZERO — the idea explained step by step

Conjugacy: some priors give a posterior of the same family. For a success rate, a Beta prior with a count of successes gives again a Beta: Beta(1 + k, 1 + n - k) after k successes in n trials.

IN OUR PROJECT — used

Our 8-bit model gets k = 2660 of n = 2,947 test windows right. The posterior Beta(2661, 288) has mean 0.9023 and a 95% interval from 0.8914 to 0.9128 (measured). This is the uncertainty from the finite test set alone.

Toss a coin ten times and see 7 heads. With a uniform prior Beta(1,1) the posterior is Beta(8,4): posterior mean $8/12 = \mathbf{0.667}$, against the MLE of 0.700. The 95% credible interval runs from about 0.39 to 0.89 - which is the honest answer after ten tosses, and exactly what a point estimate hides. Add 90 more tosses with 63 more heads and the posterior becomes Beta(71,31): mean 0.696, interval 0.60 to 0.78. **The prior's influence decays like $1/n$** ; with enough data Bayes and MLE agree, which is why the argument only matters in the small-data regime where it should.

Likelihood	Conjugate prior	Posterior update	Typical use
Bernoulli / Binomial	Beta(a, b)	a += successes, b += failures	Click-through rate, A/B tests
Categorical / Multinomial	Dirichlet(alpha)	alpha += counts	Topic models, naive Bayes smoothing
Poisson	Gamma(a, b)	a += SUM x, b += n	Counts, arrivals
Gaussian, known variance	Gaussian	Precision-weighted average	Kalman filters, sensor fusion
Gaussian, unknown both	Normal-Inverse-Gamma	Closed form	Small-sample regression

WHERE CONJUGACY EARNS ITS KEEP IN PRODUCTION

Ranking items by click-through rate with tiny denominators. A raw rate of 1/1 beats 480/1000, which is absurd. Ranking by the posterior mean of Beta(1 + clicks, 1 + misses) - or better, by a lower credible bound - fixes it with two lines of code and no model. Thompson sampling for bandits is the same posterior, sampled instead of averaged.

37.3 Latent variables and the EM algorithm

Many models are easy if you know a hidden label z and easy if you know the parameters, but hard when both are unknown: which cluster produced this point, which topic produced this word, which state emitted this observation. Expectation-Maximization alternates the two easy problems.

Goal	max_theta log p(X theta) = log SUM_z p(X, z theta)
ELBO	log p(X theta) >= E_q(z)[log p(X,z theta)] + H(q)
E-step	q(z) <- p(z X, theta_old) makes the bound tight
M-step	theta <- argmax E_q(z)[log p(X, z theta)]

Each step increases the true log-likelihood, so EM converges - to a local optimum, which is why initialization matters.

FROM ZERO — the idea explained step by step

Latent variables and the EM algorithm: alternate between guessing hidden variables and fitting parameters.

IN OUR PROJECT — partly used

Probabilistic PCA, used in the continual-learning example (PpcaReplay.c), is a latent-variable model: each window is explained by 8 hidden numbers plus noise [35].

Gaussian mixture model: the update equations

E-step (responsibility of component k for point i):

$$r_{ik} = \pi_k N(\mathbf{x}_i | \mu_k, \mathbf{S}_k) / \sum_j \pi_j N(\mathbf{x}_i | \mu_j, \mathbf{S}_j)$$

M-step:

$$N_k = \sum_i r_{ik}$$

$$\mu_k = (1/N_k) \sum_i r_{ik} \mathbf{x}_i$$

$$\mathbf{S}_k = (1/N_k) \sum_i r_{ik} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^T$$

$$\pi_k = N_k / n$$

k-means is the limit of this as all covariances shrink to zero: responsibilities become hard 0/1 assignments and the M-step

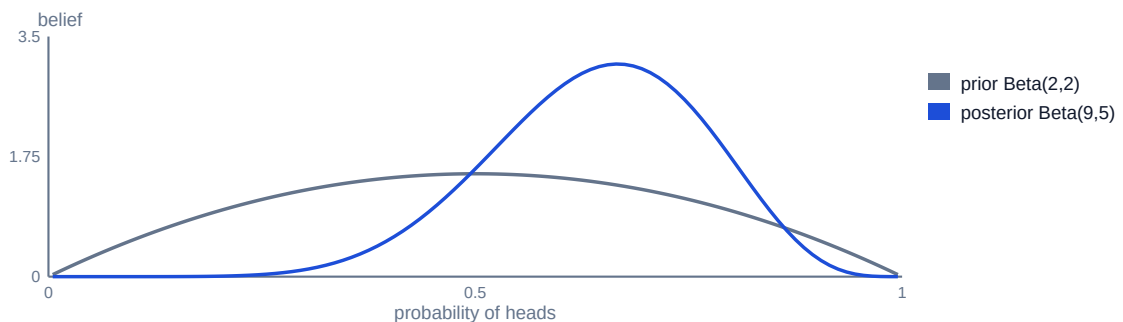
IN OUR PROJECT — Gaussian mixture model: the update equations

Not used. k-means (Chapter 12) is its hard-assignment cousin.

becomes 'average the members'.

ONE EM ITERATION BY HAND

Points 1, 2, 8, 9 on the line. Start with $\mu_1 = 2$, $\mu_2 = 7$, both variances 4, equal weights. For $x = 8$ the squared distances are 36 and 1, so the unnormalized responsibilities are $\exp(-36/8) = 0.0111$ and $\exp(-1/8) = 0.8825$; normalized, $r = 0.0124$ and 0.9876 . Repeating for all four points, the responsibility of component 2 is 0.0124, 0.0421, 0.9876, 0.9964. The M-step then gives $N_2 = 2.0385$ and $\mu_2 = (1(0.0124) + 2(0.0421) + 8(0.9876) + 9(0.9964)) / 2.0385 = 8.32$, and symmetrically $\mu_1 = 1.55$ - already almost the right answer after a single pass. Soft assignment is the only difference from k-means, and it is what lets the model report that a point between clusters is genuinely ambiguous.



Graph 37.1 — Bayesian updating with a Beta prior (computed). Start from Beta(2, 2), a mild belief that a coin is fair; see 7 heads in 10 flips; the posterior is Beta(2 + 7, 2 + 3) = Beta(9, 5), narrower and centred near 0.64.

```
# GMM in NumPy - the whole algorithm is these ten lines
for _ in range(n_iter):
    # E-step: responsibilities, shape (n, K)
    logp = np.stack([np.log(pi[k]) + gauss_logpdf(X, mu[k], S[k])
                     for k in range(K)], axis=1)
    logp -= logsumexp(logp, axis=1, keepdims=True) # stable softmax
    R = np.exp(logp)
    # M-step
    Nk = R.sum(0) + 1e-10
    mu = (R.T @ X) / Nk[:, None]
    for k in range(K):
        d = X - mu[k]
        S[k] = (R[:, k, None] * d).T @ d / Nk[k] + 1e-6 * np.eye(D)
    pi = Nk / len(X)
```

Always work in log space and always add a small ridge to the covariance: a component that captures a single point drives its covariance to zero and the likelihood to infinity. That singularity is the classic GMM failure.

37.4 Hidden Markov models and dynamic programming

An HMM adds time: a hidden state evolves as a Markov chain and each state emits an observation. It remains the right model when states are genuinely discrete and interpretable - activity recognition, keyword spotting, gene finding, fault modes - and it is far cheaper than a recurrent network.

Parameters	π (initial), A (transitions), B (emissions)
Forward	$\alpha_t(j) = b_j(o_t) \sum_i \alpha_{t-1}(i) A_{ij}$
Backward	$\beta_t(i) = \sum_j A_{ij} b_j(o_{t+1}) \beta_{t+1}(j)$
Likelihood	$p(0) = \sum_i \alpha_T(i)$
Viterbi	$d_t(j) = b_j(o_t) \max_i d_{t-1}(i) A_{ij}$ (+ backpointers)
Learning	Baum-Welch = EM with forward-backward in the E-step

Forward sums over paths; Viterbi maximizes over paths. Same recursion, different semiring - and the same trick as beam search

FROM ZERO — the idea explained step by step

A **hidden Markov model** has hidden states (e.g. activities) that change over time with fixed probabilities, and each state produces observations.

IN OUR PROJECT — not used, and why

Not used. It would fit HAR on a stream: activities last many windows, so smoothing predictions over time with an HMM would remove isolated errors.

in Chapter 24.

VITERBI ON THREE STEPS, BY HAND

States Rain and Sun; start 0.6/0.4; transitions Rain->Rain 0.7, Sun->Sun 0.6; emissions $p(\text{umbrella} | \text{Rain}) = 0.9$, $p(\text{umbrella} | \text{Sun}) = 0.2$. Observe umbrella, umbrella, no-umbrella. $t=1$: Rain $0.6 \cdot 0.9 = 0.54$, Sun $0.4 \cdot 0.2 = 0.08$. $t=2$: Rain = $0.9 \cdot \max(0.54 \cdot 0.7, 0.08 \cdot 0.4) = 0.9 \cdot 0.378 = 0.340$; Sun = $0.2 \cdot \max(0.54 \cdot 0.3, 0.08 \cdot 0.6) = 0.2 \cdot 0.162 = 0.0324$. $t=3$: Rain = $0.1 \cdot \max(0.340 \cdot 0.7, 0.0324 \cdot 0.4) = 0.0238$; Sun = $0.8 \cdot \max(0.340 \cdot 0.3, 0.0324 \cdot 0.6) = 0.8 \cdot 0.102 = 0.0816$. The best final state is Sun, and its backpointer chain gives Rain, Rain, Sun. Note that the third observation flipped the state even though Rain was far ahead - evidence propagates, which is the point of the model.

Work in log space for any real sequence: alpha underflows within a few dozen steps in single precision. The Kalman filter is the same set of recursions for continuous states with linear-Gaussian dynamics, and a particle filter is their Monte Carlo version for nonlinear ones.

37.5 Gaussian processes: distributions over functions

Instead of a prior over weights, put a prior directly over functions. A GP says: for any finite set of inputs, the function values are jointly Gaussian, with covariance given by a kernel $k(x, x')$ that encodes how similar two inputs are.

FROM ZERO — the idea explained step by step

Gaussian processes put a distribution over functions.

IN OUR PROJECT — not used, and why

Not used in the project.

Prior $f \sim \text{GP}(m(x), k(x, x'))$, usually $m = 0$
RBF kernel $k(x, x') = s_f^2 \exp(-||x - x'||^2 / (2 l^2))$
Posterior mean $\mu^* = K_* [K + s_n^2 I]^{-1} y$
Posterior cov $S^* = K_{**} - K_* [K + s_n^2 I]^{-1} K_*^T$

K is (n, n) over training inputs, K_* is (m, n) between test and training inputs, K_{**} is (m, m) over test inputs.

Two hyperparameters carry the meaning. The **lengthscale** l is how far you must move in input space before the function may change; the **signal variance** s_f^2 is how far it may move vertically. Both are fitted by maximizing the marginal likelihood, which trades data fit against model complexity automatically - Occam's razor falls out of the log-determinant term rather than being imposed.

A TWO-POINT GP, COMPUTED

Observe $f(0) = 1$ and $f(2) = 3$ with noise variance 0.01, an RBF kernel with $s_f = 1$ and $l = 1$. Then $k(0,0) = k(2,2) = 1$ and $k(0,2) = \exp(-4/2) = 0.1353$, so $K + s_n^2 I = \begin{bmatrix} 1.01 & 0.1353 \\ 0.1353 & 1.01 \end{bmatrix}$. Solving that system against $y = (1, 3)$ gives $\alpha = (0.6029, 2.8895)$. Predict at $x^* = 1$: $k_* = (\exp(-0.5), \exp(-0.5)) = (0.6065, 0.6065)$, so the posterior mean is $0.6065(0.6029 + 2.8895) = 2.12$ and the posterior variance is $1 - k_*^T K^{-1} k_* = 0.358$, a standard deviation of 0.60. Midway between two observations one lengthscale apart, the GP interpolates and still reports substantial uncertainty. At $x^* = 10$ the kernel vector is numerically zero, so the mean returns to the prior mean 0 and the variance to 1 - a GP knows when it is extrapolating, which is the property no plain neural network has for free.

Property	Gaussian process	Neural network
Cost	$O(n^3)$ to fit, $O(n^2)$ per prediction	$O(n)$ per epoch
Practical data size	Up to ~10k exactly; sparse/inducing methods beyond	Unbounded
Uncertainty	Exact and calibrated under the prior	Requires ensembles or approximations (Chapter 33)
Priors	Explicit in the kernel: smoothness, periodicity, linear trends	Implicit in architecture and initialization
High-dimensional raw input	Weak without a learned kernel	The reason deep learning exists

The killer application is **Bayesian optimization** : when each evaluation is expensive - a training run, a lab experiment, a hardware measurement - fit a GP to the results so far and pick the next point by maximizing an acquisition function such as expected improvement or the upper confidence bound $\mu(x) + \kappa \sigma(x)$. This is why GPs are the standard engine inside hyperparameter tuners for budgets of tens to a few hundred trials, where random search is wasteful and grid search is hopeless.

37.6 When the integral is intractable: VI and MCMC

Conjugacy is a luxury. For everything else there are two families of approximation, and the choice between them is the classic speed-versus-accuracy trade in probabilistic modelling.

Variational inference: pick $q(\theta)$ in a simple family,
 maximize $\text{ELBO}(q) = \mathbb{E}_q[\log p(D, \theta)] - \mathbb{E}_q[\log q(\theta)]$
 $= \log p(D) - \text{KL}(q || p(\theta|D))$
 so maximizing the ELBO minimizes KL to the true posterior.

This is the same ELBO as the VAE in Chapter 25 - there, q is produced by an encoder network instead of being optimized per

FROM ZERO — the idea explained step by step

Variational inference and MCMC approximate posteriors that cannot be computed exactly.

IN OUR PROJECT — not used, and why

Not used in the project.

data point.

	Variational inference	MCMC
Idea	Turn integration into optimization	Sample a chain whose stationary distribution is the posterior
Speed	Fast, scales to large data with mini-batches	Slow; typically thousands of passes
Bias	Biased: limited to the chosen family; mean-field q underestimates variance	Asymptotically exact
Workhorse	ADVI, Bayes-by-backprop, VAEs	Metropolis-Hastings, Gibbs, HMC / NUTS
Diagnosics	ELBO trace; compare families	R-hat < 1.01, effective sample size, divergences

```
# Metropolis-Hastings: the entire algorithm
theta = theta_init
for t in range(T):
    prop = theta + step * np.random.randn(*theta.shape)
    log_ratio = log_post(prop) - log_post(theta) # prior + likelihood
    if np.log(np.random.rand()) < log_ratio:
        theta = prop # accept
    chain.append(theta)
# Tune 'step' for an acceptance rate near 0.234 for high-dimensional
# random-walk proposals; discard the first half as burn-in.
```

THE THREE WAYS PROBABILISTIC MODELS ARE MISUSED

Reporting a posterior interval as if it were a frequentist confidence interval when the prior was chosen carelessly; running a chain for a few hundred steps and never checking R-hat, so the 'posterior' is a record of where the chain happened to start; and using mean-field VI and then quoting its variance, which is systematically too small. All three produce confident numbers that are wrong in the same direction: too certain.

37.7 Choosing your weapon

Situation	Use
Under ~1,000 rows, uncertainty matters	Bayesian model with conjugate or GP structure
Cheap ranking of items with small counts	Beta posterior mean or lower bound
Hidden discrete structure over time	HMM (Baum-Welch); a recurrent net only if you have far more data
Expensive black-box tuning, < 200 trials	GP Bayesian optimization
Millions of rows, uncertainty is secondary	Deep model plus an ensemble or temperature scaling (Chapter 33)
You need a probability you will act on financially	Full posterior if the model is small; deep ensemble plus calibration if it is not

FROM ZERO — the idea explained step by step

Choose the simplest probabilistic tool that answers the question.

IN OUR PROJECT — partly used

For the thesis: the Beta interval for accuracy, PPCA for replay memory -- nothing heavier.

Exercises

1 Derive the Beta-Bernoulli posterior from Bayes' rule and confirm that the posterior mean lies between the prior mean and the MLE. 2 Implement the ten-line GMM above and run it on the four points 1, 2, 8, 9. Verify the responsibilities quoted in the box. 3 Implement Viterbi and reproduce the umbrella example, then check that the forward algorithm gives the total probability of the observation sequence. 4 Fit a GP with an RBF kernel to five points of a sine wave and plot the mean with two-standard-deviation bands. Increase the lengthscale by 10x and explain what happens to both. 5 Take a logistic regression with 200 rows, fit it by MAP and then by Metropolis-Hastings, and compare the coefficient intervals with the bootstrap intervals from Chapter 13.

38. Time Series and Forecasting

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Our data are time series: 50 samples per second, cut into windows. The tools of this chapter -- autocorrelation, frequency, sliding windows -- describe exactly how the HAR data were prepared.

Time series break the assumption every earlier chapter rested on: the rows are not independent, and the future is not exchangeable with the past. That single change invalidates random splits, ordinary cross-validation, most feature engineering habits, and the intuition that more model capacity helps. It also creates the most common catastrophic bug in applied machine learning - a feature that quietly contains the future.

38.1 What makes a time series different

Property	Consequence
Observations are ordered and autocorrelated	Random shuffling leaks the future into training; only time-ordered splits are valid
The distribution drifts (non-stationarity)	A model fitted on 2019 may be structurally wrong in 2024, not just stale
Seasonality at several periods at once	Daily, weekly and yearly cycles superimpose; each needs its own representation
The horizon is part of the problem	Predicting one step ahead and 28 steps ahead are different tasks with different achievable errors
Data is revised after the fact	The value you have today for last Tuesday may not be the value that was available last Tuesday

FROM ZERO — the idea explained step by step

A **time series** is a sequence of measurements in time order. Neighbouring values depend on each other, so the order must not be shuffled away, and random splits leak information.

IN OUR PROJECT — used

Our data: 9 sensor signals at 50 samples per second. The UCI windows overlap by 50%, so neighbouring windows share half their samples -- a random split would put nearly identical windows in training and test. That is why the official split is by person (21 training, 9 test people, overlap 0) [9].

THE NUMBER ONE BUG IN FORECASTING

Computing any statistic over the whole series before splitting: a global mean and standard deviation for scaling, a target encoding, an imputation fill, a de-seasonalizing step. Every one of them carries future information backwards and produces a backtest that cannot be reproduced in production. Fit every transformation on the training window only, then apply it forward.

38.2 Decomposition: trend, seasonality, remainder

Additive $y_t = T_t + S_t + R_t$
Multiplicative $y_t = T_t * S_t * R_t$ (log => additive)

Use multiplicative when the seasonal swing grows with the level - retail sales, traffic. Taking logs first is usually simpler than

FROM ZERO — the idea explained step by step

Decompose a series into trend + repeating (seasonal) part + remainder.

IN OUR PROJECT — partly used

The total acceleration of HAR is gravity (a slow "trend", depending on posture) + body motion (the fast part). The dataset already separates them with a low-pass filter: `total_acc = body_acc + gravity` [9].

changing model families.

STL decomposition (seasonal-trend by LOESS) is the workhorse: robust to outliers, handles a changing seasonal shape, and gives you three series to inspect. Plot them before modelling anything. Most 'the model is broken' reports resolve into 'the seasonality changed in March' once the decomposition is on screen.

38.3 Stationarity, differencing, ACF and PACF

A series is (weakly) stationary if its mean, variance and autocovariance do not depend on t . Classical models require it; ML models tolerate non-stationarity only if you give them the right features. Differencing is the standard cure.

First difference	$z_t = y_t - y_{(t-1)}$	removes a linear trend
Seasonal difference	$z_t = y_t - y_{(t-m)}$	removes a period- m cycle
ACF(k)	$= \text{corr}(y_t, y_{(t-k)})$	total correlation at lag k
PACF(k)	$= \text{corr of the part of } y_t \text{ and } y_{(t-k)} \text{ not explained by lags } 1..k-1$	

FROM ZERO — the idea explained step by step

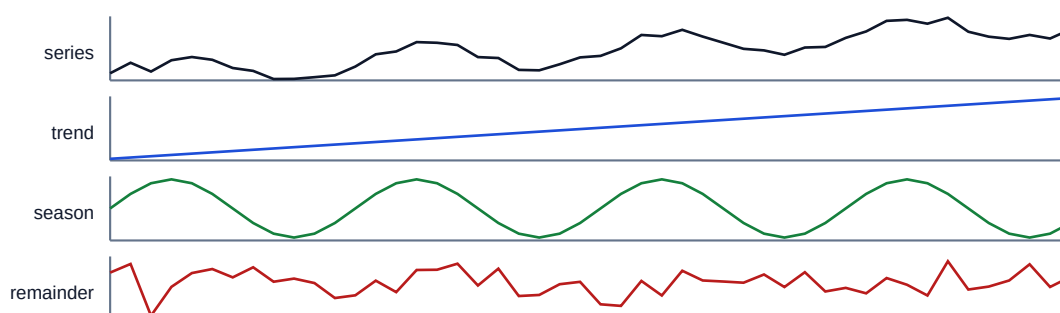
Autocorrelation (ACF): how similar is the series to itself shifted by a lag? Peaks show repeating patterns.

IN OUR PROJECT — used

For walking the autocorrelation peaks at a lag of 29 samples = 0.58 s -- one step. The frequency spectrum agrees: the main peak is at 1.9531 Hz (frequency resolution 0.3906 Hz, measured).

WORKED ON OUR PROJECT — The walking rhythm in the frequency domain

1. Take the `body_acc_x` signal of every walking window, remove its mean, and average the magnitudes of the FFT (128 points at 50 Hz, frequency resolution $50 / 128 = 0.3906$ Hz).
2. The strongest frequency is 1.9531 Hz: about two steps per second, a normal walking cadence.
3. This is why a kernel covering about 0.5 s (conv3's receptive field, Chapter 21) is a sensible scale for this data.
4. Classical HAR features (the 561 features shipped with the dataset) include exactly such FFT energies [9].



Graph 38.1 — A time series split into its parts (made-up monthly data): a slow trend, a repeating seasonal pattern, and what is left over. Forecasting models handle each part differently.

Pattern in the plots	Reading
ACF decays slowly, almost linearly	Non-stationary: difference it
ACF cuts off after lag q , PACF decays	MA(q)

Pattern in the plots	Reading
PACF cuts off after lag p, ACF decays	AR(p)
Both decay	ARMA(p,q); choose orders by AIC
Spike at lag m and its multiples	Seasonal component of period m

The augmented Dickey-Fuller test formalizes the first row, but the plot is usually enough and always more informative. Note that differencing twice when once was enough injects noise - prefer the smallest d that makes the ACF decay quickly.

38.4 The classical families

AR(p) $y_t = c + \sum_{i=1..p} \phi_i y_{(t-i)} + e_t$
MA(q) $y_t = c + e_t + \sum_{j=1..q} \theta_j e_{(t-j)}$
ARIMA(p,d,q) = ARMA(p,q) applied to the d-times differenced series
SARIMA(p,d,q)(P,D,Q)_m adds the same structure at the seasonal lag m
ETS exponential smoothing of Error, Trend, Seasonal components

FROM ZERO — the idea explained step by step

Classical models: ARIMA, exponential smoothing.

IN OUR PROJECT — not used, and why

Not used; the task is classification, not forecasting.

AR(1) BY HAND, AND WHAT ITS COEFFICIENT MEANS

Series 10, 12, 11, 13, 12 with mean 11.6. Centred: -1.6, 0.4, -0.6, 1.4, 0.4. The lag-1 autocovariance is the average of the products of consecutive centred values: $(-1.6)(0.4) + (0.4)(-0.6) + (-0.6)(1.4) + (1.4)(0.4) = -0.64 - 0.24 - 0.84 + 0.56 = -1.16$, divided by 5 gives -0.232. The variance is $(2.56 + 0.16 + 0.36 + 1.96 + 0.16)/5 = 1.04$. So $\phi = -0.232/1.04 = -0.223$: a negative coefficient, meaning the series alternates - a high value tends to be followed by a low one. The forecast for $t = 6$ is $11.6 + (-0.223)(12 - 11.6) = 11.51$, and every further step decays geometrically back to the mean. An AR(1) forecast reverts to the mean at rate ϕ^h ; if your business problem needs a long horizon, an AR model will give you a flat line and be right to do so.

Holt-Winters (ETS with additive trend and seasonality) remains an excellent default for short, seasonal, low-noise series and takes three smoothing parameters. In the M4 and M5 forecasting competitions, simple statistical methods and their combinations beat most sophisticated entries; the winning entries were hybrids that used a global learned model on top of classical structure. **Always fit the classical baseline first.**

38.5 The machine-learning approach: supervised windows

Reframe forecasting as regression: build a table whose row t contains features known strictly before t and whose target is y at $t + h$. This is the approach that wins most business forecasting problems, because it absorbs covariates - price, promotions, weather, holidays - that classical models handle awkwardly.

FROM ZERO — the idea explained step by step

The ML approach: cut the series into fixed-length **windows** and treat each window as one example.

IN OUR PROJECT — used

Exactly our input: windows of 128 samples (2.56 s), 9 channels, one label per window [9].

```
# Direct multi-horizon framing: one model per horizon h
def make_features(df, h):
    X = pd.DataFrame(index=df.index)
    for lag in [h, h+1, h+2, h+7, h+14, h+28]:      # never lag < h
        X[f'lag_{lag}'] = df['y'].shift(lag)
    for w in [7, 28]:
        X[f'roll_mean_{w}'] = df['y'].shift(h).rolling(w).mean()
        X[f'roll_std_{w}'] = df['y'].shift(h).rolling(w).std()
    X['dow'] = df.index.dayofweek
    X['month'] = df.index.month
    X['is_holiday'] = df['holiday'].astype(int)      # known in advance
    y = df['y'].shift(-0)                          # target at time t
    return X, y
```

The shift(h) on every rolling statistic is the whole game: at the moment of prediction you know values up to $t - h$, and nothing after.

Write this function once, review it twice, and never compute a rolling window without an accompanying shift.

Strategy	How	Trade-off
Recursive	One one-step model, feed predictions back in	Cheap; errors compound over the horizon
Direct	One model per horizon h	No compounding; h models to train and maintain
Multi-output	One model predicting the whole horizon vector	Shares structure; needs a model that supports vector targets

LOCAL VERSUS GLOBAL MODELS

A local model is fitted per series (one ARIMA per SKU). A global model is one model fitted across all series with the series identity as a feature or embedding. Global models win when you have many related series with short histories - they borrow strength - and they are how gradient boosting and modern deep forecasters take retail problems with 50,000 SKUs. Local models win when series are few, long and genuinely unrelated.

38.6 Deep learning for forecasting

Model	Idea	When it is worth it
DeepAR	Autoregressive RNN emitting distribution parameters per step	Many related series, probabilistic output required
N-BEATS / N-HiTS	Deep stacks of basis expansions with backcast/forecast residuals	Univariate benchmarks, no covariates needed
Temporal Fusion Transformer	Attention over time plus gated variable selection	Many covariates, and you need interpretable attention over them
PatchTST / Transformers	Patch the series into tokens, apply a Transformer	Long horizons with abundant history
Foundation forecasters	Pretrained across millions of series, used zero-shot	Cold-start series and rapid prototyping

FROM ZERO — the idea explained step by step

Deep models for series: 1-D CNNs, RNNs, Transformers.

IN OUR PROJECT — used

Our model is a 1-D CNN (Chapter 21).

THE HONEST STATE OF THE ART

On standard long-horizon benchmarks, a well-tuned linear model on lagged inputs matches or beats several published Transformer forecasters, a result that survived careful replication. Deep forecasting earns its keep with many series, rich covariates, and distributional output - not with a single univariate series, where ETS or gradient boosting on lag features is usually both better and a hundred times cheaper.

38.7 Backtesting: the only evaluation that counts

```
|----- train -----|gap|-- test --|          fold 1
|----- train -----|gap|-- test --|          fold 2
|----- train -----|gap|-- test --|          fold 3
expanding window (above) keeps all history;
a rolling window drops the oldest data instead - use it when
the process changes and old data actively misleads.
```

Rolling-origin evaluation. The gap must be at least the forecast horizon, so no training row is closer to a test row than the horizon

FROM ZERO — the idea explained step by step

Backtesting: test only on data that come after the training data in time.

IN OUR PROJECT — partly used

For HAR the equivalent is testing on unseen people; both avoid leaking near-copies of training data into the test.

WORKED ON OUR PROJECT — Why windows must not be shuffled across time for forecasting

1. For classification of independent windows, shuffling is fine. For forecasting (predicting the next seconds), training on later windows and testing on earlier ones leaks the future.
2. Backtesting splits by time: train on minutes 0-10, test on 10-12, then move forward.
3. The overlap measured in Chapter 3 (92.1% of neighbouring windows share half their samples) makes this even more important.

you claim to predict.

Rules that make a backtest believable: the test folds are contiguous and in the future; every transformation is refitted inside each fold; the horizon in the backtest equals the horizon in production; and the number of folds is large enough that you are averaging over several regimes, not one lucky quarter. Report the metric per fold, not only its mean - the spread across folds is the number that predicts how the model will behave next quarter.

38.8 Metrics, and the one that is quietly broken

```
MAE   = mean |y - yhat|          units of y, robust
RMSE  = sqrt( mean (y - yhat)^2 ) punishes large misses
MAPE  = mean |y - yhat| / |y| * 100 BREAKS when y ~ 0
sMAPE = mean 2|y - yhat| / (|y| + |yhat|) asymmetric in practice
MASE  = MAE(model) / MAE(seasonal naive on the training set)
Pinball_q = mean [ q(y - yhat) if y >= yhat else (1-q)(yhat - y) ]
```

MASE is the safe default: scale-free, defined at zero, and below 1 exactly when you beat the naive baseline.

FROM ZERO — the idea explained step by step

Forecasting metrics (MAE, MAPE, MASE) and their pitfalls.

IN OUR PROJECT — not used, and why

Not used; we report accuracy.

MASE COMPUTED

A daily series where the seasonal-naive (last week's same weekday) forecast has MAE 12.0 on the training set. Your model scores MAE 9.6 on the backtest. $MASE = 9.6 / 12.0 = 0.80$: you are 20% better than doing nothing clever. If $MASE \geq 1$ - which happens more often than anyone admits - the model is worse than a one-line baseline and should not ship. Always report the naive baseline next to the model.

For probabilistic forecasts, evaluate the interval, not just the point: report pinball loss at the quantiles you actually use, and check **coverage** - the fraction of actuals falling inside the 80% interval should be 80%, not 55%. Inventory, staffing and capacity decisions all consume a quantile, not a mean, so a model with a good mean and a miscalibrated tail is the wrong model.

38.9 Hierarchies, intermittency and anomalies

- **Hierarchical forecasting.** Store-level forecasts must sum to region-level and national forecasts. Forecast every level independently, then reconcile - MinT reconciliation is the standard method and usually improves accuracy at every level, not only consistency.

FROM ZERO — the idea explained step by step

Hierarchies, intermittent series and anomalies in series.

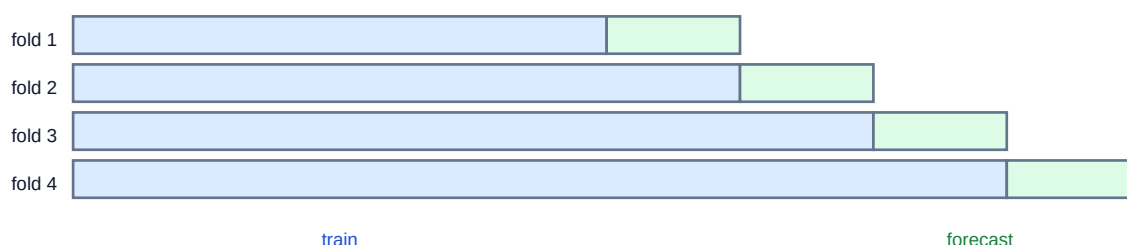
IN OUR PROJECT — partly used

Anomalies in a series: the ECG autoencoder example flags abnormal heartbeats [35].

- **Intermittent demand.** Series that are zero most days (spare parts) break MAPE and mislead RMSE. Croston's method or a two-part model (probability of a nonzero day x expected size) is the right shape.
- **Anomaly detection.** Fit a forecast, then flag points whose residual exceeds a robust threshold, for example a multiple of the median absolute deviation. Seasonal-hybrid ESD is the classical choice. Anomalies must then be replaced in the training window or the next forecast will chase them.
- **Change points.** A structural break - a pricing change, a pandemic, a new competitor - is not an outlier to be smoothed. Detect it, and either restrict the training window or add an indicator feature.

Exercises

1 Take any daily series, plot the STL decomposition, and write one sentence describing each of the three components. 2 Compute the AR(1) coefficient by hand on the five numbers in the box, then confirm it with a library fit.



Graph 38.2 — Backtesting with a rolling origin. Train on the past up to a cut-off, forecast the next block, move the cut-off forward, repeat. Never train on anything that comes after the block being tested.

3 Build the supervised window table for $h = 7$, then deliberately remove the $\text{shift}(h)$ from one rolling feature and measure how much the backtest score improves. That improvement is the size of the leak you would have shipped. 4 Run a five-fold rolling-origin backtest of a seasonal-naive baseline and a gradient-boosted model, and report MASE per fold for both. 5 Produce 10th, 50th and 90th percentile forecasts, measure empirical coverage of the 80% interval, and calibrate it if it is off.

39. Recommender Systems

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Recommender systems are not used in the project.

Recommendation is the highest-revenue application of machine learning in existence and the one whose textbook description least resembles its practice. The differences: the label depends on what you showed, so the data is generated by your own past model; the item catalogue is too large to score exhaustively; and the offline metric agrees with the online outcome only sometimes.

39.1 The industrial shape of the problem

```

millions of items
|
[ RETRIEVAL ]  cheap, recall-oriented, many sources in parallel
|              (two-tower ANN, co-visitation, popularity, rules)
~ 500 candidates
|
[ RANKING ]    expensive model, rich features, one score per item
|              (GBDT or deep ranker, tens of ms budget)
~ 50 ranked
|
[ RE-RANKING ] diversity, freshness, business rules, dedup, ads
|
10 shown
  
```

The funnel. Each stage has a different objective and a different cost budget; confusing them is the most common architectural

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to recommender systems.

IN OUR PROJECT — not used, and why

Not used in the project.

mistake.

Stage	Objective	Latency budget	Metric
Retrieval	Do not lose the good items	1-10 ms over millions	Recall@k
Ranking	Order the survivors correctly	10-50 ms over hundreds	NDCG, AUC, calibrated CTR
Re-ranking	Make the list good as a set	< 5 ms	Diversity, coverage, business KPIs

39.2 Feedback data and what it really means

Signal	Density	Bias
Explicit ratings	Very sparse (< 1% of pairs)	Only motivated users rate; J-shaped distribution
Clicks	Dense	Position bias, popularity bias, clickbait
Dwell time / completion	Dense	Better proxy for satisfaction; content-length confound
Purchases / subscriptions	Sparse but decisive	Delayed, and attribution is contested
Explicit negatives (hide, not interested)	Very sparse	High precision, worth weighting heavily

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to recommender systems.

IN OUR PROJECT — not used, and why

Not used in the project.

MISSING IS NOT NEGATIVE

A user did not click an item either because they disliked it or because they never saw it. Treating all non-interactions as negatives biases the model towards whatever your previous system happened to show. The standard corrections are confidence weighting (implicit ALS), sampled negatives drawn from the catalogue rather than from impressions, and inverse-propensity weighting by the probability the item was shown.

39.3 Neighbourhood collaborative filtering

Item-item score:

$$s(u, i) = \frac{\sum_{j \text{ in items rated by } u} \text{sim}(i, j) r_{uj}}{\sum_j |\text{sim}(i, j)|}$$

Cosine similarity: $\text{sim}(i, j) = (r_i \cdot r_j) / (||r_i|| ||r_j||)$

Item-item beats user-user in practice: item vectors are denser, more stable over time, and precomputable.

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to recommender systems.

IN OUR PROJECT — not used, and why

Not used in the project.

ITEM-ITEM ON A TINY MATRIX

Three users, ratings on items A, B, C: $u_1 = (5, 4, ?)$, $u_2 = (4, 5, 2)$, $u_3 = (1, 2, 5)$. Column A = (5,4,1), B = (4,5,2), C = (?,2,5) - use (0,2,5) for the cosine over co-rated users only. $\text{sim}(A,B) = (20+20+2)/(\sqrt{42} \sqrt{45}) = 42/43.5 = 0.966$; $\text{sim}(A,C)$ using users 2 and 3 = $(8+5)/(\sqrt{17} \sqrt{29}) = 13/22.2 = 0.586$. Predicting C for u_1 from their ratings of A and B: $(0.586(5) + \text{sim}(B,C)(4)) / (0.586 + \text{sim}(B,C))$. Over users 2 and 3, $\text{sim}(B,C) = (10 + 10)/(\sqrt{29} \sqrt{29}) = 0.690$, so the prediction is $(2.93 + 2.76)/1.276 = 4.46$ - which is too high, and usefully so: u_1 looks like u_2 , who rated C low. The fix is to centre each user's ratings before computing similarities, which removes the 'this user rates everything highly' effect. Mean-centring is not optional in neighbourhood CF.

39.4 Matrix factorization, the model that won Netflix

```
rhat_ui = mu + b_u + b_i + p_u . q_i
min SUM_(u,i in observed) (r_ui - rhat_ui)^2
    + lambda ( ||p_u||^2 + ||q_i||^2 + b_u^2 + b_i^2 )
SGD updates, with e_ui = r_ui - rhat_ui:
b_u <- b_u + lr (e_ui - lambda b_u)
p_u <- p_u + lr (e_ui q_i - lambda p_u)
q_i <- q_i + lr (e_ui p_u - lambda q_i)
```

The bias terms alone - global mean, user leniency, item popularity - capture most of the achievable improvement over the global

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to recommender systems.

IN OUR PROJECT — not used, and why

Not used in the project.

WORKED ON OUR PROJECT — A subject x activity table, and why it is almost rank 1

1. Count training windows per person and activity: 21 people x 6 activities. First person: WALKING 95, UPSTAIRS 53, DOWNSTAIRS 49, SITTING 47, STANDING 53, LAYING 50.
2. The largest singular value carries 98.5% of the table's energy: the table is almost "person size x activity profile".
3. In recommender terms (Section 39.4), a rank-1 factorisation already explains it: everybody did the same protocol, for different lengths of time.
4. The project does not use recommenders; the example shows what a low-rank structure looks like on familiar data.

average. Fit them before adding factors.

ONE SGD STEP, ARITHMETIC INCLUDED

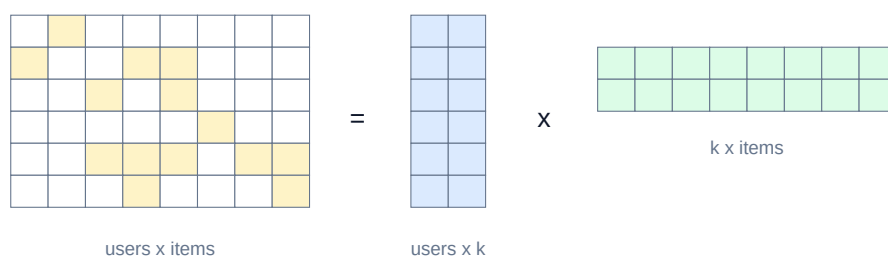
$\mu = 3.5$, $b_u = 0.2$, $b_i = -0.3$, $p_u = (0.1, 0.4)$, $q_i = (0.5, -0.2)$, $lr = 0.01$, $\lambda = 0.05$, true rating 5. Prediction $= 3.5 + 0.2 - 0.3 + (0.05 - 0.08) = 3.37$. Error $e = 1.63$. Then b_u becomes $0.2 + 0.01(1.63 - 0.05(0.2)) = 0.2162$; p_u becomes $(0.1, 0.4) + 0.01(1.63(0.5, -0.2) - 0.05(0.1, 0.4)) = (0.1 + 0.00810, 0.4 - 0.00346) = (0.1081, 0.3965)$. The first component of p_u moved towards q_i because the item was under-predicted; the second moved away because q_i 's second component is negative. That is all matrix factorization does, a hundred million times.

For implicit feedback, the objective changes shape: every unobserved pair enters the sum with low confidence rather than being ignored.

```
Implicit ALS: min SUM_(all u,i) c_ui ( pref_ui - p_u . q_i )^2 + reg
pref_ui = 1 if any interaction else 0
c_ui    = 1 + alpha * count_ui      (confidence grows with count)
BPR (pairwise): max SUM log sigmoid( p_u.q_i - p_u.q_j )
i = an item the user interacted with, j = a sampled unseen item
```

ALS is embarrassingly parallel and closed-form per user; BPR optimizes ranking directly and is the better choice when you only care about the order of the top few.

39.5 Neural retrieval and sequential models



Graph 39.1 — Matrix factorisation for recommenders. The big, mostly-empty users x items rating matrix is approximated by a thin users x k matrix times a thin k x items matrix; the product fills in the empty cells as predictions.

The **two-tower** model is the modern retrieval workhorse: one encoder for the user context, one for the item, trained so that the dot product of their embeddings predicts interaction. Because the item tower does not see the user, all item embeddings can be precomputed and indexed; retrieval becomes approximate nearest-neighbour search.

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to recommender systems.

IN OUR PROJECT — not used, and why

Not used in the project.

```
# Two-tower training with in-batch negatives (sampled softmax)
u = user_tower(user_features)          # (B, d)
v = item_tower(item_features)          # (B, d), the positives
logits = u @ v.T / temperature          # (B, B); diagonal = positives
# correct for popularity: subtract log of sampling probability
logits = logits - torch.log(item_freq).unsqueeze(0)
loss = F.cross_entropy(logits, torch.arange(len(u), device=u.device))
```

Every other item in the batch serves as a negative, which is why large batches matter here. The log-frequency correction is essential: without it the model learns to retrieve popular items and nothing else.

Index	Structure	Trade-off
Flat / brute force	Exact dot product over all items	Perfect recall; fine up to ~1M items with a GPU
IVF	Cluster items, search a few clusters	Tunable via nprobe; needs training
HNSW	Navigable small-world graph	Best latency/recall; high memory, slow to build
Product quantization	Compress vectors to codes	8-32x memory saving; some recall loss - the same trade as Chapter 28

Sequential recommenders treat a user's history as a sequence and predict the next item: GRU4Rec used a recurrent net, SASRec a causal Transformer, BERT4Rec a masked one. They are the strongest family when order carries meaning - sessions, media consumption, learning paths - and they connect this chapter directly to Chapters 22 to 24: the architecture is identical, only the vocabulary is items instead of tokens.

39.6 Ranking and its metrics

The ranker sees hundreds of candidates and rich features: user history aggregates, item statistics, context (time, device, position), and cross features. Gradient-boosted trees with a ranking objective (LambdaMART) remain extremely strong; deep rankers (Wide and Deep, DLRM, DCN) win when there are many high-cardinality categorical features whose interactions matter.

```
Precision@k = (relevant items in top k) / k
Recall@k    = (relevant items in top k) / (all relevant items)
DCG@k       = SUM_(i=1..k) rel_i / log2(i + 1)
NDCG@k      = DCG@k / IDCG@k      (IDCG = DCG of the perfect ordering)
MRR         = mean of 1 / (rank of the first relevant item)
```

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to recommender systems.

IN OUR PROJECT — not used, and why

Not used in the project.

NDCG@3 BY HAND

Relevances of the three items you showed, in the order you showed them: 1, 0, 2. $DCG = 1/\log_2(2) + 0/\log_2(3) + 2/\log_2(4) = 1/1 + 0 + 2/2 = 2.00$. The ideal ordering is 2, 1, 0, giving $IDCG = 2/1 + 1/1.585 + 0 = 2 + 0.631 = 2.631$. $NDCG@3 = 2.00/2.631 = 0.760$. Now swap your first two items to get 0, 1, 2: $DCG = 0 + 1/1.585 + 1 = 1.631$ and $NDCG$ falls to 0.620. The logarithmic discount is what makes the metric care about the top of the list - moving a relevant item from position 2 to position 1 is worth far more than moving one from position 9 to position 8, which is exactly how users behave.

OFFLINE METRICS DISAGREE WITH ONLINE OUTCOMES

An offline $NDCG$ improvement can lose an A/B test, routinely. The three reasons: your offline data only contains items the old system showed, so a genuinely better retrieval looks worse; position bias inflates whatever the old ranker put on top; and the online objective (long-term retention) is not the offline label (this session's click). Treat offline metrics as a filter that decides what is worth testing, never as the decision itself.

39.7 The problems that only recommenders have

Problem	What happens	Standard treatment
Cold start (item)	New items have no interactions and are never shown	Content features in the item tower; explicit exploration budget
Cold start (user)	No history to personalize on	Popularity by segment, onboarding preferences, contextual bandits
Position bias	Top slots get clicks regardless of relevance	Inverse-propensity weighting; randomized-slot data collection
Popularity bias	The rich get richer; the tail dies	Frequency-corrected sampling, exposure-aware objectives
Feedback loop	Model trains on its own recommendations and narrows	Logged exploration data, off-policy correction, diversity constraints
Filter bubble / harms	Users are pushed towards ever narrower or more extreme content	Diversity and novelty objectives, content-quality classifiers, explicit ranking policy

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to recommender systems.

IN OUR PROJECT — not used, and why

Not used in the project.

Off-policy evaluation gives a partial answer to 'how would the new ranker have done?' using logged data, by importance-weighting each logged interaction by the ratio of new-policy to logging-policy probability. Inverse propensity scoring is unbiased but high-variance; clipped and doubly-robust estimators trade a little bias for a lot of variance. All of them require that the logging policy was **stochastic** and that its probabilities were logged - a decision you must make before you need it, not after.

THE ONE-WEEK RECOMMENDER

Day 1: popularity baseline by segment, measured. Day 2: item-item co-visitation from the last 30 days - typically beats popularity by a wide margin and costs nothing. Day 3-4: implicit ALS or a two-tower model for retrieval. Day 5: a GBDT ranker over the candidates with twenty features. Day 6: diversity and dedup rules. Day 7: A/B test with a holdout that never receives personalization, so you can measure the value of the whole system for as long as it runs.

Exercises

1 Compute NDCG@5 for two orderings of the same five items by hand, then verify with a library. 2 Build the item-item co-visitation baseline on any interaction log and measure Recall@20 against a time-based holdout. 3 Implement the matrix-factorization SGD update and reproduce the arithmetic in the worked box exactly. 4 Train a two-tower model with and without the log-frequency correction and compare the popularity distribution of what each retrieves. 5 Take a logged ranking dataset, estimate position bias by comparing click rates of the same item in different slots, and re-weight your training data accordingly.

40. Computer Vision Beyond Classification

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Detection and segmentation are image tasks; the project works on 1-D signals. One idea carries over: segmenting a stream in time.

Chapter 21 stopped at 'which class is this image?'. Almost no real vision product stops there. This chapter covers what the other tasks are, the geometry and matching machinery they need, the metrics that are easy to misreport, and the deployment details that decide whether a detector runs at 5 or 50 frames per second.

40.1 The task taxonomy

Task	Output per image	Typical metric
Classification	One label (or a multi-label vector)	Accuracy, mAP for multi-label
Object detection	A set of boxes with classes and scores	mAP@[.50:.95]
Semantic segmentation	A class for every pixel	mIoU
Instance segmentation	A mask per object instance	Mask AP
Panoptic segmentation	Instances for things, regions for stuff	Panoptic Quality
Keypoints / pose	Ordered landmark coordinates	OXS-based AP
Depth / normals	A continuous value per pixel	Abs-rel error, delta < 1.25
Multi-object tracking	Boxes plus consistent identities over time	MOTA, IDF1, HOTA

FROM ZERO — the idea explained step by step

Vision tasks: classification, detection, segmentation.

IN OUR PROJECT — not used, and why

Our task is the 1-D analogue of classification: one label per window.

THE STRUCTURAL DIFFERENCE FROM CLASSIFICATION

The output is a set of variable size, not a fixed vector. That single fact forces everything that follows: a matching step between predictions and ground truth, a rule for suppressing duplicates, and a metric that must integrate over both confidence and overlap thresholds.

40.2 IoU, the quantity everything is built on

$$\begin{aligned} \text{IoU}(A, B) &= \text{area}(A \cap B) / \text{area}(A \cup B) \\ &= I / (\text{area}(A) + \text{area}(B) - I) \end{aligned}$$

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to computer vision.

IN OUR PROJECT — not used, and why

Not used in the project.

WORKED ON OUR PROJECT — IoU in time for activity segments

1. Detection on a stream means finding intervals. True walking interval: 10.0-20.0 s. Predicted: 12.56-21.76 s.
2. Intersection $12.56-20.0 = 7.44$ s. Union $10.0-21.76 = 11.76$ s. $\text{IoU} = 7.44 / 11.76 = 0.63$.
3. With window step 1.28 s, predicted boundaries can only land on multiples of 1.28 s -- that limits the best possible IoU for short activities.

IOU COMPUTED ON TWO BOXES

Prediction $(x1,y1,x2,y2) = (10, 10, 50, 50)$, ground truth $= (30, 20, 70, 60)$. The intersection spans x from $\max(10,30) = 30$ to $\min(50,70) = 50$ and y from $\max(10,20) = 20$ to $\min(50,60) = 50$, so it is $20 \times 30 = 600$. The areas are $40 \times 40 = 1600$ each, so the union is $1600 + 1600 - 600 = 2600$ and $\text{IoU} = 600/2600 = 0.231$. That box would count as a false positive at the standard 0.5 threshold despite overlapping substantially - which is why 'the model found the object' and 'the model scored a true positive' are different claims.

Non-maximum suppression turns a dense score map into a set: sort boxes by score, keep the top one, delete every remaining box whose IoU with it exceeds a threshold, repeat. Its two failure modes are worth knowing: crowded scenes lose genuine overlapping objects (soft-NMS decays scores instead of deleting), and NMS is often the latency bottleneck on device because it is sequential and data-dependent.

40.3 Detector families

Family	Representative	Mechanism	Trade-off
Two-stage	Faster R-CNN	A region proposal network proposes, a head classifies and refines with RoIAlign	Most accurate per FLOP historically; slower and more complex
One-stage anchored	RetinaNet, SSD, YOLOv3	Dense predictions over predefined anchor boxes	Fast; needs focal loss or hard negative mining for the 1000:1 background imbalance
Anchor-free	FCOS, CenterNet	Predict object centres and distances to box sides	No anchor hyperparameters to tune; simpler heads
Set prediction	DETR, DINO	Transformer decoder with Hungarian matching to ground truth	No NMS and no anchors; slow to converge originally, largely fixed by later variants
Modern real-time	YOLOv8-class, RT-DETR	Anchor-free heads, heavy augmentation, distillation	The practical default for deployment today

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to computer vision.

IN OUR PROJECT — not used, and why

Not used in the project.



Graph 40.1 — IoU of the two boxes in 40.2: prediction $(10, 10, 50, 50)$, truth $(30, 20, 70, 60)$. They overlap in a $20 \times 30 = 600$ area; the union is $1,600 + 1,600 - 600 = 2,600$; $\text{IoU} = 0.231$ -- below the usual 0.5, so this counts as a miss.

Focal loss $FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$
 $\gamma = 2$ typical: an easy background example with $p_t = 0.99$
 is down-weighted by $(0.01)^2 = 1e-4$ relative to plain cross-entropy

The whole idea: with 100,000 anchors and 5 objects, the sum of many tiny easy-negative losses drowns out the few that matter.

Hungarian matching in DETR replaces both anchors and NMS: predictions and ground-truth objects are matched one-to-one by minimizing a cost combining class probability and box distance, so duplicates are penalized by construction rather than removed afterwards. It is the clearest example in vision of replacing a hand-designed post-processing step with a learned, end-to-end one.

40.4 Segmentation

U-Net: contracting path + expanding path with skip connections
 input --> [conv] --> [conv] --> [conv] ---bottleneck---
 | | | |
 |skip |skip |skip |
 v v v v
 output <-- [up] <---- [up] <---- [up] <-----
 The skips carry high-resolution detail that pooling destroyed;
 without them the mask boundaries are blurred beyond usefulness.

U-Net remains the default for medical and scientific segmentation with small datasets, fifteen years of newer architectures

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to computer vision.

IN OUR PROJECT — not used, and why

Not used in the project.

notwithstanding.

- **FCN** replaced the classifier head with 1x1 convolutions and upsampling - the idea that started the field.
- **U-Net** added symmetric skip connections; it is data-efficient and trains on a few hundred annotated images.
- **DeepLab** used atrous (dilated) convolutions and ASPP to enlarge the receptive field without losing resolution.
- **Mask R-CNN** added a mask head to Faster R-CNN, making instance segmentation a small increment over detection.
- **Transformer-based** (SegFormer, Mask2Former) unified semantic, instance and panoptic segmentation as mask classification.
- **Promptable segmentation** (SAM-style) produces masks from a point or box prompt with no task-specific training, which turns annotation from hours of polygon drawing into seconds of clicking.

mIoU = mean over classes of $TP / (TP + FP + FN)$ at pixel level
Dice = $2 TP / (2 TP + FP + FN)$ (= F1 on pixels)
PQ = $(SUM_matched IoU) / (TP + 0.5 FP + 0.5 FN)$

Dice and IoU are monotonically related but Dice is more forgiving on small objects, which is why medical papers quote it. Report both if your classes are very imbalanced in area.

40.5 Mean average precision, computed honestly

AP is the area under the precision-recall curve for one class at one IoU threshold; mAP averages over classes, and COCO-style mAP averages further over IoU thresholds from 0.50 to 0.95 in steps of 0.05. That second average is why COCO mAP numbers look low: a detector with human-level box placement still loses points at IoU 0.95.

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to computer vision.

IN OUR PROJECT — not used, and why

Not used in the project.

AP FROM FIVE DETECTIONS

One class, 3 ground-truth objects. Sorted by confidence, the detections are TP, FP, TP, TP, FP. Cumulative precision and recall after each: $(1/1, 1/3) = (1.000, 0.333)$; $(1/2, 1/3) = (0.500, 0.333)$; $(2/3, 2/3) = (0.667, 0.667)$; $(3/4, 3/3) = (0.750, 1.000)$; $(3/5, 1.000) = (0.600, 1.000)$. Interpolated precision (the maximum precision at or beyond each recall level) is 1.000 at recall 1/3 and 0.750 at recalls 2/3 and 1. The area, summing precision times the recall increments, is $(1/3)(1.000) + (1/3)(0.750) + (1/3)(0.750) = 0.833$. Note the fourth detection - a low-confidence true positive - raised AP even though it lowered precision at the top of the list, because recall matters as much as precision in this metric.

THREE WAYS MAP GETS MISREPORTED

Quoting $mAP@0.5$ and comparing it against someone else's $mAP@[.5:.95]$ (the first is typically 15-25 points higher); evaluating at a confidence threshold rather than over the whole curve; and reporting on a test set whose images share scenes with the training set, which is endemic in datasets scraped from video.

40.6 Video, tracking and 3D

- **Tracking-by-detection** is the dominant pipeline: run a detector per frame, then associate boxes across frames. SORT uses a Kalman filter for motion plus Hungarian matching on IoU; DeepSORT adds an appearance embedding so identities survive occlusion. The metric family (MOTA, IDF1, HOTA) separates detection quality from association quality - report HOTA if you can, since MOTA is dominated by detection errors.

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to computer vision.

IN OUR PROJECT — not used, and why

Not used in the project.

- **Video understanding** needs temporal modelling: 3D convolutions (C3D, I3D), two-stream networks (appearance plus optical flow), and now video Transformers with spatio-temporal attention. Cost is the binding constraint - a 16-frame clip is 16x the pixels.
- **Depth and 3D** : monocular depth estimation is now a strong zero-shot capability; point-cloud networks (PointNet, sparse convs) handle lidar; and neural rendering (NeRF, Gaussian splatting) reconstructs a scene from posed images. These are separate sub-fields, but they share this book's core machinery entirely.

40.7 Training and deploying detectors in practice

Concern	What to do
Small objects dominate the errors	Higher input resolution beats a bigger backbone; use feature pyramids; tile large images at inference
Heavy augmentation is the norm	Mosaic, scale jitter, random crop, colour jitter - and turn mosaic off for the last few epochs, which reliably gains a point
Class imbalance across the catalogue	Repeat-factor sampling for rare classes; do not simply oversample images

Concern	What to do
Annotation quality dominates everything	Measure inter-annotator IoU before blaming the model; a 0.8 agreement ceiling caps your mAP
Latency on device	Fuse conv-bn, use INT8 (Chapter 28), reduce input resolution first, and profile NMS separately - it is often 30% of the budget
Preprocessing mismatch	Letterboxing, channel order and normalization constants must be byte-identical between training and the deployed runtime; this is the most common cause of 'it was fine in Python'

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to computer vision.

IN OUR PROJECT — not used, and why

Not used in the project.

Exercises

- 1 Compute IoU by hand for three pairs of boxes, including one with no overlap, then implement it and check.
- 2 Implement NMS in twenty lines and run it on a synthetic set of overlapping boxes at three IoU thresholds. Describe what changes.
- 3 Reproduce the AP calculation in the worked box, then recompute it with the last two detections swapped.
- 4 Fine-tune a small pretrained detector on 200 annotated images and report mAP@0.5 and mAP@[.5:.95] side by side.
- 5 Measure your detector's end-to-end latency split into preprocessing, backbone, head and NMS. Optimize the largest term first.

41. Natural Language Processing: From Counting Words to Retrieval

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Text processing is not used in the project.

Chapter 24 covered large language models. This chapter covers everything that comes before and around them - representations that still win on small data, the tasks with their own structure, retrieval systems that give a model access to your documents, and the evaluation problem that makes language harder to measure than vision.

41.1 The classical pipeline, and why it still matters

- **Normalization** : case folding, Unicode NFKC, stripping or keeping accents and emoji. Every choice is a modelling decision; document it.

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to language processing.

IN OUR PROJECT — not used, and why

Not used in the project.

- **Tokenization** : whitespace and rules for classical models; subword (BPE, WordPiece, Unigram) for neural ones. Languages without spaces (Chinese, Japanese, Thai) need segmentation, and morphologically rich languages (Finnish, Turkish, Arabic) suffer most from a poorly-fitted vocabulary.
- **Stemming and lemmatization** : crude versus linguistic reduction to a base form. Useful for search and for bag-of-words models; harmful before a pretrained model, whose tokenizer expects raw text.
- **Stop words** : removing them helps TF-IDF retrieval and hurts anything that needs syntax. 'To be or not to be' is entirely stop words.

A BASELINE THAT EMBARRASSES EXPENSIVE MODELS

TF-IDF features plus linear logistic regression, on a few thousand labelled documents, trains in one second and frequently lands within a couple of points of a fine-tuned Transformer on topic classification. Run it first, always. If a large model cannot beat it by a margin that justifies its serving cost, it should not ship.

41.2 Counting: TF-IDF and BM25

```
tf(t,d)    = count of term t in document d (or 1 + log count)
idf(t)     = log( N / df(t) )           or log( (N + 1)/(df(t) + 1) ) + 1
tfidf(t,d) = tf(t,d) * idf(t), then L2-normalize each document vector
```

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to language processing.

IN OUR PROJECT — not used, and why

Not used in the project.

TF-IDF ON FOUR DOCUMENTS

$N = 4$ documents. The word 'the' appears in all four, so $\text{idf} = \log(4/4) = 0$ - it contributes nothing, which is why stop-word removal is optional once you use IDF. The word 'quantization' appears in one, so $\text{idf} = \log(4/1) = 1.386$. In a document where 'quantization' occurs 3 times and 'the' 12 times, the raw TF-IDF weights are $3 \times 1.386 = 4.16$ and $12 \times 0 = 0$. Rarity, not frequency, is what carries information - the single most transferable idea in information retrieval.

BM25 is TF-IDF's better-behaved successor and remains the strongest sparse retriever: it saturates term frequency (the tenth occurrence adds almost nothing) and normalizes by document length. In modern retrieval stacks BM25 is not a baseline to be replaced but a component to be combined - hybrid BM25-plus-dense retrieval beats either alone on almost every benchmark, because sparse matching catches exact names, codes and rare terms that embeddings blur.

41.3 Word embeddings: dense meaning from co-occurrence

Word2vec's skip-gram model predicts context words from a centre word. The full softmax over a 100,000-word vocabulary is too expensive, so negative sampling replaces it with a set of binary decisions:

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to language processing.

IN OUR PROJECT — not used, and why

Not used in the project.

WORKED ON OUR PROJECT — Turning windows into "words"

1. Cluster all training windows into, say, 64 prototypes with k-means; each window becomes the index of its nearest prototype -- a "motion word".
2. A long recording becomes a sentence of motion words; bag-of-words or TF-IDF counts over a minute describe the activity mix.
3. This is how classical text methods can be applied to sensor streams; the project classifies each window directly instead.

Skip-gram with negative sampling, for centre c and context o :

```
L = -log sigmoid( v_o . u_c )
    - SUM_(k=1..K) log sigmoid( -v_k . u_c )    k sampled from P_n
P_n(w) proportional to freq(w)^0.75    (the 3/4 power matters:
    it samples rare words more often than frequency alone)
```

Gradients: $dL / d u_c = (\text{sigmoid}(v_o \cdot u_c) - 1) v_o + \text{SUM}_k \text{sigmoid}(v_k \cdot u_c) v_k$. Two vectors per word - as centre and as context - and it is standard to keep or average them.

Method	What it optimizes	Notable property
word2vec (SGNS)	Local context prediction	Fast; famous analogy arithmetic king - man + woman ~ queen
GloVe	Weighted least squares on log co-occurrence counts	Uses global statistics directly; similar quality
FastText	Skip-gram over character n-grams	Handles out-of-vocabulary and rich morphology; the right default for non-English
Contextual (BERT-style)	Masked language modelling	One vector per occurrence, so 'bank' differs by sentence

TWO THINGS STATIC EMBEDDINGS GET WRONG

They give one vector per word, so every sense of a polysemous word is averaged into a single point - 'bank' sits between rivers and finance and is a good representation of neither. And they encode the social biases of the corpus in a directly measurable way: occupation words align with gender directions, and models built on them inherit that alignment. Both are reasons the field moved to contextual representations, and the second does not disappear there.

41.4 Tasks with structure

Task	Shape	Standard approach
Text classification	Document -> label	TF-IDF + linear, or fine-tuned encoder
Sequence labelling (NER, POS)	Token -> tag, with BIO encoding	Encoder + per-token head, optionally a CRF layer
Span extraction (QA)	Text -> (start, end)	Two heads predicting start and end positions
Sentence pair (NLI, dedup)	Two texts -> relation	Cross-encoder for accuracy, bi-encoder for speed
Generation (summarize, translate)	Text -> text	Encoder-decoder or decoder-only LLM
Retrieval	Query -> ranked documents	Hybrid BM25 + dense bi-encoder, then a cross-encoder reranker

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to language processing.

IN OUR PROJECT — not used, and why

Not used in the project.

BIO tagging is worth a line because it is where subtle bugs live: B-PER marks the beginning of a person entity, I-PER its continuation, O everything else. Entity-level F1 - not token accuracy - is the metric, because getting 4 of 5 tokens of a name right is not getting the name right. A CRF layer on top enforces valid transitions (an I- tag cannot follow O), which is worth one to three F1 points and costs almost nothing.

41.5 Retrieval-augmented generation, done properly

```
query --> [rewrite] --> [BM25 search] ---.
      \-> [dense search] ---+--> [fuse/RRF] --> top 50
                              |
                              [cross-encoder rerank] -> top 5
                              |
[prompt: question + passages] --> LLM
                              |
                        answer + citations to passages
```

The retrieval half of RAG is a search engineering problem, and it is where nearly all quality comes from. The generator can only

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to language processing.

IN OUR PROJECT — not used, and why

Not used in the project.

be as good as the passages it is handed.

Decision	Guidance
Chunk size	200-500 tokens with 10-20% overlap; split on structure (headings, paragraphs) rather than fixed windows
What to embed	The chunk plus its document title and section path; a bare chunk loses the context that makes it findable
Hybrid or dense only	Hybrid, almost always. Dense retrieval misses exact identifiers, part numbers and rare names
Reranking	A cross-encoder over the top 50 is the highest value-per-millisecond component in the whole stack
Evaluation	Measure retrieval separately: Recall@k and MRR on questions with known answer passages, before judging the answers
Grounding	Require citations and check that cited passages actually contain the claim; unfaithful summaries of correct passages are the dominant failure

41.6 Evaluating language output

Metric	Measures	Weakness
BLEU	n-gram precision against references, with brevity penalty	Insensitive to meaning; punishes valid paraphrase
ROUGE-L	Longest common subsequence recall	Rewards copying; standard for summarization anyway
chrF	Character n-gram F-score	Better for morphologically rich languages
BERTScore	Similarity of contextual embeddings	Correlates better with humans; depends on the scoring model
Exact match / F1 (QA)	String overlap with the gold answer	Brittle to formatting; use with normalization
LLM-as-judge	A model scores outputs against a rubric	Position bias, verbosity bias, self-preference; needs paired randomized comparisons and a human-labelled calibration set

FROM ZERO — the idea explained step by step

See the original text below; this topic is specific to language processing.

IN OUR PROJECT — not used, and why

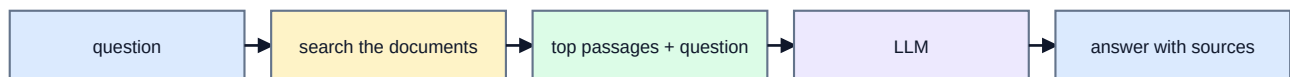
Not used in the project.

THE EVALUATION TRAP THAT COSTS THE MOST TIME

Benchmark contamination. If the test set existed on the public web before the model was pretrained, a high score measures memorization. For any claim that matters, build a small private evaluation set from your own data, keep it out of every prompt and log, and treat public leaderboard numbers as advertising.

Exercises

1 Compute TF-IDF vectors by hand for four short documents and verify that a term appearing everywhere gets weight zero. 2 Train word2vec on any corpus of a few million words and inspect the ten nearest neighbours of five words, including one ambiguous one. 3 Build a TF-IDF plus logistic-regression baseline and a fine-tuned encoder for the same classification task; report accuracy, training time and inference cost for both.



Graph 41.1 — Retrieval-augmented generation. The question first searches a document store; the best passages are pasted into the prompt; the language model answers from them, so the answer can cite sources and stay current.

4 Build a minimal RAG pipeline over 500 of your own documents, then measure Recall@10 of the retriever alone on 30 hand-written questions. 5 Take 50 generated summaries, score them with ROUGE and with an LLM judge, and compute the correlation of each with your own manual ratings.

42. Speech, Audio and Multimodal Models

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The project has two speech examples (keyword spotting): `kws_mfcc` uses the classic MFCC features of 42.1, `kws_raw` learns from the raw waveform.

Audio is the modality where classical signal processing and deep learning meet most directly: the front end is a hundred years of Fourier analysis, and everything after it is a neural network. Multimodal models then join audio, vision and text in a shared space, which is the mechanism behind zero-shot classification, cross-modal search, and the vision-language assistants now built on top of Chapter 24's LLMs.

42.1 From waveform to features

Sample rate	16 kHz for speech, 44.1/48 kHz for music
Frame	25 ms window, 10 ms hop -> 100 frames per second
STFT	$X(f, t) = \text{FFT}(\text{window} * \text{frame}_t)$
Power spectrum	$ X(f, t) ^2$
Mel filterbank	80 triangular filters spaced by $\text{mel}(f) = 2595 \log_{10}(1 + f/700)$
Log-mel	$\log(\text{mel_energies} + \text{eps})$ <- the modern default
MFCC	$\text{DCT}(\text{log-mel})$, keep 13 coefficients <- classical

A 1-second clip at 16 kHz is 16,000 numbers; its log-mel spectrogram is $100 \times 80 = 8,000$ - and a far easier learning problem,

FROM ZERO — the idea explained step by step

Audio is a very long 1-D signal (16,000 samples per second). **MFCC** features compress it: cut into short frames, compute the spectrum of each, group the frequencies like the human ear does (mel scale), take the log and a cosine transform. The result is a small table: coefficients x frames.

IN OUR PROJECT — used

Used in the `kws_mfcc` example: each 1-second clip becomes a table of 40 MFCC coefficients x 32 frames, computed once in `prepare_data.py`, then a small 1-D CNN classifies 6 keywords (yes, no, up, down, silence, unknown) [35].

`kws_raw` skips MFCC and learns from the raw waveform with 3 convolution + LayerNorm blocks (Chapter 18) [35]. The HAR data are the same kind of problem at a lower rate (50 Hz, 9 channels).

WORKED ON OUR PROJECT — From raw signal to features, as in audio

1. Audio front ends turn a waveform into a spectrogram. For HAR the same idea gives the walking peak at 1.9531 Hz (Chapter 38 example).
2. The CNN learns its own filters: conv1's 16 kernels of length 7 (0.14 s) act as learned band-pass filters on the raw signal [25].
3. Learning the front end instead of fixing it is the main difference between the CNN (~0.90) and the hand-made-feature models of Chapters 6-11 (0.76-0.84).

because the network no longer has to discover the Fourier transform.

The mel scale exists because human pitch perception is roughly logarithmic: the difference between 200 and 300 Hz is obvious, between 5,000 and 5,100 Hz inaudible. MFCCs additionally decorrelate the filterbank outputs with a DCT, which mattered when models were Gaussian mixtures and matters little for convolutional or Transformer front ends - **use log-mel for neural models, MFCC only for classical pipelines or very tight compute budgets**.

Augmentation	What it simulates	Note
Additive noise at a target SNR	Real environments	Use a noise corpus, not white noise
Room impulse response convolution	Reverberation	The single most valuable augmentation for far-field audio
Speed / tempo perturbation (0.9x, 1.1x)	Speaker rate variation	Cheap and reliably helpful for ASR
SpecAugment: time and frequency masking	Occlusion in the spectrogram	Applied to the features, not the audio; near-free and very effective
Gain, clipping, codec round-trip	Device and transport	Match the codecs your product actually uses

42.2 Recognition: CTC, attention and transducers

Speech recognition has a structural problem: the input has 100 frames per second, the output has perhaps 3 characters per second, and no alignment between them is given. The three solutions define the field.

CTC: add a blank symbol, sum over all alignments that collapse to the target after removing blanks and repeats.
 $p(y | x) = \text{SUM over valid alignments } a \text{ of } \text{PROD}_t p(a_t | x)$
 computed by the same forward algorithm as an HMM (Chapter 37)
Assumption: outputs are conditionally independent given x.

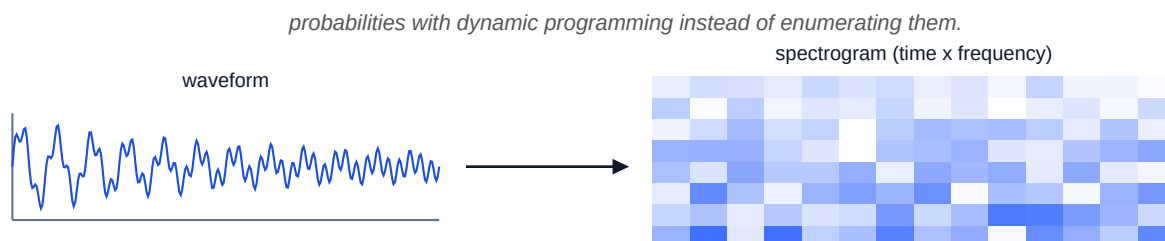
For target 'CAT' over 5 frames, valid alignments include C-A-T-_-_, CC-A-T-_, C-_-A-T-T and dozens more; CTC sums their

FROM ZERO — the idea explained step by step

Speech recognition: CTC, attention, transducers.

IN OUR PROJECT — not used, and why

Not used; keyword spotting is classification.



Graph 42.1 — From sound to features. The waveform is cut into short overlapping frames (about 25 ms), each frame is turned into its frequency content, and the result -- a spectrogram -- is what the model sees: time across, frequency up.

Approach	Streaming	Strength	Weakness
CTC	Yes, naturally	Simple, fast, monotonic by construction	No output-to-output dependency; needs an external language model
Attention encoder-decoder	No (needs the full utterance)	Best accuracy offline; implicit language model	Can hallucinate or loop; not monotonic
RNN-Transducer (RNN-T)	Yes	Streaming plus an internal prediction network - the standard for on-device assistants	More complex loss, heavier training
Whisper-style encoder-decoder	No (30 s windows)	Multilingual, robust, trained weakly-supervised at scale	Latency and hallucination on silence

WORD ERROR RATE, COMPUTED

WER = (substitutions + insertions + deletions) / words in the reference, computed by Levenshtein alignment. Reference: 'the quick brown fox jumps' (5 words). Hypothesis: 'the quick brown box jump over' - one substitution (fox -> box), one substitution (jumps -> jump), one insertion (over). WER = 3/5 = 0.60, or 60%. Note that WER can exceed 100% because of insertions, and that it weights a function word the same as the one content word your product actually needed - which is why keyword-spotting products report false accepts per hour and false rejects at a fixed threshold instead.

Decoding matters as much as the acoustic model. Greedy CTC decoding is the floor; beam search with an n-gram or neural language model typically removes 15-30% of the remaining errors on domain text, because the acoustic model has no idea that your product's proper nouns exist. Shallow fusion - adding lambda times the language-model log-probability to the beam score - is the standard mechanism, and a per-user contextual biasing list is how contact names get recognized.

42.3 Speaking, separating and self-supervising

- **Text to speech** is now two stages: an acoustic model (Tacotron, FastSpeech) turning text into a mel spectrogram, and a neural vocoder (HiFi-GAN, WaveRNN) turning that into a waveform. Non-autoregressive acoustic models with explicit duration prediction are the deployment default because they are fast and cannot loop.

FROM ZERO — the idea explained step by step

Speech synthesis, source separation, self-supervised audio.

IN OUR PROJECT — not used, and why

Not used in the project.

- **Neural audio codecs** (SoundStream, EnCodec) compress audio to discrete tokens, which lets a language model generate speech and music with exactly the machinery of Chapter 24. This is the architecture behind current speech-to-speech systems.
- **Speaker tasks** : an embedding network (x-vector, ECAPA) trained with a margin softmax gives a vector per utterance; verification is a cosine threshold, and diarization is clustering those embeddings over time.
- **Self-supervised pretraining** (wav2vec 2.0, HuBERT, WavLM) is the reason low-resource ASR became feasible: pretrain on tens of thousands of unlabelled hours, fine-tune on ten labelled hours, and beat a system trained on a thousand labelled hours the old way.
- **Enhancement and separation** (speech denoising, source separation) operate on masks over the spectrogram or directly on the waveform, and are usually a preprocessing stage whose benefit must be measured end-to-end - a denoiser that improves human listening can hurt an ASR model that was trained on noisy audio.

ON-DEVICE KEYWORD SPOTTING, THE CANONICAL TINY-ML TASK

A wake-word model runs continuously on a battery: budgets are tens of kilobytes of weights and single-digit milliwatts. The recipe is a small depthwise-separable CNN or tiny Transformer over log-mel features, INT8 quantized (Chapter 28), pruned (Chapter 29), and cascaded - a 1-kilobyte always-on stage gates a larger verifier. This is where Chapters 28 to 31 stop being theory.

42.4 Multimodal models: one space for two modalities

CLIP-style contrastive objective (InfoNCE), batch of N pairs:

```
s_ij = cos( image_emb_i , text_emb_j ) / temperature
L = 0.5 * [ CE(rows of s, diagonal) + CE(columns of s, diagonal) ]
```

The matched pair is the positive; the other N-1 texts (and images) in the batch are the negatives.

Large batches are not an optimization detail here - they are the source of the negatives, which is why CLIP-class models are

FROM ZERO — the idea explained step by step

Multimodal models put images and text into one space.

IN OUR PROJECT — not used, and why

Not used in the project.

trained with batches in the tens of thousands.

Once image and text live in one space, zero-shot classification is retrieval: embed the candidate class names as sentences ('a photo of a {class}'), embed the image, take the nearest. Cross-modal search, duplicate detection and content moderation all become nearest-neighbour queries in the same index (Chapter 39's machinery, unchanged).

```

Vision-language model (LLaVA-style):
image --> [frozen vision encoder] --> patch embeddings
                                   |
                                   [projector: MLP or resampler]
                                   |
text  --> [tokenizer] --> token embeddings + projected image tokens
                                   |
                                   [ pretrained LLM decoder ]
                                   |
                                   answer

```

The projector is the only part trained in stage one; the LLM is then fine-tuned on instruction data in stage two. The image becomes, quite literally, a few hundred extra tokens in the context.

Family	Example capability	Main failure mode
Contrastive dual encoder	Zero-shot classification, retrieval	No compositional reasoning: 'dog left of cat' matches 'cat left of dog'
Vision-language decoder	Describe, answer, read charts and documents	Object hallucination - naming objects that are not present
Audio-language	Speech instruction following, audio captioning	Degrades under noise far faster than a dedicated ASR model
Any-to-any	Text, image, audio in and out	Uneven quality across modalities; evaluation is largely unsolved

EVALUATING MULTIMODAL SYSTEMS

Captioning metrics (CIDEr, SPICE) and VQA accuracy reward safe, generic answers. Hallucination needs its own measurement: ask about objects that are absent (POPE-style probing) and report the false positive rate. And check the training data question - many public benchmarks appear in the pretraining corpora of the models being scored on them.

Exercises

1 Compute the number of frames, and the log-mel feature-tensor shape, for a 3-second clip at 16 kHz with 25 ms windows and 10 ms hops. 2 Implement WER with a Levenshtein alignment and reproduce the 0.60 in the worked box. 3 Enumerate every valid CTC alignment of the target 'AB' over 4 frames and confirm the count against the dynamic-programming recursion. 4 Fine-tune a pretrained self-supervised speech model on one hour of labelled audio and compare with a model trained from scratch on the same hour. 5 Build a cross-modal search demo: embed 1,000 images with a CLIP-class model and retrieve them by text query. Then find a query where the compositional failure in the table above is visible.

43. Systems and Scale: Hardware, Memory and Distributed Training

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The project runs at the other end of the scale from GPUs: a single microcontroller core. The memory accounting of 43.2 is still exactly how the thesis counts its RAM.

Beyond a certain size, machine learning stops being a modelling problem and becomes a systems problem: how many bytes fit, how fast they move, and how many machines can cooperate without spending all their time talking. This chapter gives the arithmetic to answer those questions before you rent the hardware.

43.1 What a GPU actually is, for our purposes

Component	Role	Order of magnitude (data-centre GPU)
Tensor cores	Matrix-multiply-accumulate in low precision	Hundreds of TFLOP/s in bf16, more in fp8
HBM (device memory)	Where weights and activations live	40-192 GB at 1.5-8 TB/s
SRAM / shared memory	On-chip scratchpad per streaming multiprocessor	Tens of MB total, ~10x the bandwidth of HBM
Interconnect (NVLink)	GPU-to-GPU inside a node	Hundreds of GB/s
Network (InfiniBand/Ethernet)	Node-to-node	25-400 Gb/s - roughly an order of magnitude below NVLink

FROM ZERO — the idea explained step by step

A GPU runs thousands of simple operations in parallel.

IN OUR PROJECT — not used, and why

The project's target is the opposite: one Cortex-M7 core at 216 MHz, no GPU. The PyTorch reference runs on a PC CPU.

Arithmetic intensity $I = \text{FLOPs performed} / \text{bytes moved}$
Roofline: $\text{achievable FLOP/s} = \min(\text{peak FLOP/s}, I * \text{memory bandwidth})$
Ridge point: $I^* = \text{peak FLOP/s} / \text{bandwidth}$ (often 100-400 FLOP/byte)

Below the ridge point you are memory-bound and adding compute does nothing. This one inequality explains most disappointing benchmarks.

WHERE THE COMMON OPERATIONS LAND

A large matrix multiply has intensity proportional to the shared dimension: compute-bound, which is why training is dominated by GEMMs and why bigger batches help. Elementwise operations (activation, normalization, dropout, residual add) read and write more bytes than they compute on: memory-bound, which is why kernel fusion - doing several of them in one pass over the data - is where framework compilers get their speedups. Single-token LLM decoding reads the whole weight matrix to compute one vector product: severely memory-bound, which is why decode throughput tracks memory bandwidth and not FLOP/s, and why quantization (Chapter 28) speeds up generation even when the arithmetic stays in higher precision.

43.2 Training memory, accounted exactly

Per parameter, mixed-precision training with Adam:

bf16 weights	2 bytes
bf16 gradients	2 bytes
fp32 master weights	4 bytes
fp32 Adam m, v	8 bytes

TOTAL	16 bytes per parameter

FROM ZERO — the idea explained step by step

Training memory = weights + gradients + optimiser state + activations kept for the backward pass + temporary buffers.

IN OUR PROJECT — used

The thesis counts exactly these on the board. Float HAR: weights 40,856 B, gradients 40,856 B, SGD momentum 40,856 B (AdamW: 81,712 B), plus activations for one window. Measured total training state 209,680 B + 27,768 B stack = 237,448 B, 72.5% of the 327,680 B of RAM [35].

WORKED ON OUR PROJECT — Training memory accounted exactly, on the board

1. Parameters: $10,214 \times 4 = 40,856$ B. Gradients: 40,856 B. Momentum: 40,856 B. Together 122,568 B [35].
2. The rest of the 209,680 B static state holds activations for the backward pass, the batch buffers, and the masks (1,262 B at 90% RigL sparsity).
3. Stack peak 27,768 B. Total 237,448 B of 327,680 B = 72.5% [35].
4. int8 storage of weights (10,214 B) would save 30,642 B; int8 gradients as in [3] would save as much again.

Activations (per layer, per sample) ~ batch * seq * hidden * bytes
 * a constant of roughly 10-20 for a Transformer block

The 16-bytes-per-parameter rule is the single most useful number in large-model engineering.

CAN I FINE-TUNE A 7-BILLION-PARAMETER MODEL ON ONE 80 GB GPU?

States: $7e9 \times 16$ bytes = 112 GB. That exceeds 80 GB before a single activation is stored, so full fine-tuning does not fit. The options, in increasing order of intrusiveness: shard the optimizer states across GPUs (ZeRO-1 across 4 GPUs brings it to $7e9 \times (4 + 12/4) = 49$ GB each); offload optimizer states to CPU; use 8-bit Adam (saves 6 bytes per parameter, giving 70 GB); or use LoRA (Chapter 24) where the base weights are frozen at 2 bytes and only ~0.1% of parameters have optimizer state - $7e9 \times 2 + \text{tiny} = \text{about } 15$ GB, which fits comfortably and is why LoRA became the default. Inference alone needs 14 GB in bf16, or 7 GB in INT8, or 3.5 GB in INT4.

Activations are the other half and the one people forget. **Activation checkpointing** (gradient checkpointing) stores only the inputs of each block and recomputes the interior during the backward pass: memory falls from $O(L)$ to about $O(\sqrt{L})$ segments, at a cost of roughly 30% extra compute. On any model that nearly fits, it is the first lever to pull.

43.3 Precision

Format	Bits (E/M)	Property	Use
fp32	8/23	The reference	Master weights, reductions, loss
fp16	5/10	Narrow range; overflows	Needs dynamic loss scaling; legacy hardware
bf16	8/7	fp32's range, less mantissa	The training default today - no loss scaling needed

Format	Bits (E/M)	Property	Use
fp8 (E4M3/E5M2)	4/3 and 5/2	Per-tensor scaling required	Forward and backward GEMMs on recent hardware
INT8 / INT4	integer	Post-training or QAT (Chapter 28)	Inference, and increasingly KV caches

FROM ZERO — the idea explained step by step

Precision: float32, float16, bfloat16, int8 -- fewer bits, less memory and faster arithmetic.

IN OUR PROJECT — used

The project: weights in 8 or 4 bits, forward operands of at most 12 bits, sums in int32 [35].

The rule that keeps mixed precision stable: **compute in low precision, accumulate and update in high precision**. Matrix multiplies run in bf16 or fp8; their accumulators are fp32; the master weights and the optimizer moments stay fp32; loss and softmax reductions stay fp32. Violating the last point is the usual cause of a training run that diverges only at large batch sizes.

43.4 The four axes of parallelism

Axis	What is split	Communication	When to use
Data parallel	The batch; every GPU holds all weights	All-reduce of gradients once per step	Always, first, until the model stops fitting
ZeRO / FSDP	Optimizer states, then gradients, then parameters	All-gather of parameters per layer, reduce-scatter of gradients	The standard way to train a model that does not fit, with minimal code change
Tensor parallel	Individual matrices, across GPUs	All-reduce twice per Transformer block - very heavy	Inside one node only, over NVLink
Pipeline parallel	Layers, across GPUs	Point-to-point activations at stage boundaries	Across nodes, when depth is large
Sequence / context parallel	The sequence dimension	Ring exchange of keys and values	Very long contexts, where activations dominate
Expert parallel (MoE)	Experts across devices	All-to-all of tokens	Sparse models with many experts

FROM ZERO — the idea explained step by step

Parallelism across many GPUs: data, tensor, pipeline, sequence.

IN OUR PROJECT — not used, and why

Not used; one device.

16 bytes per parameter

bf16 weights 2	grads 2	fp32 master 4	Adam m, v 8
----------------	---------	---------------	-------------

Graph 43.1 — Training memory per parameter with Adam in mixed precision (the usual accounting): 2 bytes of bf16 weights, 2 of gradients, 4 of fp32 master weights, and 8 for Adam's two moments -- 16 bytes per parameter before any activations. A 7-billion-parameter model needs 112 GB for these alone.

Ring all-reduce cost for S bytes over N workers:

$\text{time} \sim 2(N-1)/N * S / \text{bandwidth} \rightarrow \sim 2S/\text{BW}$ for large N

Pipeline bubble fraction with P stages and M microbatches:

$\text{bubble} = (P-1) / (M+P-1) \rightarrow \text{keep } M \gg P$

With 4 pipeline stages and 8 microbatches, 27% of the time is bubble; with 64 microbatches it is 4.5%.

3D parallelism composes them: tensor-parallel within a node, pipeline-parallel across a small group of nodes, data-parallel across the rest. The ordering is not arbitrary - it puts the chattiest axis on the fastest link. A practical rule: use FSDP/ZeRO-3 alone until communication becomes the bottleneck, then introduce tensor parallelism inside the node, and only then pipelining.

```
# Data-parallel training, the modern minimal form
model = FSDP(model, sharding_strategy=FULL_SHARD,
              mixed_precision=MixedPrecision(param_dtype=torch.bfloat16,
                                              reduce_dtype=torch.float32))

for step, batch in enumerate(loader):
    with torch.autocast('cuda', dtype=torch.bfloat16):
        loss = model(batch).loss / accum_steps
    loss.backward() # gradients reduce-scattered
    if (step + 1) % accum_steps == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step(); opt.zero_grad(set_to_none=True)
```

Gradient accumulation is the free way to raise the effective batch size when memory is the limit; note that the loss must be divided by the number of accumulation steps, and that gradient clipping belongs after the last accumulation, not inside the loop.

43.5 Measuring whether your run is any good

Model FLOPs Utilization (MFU):

$\text{MFU} = (6 * N_{\text{params}} * \text{tokens_per_second}) / (\text{GPUs} * \text{peak FLOP/s})$

The 6ND rule from Chapter 36, turned into a hardware efficiency number.

FROM ZERO — the idea explained step by step

Measure whether hardware is used well (utilisation, throughput).

IN OUR PROJECT — partly used

On the board the equivalent is time (or CPU cycles) per training step; the thesis budget is set by memory first.

WORKED ON OUR PROJECT — Throughput: compute versus memory on the M7

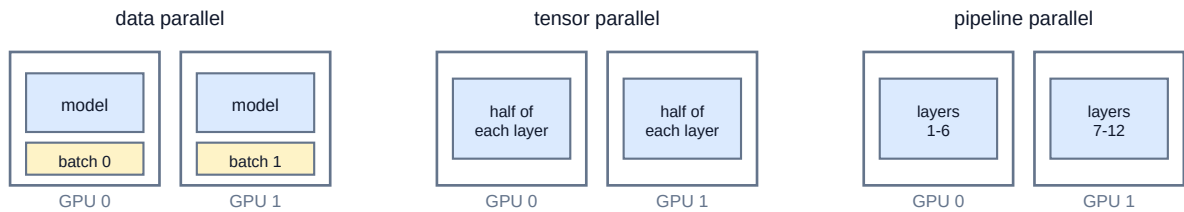
1. One training step on a window is about 1.5 million MACs (Chapter 14). At 216 MHz and roughly one MAC per cycle, that is about 7 ms per window.
2. An epoch of 6,617 windows is then roughly 46 s; 20 epochs about 15 minutes -- if memory traffic does not dominate [7].
3. The feasibility question Q3 of the thesis measures this on the real board.

AN MFU CALCULATION

A 7B model training at 12,000 tokens/second on 8 GPUs with 400 TFLOP/s peak bf16 each. Numerator: $6 \times 7e9 \times 12,000 = 5.04e14$ FLOP/s. Denominator: $8 \times 4e14 = 3.2e15$. MFU = 15.8%. That is poor - a well-tuned dense Transformer run reaches 35-55%. The checklist, in the order that usually pays: is the data loader starving the GPU (profile it); is the sequence length so short that kernels are launch-bound; is activation checkpointing on when it need not be; is tensor parallelism spanning nodes; are you in fp32 by accident.

Symptom	Likely cause	Check
GPU utilization spiky, 40-70%	Input pipeline	Time one epoch of the loader with the model removed
Utilization high, MFU low	Memory-bound kernels, small shapes	Profile; fuse; raise batch or sequence length
Scaling breaks past 8 GPUs	Gradient all-reduce over slow network	Measure interconnect bandwidth; enable bucketing and overlap
Loss spikes then diverges at scale	fp16 range, or a bad LR warmup	Switch to bf16; lengthen warmup; clip gradients
Throughput falls over hours	Thermal throttling or a straggler rank	Log per-rank step time; the slowest rank sets the pace

43.6 Serving: a different bottleneck



Graph 43.2 — The ways to split training across GPUs. Data parallel: every GPU has the whole model, each takes part of the batch. Tensor parallel: each layer is split across GPUs. Pipeline parallel: different layers on different GPUs.

Inference for autoregressive models has two phases with opposite characteristics, and conflating them is the classic mistake.

FROM ZERO — the idea explained step by step

Serving large models is limited by memory bandwidth.

IN OUR PROJECT — not used, and why

Not relevant at our scale.

Phase	Work	Bound by	Lever
Prefill (prompt)	One big matrix multiply over all prompt tokens	Compute	Batching helps little; chunk long prompts
Decode (generation)	One token at a time, reading all weights and the KV cache	Memory bandwidth	Batching helps enormously; quantization helps directly

- **Continuous batching** admits new requests into the running batch as others finish, instead of waiting for a whole batch to complete - typically a 2-4x throughput gain at the same latency.
- **Paged attention** stores the KV cache in fixed-size blocks like virtual memory pages, removing the fragmentation that otherwise wastes half the cache.
- **Speculative decoding** drafts several tokens with a small model and verifies them in one pass of the large one; acceptance rates of 60-80% give 2-3x lower latency with **identical** output distribution.
- **Quantized serving** (INT8, INT4, fp8 weights, and increasingly quantized KV caches) reduces both the memory read per token and the cache footprint, so it raises the batch size you can hold as well as the speed per token.

SIZING A SERVING DEPLOYMENT IN FIVE MINUTES

Weights: $\text{params} \times \text{bytes-per-weight}$. KV cache per token: $2 \times \text{layers} \times \text{kv_heads} \times \text{head_dim} \times \text{bytes}$. Multiply by the average sequence length and the number of concurrent requests. Add 10-20% overhead. If the total exceeds device memory, the fix is a smaller quantization, grouped-query attention (fewer kv_heads), a shorter context, or more GPUs - in that order of cost-effectiveness. Then measure tokens/second at your target p95 latency, not at maximum throughput, because those two operating points can differ by 5x.

Exercises

1 Compute the training memory for a 1.3B model with Adam in mixed precision, then recompute it with 8-bit Adam and with LoRA. 2 Measure the MFU of a training run you have access to, then remove the data loader (feed synthetic tensors) and measure again. The difference is your input pipeline's cost. 3 Derive the pipeline bubble fraction and plot it for $P = 4$ and M from 4 to 128. 4 Serve a small model with and without continuous batching and compare throughput at fixed p95 latency. 5 Take the roofline model and classify five operations in your own network as compute- or memory-bound, then verify with a profiler.

44. Causal Inference and Experimentation

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Causal inference is not used; one section connects to how the thesis compares configurations.

Every model in this book answers 'what is associated with what'. Almost every decision made with a model asks 'what happens if I intervene'. Those are different questions, and no amount of predictive accuracy converts one into the other. A model can predict hospital readmission perfectly and still be useless for deciding who to treat - and a feature with a large importance score can have the opposite sign as a causal effect.

44.1 Prediction is not intervention

SIMPSON'S PARADOX, WITH THE CLASSIC NUMBERS

Two treatments for kidney stones. Treatment A succeeds in 81 of 87 cases with small stones (93%) and 192 of 263 with large stones (73%). Treatment B succeeds in 234 of 270 small (87%) and 55 of 80 large (69%). A is better in both groups. Yet overall A is $273/350 = 78\%$ and B is $289/350 = 83\%$ - B looks better. The reason is that doctors gave A to the hard cases: stone size is a confounder, causing both the treatment assignment and the outcome. Any model fitted on this data without stone size will confidently recommend the worse treatment. No sample size fixes this; it is not noise, it is structure.

FROM ZERO — the idea explained step by step

Predicting what happens is different from knowing what would happen if we changed something.

IN OUR PROJECT — not used, and why

Not used in the project.

Potential outcomes:	$Y(1)$ = outcome if treated	
	$Y(0)$ = outcome if not treated	
Individual effect	$\tau_i = Y_i(1) - Y_i(0)$	NEVER observed
ATE	$= E[Y(1) - Y(0)]$	average over everyone
ATT	$= E[Y(1) - Y(0) \mid T = 1]$	among the treated
CATE	$= E[Y(1) - Y(0) \mid X = x]$	conditional on features

The fundamental problem of causal inference: for each unit you observe exactly one of the two potential outcomes. Causal inference is a missing-data problem.

Randomization solves it in one stroke: if treatment is assigned by a coin flip, then the treated and untreated groups differ only by chance in **every** variable, observed or not, so the difference in average outcomes estimates the ATE without bias. That is the entire justification for the A/B test, and it is why an experiment beats any observational analysis that is available.

44.2 Running an experiment that means something

Sample size per arm, two-sided test at level α with power $1-\beta$:

$$n = 2 (z_{(1-\alpha/2)} + z_{(1-\beta)})^2 \cdot \sigma^2 / \delta^2$$

For a proportion: $\sigma^2 = p (1 - p)$

$$z_{0.975} = 1.960, \quad z_{0.80} = 0.8416$$

FROM ZERO — the idea explained step by step

A good experiment randomises and compares against a control.

IN OUR PROJECT — partly used

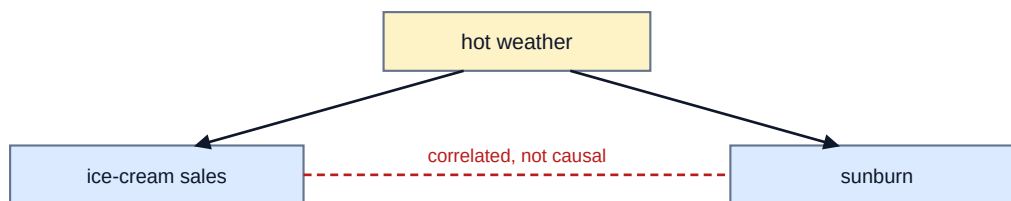
The seed sweeps are a controlled experiment: only the configuration changes, the seeds are the random repetition.

WORKED ON OUR PROJECT — Stochastic rounding versus round-to-nearest as a controlled experiment

1. Question: does stochastic rounding cause better accuracy, or does it only correlate with other settings?
2. Design: same model, same data, same seeds; only the rounding mode differs. Pair the runs by seed and look at the per-seed differences [35].
3. Because the rounding mode is assigned by the experimenter, the difference is a causal effect of the rounding, not a correlation.
4. Warning from the C book (2.7): if the two arms draw rounding decisions from the same seeded stream in different ways, the "same seed" is not the same experiment -- check which RNG each decision uses.

HOW MUCH TRAFFIC DO I NEED?

Baseline conversion 5%, and you want to detect a relative lift of 2%, i.e. $\delta = 0.001$ in absolute terms. $\sigma^2 = 0.05(0.95) = 0.0475$. Then $n = 2(1.960 + 0.8416)^2 (0.0475) / (0.001)^2 = 2(7.849)(0.0475)/1e-6 = 746,000$ users per arm. At 50,000 users a day that is a 30-day experiment. Detecting a 10% relative lift instead needs 1/25 of that - about 30,000 per arm, or under two days. Power scales with the square of the effect you are chasing, which is why 'we will just run it and see' is not a plan and why small effects are usually undetectable at any realistic traffic.



Graph 44.1 — Why prediction is not intervention. Hot weather drives both ice-cream sales and sunburn, so the two are correlated -- but banning ice cream would not reduce sunburn. A confounder like this must be measured or randomised away.

Threat to validity	What goes wrong	Treatment
Peeking	Checking daily and stopping at significance inflates the false-positive rate far above 5%	Fix the horizon in advance, or use a sequential test (always-valid p-values, group-sequential boundaries)
Multiple metrics	Twenty metrics give one 'significant' result by chance	Declare one primary metric; treat the rest as guardrails with corrected thresholds
Interference (SUTVA violation)	One user's treatment affects another - marketplaces, social graphs, shared inventory	Cluster randomization, switchback designs, or budget-split designs
Novelty and primacy effects	Behaviour changes because the thing is new, then reverts	Run long enough; analyse new versus returning users separately
Sample-ratio mismatch	Arms receive unequal traffic, revealing a broken assignment	Chi-square test on the split every day; an SRM invalidates the experiment, full stop
Under-powered launch decisions	A non-significant result is read as 'no effect'	Report the confidence interval; 'we could not detect an effect smaller than 3%' is the honest statement

CUPED: THE FREE VARIANCE REDUCTION

If you have a pre-experiment covariate X correlated with the outcome Y (typically the same metric measured before the experiment), analyse $Y_{\text{adj}} = Y - \theta(X - E[X])$ with $\theta = \text{Cov}(X, Y) / \text{Var}(X)$. The adjusted metric has variance reduced by a factor $(1 - \rho^2)$. With a correlation of 0.7, that is half the variance and therefore half the required sample size - the cheapest experimental improvement available, and it is unbiased because X predates the assignment.

44.3 When you cannot randomize

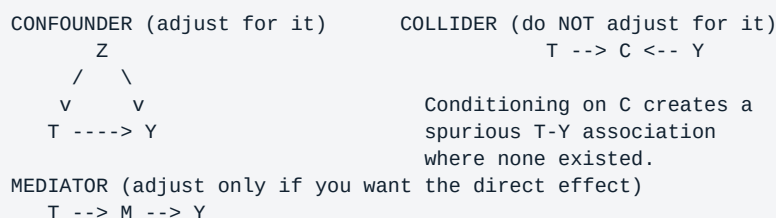
Sometimes the intervention is already deployed, unethical to withhold, or outside your control. Then you must adjust for confounding explicitly, and the first step is to draw the graph.

FROM ZERO — the idea explained step by step

Methods for when randomisation is impossible.

IN OUR PROJECT — not used, and why

Not used in the project.



The backdoor criterion: to estimate the effect of T on Y , block every path that enters T through an arrow into it, and never open a path by conditioning on a collider or its descendants.

'CONTROL FOR EVERYTHING' IS WRONG, NOT JUST WASTEFUL

Adding every available variable to a regression is standard practice and can move an estimate away from the truth. Conditioning on a collider manufactures correlation; conditioning on a mediator removes the very effect you were measuring; conditioning on a post-treatment variable does both. Which variables to adjust for is a question about the world, answerable only with domain knowledge - the data cannot tell you.

Design	Assumption it needs	Typical use
Propensity matching / weighting	No unmeasured confounders; overlap between groups	Observational program evaluation
Doubly robust (AIPW)	Either the outcome model or the propensity model is correct	The default estimator when you must adjust
Difference-in-differences	Parallel trends absent treatment	A change rolled out to some regions or at some date
Instrumental variables	The instrument affects treatment only, not the outcome directly	Encouragement designs, imperfect compliance, natural experiments
Regression discontinuity	Units just above and below a cutoff are comparable	Eligibility thresholds, score cutoffs
Synthetic control	A weighted combination of untreated units tracks the treated one before treatment	One treated region or market

44.4 Heterogeneous effects and uplift modelling

The average effect answers 'should we launch this'. The conditional effect answers 'for whom' - which is the question that makes targeting profitable. Note the four-way segmentation that makes uplift different from response modelling:

FROM ZERO — the idea explained step by step

Effects that differ between groups; uplift models.

IN OUR PROJECT — not used, and why

Not used in the project.

Segment	Treated outcome	Untreated outcome	Action
Persuadables	Converts	Does not	Target these
Sure things	Converts	Converts	Waste of budget
Lost causes	Does not	Does not	Waste of budget
Sleeping dogs	Does not	Converts	Actively harmful to target

A response model that predicts $P(\text{convert} | \text{treated})$ targets sure things, because they are the easiest to predict. Uplift models estimate the difference instead:

- **S-learner** : one model with treatment as a feature; simple, but the model may ignore a weak treatment feature entirely.
- **T-learner** : separate models for treated and control; unbiased in structure, but the difference of two noisy models is noisier still.
- **X-learner** : impute each unit's counterfactual with the opposite model, then regress; much better with unbalanced arms.
- **Causal forests / DR-learner** : honest sample splitting with doubly robust scores; the current default for credible CATE estimates.

Evaluate with the **Qini or uplift curve** : sort by predicted uplift, and plot the cumulative incremental conversions against the fraction targeted. The area over the random line is the value of the targeting. Ordinary AUC is meaningless here - you can never observe the individual-level label being predicted.

THE RULE THAT SAVES THE MOST MONEY

Never evaluate a targeting policy on the same experiment that trained it, and never evaluate it on outcome prediction accuracy. Hold out a randomized slice permanently, and measure the incremental effect of the policy in that slice. Almost every 'our model increased revenue 40%' claim that later evaporated was an uplift claim measured with a response metric.

Exercises

1 Reproduce the kidney-stone numbers and confirm both the within-group and the aggregate comparison. 2 Compute the required sample size for your own product's baseline conversion rate and the smallest lift worth shipping. 3 Simulate peeking: generate 1,000 A/A experiments, test daily for 14 days, and count how often you would have declared significance. 4 Apply CUPED to a past experiment using the pre-period metric and report the variance reduction achieved. 5 Take an observational dataset, draw the DAG, and estimate an effect twice: once adjusting for a collider and once not. Report both.

45. Privacy, Security and Governance

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

Privacy is one of the main reasons for on-device training: the movement data never leave the device.

A model is a lossy, queryable copy of its training data, deployed at an endpoint anyone can probe. That framing makes the risks concrete: data can leak out of the weights, behaviour can be corrupted by data going in, and the interface itself is an attack surface. This chapter covers the mechanisms and the obligations, both of which now appear in procurement questionnaires and in law.

45.1 The threat model

Attack	What the adversary gets	Practical defence
Membership inference	Whether a specific record was in training	Reduce overfitting, limit confidence output, differential privacy
Training-data extraction	Verbatim memorized secrets from a generative model	Deduplicate the corpus, scan and filter secrets, DP fine-tuning, output filters
Model inversion	Reconstruction of typical class inputs	Restrict query volume and returned detail
Model extraction / stealing	A functional copy from the API	Rate limits, monitoring for systematic querying, watermarking
Data poisoning / backdoor	A trigger that flips predictions	Provenance for training data, anomaly screening, hold out a trusted clean evaluation set
Adversarial examples	Wrong output from a tiny perturbation	Adversarial training, input sanitization, ensembling (Chapter 33)
Prompt injection	An LLM agent follows instructions from retrieved content	Treat all retrieved text as untrusted data; constrain tools; require confirmation for consequential actions
Supply chain	Malicious code in weights or a dependency	Load only safetensors-style formats, never arbitrary pickles; pin and verify hashes

FROM ZERO — the idea explained step by step

A threat model lists who could attack, what they want, and how.

IN OUR PROJECT — partly used

For HAR, movement data can reveal habits and health; keeping them on the device removes the main threat.

MEMORIZATION IS MEASURABLE, SO MEASURE IT

Large models reproduce rare training strings verbatim, and the rate rises with model size, with duplication in the corpus, and with the length of the prefix you supply. Before releasing a model trained on internal data, run the extraction test yourself: prompt it with prefixes of known-sensitive records and check what completes. Deduplication of the training corpus is the single most effective mitigation, and it improves quality at the same time.

45.2 Differential privacy, precisely enough to use

A randomized mechanism M is (ϵ, δ) -differentially private if for any two datasets D, D' differing in one record, and any output S :

FROM ZERO — the idea explained step by step

Differential privacy adds calibrated noise so no single person can be identified from the result.

IN OUR PROJECT — not used, and why

Not used. With on-device training and no data upload, it is not needed for the thesis scenario.

$$P[M(D) \text{ in } S] \leq e^{\epsilon} * P[M(D') \text{ in } S] + \delta$$

Interpretation: whatever anyone can conclude about you from the output, they could almost equally have concluded had you not participated at all.

epsilon is a privacy budget: smaller is stronger, and budgets compose additively across queries.

```
# DP-SGD: two changes to an ordinary training step
for batch in loader:
    per_sample_grads = compute_per_sample_gradients(model, batch)
    # 1. clip each example's gradient to a fixed L2 norm C
    clipped = [g * min(1.0, C / (g.norm() + 1e-6))
               for g in per_sample_grads]
    g_sum = sum(clipped)
    # 2. add Gaussian noise calibrated to C and the noise multiplier
    g_noisy = (g_sum + torch.normal(0., sigma * C, g_sum.shape)) / B
    optimizer.step_with(g_noisy)
# The accountant tracks epsilon from (sigma, sampling rate, steps).
```

Clipping bounds any single example's influence; the noise hides it. The cost is real: per-sample gradients are memory-hungry, and accuracy falls - typically a few points at epsilon around 8, more on small or imbalanced datasets.

epsilon	Reading	Where it is seen
< 1	Strong formal guarantee	Aggregate statistics, census-style releases
1 - 10	Meaningful in practice; the usual operating range	DP-SGD fine-tuning, telemetry collection
> 20	Little formal meaning; still blocks naive memorization	Reported honestly, or not reported at all

Federated learning (Chapter 31) is **not** privacy on its own: raw gradients can reveal their inputs, sometimes reconstructing images exactly. It becomes a privacy mechanism when combined with secure aggregation (the server sees only the sum of many updates) and user-level DP noise. State which of the three you have; they are routinely conflated in marketing material.

45.3 Handling personal data

- **Minimize** : the strongest protection for a field is not collecting it. Ask what decision each feature supports before it enters the warehouse.

FROM ZERO — the idea explained step by step

Handle personal data carefully: collect little, keep it locally, delete it when done.

IN OUR PROJECT — used

On-device training is data minimisation: the wearer's windows are used for training and never leave the board.

WORKED ON OUR PROJECT — Why on-device training helps privacy

1. The raw windows describe a person's movements through the day -- personal data.
2. Training on the device means those windows never leave it; only model updates (or nothing) are sent [20].
3. Model updates can still leak information; differential privacy adds calibrated noise to the updates to bound that leak [32].
4. The UCI HAR data are anonymised by subject number only; in a product the subject identity must be protected as well [9].

- **Separate identity from behaviour** : pseudonymize with keyed hashes, hold the key separately, and rotate it. Note that pseudonymized data is still personal data under most regimes.
- **Do not rely on k-anonymity alone** : high-dimensional behavioural data re-identifies easily - a handful of timestamped locations or ratings is usually unique to one person.
- **Retention and deletion** : define a retention period per dataset and enforce it automatically. A deletion request must reach backups, feature stores, logs and derived datasets, not just the primary table.
- **Unlearning** : removing a record's influence from trained weights is genuinely hard. The practical answers are retraining on a schedule, sharded training so only one shard must be retrained, or DP training that bounds any single record's influence in advance. Decide which before you promise deletion.
- **Cross-border and vendor flows** : sending data to an external API is a transfer; check what the provider retains and for how long, and record that decision where an auditor can find it.

45.4 Securing a deployed model, and an agent

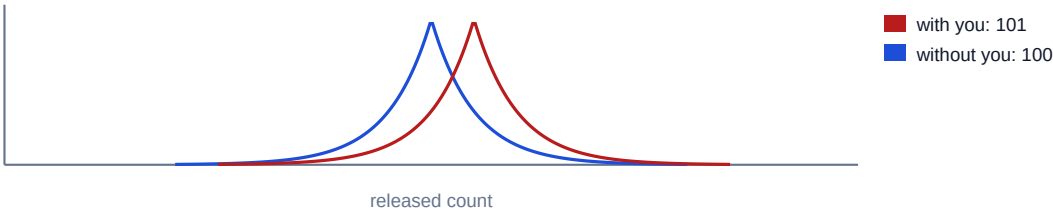
Layer	Control
Input	Schema and range validation, size limits, content-type checks; reject rather than coerce
Rate	Per-key quotas and anomaly detection on query patterns - the defence against extraction and inversion
Output	Confidence rounding, refusal of out-of-scope requests, PII and secret filters on generated text
Tools (agents)	Least privilege per tool, allowlists for domains and commands, no ambient credentials, human confirmation for irreversible actions
Context (agents)	Every retrieved document, web page, email and tool result is untrusted data, never instructions; keep system policy outside the retrievable corpus
Artefacts	Signed model files, hash-pinned dependencies, no arbitrary code execution on load, provenance recorded in the registry
Monitoring	Log inputs and outputs (with privacy controls), alert on distribution shifts and on refusal-rate changes, and keep an incident runbook with a rollback path

FROM ZERO — the idea explained step by step

Securing a deployed model against extraction and misuse.

IN OUR PROJECT — not used, and why

Not studied in the project.



Graph 45.1 — Differential privacy in one picture. A count is released with random noise added, large enough that the answer barely changes whether or not any one person is in the data. Smaller epsilon means more noise and more privacy.

THE AGENT SECURITY RULE, STATED ONCE

An LLM cannot reliably distinguish instructions written by your user from instructions embedded in content it reads. Therefore security cannot live in the prompt. It must live in the capabilities: what the tools can do, what credentials they carry, and which actions require a human. Design as if the model will at some point follow a malicious instruction, because eventually it will.

45.5 Governance: the paperwork that is actually load-bearing

Artefact	Contents	Why it pays for itself
Datasheet for the dataset	Provenance, consent basis, collection period, known gaps, licence	Answers the question that stalls every deployment review
Model card	Intended use, out-of-scope use, training data summary, metrics by slice, limitations	Forces the slice evaluation that finds the failure before users do
Evaluation record	Frozen test set, per-slice results, calibration, robustness and safety probes	The evidence when a regression is disputed
Risk classification	The decision the model influences, who is affected, reversibility, human oversight	Determines how much of the rest is required at all
Change log and registry	Versions, data snapshots, approvals, rollbacks	Makes incident response minutes rather than days

FROM ZERO — the idea explained step by step

Governance: documentation of data, model and limits.

IN OUR PROJECT — partly used

The project documents its data sources (UCI HAR [9]) and known limits in its docs/ folder [35].

Regimes differ in detail and change often, but the shape is now stable across them: obligations scale with the **risk of the use case**, not with the size of the model. Systems that affect employment, credit, education, essential services, health or law enforcement attract documentation, human-oversight, accuracy and record-keeping duties; content generation attracts disclosure duties; most internal tooling attracts little beyond ordinary data protection. Classify your use case early - it is cheap then, and expensive after the architecture is fixed.

Before a model touches real users

The legal basis for every training data source is written down.

Personal fields are minimized, pseudonymized, and on a retention clock.

A memorization or membership probe has been run for generative or small-data models.

Slice metrics exist for the groups the system can plausibly harm.

Input validation, rate limits and output filters are deployed, not planned.

Agent tools follow least privilege, with confirmation on irreversible actions.

The model card, datasheet and evaluation record exist and are current.

A rollback path and an incident owner are named and have been rehearsed.

Exercises

1 Run a membership-inference check on a model you trained: compare the loss distribution on training rows against held-out rows. A visible gap is the attack surface. 2 Fine-tune a small model with and without DP-SGD at epsilon around 8 and report the accuracy cost. 3 Take a retrieval-augmented assistant and write three prompt-injection payloads into the corpus it reads. Then fix it at the capability layer rather than in the prompt. 4

Write the model card and datasheet for a model you have already shipped. Note every question you cannot answer - each one is a gap.

5 Trace a deletion request end to end through your systems and list every store that would retain the record.

46. A Complete Project, Step by Step

HOW THIS CHAPTER CONNECTS TO OUR PROJECT

The complete project of this chapter can be followed with the thesis project side by side. Worked project B (46.15), a deep model for sensor data on a device, is almost our project.

Every previous chapter taught a piece. This one puts them in order. It is a single continuous procedure - thirteen stages from an unclear business request to a monitored production model - written so that you can follow it literally on your own data. Two full worked projects close the chapter: a tabular classifier and a deep-learning sensor model, each with the code that actually runs.

0. FRAME	what is predicted, what a mistake costs, what 'done' means
1. SET UP	repository, environment, seeds, experiment log
2. DATA	collect, document, and split BEFORE looking
3. EXPLORE	on the training split only
4. BASELINE	trivial, then simple - never skip this
5. PIPELINE	leak-free preprocessing as one object
6. MODEL	classical track first, deep track if justified
7. TUNE	search the few hyperparameters that matter
8. EVALUATE	slices, calibration, error analysis, confidence intervals
9. COMPRESS	quantize / prune / distil to the deployment budget
10. PACKAGE	freeze artefacts, write the inference path, test it
11. DEPLOY	shadow, canary, ramp, with rollback ready
12. MONITOR	drift, performance, cost - and retrain on a trigger

Figure 36.1 - The thirteen stages. Work them in order; the expensive mistakes come from skipping forward.

46.1 Stage 0 - Frame the problem before touching data

The most costly errors in machine learning are made before any code is written, and they all have the same shape: building a model that works but answers the wrong question. Write a one-page charter and get it agreed. If you cannot fill in every row below, you are not ready to start.

FROM ZERO — the idea explained step by step

Frame the problem: what is predicted, from what, with what budget, judged how.

IN OUR PROJECT — used

The thesis frame: predict 1 of 6 activities from 2.56 s of 9 sensor signals, train on a 320 KB microcontroller, judged by test accuracy on unseen people.

WORKED ON OUR PROJECT — The thesis mapped onto the 13 stages

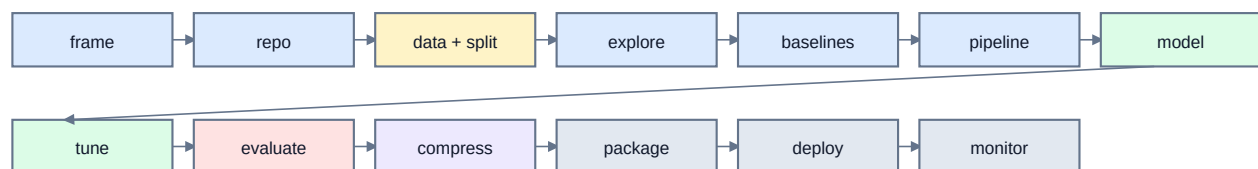
1. Frame: classify 6 activities on the STM32F746 while training on the device; the measure is test accuracy on new people and memory/time on the board.
2. Data: UCI HAR [9], split by the dataset's own subject split for testing; note the leaky random validation split (Chapter 3).
3. Baseline: float CNN 0.8953 +/- 0.0050 (10 seeds) [35]; classical baselines 0.76-0.84 (this appendix).
4. Compress: int8 SYM storage, FQT with 12/16-bit operands/gradients [3], RigL sparsity with ERK [4].
5. Evaluate: 10 seeds, paired comparisons, parity with PyTorch; package and deploy on the board; monitor via the run logs [35].

Question	Why it decides everything downstream	Example answer
What exactly is predicted, for what entity, at what moment?	Defines the unit of analysis and the exact timestamp at which features must be available - i.e. what counts as leakage	For each transaction, at authorisation time, the probability it is fraudulent
What decision does the prediction drive?	A number nobody acts on has no value; the action determines the metric	Send to a human analyst, or auto-decline
What does each kind of error cost?	Sets the metric and the decision threshold (Chapter 13)	False positive: 20 EUR of analyst time. False negative: ~500 EUR average loss
What is the current baseline?	You must beat something that already exists, or the project has no measurable value	A hand-written rule set catching 55% of fraud with 8% precision
What would make this a success?	The go/no-go number, agreed in advance, in business units	Same recall at 3x the precision, evaluated on next quarter's data
What are the constraints?	Latency, memory, privacy, regulation, explainability - each one removes options	Under 50 ms at p99; must run on-premise; the reason for a decline must be explainable
Who owns it after launch?	An unowned model rots within months	The risk team, with a monthly review

THREE FRAMINGS THAT SINK PROJECTS

PREDICTING SOMETHING NOBODY CAN ACT ON: a churn score delivered the day the customer leaves. THE TARGET IS A PROXY THAT DRIFTS FROM THE GOAL: optimising click-through when the goal is satisfaction. THE LABEL DOES NOT EXIST YET: 'predict which customers will love the new product' when the product has never shipped. In all three cases the modelling will succeed and the project will fail.

46.2 Stage 1 - Set up the repository and the environment



Graph 46.1 — The stages of a complete project, from the chapter. Each stage produces something the next one checks; the test set is only opened once, at evaluation.

Do this on day one, not after the first promising result. The cost is thirty minutes; the cost of not doing it is being unable to reproduce the result that mattered.

FROM ZERO — the idea explained step by step

Set up the repository and environment first.

IN OUR PROJECT — used

OnDeviceTraining: C library (src/), examples/, test/, CMake presets, uv for Python, CI on GitHub [35].

```

project/
|-- data/
|   |-- raw/           # immutable. NEVER write here after download
|   |-- interim/       # intermediate, reproducible from raw
|   +-- processed/     # model-ready tables/tensors
|-- notebooks/         # exploration only; nothing importable lives here
|-- src/
|   |-- data.py        # loading, splitting, validation
|   |-- features.py    # the preprocessing pipeline (one object)
|   |-- train.py       # takes a config, writes a run directory
|   |-- evaluate.py    # metrics, slices, plots
|   +-- serve.py       # the inference path used in production
|-- configs/           # one YAML per experiment, version-controlled
|-- models/            # saved artefacts, one folder per run id
|-- reports/           # figures, model cards, results tables
|-- tests/             # data validation + behavioural tests
|-- requirements.txt   # pinned versions, not ranges
+-- README.md          # how to reproduce, in five commands or fewer

```

Listing 36.1 - A layout that scales from a notebook to a team.

```

# src/utils.py - the reproducibility preamble every run should call
import os, random, json, subprocess, datetime
import numpy as np, torch
def set_seed(seed: int = 42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    # bit-exactness costs speed; enable only when you need it
    # torch.use_deterministic_algorithms(True)
def run_metadata(cfg: dict) -> dict:
    return {
        'timestamp': datetime.datetime.now().isoformat(timespec='seconds'),
        'git_commit': subprocess.check_output(
            ['git', 'rev-parse', 'HEAD']).decode().strip(),
        'git_dirty': bool(subprocess.check_output(
            ['git', 'status', '--porcelain']).decode().strip()),
        'config': cfg,
        'data_hash': cfg.get('data_hash'),
    }
# At the end of training, write this next to the weights:
# json.dump(run_metadata(cfg), open(run_dir / 'meta.json', 'w'), indent=2)

```

Listing 36.2 - Seeds, commit hash and config saved with every run. A checkpoint whose hyperparameters are unknown is nearly worthless.

Day-one setup

Git repository initialised, with data/ in .gitignore.

Virtual environment with pinned versions; the exact Python version recorded.

A make train or python -m src.train --config configs/baseline.yaml entry point that runs end to end on a 1% sample in under a minute.

An experiment log - MLflow, Weights and Biases, or a CSV that every run appends to.

The README explains how to reproduce the current best result from a clean checkout.

46.3 Stage 2 - Get the data, document it, and split it first

The split comes before exploration, not after. Every look at the test set spends a little of its value, and the first look is the one that biases every later decision.

FROM ZERO — the idea explained step by step

Get the data, document them, and split them before looking.

IN OUR PROJECT — used

UCI HAR: 6617 training / 735 validation / 2947 test windows; the test people are separate (overlap 0); no missing values (0 NaN, measured).

1 Choose the split axis from the deployment reality. If the model will see future data, split by time. If it will see new users, patients or devices, split by that entity. Random splitting is correct only when rows are genuinely independent - which, in practice, is rarer than it looks. **2 Freeze the test set.** Write it to disk with a hash, and do not regenerate it when the data is refreshed - or your numbers stop being comparable across weeks. **3 Record provenance:** where each field came from, when it was collected, its licence and consent basis, and known gaps in coverage. **4 Validate the schema automatically** so that a silent upstream change fails the pipeline instead of quietly degrading the model.

```
# src/data.py
import hashlib, pandas as pd
from sklearn.model_selection import StratifiedGroupKFold, train_test_split

def load_raw(path):
    df = pd.read_parquet(path)
    h = hashlib.sha256(pd.util.hash_pandas_object(df).values).hexdigest()
    return df, h[:12] # goes into the run metadata

def split_by_time(df, ts_col, val_frac=0.15, test_frac=0.15):
    df = df.sort_values(ts_col)
    n = len(df); i_te = int(n * (1 - test_frac))
    i_va = int(i_te * (1 - val_frac))
    return df.iloc[:i_va], df.iloc[i_va:i_te], df.iloc[i_te:]

def split_by_group(df, group_col, y_col, seed=42):
    # keeps every row of one subject/user/device on one side only
    g = df[group_col].unique()
    tr_g, te_g = train_test_split(g, test_size=0.2, random_state=seed)
    tr_g, va_g = train_test_split(tr_g, test_size=0.2, random_state=seed)
    m = lambda ids: df[df[group_col].isin(ids)]
    return m(tr_g), m(va_g), m(te_g)

def assert_no_overlap(tr, va, te, key):
    a, b, c = set(tr[key]), set(va[key]), set(te[key])
    assert not (a & b) and not (a & c) and not (b & c), 'split leak!'
```

Listing 36.3 - The three splits you will actually need, plus the assertion that catches the mistake.

THE LEAKAGE AUDIT, ONE QUESTION PER FEATURE

Go through every column and ask: at the exact moment I must make this prediction, would this value be known, and would it have this value? A column such as `total_amount_paid` fails for a default-prediction model because it is filled in afterwards. `account_status` fails if it is overwritten to 'closed' when fraud is confirmed. This audit takes an hour and routinely saves a project.

46.4 Stage 3 - Explore, on the training split only

FROM ZERO — the idea explained step by step

Explore on the training split only.

IN OUR PROJECT — used

E.g. per-channel statistics and class counts of the training split: classes are balanced within a ratio of 1.4478 (measured).

The exploration pass that pays for itself

Shape, dtypes, memory. Does the row count match what you were told?

Target distribution. What is the positive rate, or the shape of y ?

Missing-value rate per column; is missingness correlated with the target?

Cardinality of every categorical; are there near-duplicate levels ('watch_v2' vs 'Watch V2')?

Numeric ranges and impossible values; physical or business plausibility.

Duplicates, exact and near.

Time coverage: gaps, seasonality, a regime change halfway through.

Correlations with the target - and be suspicious of anything above 0.95, which usually means leakage rather than luck.

Look at 50 raw samples with your own eyes. Every single time.

THE SUSPICIOUSLY GOOD FEATURE

A single feature with an AUC of 0.99 on its own is almost never a discovery; it is almost always leakage, a duplicate of the label under another name, or an artefact of how the dataset was assembled. Treat it as a bug report and go find the cause before celebrating.

46.5 Stage 4 - Establish baselines before modelling

You need two numbers before any real model: what you get for free, and what you get for very little. Without them you cannot tell whether a later result is good.

FROM ZERO — the idea explained step by step

Establish simple baselines first.

IN OUR PROJECT — used

Our CNN trained by the C library reaches 0.9002 on the new people (one run; 0.8953 +/- 0.0050 over 10 seeds in the project's sweep [35]).

For comparison, simple baselines on 18 hand-made features, computed in Python with scikit-learn (they are not part of the project): naive Bayes 0.7577, softmax regression 0.7869, SVM 0.8392, random forest 0.8096.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

Baseline	Classification	Regression	Why it matters
Trivial	Always predict the majority class	Always predict the mean or median	Any metric must beat this or the model has learned nothing
Naive domain rule	The existing hand-written rules	Last value carried forward	This is what you are actually replacing
Simple model	Logistic regression on the raw columns	Ridge regression	Fast, interpretable, and surprisingly hard to beat; it also smoke-tests the whole pipeline
Strong classical	Gradient boosting with defaults	Gradient boosting with defaults	On tabular data this is often the final model; anything fancier must beat it

```

from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import average_precision_score
for name, model in [
    ('trivial', DummyClassifier(strategy='prior')),
    ('logistic', make_pipeline(pre, LogisticRegression(max_iter=2000))),
    ('gbdt', make_pipeline(pre, HistGradientBoostingClassifier())),
]:
    model.fit(X_tr, y_tr)
    ap = average_precision_score(y_va, model.predict_proba(X_va[:, 1]))
    print(f'{name:9s} val AP = {ap:.4f}')
# Typical output on an imbalanced problem:
# trivial    val AP = 0.0310      <- this is the floor (the positive rate)
# logistic   val AP = 0.2840
# gbdt       val AP = 0.4120      <- the number to beat from now on

```

Listing 36.4 - Three baselines in fifteen lines. Run this on day two.

46.6 Stage 5 - Build the preprocessing pipeline as one object

Everything data-dependent - imputation, scaling, encoding, feature selection, resampling - must live inside a single fitted object that is refitted on each training fold and saved with the model. This is not style; it is the structural defence against the leakage of Chapter 3 and against training/serving skew in Chapter 34.

FROM ZERO — the idea explained step by step

Put all preprocessing into one object so training and use are identical.

IN OUR PROJECT — used

In the project, `prepare_data.py` produces `.npy` files that both PyTorch and C read, so both see identical inputs [35].

```

# src/features.py
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
NUM = ['amount', 'age_days', 'n_prior_tx', 'hour_sin', 'hour_cos']
CAT = ['country', 'device', 'merchant_category']
def build_preprocessor():
    num = Pipeline([
        ('imp', SimpleImputer(strategy='median', add_indicator=True)),
        ('sc', StandardScaler()),
    ])
    cat = Pipeline([
        ('imp', SimpleImputer(strategy='constant', fill_value='__NA__')),
        ('oh', OneHotEncoder(handle_unknown='ignore', min_frequency=20)),
    ])
    return ColumnTransformer([('num', num, NUM), ('cat', cat, CAT)],
                             remainder='drop') # be explicit, always
def add_derived(df):
    # derived features computed from ONE row's own past only
    df = df.copy()
    df['hour_sin'] = np.sin(2*np.pi*df['ts'].dt.hour/24)
    df['hour_cos'] = np.cos(2*np.pi*df['ts'].dt.hour/24)
    df['amount_per_prior'] = df['amount'] / (df['n_prior_tx'] + 1)
    return df

```

Listing 36.5 - One preprocessor object, used identically in training, cross-validation and serving.

‘REMAINDER=‘DROP’ IS A SAFETY FEATURE

Listing every column you intend to use, and dropping the rest, means that a new column appearing upstream cannot silently enter the model - and that an ID column cannot accidentally become a feature. The default of silently passing everything through is how identifiers end up as predictors.

46.7 Stage 6 - Model iteration: two tracks

Choose the track from the data, not from ambition. On tabular data with fewer than a million rows, the classical track usually wins and always wins on effort per point of accuracy. On perceptual data - images, audio, text, raw waveforms - the deep track wins decisively, and you should start from pretrained weights.

FROM ZERO — the idea explained step by step

Iterate on the model, one change at a time.

IN OUR PROJECT — used

The thesis configurations change one thing each: optimiser, bits, learning-rate schedule, sparsity [35].

Signal	Take the classical track	Take the deep track
Data type	Tabular, mixed numeric/categorical	Images, audio, text, raw sensor streams
Rows	10^3 - 10^6	10^4 + from scratch, or 10^2 + with a pretrained backbone
Feature semantics	Meaningful named columns	Raw signal where useful features are unknown
Constraints	Explainability, tiny training budget, CPU-only	GPU available, latency budget allows a network
First model	HistGradientBoosting / LightGBM defaults	A pretrained backbone plus a new head

The iteration loop, one change at a time

1 Form a hypothesis: 'adding 7-day rolling aggregates will help because fraud is bursty'. 2 Change exactly one thing. Two changes in one run teach you nothing about either. 3 Evaluate on the validation split (or by cross-validation), with the same seed policy as your baseline.

4 Record the result in the experiment log, including failures - the log of what did not work is what stops you retrying it in six weeks. 5 Keep the change only if it beats the current best by more than the seed-to-seed noise you measured in stage 4.

THE ORDER THAT FINDS ACCURACY FASTEST

1. Fix data problems - label noise, leakage, wrong splits. Nothing else matters until these are clean. 2. Add features (classical) or improve augmentation and the backbone (deep). 3. Tune the hyperparameters that actually move the metric. 4. Ensemble. Most people run this list backwards and spend a week on step 4 for a quarter of the gain step 1 would have given them.

46.8 Stage 7 - Tune only what matters

Model	Tune first	Tune next	Leave alone
Gradient boosting	learning_rate + n_estimators (with early stopping)	num_leaves / max_depth, min_samples_leaf	Everything else, usually
Random forest	max_features	min_samples_leaf, n_estimators	Criterion
Linear / logistic	C or alpha (log scale)	Penalty type	Solver, unless it fails to converge

Model	Tune first	Tune next	Leave alone
Neural network	learning rate + schedule	Batch size, weight decay, augmentation strength, width/depth	Optimiser betas, epsilon, activation function

FROM ZERO — the idea explained step by step

Tune only what matters most (usually the learning rate).

IN OUR PROJECT — used

The project tuned the learning rate per optimiser (SGD 0.01, AdamW 0.001) and the schedule for 4 bits [35].

Use random or Bayesian search over log-uniform ranges (Chapter 7), give the search a fixed budget decided in advance, and always include the default configuration as one of the trials - it wins more often than people expect, and if it does, you have learned that tuning is not your bottleneck.

46.9 Stage 8 - Evaluate like a sceptic

FROM ZERO — the idea explained step by step

Evaluate like a sceptic: unseen data, spreads, per-class errors.

IN OUR PROJECT — used

10 seeds per configuration, test on unseen people, confusion matrix per class (Chapter 13) [35].

Before you claim a result

Metric matches the decision and its costs, chosen in stage 0 - not accuracy by default.

Compared against the trivial and the existing baseline, on the same split.

Averaged over 3-5 seeds, with the spread reported.

Bootstrap confidence interval on the test metric.

Per-slice metrics: by class, by subgroup, by data source, by time period, by difficulty.

Calibration checked, and fixed with temperature or isotonic scaling if the probability is used downstream.

Decision threshold chosen on validation using the cost matrix, not left at 0.5.

Error analysis: read 50 of the worst errors and categorise them. This is where the next feature idea comes from.

Test set touched exactly once, at the very end.

```
# src/evaluate.py - the error-analysis step people skip
import numpy as np, pandas as pd
proba = model.predict_proba(X_va)[: , 1]
err = pd.DataFrame({'y': y_va, 'p': proba}).assign(
    loss=lambda d: -(d.y*np.log(d.p+1e-9) + (1-d.y)*np.log(1-d.p+1e-9)))
worst = err.sort_values('loss', ascending=False).head(50).index
review = X_va.loc[worst].copy()
review['true'] = y_va.loc[worst]; review['pred'] = proba[worst]
review.to_csv('reports/worst_50.csv')
# Now READ it. Group the failures into 3-5 named categories and count
# them. Each category is either a data fix, a feature idea, or a
# genuine limit of the problem - and you cannot tell which from a
# metric alone.
```

Listing 36.6 - Error analysis. Half an hour here beats a day of tuning.

46.10 Stage 9 - Compress to the deployment budget

If the model runs in the cloud with generous latency, skip this stage. If it runs on a phone, a vehicle, a wearable or a microcontroller, this is where Chapters 28-31 are applied, in this order:

FROM ZERO — the idea explained step by step

Compress to the deployment budget.

IN OUR PROJECT — used

Quantisation (Chapter 28) and sparsity (Chapter 29) bring training within 72.5% of the board's RAM [35].

Step	Action	Typical result	Check afterwards
1	Measure the dense baseline ON THE TARGET DEVICE	The only latency number that means anything	p50 and p99, not the mean
2	Distil into a smaller architecture if the gap allows	2-10x smaller at a small accuracy cost	Accuracy against the teacher
3	Structured pruning of channels, heads or blocks	30-50% fewer parameters, real speedup	Rebuild the model with smaller layers, then re-measure
4	INT8 post-training quantization with a calibration set	4x smaller, 2-4x faster	Evaluate with the target RUNTIME, not the simulator
5	Quantization-aware fine-tuning if PTQ lost too much	Recovers most of the gap	Per-class accuracy, not just the average
6	Re-measure everything on the device	The number you report	Memory, energy, and thermal behaviour under sustained load

COMPRESSION DAMAGE IS NOT UNIFORM

Quantization and pruning almost always hurt rare classes and unusual inputs more than the average. A model that loses 0.4 points overall can easily lose 6 points on the smallest class. Always compare the per-slice table before and after, and treat a large per-slice drop as a blocker even when the headline number looks fine.

46.11 Stage 10 - Package the artefact

What ships is not the notebook. It is a directory containing everything needed to reproduce a prediction, and an inference function that is tested.

FROM ZERO — the idea explained step by step

Package the model with everything needed to run it.

IN OUR PROJECT — used

The ODT library serialises models (src/serial) and loads PyTorch weights (StateDictApi) [35].

```
models/2024-11-03_gbdt_v7/
|-- model.joblib          # the FULL pipeline: preprocessing + estimator
|-- meta.json             # git commit, config, data hash, seeds
|-- metrics.json          # test metrics, per slice, with CIs
|-- threshold.json        # the operating point and how it was chosen
|-- feature_schema.json   # names, dtypes, allowed ranges
+-- model_card.md         # intended use, data, limits, ethics
```

Listing 36.7 - A deployable run directory.

```
# src/serve.py - the one code path production uses
import joblib, json, numpy as np, pandas as pd
class Predictor:
    def __init__(self, run_dir):
        self.pipe = joblib.load(f'{run_dir}/model.joblib')
        self.schema = json.load(open(f'{run_dir}/feature_schema.json'))
        self.thr = json.load(open(f'{run_dir}/threshold.json'))['value']
    def _validate(self, df):
        missing = set(self.schema['columns']) - set(df.columns)
        if missing:
            raise ValueError(f'missing columns: {sorted(missing)}')
        for c, (lo, hi) in self.schema.get('ranges', {}).items():
            bad = df[c].dropna().between(lo, hi).eq(False).sum()
            if bad:
                raise ValueError(f'{bad} out-of-range values in {c}')
    def predict(self, records: list[dict]) -> list[dict]:
        df = add_derived(pd.DataFrame(records))
        self._validate(df)
        p = self.pipe.predict_proba(df)[:, 1]
        return [{'score': float(s), 'decision': bool(s >= self.thr)}
                for s in p]
```

Listing 36.8 - Inference with input validation. The validation is not optional: silent schema drift is the most common production failure.

```
# tests/test_serve.py
def test_known_case():
    out = predictor.predict([GOLDEN_RECORD])[0]
    assert abs(out['score'] - 0.8123) < 1e-4 # regression guard
def test_invariance():
    a = predictor.predict([REC])[0]['score']
    b = predictor.predict([**REC, 'customer_name': 'different'])[0]['score']
    assert a == b # an irrelevant field must not move the score
def test_direction():
    low = predictor.predict([**REC, 'amount': 10])[0]['score']
    high = predictor.predict([**REC, 'amount': 10_000])[0]['score']
    assert high > low # domain knowledge, encoded as a test
def test_rejects_bad_input():
    with pytest.raises(ValueError):
        predictor.predict([**REC, 'amount': -5])
```

Listing 36.9 - Behavioural tests. These catch the failures that unit tests on training code never will.

46.12 Stage 11 - Deploy carefully

- **Shadow:** run the new model on live traffic, log its output, act on nothing. Compare its score distribution against the offline distribution - a mismatch here means training/serving skew, and it is common.

FROM ZERO — the idea explained step by step

Deploy carefully, step by step.

IN OUR PROJECT — partly used

For the thesis: PC parity first, then the board, checking memory and timing on the hardware.

- **Canary:** route 1-5% of traffic to it, watch the operational and business metrics for a defined period.
- **Ramp:** 25%, 50%, 100%, with a documented rollback trigger at each step.
- **Keep the previous model warm** and rehearse the rollback before you need it.
- **A/B test against the business metric**, not the offline metric - they disagree more often than they agree.

46.13 Stage 12 - Monitor, and plan for decay

A deployed model is a photograph of the past. Assume it will degrade and instrument for it from the first day.

FROM ZERO — the idea explained step by step

Monitor and plan for decay: models get worse as the world changes.

IN OUR PROJECT — used

On-device training is the thesis answer to decay: the model follows its wearer.

Signal	How to measure	Alert when
Input drift	PSI or KS per feature against a training reference	PSI > 0.2 on an important feature
Prediction drift	Score distribution over time	Mean score moves beyond its historical band
Performance	Metrics on delayed labels	Below the agreed floor for two consecutive periods
Data health	Null rates, schema, cardinality	Any schema violation - fail loudly, never silently impute
Operational	Latency p99, error rate, cost per 1k predictions	Breaches the SLA
Business	The metric from stage 0	Any sustained decline

Decide the retraining policy in advance: scheduled (simplest and usually adequate), or triggered by drift. Whichever you choose, a retrained model is promoted only after beating the incumbent on a frozen comparison set - automatic training is fine, automatic promotion is not.

46.14 Worked project A - a tabular classifier, end to end

Fraud detection on transactions: 500,000 rows, 3% positive, 40 columns, a 50 ms latency budget, and a requirement that any decline be explainable. Here is the whole project as one script, with the decisions annotated.

FROM ZERO — the idea explained step by step

Worked project A: a tabular classifier.

IN OUR PROJECT — partly used

Our feature baselines (random forest 0.8096) are a tabular project of this kind on HAR data.

Numbers marked with this note were computed in Python (scikit-learn) on 18 hand-made features from the project's data files. They illustrate the method; the algorithm is not part of the project's C code.

```

# 0. FRAME -> metric: average precision; threshold from the cost matrix
#           (FP = 20 EUR analyst time, FN = 500 EUR loss)
# 2. SPLIT -> by TIME: fraud patterns evolve, so future data is the
#           honest test. Never random-split transactional data.
df, data_hash = load_raw('data/raw/tx.parquet')
df = add_derived(df)
tr, va, te = split_by_time(df, 'ts')
X_tr, y_tr = tr.drop(columns=['is_fraud']), tr['is_fraud']
X_va, y_va = va.drop(columns=['is_fraud']), va['is_fraud']
# 4. BASELINES
#   trivial AP = 0.031   logistic AP = 0.284   gbdt AP = 0.412
# 5-6. PIPELINE + MODEL
import lightgbm as lgb
pre = build_preprocessor()
Xt, Xv = pre.fit_transform(X_tr), pre.transform(X_va)
model = lgb.LGBMClassifier(
    objective='binary', n_estimators=5000, learning_rate=0.03,
    num_leaves=63, min_child_samples=50, subsample=0.8,
    colsample_bytree=0.8, reg_lambda=1.0,
    scale_pos_weight=1.0) # prefer threshold tuning to reweighting
model.fit(Xt, y_tr, eval_set=[(Xv, y_va)], eval_metric='average_precision',
    callbacks=[lgb.early_stopping(200), lgb.log_evaluation(0)])
# 8. THRESHOLD from costs, chosen on VALIDATION
import numpy as np
p_va = model.predict_proba(Xv)[:, 1]
ths = np.linspace(0.01, 0.99, 99)
cost = [(20 * ((p_va >= t) & (y_va == 0)).sum() +
    500 * ((p_va < t) & (y_va == 1)).sum()) for t in ths]
t_star = ths[int(np.argmin(cost))]
print('threshold', round(t_star, 3), 'expected cost', min(cost))
# 8. SLICES - the table that decides whether this ships
for col in ['country', 'device', 'merchant_category']:
    for lvl, idx in X_va.groupby(col).groups.items():
        if len(idx) < 200: continue
        ap = average_precision_score(y_va.loc[idx], p_va[X_va.index.get_indexer(idx)])
        print(f'{col}={lvl:20s} n={len(idx):6d} AP={ap:.3f}')
# 8. EXPLAINABILITY - required by stage 0, so it is part of the model
import shap
explainer = shap.TreeExplainer(model)
def reason_codes(x_row, k=3):
    sv = explainer.shap_values(x_row)
    order = np.argsort(-np.abs(sv))[:k]
    return [(feature_names[i], float(sv[i])) for i in order]

```

Listing 36.10 - Project A. Note what is NOT here: no neural network, no exotic sampling, no stacking. On tabular data those come last, if at all.

The experiment log for this project is reproduced below. The numbers are from one representative run; what matters is the SHAPE of the table - which changes paid, which did not, and which decision each result triggered.

Stage	Result	Decision taken
Baselines	trivial 0.031, logistic 0.284, GBDT 0.412	GBDT is the track; logistic stays as the sanity check
+ rolling 7-day aggregates per card	0.468	Kept - largest single gain, and it matches the domain story
+ 60 tuning trials (Optuna)	0.487	Kept, but note it is worth less than one good feature
+ 5-seed ensemble	0.494	Rejected - 0.007 AP is not worth 5x the serving cost at a 50 ms budget
Threshold from costs	$t^* = 0.19$	Expected cost falls 41% versus the 0.5 default, with no model change
Slice check	AP 0.31 on one country with 4% of traffic	Blocker - investigated, traced to a missing merchant category mapping, fixed in the pipeline

Stage	Result	Decision taken
Final on frozen test	AP 0.481 (95% CI 0.462-0.499)	Ships. Beats the rule set at 3.1x the precision at equal recall

46.15 Worked project B - a deep model for sensor data, on-device

Human activity recognition from a wrist accelerometer: 30 subjects, 50 Hz, 6 activities, and the model must run on the watch itself in under 20 ms with less than 256 KB of RAM. This is the project this repository is about, and every constraint changes a decision.

FROM ZERO — the idea explained step by step

Worked project B: a deep model for sensor data, deployed on a device.

IN OUR PROJECT — used

This is our project: a 1-D CNN on 9 IMU channels, 10,214 parameters, quantised to 8 bits, trained with RigL sparsity, on an STM32F746ZG. Read this section next to the PREFACE box "Meet the project".

```
# 2. SPLIT BY SUBJECT. A random split here would let the model
# recognise the person instead of the activity and would report
# ~99% accuracy that collapses on a new wearer.
tr, va, te = split_by_group(df, group_col='subject_id', y_col='activity')
assert_no_overlap(tr, va, te, 'subject_id')
# 3-5. WINDOWING - 2 s windows, 50% overlap, computed WITHIN each split
Xtr, ytr = make_windows(tr, fs=50, sec=2.0, overlap=0.5) # (N, 100, 3)
Xva, yva = make_windows(va, fs=50, sec=2.0, overlap=0.0) # no overlap
# 4. BASELINE FIRST - classical features + GBDT, 30 seconds to train
# accuracy 0.883. A deep model must beat this to justify itself.
# 6. DEEP TRACK - a small 1D CNN sized for the device from the start
import torch.nn as nn
def block(cin, cout, k=5, s=2):
    return nn.Sequential(
        nn.Conv1d(cin, cin, k, s, k//2, groups=cin, bias=False), # depthwise
        nn.Conv1d(cin, cout, 1, bias=False), # pointwise
        nn.BatchNorm1d(cout), nn.ReLU())
model = nn.Sequential(
    nn.Conv1d(3, 32, 7, 2, 3, bias=False), nn.BatchNorm1d(32), nn.ReLU(),
    block(32, 64), block(64, 64), block(64, 128),
    nn.AdaptiveAvgPool1d(1), nn.Flatten(),
    nn.Dropout(0.3), nn.Linear(128, 6))
# 17,158 parameters -> 67 KB in fp32, 17 KB in INT8. Weights fit the
# budget with room left for activations, which for a (100, 3) input and
# these widths peak at about 25 KB - the number that actually binds.
# 6. AUGMENTATION that encodes true invariances for this sensor
# jitter, scaling, time warping, small rotations of the sensor frame.
# NOT horizontal flip - reversing time is not an invariance here.
# 9. COMPRESS to the device budget. Sizes below are computed from the
# parameter counts; the latency and accuracy columns are the shape
# of a representative run - measure your own on the real device.
#
#           weights    latency    accuracy
# fp32 dense           67 KB      31 ms      0.921
# + 30% channel prune + FT  38 KB      22 ms      0.916
# + INT8 QAT            10 KB       9 ms      0.913
# + distil from the teacher 10 KB       9 ms      0.919 <- ships
```

Listing 36.11 - Project B. The architecture was chosen against the memory budget before training started, not shrunk afterwards.

WHAT DIFFERED BETWEEN THE TWO PROJECTS

Project A split by time, project B by subject - both because of how the model will be used, not because of a convention. A used no deep learning at all; B used a small one and still had to beat a classical baseline to justify it. A's hard constraint was explainability, which kept it on trees; B's was 256 KB of RAM, which decided the architecture before the first epoch. The thirteen stages were identical; every decision inside them came from stage 0.

46.16 Time budget and the failure timeline

Week	What a healthy project is doing	What a failing one is doing
1	Charter agreed; repository set up; data loaded and split; baselines running	Reading papers; choosing a framework; building a model with no baseline
2	Leakage audit; exploration; first real model beating the baseline	Tuning a model whose split is wrong
3-4	Feature or augmentation iteration; error analysis driving the next change	Trying a fifth architecture; the metric has not moved since week 2
5	Slice evaluation; calibration; threshold from costs; test set touched once	Discovering the test set has been used for selection all along
6	Packaging, behavioural tests, shadow deployment	Realising the features cannot be computed at serving time
7+	Canary, ramp, monitoring, first retraining rehearsal	A model that scores well and has never seen a real request

FROM ZERO — the idea explained step by step

Time budget: where the time of a project really goes.

IN OUR PROJECT — partly used

In the thesis most time went to the C implementation and to proving it matches PyTorch.

46.17 The one-page master checklist

FROM ZERO — the idea explained step by step

A one-page checklist for any project.

IN OUR PROJECT — used

Each line of it can be ticked for the thesis with the numbers in these boxes.

Framing and data

The prediction target, the entity and the decision moment are written down.

The cost of each error type is quantified.

The baseline being replaced is measured.

The success criterion was agreed before modelling started.

The split axis matches deployment (time / group / random).

The test set is frozen and hashed.

Every feature passed the 'available at prediction time' audit.

Modelling and evaluation

Trivial and simple baselines exist and are beaten.

All preprocessing lives inside the pipeline object.

One change per experiment; every run logged with its config and commit.

Results averaged over seeds, with the spread reported.

Per-slice metrics computed; no slice regressed unacceptably.

Calibration checked; the threshold was chosen from costs on validation.

50 worst errors were read and categorised.

The test set was used exactly once.

Shipping and keeping it alive

Latency, memory and energy measured on the target device with the target runtime.

The artefact directory contains model, metadata, metrics, threshold, schema and model card.

Input validation and behavioural tests exist and run in CI.

Shadow and canary stages completed; rollback rehearsed.

Drift, performance and cost monitoring are live with owned alerts.

The retraining policy and the promotion gate are written down.

An owner is named.

IF YOU REMEMBER ONLY ONE THING FROM THIS CHAPTER

Stage 0 and stage 4 - a written charter and an honest baseline - cost one day between them and determine whether the other twelve stages produce anything of value. Teams that skip them build models that are technically excellent answers to questions nobody asked, measured against nothing.

Exercises

1 Take a dataset you already have and write its stage-0 charter. If you cannot fill in the cost of a false positive, go and find out - that conversation is the exercise. 2 Reproduce the thirteen stages on a public dataset, timing each one. Most people are surprised by how much of the total lands in stages 2-3. 3 Take a model you trained earlier in this book, package it as in stage 10, and write four behavioural tests for it. 4 Deliberately choose the wrong split axis for a grouped dataset, then the right one, and report both numbers. Keep the pair - it is the most convincing argument you will ever have for careful splitting.

APPENDICES

Appendices

A compact mathematical reference, a glossary of every term used in this book, and a study roadmap with resources.

Appendix A. Mathematical Reference

Linear algebra

Dot product	$a \cdot b = \sum_i a_i b_i$
Matrix product	$(AB)_{ij} = \sum_k A_{ik} B_{kj} \quad (m,k)(k,n) \rightarrow (m,n)$
Transpose	$(AB)^T = B^T A^T$
Inverse	$(AB)^{-1} = B^{-1} A^{-1}$
Trace	$\text{tr}(AB) = \text{tr}(BA), \quad \text{tr}(A) = \sum_i \lambda_i$
Determinant	$\det(AB) = \det(A)\det(B), \quad \det(A) = \prod_i \lambda_i$
Norms	$\ x\ _1, \ x\ _2, \ x\ _\infty$
Orthogonal Q	$Q^T Q = I, \quad \ Qx\ = \ x\ $
Eigen	$A v = \lambda v; \quad \text{symmetric } A = Q L Q^T$
SVD	$A = U S V^T \quad (\text{always exists})$
PSD	$x^T A x \geq 0 \text{ for all } x \Leftrightarrow \text{all eigenvalues } \geq 0$

Matrix calculus (denominator layout)

$d(a^T x)/dx$	$= a$
$d(x^T A x)/dx$	$= (A + A^T) x = 2Ax \text{ if } A \text{ symmetric}$
$d(\ x\ ^2)/dx$	$= 2x$
$d(\ Ax - b\ ^2)/dx$	$= 2 A^T (Ax - b)$
$d(\text{tr}(AB))/dA$	$= B^T$
$d(\log \det A)/dA$	$= A^{-T}$
Chain rule	$dz/dx = (dz/dy)(dy/dx)$

Probability

Bayes	$P(H D) = P(D H)P(H)/P(D)$
Expectation	$E[aX + bY] = aE[X] + bE[Y] \quad (\text{always})$
Variance	$\text{Var}[X] = E[X^2] - E[X]^2$ $\text{Var}[aX] = a^2 \text{Var}[X]$ $\text{Var}[X+Y] = \text{Var}[X] + \text{Var}[Y] \text{ if independent}$
Covariance	$\text{Cov}[X,Y] = E[XY] - E[X]E[Y]$
Gaussian	$p(x) = (1/\sqrt{2\pi s^2}) \exp(-(x-\mu)^2 / (2 s^2))$
CLT	mean of n iid samples $\rightarrow N(\mu, \sigma^2/n)$
Jensen	$f \text{ convex} \Rightarrow f(E[X]) \leq E[f(X)]$

Information theory

$H(p)$	$= -\sum p \log p$
$H(p,q)$	$= -\sum p \log q \quad (\text{cross-entropy})$
$KL(p q)$	$= \sum p \log(p/q) \geq 0 \quad (0 \text{ iff } p = q)$
$H(p,q)$	$= H(p) + KL(p q)$
$I(X;Y)$	$= H(X) - H(X Y) = KL(p(x,y) p(x)p(y))$

Key derivatives used in this book

```

sigmoid s(z)      s'(z) = s(z)(1 - s(z))
tanh(z)           1 - tanh^2(z)
ReLU(z)           1 if z > 0 else 0
GELU(z)           Phi(z) + z phi(z)
softmax_i wrt z_j  p_i(delta_ij - p_j)
CE(softmax) wrt z  p - y          <- the identity to remember
BCE(sigmoid) wrt z  p - y          <- the same identity
MSE wrt y_hat      2(y_hat - y)/n

```

Complexity cheat-sheet

Operation	Cost
Dense layer forward, batch n	$O(n * d_{in} * d_{out})$
Backward pass	About 2x the forward cost
Conv layer	$O(K^2 * C_{in} * C_{out} * H * W)$
Self-attention	$O(n^2 d)$ time, $O(n^2)$ memory naively
Normal equations	$O(n d^2 + d^3)$
SVD of (n,d)	$O(n d \min(n,d))$
k-NN query, brute force	$O(n d)$
Decision tree training	$O(n d \log n)$
k-means iteration	$O(n k d)$

Useful numbers

Quantity	Value
FP32 / FP16 / INT8 bytes per value	4 / 2 / 1
Adam training memory	~16 bytes per parameter (fp32) before activations
Cross-entropy at random init, K classes	$\log K$ (2.30 for K=10, 6.91 for K=1000)
Bootstrap sample coverage	63.2% of rows appear at least once
Chinchilla-optimal tokens per parameter	~20
KV cache bytes	$2 * \text{layers} * \text{kv_heads} * \text{head_dim} * \text{seq} * \text{batch} * \text{bytes_per_value}$
Rule of thumb, samples needed	10-100x the number of free parameters for classical models; far less with pretraining

Appendix B. Glossary

Term	Definition
A/B test	A randomised controlled experiment on live traffic; the only design that estimates a causal effect without assumptions.
Activation function	Non-linearity applied to a neuron's weighted sum; without it, depth adds nothing.
AdamW	Adam with decoupled weight decay; the default optimiser for Transformers.
Arithmetic intensity	FLOPs performed per byte moved; decides whether an operation is compute-bound or memory-bound.
ATE / CATE	Average treatment effect over everyone, and the conditional version for a given feature vector.

Term	Definition
Attention	A mechanism that computes a weighted average of values, where the weights come from query-key similarity.
Autoregressive	Generating a sequence one element at a time, each conditioned on all previous ones.
Backdoor criterion	The graphical rule for choosing which variables to adjust for when estimating a causal effect.
Backpropagation	Reverse-mode automatic differentiation applied to a neural network.
Bagging	Training models on bootstrap resamples and averaging them to cut variance.
Batch normalisation	Normalising activations using statistics of the current mini-batch, with learned scale and shift.
Bias (parameter)	The additive constant term in a linear or convolutional layer.
Bias (statistical)	Systematic error from a model class too simple to represent the truth.
BM25	A length-normalised, frequency-saturating sparse retrieval score; still the strongest non-neural retriever.
Boosting	Sequentially adding weak models, each correcting the current ensemble's errors.
BPR	Bayesian Personalised Ranking; a pairwise loss that trains a recommender to order a seen item above a sampled unseen one.
Calibration	Agreement between predicted probabilities and observed frequencies.
Capacity	How complex a function a model class can represent.
Catastrophic forgetting	Loss of previously learned ability when fine-tuning on new data.
Checkpointing (gradient)	Recomputing activations in the backward pass to save memory.
Collider	A variable caused by two others; conditioning on it creates a spurious association between them.
Contrastive learning	Training representations by pulling matching pairs together and pushing others apart.
Convolution	A weight-sharing local operation that slides a kernel over the input.
Cross-entropy	The standard classification loss; equals KL divergence from the label distribution up to a constant.
Cross-validation	Repeated train/validate splits used to estimate generalisation.
CTC	Connectionist Temporal Classification; a loss that sums over all alignments of an output sequence to a longer input.
CUPED	Variance reduction in experiments using a pre-experiment covariate; typically halves the required sample size.
Differential privacy	A formal guarantee, parameterised by epsilon, that one record's presence barely changes the output distribution.
Diffusion model	A generative model that learns to reverse a gradual noising process.
Distillation	Training a small student to imitate a large teacher's outputs.
Distribution shift	The deployment data distribution differing from the training one.
Double descent	The modern test-error curve: rising to a peak at the interpolation threshold, then falling again as capacity grows.
DP-SGD	Training with per-example gradient clipping plus calibrated Gaussian noise, giving a differential-privacy bound.
Dropout	Randomly zeroing units during training as a regulariser.
Early stopping	Halting training when validation performance stops improving.
ELBO	Evidence lower bound; the objective maximised by variational inference and by variational autoencoders.
EM algorithm	Alternating expected-assignment and parameter-update steps for models with latent variables.

Term	Definition
Embedding	A learned dense vector representing a discrete item.
Ensemble	A combination of several models' predictions.
Epoch	One full pass over the training set.
Feature	One measured input variable.
Fine-tuning	Continuing training of a pretrained model on a target task.
FLOPs	Floating-point operations; a compute measure that is a poor proxy for latency.
FSDP / ZeRO	Sharding optimiser states, gradients and parameters across data-parallel workers so a model too large for one device fits.
Gaussian process	A prior over functions defined by a kernel; gives exact posterior uncertainty and drives Bayesian optimisation.
Generalisation	Performance on data not used for training.
Gradient clipping	Rescaling gradients whose norm exceeds a threshold.
Gradient descent	Iteratively stepping parameters against the gradient of the loss.
Hidden Markov model	A latent discrete state sequence with observations; solved by forward-backward and Viterbi.
Hyperparameter	A setting chosen before training rather than learned from the training loss.
Inductive bias	The assumptions a model makes that let it prefer some solutions over others.
Inference	Running a trained model to obtain predictions.
IoU	Intersection over union of two regions; the matching criterion behind detection and segmentation metrics.
KV cache	Stored keys and values from previous tokens, so generation does not recompute them.
Label smoothing	Softening one-hot targets to reduce overconfidence.
Latent variable	An unobserved variable inferred by the model.
Layer normalisation	Normalising across features within one sample; batch-independent.
Learning rate	The step size in gradient descent; the most important hyperparameter.
Log-mel spectrogram	Log energies in mel-spaced frequency bands; the standard input representation for speech models.
Logit	An unbounded score before sigmoid or softmax.
LoRA	Low-rank adaptation: training small low-rank updates to frozen weights.
Loss function	The scalar measure of error that training minimises.
mAP	Mean average precision: area under the precision-recall curve, averaged over classes and (in COCO style) IoU thresholds.
MASE	Mean absolute scaled error; forecast error divided by the naive baseline's error, so values below 1 mean you beat it.
Matrix factorisation	Representing a user-item matrix as the product of low-rank user and item factors plus bias terms.
MFU	Model FLOPs utilisation: achieved model FLOPs divided by the hardware's peak; 35-55% is a healthy dense training run.
Mixed precision	Training with 16-bit compute and 32-bit master weights.
MLP	Multilayer perceptron; a stack of fully connected layers.
Momentum	Accumulating a velocity of past gradients to smooth and accelerate descent.
NDCG	Normalised discounted cumulative gain; the standard ranking metric, discounting relevance logarithmically by position.
Non-parametric	A model whose complexity grows with the data, e.g. k-NN.

Term	Definition
Overfitting	Fitting noise in the training data, so test error rises.
Parameter	A number learned from data during training.
Perplexity	$\exp(\text{cross-entropy})$; the standard language-model metric.
Pipeline bubble	Idle time in pipeline parallelism, equal to $(P-1)/(M+P-1)$ for P stages and M microbatches.
Position bias	The tendency of users to click higher-ranked items regardless of relevance; must be corrected before training on clicks.
Potential outcomes	The pair of results a unit would have under treatment and under control; only one is ever observed.
Prompt injection	Instructions hidden in retrieved content that an LLM agent follows; mitigated at the capability layer, not the prompt.
Propensity score	The probability of receiving treatment given covariates; used for matching, weighting and off-policy estimates.
Pruning	Removing weights, channels or blocks from a trained network.
Quantization	Representing weights and activations with fewer bits.
Rademacher complexity	The expected ability of a class to fit random signs on your sample; a data-dependent complexity measure.
RAG	Retrieval-augmented generation: inserting retrieved documents into a model's prompt.
RAG	Retrieval-augmented generation: retrieve passages, then condition the generator on them and cite them.
Receptive field	The region of input that influences one output unit.
Regularisation	Any change intended to reduce test error rather than training error.
Reinforcement learning	Learning a policy from rewards received while acting.
Residual connection	$y = x + F(x)$; preserves gradient flow through depth.
Roofline	The bound $\min(\text{peak FLOP/s}, \text{intensity} \times \text{bandwidth})$ that explains most disappointing hardware benchmarks.
RoPE	Rotary positional embedding; encodes position by rotating queries and keys.
Scaling law	An empirical power law relating loss to parameters, data and compute; the basis of Chinchilla-optimal training.
Self-supervised learning	Supervised learning on labels derived automatically from the data itself.
Semi-supervised learning	Using labelled and unlabelled data together.
SGD	Stochastic gradient descent: updating from mini-batch gradients.
Softmax	Turning a vector of logits into a probability distribution.
Sparsity	The fraction of parameters or activations that are zero.
Speculative decoding	Drafting tokens with a small model and verifying them with the large one, preserving the output distribution.
Stationarity	Constant mean, variance and autocovariance over time; required by classical time-series models, obtained by differencing.
STL decomposition	Splitting a series into seasonal, trend and remainder components using local regression.
Straight-through estimator	Treating a non-differentiable forward op as the identity in the backward pass.
Supervised learning	Learning a mapping from labelled input-output pairs.
Tensor	An n -dimensional array, with autograd support in deep-learning frameworks.
Tokenisation	Splitting text into subword units a model can embed.
Transfer learning	Reusing knowledge from one task to improve another.
Transformer	An architecture built from self-attention and position-wise feedforward blocks with residual connections.

Term	Definition
Two-tower model	Separate user and item encoders scored by dot product, allowing item embeddings to be indexed for fast retrieval.
Underfitting	The model is too simple; both training and test error are high.
Unsupervised learning	Finding structure in data with no labels.
Uplift model	A model of the treatment effect rather than the outcome; targets persuadables and avoids sleeping dogs.
Validation set	Held-out data used to choose hyperparameters and stop training.
Vanishing gradient	Gradient magnitudes shrinking exponentially with depth, stalling early layers.
Variance (statistical)	Sensitivity of the fitted model to which training sample was drawn.
VC dimension	The largest number of points a hypothesis class can label in every possible way; controls the classical generalisation bound.
Viterbi algorithm	Dynamic programming for the most probable hidden state sequence in an HMM.
Weight decay	Shrinking weights towards zero each step; L2 regularisation, decoupled in AdamW.
WER	Word error rate: substitutions plus insertions plus deletions over reference words; can exceed 100%.
Zero-shot	Performing a task with no task-specific training examples.

Appendix C. Study Roadmap, Projects and Resources

A 30-week study plan

Weeks	Focus	Deliverable
1-2	Python, NumPy, pandas, plotting; Chapters 1-2	Load a dataset, clean it, plot five informative figures
3-4	Chapters 3-4: generalisation, splits, leakage, features	A leak-free pipeline with cross-validation
5-6	Chapters 5-7: linear and logistic regression, regularisation	Both implemented from scratch, matching scikit-learn
7-8	Chapters 8-11: k-NN, trees, SVM, ensembles	A tuned gradient-boosting model on a real tabular dataset
9-10	Chapters 12-13: unsupervised learning, evaluation	A clustering study and a full evaluation report with calibration
11-13	Chapters 14-17: MLP, backprop, activations, optimisers	Backpropagation in NumPy, 97%+ on MNIST
14-15	Chapters 18-20: normalisation, regularisation, debugging	A CNN on CIFAR-10 above 90% with a documented recipe
16-17	Chapter 21: CNNs and transfer learning	Fine-tune a pretrained backbone on your own images
18-20	Chapters 22-24: sequences, Transformers, LLMs	A character-level Transformer trained from scratch, plus a RAG demo
21-22	Chapters 25-27: generative and self-supervised models	A VAE and a small diffusion model on 32x32 images
23-24	Chapters 28-31, 34: efficiency and deployment	Quantize, prune and deploy one model to a device or an API, with measured latency
25-26	Chapters 36-37: learning theory, probabilistic modelling	Run the label-randomisation test on two models; fit a GP and a GMM from scratch
27-28	Two domain chapters of your choice from 38-42	One end-to-end project in that domain with the domain's own metric reported against a naive baseline
29	Chapter 43: hardware, memory and distributed training	Measure MFU on a real run and account for every gigabyte of memory it uses

Weeks	Focus	Deliverable
30	Chapters 45-46: causal inference, then the full project walkthrough	Design an A/B test with a power calculation, then run the thirteen stages end to end

Portfolio projects worth building

- **End-to-end tabular:** a leak-free pipeline, tuned gradient boosting, SHAP explanations, calibrated probabilities, a threshold chosen from costs, and a deployed API. This single project demonstrates most of Parts I-II.
- **Vision transfer:** collect a few thousand of your own images, fine-tune a pretrained backbone, evaluate per class, and ship it to a phone with INT8 quantization.
- **From-scratch Transformer:** a character-level model trained on a text corpus you care about, with your own attention implementation.
- **Sensor / time-series:** windowing, leakage-safe group splits, a 1D-CNN or GRU baseline, and an on-device deployment with measured energy.
- **Compression study:** take one model, apply distillation, pruning and quantization, and produce the accuracy/size/latency Pareto curve on real hardware.
- **RAG assistant:** over documents you own, with an evaluation set you wrote and honest accuracy numbers.

Books

Book	Best for
Hands-On Machine Learning (Geron)	The best practical first book; scikit-learn and Keras
An Introduction to Statistical Learning (James et al.)	Classical ML with clear statistics; free PDF
The Elements of Statistical Learning (Hastie et al.)	The rigorous companion to the above
Deep Learning (Goodfellow, Bengio, Courville)	Foundational theory; free online
Dive into Deep Learning (Zhang et al.)	Free, interactive, code-first, kept current
Pattern Recognition and Machine Learning (Bishop)	The Bayesian perspective
Probabilistic Machine Learning (Murphy)	Comprehensive modern reference in two volumes
Reinforcement Learning: An Introduction (Sutton & Barto)	The RL text; free online
Designing Machine Learning Systems (Huyen)	Production and MLOps
Efficient Deep Learning / TinyML literature	Quantization, pruning and edge deployment

Courses, tools and venues

- **Courses:** Andrew Ng's Machine Learning Specialization and Deep Learning Specialization; fast.ai Practical Deep Learning; Stanford CS231n (vision), CS224n (NLP), CS234 (RL); MIT 6.5940 (efficient deep learning); Hugging Face courses (NLP, diffusion, RL).
- **Libraries:** NumPy, pandas, scikit-learn, PyTorch, JAX, LightGBM/XGBoost/CatBoost, Hugging Face transformers/datasets/peft, Optuna, ONNX Runtime, ExecuTorch, TFLite, vLLM.
- **Data and practice:** Kaggle, OpenML, UCI, Hugging Face Datasets, Papers with Code.
- **Venues to skim:** NeurIPS, ICML, ICLR (general); CVPR, ICCV, ECCV (vision); ACL, EMNLP (language); MLSys (systems); arXiv cs.LG and cs.CV for preprints.

THE LAST PIECE OF ADVICE

Depth beats breadth. One project finished end to end - data collected, model trained, honestly evaluated, deployed, monitored, and improved after it failed in production - teaches more than twenty tutorials. The field will keep producing new architectures; the habits in Chapters 3, 13 and 20 will still be what separates a working system from a good validation score.

References

Cited as [n] in the text. (folder) = the PDF is in the thesis folder "text books".

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. (folder)
- [2] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*. Cambridge University Press, 2023. (folder)
- [3] M. Deutel, F. Hannig, C. Mutschler, and J. Teich, "On-device training of fully quantized deep neural networks on Cortex-M microcontrollers," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 4, Apr. 2025; arXiv:2407.10734. (folder)
- [4] U. Evci, T. Gale, J. Menick, P. S. Castro, and E. Elsen, "Rigging the lottery: Making all tickets winners," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, 2020. (folder)
- [5] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256KB memory," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022; arXiv:2206.15472. (folder)
- [6] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks," *J. Machine Learning Research*, 2021. (folder)
- [7] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, *Efficient Processing of Deep Neural Networks*. Morgan & Claypool, 2020; and "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, 2017. (folder)
- [8] C. Ünsalan, B. Höke, and E. Atmaca, *Embedded Machine Learning with Microcontrollers*. Cham: Springer, 2025. (folder)
- [9] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. ESANN*, 2013. (the UCI HAR dataset used by the project)
- [10] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF CVPR*, 2018.
- [11] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. ICML*, 2015.
- [12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE ICCV*, 2015.
- [15] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, 2015.
- [16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Machine Learning Research*, vol. 15, 2014.
- [17] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. ICLR*, 2019.
- [18] D. C. Mocanu *et al.*, "Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science," *Nature Communications*, vol. 9, 2018.
- [19] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.
- [20] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.
- [21] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [22] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [23] T. Masters, *Deep Belief Nets in C++ and CUDA C*, vols. 1-3. Apress, 2018. (folder)
- [24] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. AISTATS*, 2010.
- [25] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, 1998.
- [26] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, 1997.
- [27] A. Vaswani *et al.*, "Attention is all you need," in *Advances in NeurIPS*, 2017.
- [28] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [29] L. Breiman, "Random forests," *Machine Learning*, vol. 45, 2001.

- [30] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, 2001.
- [31] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, 1995.
- [32] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, 2014.
- [33] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *J. American Statistical Association*, vol. 58, 1963.
- [34] A. Amini, A. Amini, *et al.*, *MIT 6.S191: Introduction to Deep Learning*, lecture slides L1-L6. (folder)
- [35] OnDeviceTraining repository (this thesis): *examples/har_classifier* (train_c.c, train_pytorch.py, README sweep tables) and *src/*; and the companion documents ODT_RigL_Complete.pdf and the C book.